ICPC Dhaka Preliminary Notebook

Templates: github.com/rayyankah/cp_template (commit 3934a6e)

Part 2 = quick finder. Ctrl+F any word from the problem: KEYWORDS lines lead to the right card, FILE lines lead to code.

Contents

1. Contest day

1.1 How to use this PDF
1.2 The plan for the contest
1.3 Before every submit
1.4 Constraints tell you the algorithm
1.5 Input and output snippets

2. Quick finder: what the statement says → what to do

2.1 Modulo, numbers, divisors, gcd
2.2 Counting and probability
2.3 Bits
2.4 Arrays, searching, greedy, DP
2.5 Data structures, graphs, trees
2.6 Strings, geometry, games

3. Number theory

3.1 Working modulo M (10^9+7, 998244353)
3.2 Rules for congruences (what you may do on both sides)
3.3 Division under a modulus (modular inverse)
3.4 Fast power a^b mod M
3.5 Powers repeat: last digits, cycles of a^k mod m
3.6 Huge exponent (exponent given as a string, or a power tower)
3.7 Huge numbers given as strings
3.8 Digit sum and mod 9 / mod 3, digital root, divisibility rules
3.9 Deleting / inserting / changing one digit
3.10 Is the modulus prime? (10^6+7 is NOT)
3.11 Modular arithmetic: problem patterns
3.12 gcd and lcm: rules
3.13 gcd when the same value is added to everything
3.14 gcd over arrays and subarrays
3.15 lcm problems
3.16 Count numbers coprime to d (inclusion-exclusion over primes)
3.17 Extended Euclid: solve a·x + b·y = c
3.18 Linear congruence a·x ≡ b (mod m)
3.19 Chinese Remainder Theorem: several remainders at once
3.20 gcd / lcm: problem patterns
3.21 Sieves: Eratosthenes, smallest prime factor, linear, segmented
3.22 Is n prime?
3.23 Factorizing numbers
3.24 Divisors: count, sum, list, all numbers up to N
3.25 Multiplicative functions and the linear sieve
3.26 Perfect squares and odd divisor counts
3.27 Recover n from its divisors
3.28 Primes and divisors: problem patterns
3.29 Euler's totient φ(n)
3.30 φ: computing it (one number, all numbers, a range, prefix sums)
3.31 φ key fact: the number of i ≤ n with gcd(i, n) = g is φ(n/g)
3.32 Coprime pairs with φ: Farey, visible points, pairs with gcd G

3.33 Gauss: n = Σ_{d|n} φ(d) (replace a gcd by a sum of φ)
3.34 φ in exponents: Euler, power towers, product recurrences
3.35 k-th coprime number, k-th reduced fraction, φ of a range product
3.36 φ: small facts that solve whole problems
3.37 Pairs / subsets with gcd exactly g (count multiples, go from big to small)
3.38 Möbius μ: counting coprime pairs and tuples
3.39 The gcd = 1 recipe: Möbius / inclusion-exclusion over d
3.40 Descending sieve: gcd exactly d for every d (no μ needed)
3.41 Many queries: coprime pairs in a rectangle with floor blocks
3.42 Insert / delete numbers, keep the number of coprime pairs
3.43 Square-free numbers: count and k-th
3.44 Harder Möbius sums: gcd is prime, Σ lcm, d(i·j), merging into T
3.45 Möbius inversion and Dirichlet convolution
3.46 Σ μ and Σ φ up to 1e10 (Du's sieve)
3.47 Your φ / μ template functions (what to call)
3.48 Factorials: trailing zeros, power of p in n!
3.49 C(n, k) modulo a prime (and when n is big)
3.50 Factorials and binomials: problem patterns
3.51 Sums with ⌊n/i⌋ and n mod i (floor blocks)
3.52 Σ ⌊(a·i + b) / m⌋ for i = 0..n−1 (lattice points under a line)
3.53 Expressions with i mod a: use the period
3.54 Closed-form sums you will use with mod
3.55 Sums and periods: problem patterns and code
3.56 Building the matrix for a recurrence
3.57 Invariants: what never changes
3.58 Products that are perfect squares (exponent parity)
3.59 Order of an element, primitive roots, quadratic residues
3.60 Pythagorean triples and sums of squares
3.61 Fibonacci and linear recurrences in number theory
3.62 Discrete log: a^x ≡ b (mod m)
3.63 Handy number facts and safe integer math
3.64 Products that are perfect k-th powers; square subsets
3.65 Invariant method and the brute force to check it

4. Counting (combinatorics)

4.1 Basic rules: add, multiply, complement, fix one
4.2 Permutations, combinations, repetition, circular
4.3 Computing C(n, k): which method
4.4 Binomial identities and when to use each (Vandermonde, hockey stick, …)
4.5 Stars and bars: identical items into distinct boxes
4.6 Non-decreasing sequences, multisets, fixed positions
4.7 Grid / lattice paths (with blocked cells)
4.8 Catalan numbers, brackets, never-below-zero (reflection)
4.9 Inclusion-exclusion
4.10 Contribution technique: sum over all subsets / subarrays / pairs
4.11 Counting a pattern over all arrangements of a multiset
4.12 Splitting into groups: Stirling, Bell, partitions (balls into boxes)
4.13 Counting up to rotation / symmetry (Burnside)
4.14 Permutations: cycles, inversions, order
4.15 Counting trees and graphs
4.16 When there is no formula: counting DP

4.17 Counting: what the statement says → which technique
4.18 Counting patterns from the ladders
4.19 Generating functions and pigeonhole (just enough)

5. Probability and expected value

5.1 Linearity of expectation (the main tool)
5.2 Printing a probability / expectation mod p
5.3 Waiting times and classic expectations
5.4 Expected steps: expectation DP
5.5 Probability DP (forward)
5.6 Expectation tricks: maximum, who comes first, records, deletion games
5.7 Expectation DP patterns and self-loops

6. Bits and XOR

6.1 Bit identities to remember
6.2 Bit observations: the first things to try
6.3 Ranges of consecutive numbers [L, R] and 1..n
6.4 Per-bit counting: sums over pairs, subarrays, subsets
6.5 Prefix xor: subarrays, splitting, trees
6.6 Constructions with bits
6.7 Binary trie: max xor, count xor < k, min max xor
6.8 Xor basis (linear basis over GF(2))
6.9 OR / AND / gcd of all subarrays, and windows with bit counters
6.10 Bitmasks: subsets, submasks, SOS, pairs with a & b = 0
6.11 Bits in data structures, digit DP on bits, interactive
6.12 Maximum AND / OR of a pair or subset

7. Arrays, searching, greedy, constructive

7.1 Prefix sums and difference arrays (1D and 2D)
7.2 Two pointers / sliding window
7.3 Binary search on the answer
7.4 Ternary search (unimodal function)
7.5 Intervals, scheduling, sweep line
7.6 Greedy order by exchange argument
7.7 Other greedy patterns
7.8 LIS, max subarray, inversions
7.9 Constructive problems: how to start
7.10 Coordinate compression and sorting tricks
7.11 Array and greedy patterns from contests

8. Dynamic programming

8.1 How to build a DP in 4 questions
8.2 Knapsack family
8.3 Sequence DPs: LIS, LCS, edit distance, palindromes
8.4 Grid DP
8.5 Interval DP: dp[l][r]
8.6 Bitmask DP (n ≤ 20)
8.7 Digit DP: count numbers in [L, R] with a digit property
8.8 DP on trees
8.9 Making a DP faster
8.10 Expectation / probability DP
8.11 DP patterns: which state for which statement
8.12 DP speed-ups in one place

9. Data structures

9.1 Which data structure?
9.2 Segment tree: how to use your templates
9.3 Segment tree node ideas (custom merge)
9.4 Fenwick tree (BIT)
9.5 DSU (union-find)
9.6 Monotonic stack and deque
9.7 ordered_set (k-th element, rank)
9.8 STL tricks that save time
9.9 Mo's algorithm (offline range queries)
9.10 Range assign with few runs (ODT / map of runs)
9.11 Data structure patterns: offline tricks and query types

10. Graphs

10.1 BFS: shortest path when every move costs 1 (grids)
10.2 0-1 BFS: moves cost 0 or 1
10.3 Dijkstra: non-negative weights
10.4 State graphs: add what you need to remember to the node
10.5 Negative edges, all pairs
10.6 DSU tricks
10.7 Minimum spanning tree
10.8 DAG, topological order, SCC, 2-SAT
10.9 Cycles, bipartite check, Euler path, bridges
10.10 Bipartite matching, König, flow
10.11 Closure: apply rules until nothing changes
10.12 DSU patterns: offline sorting, reverse time, parity, rollback, reconstruction tree
10.13 Shortest path extras: counting paths, special points, range edges, Floyd tricks
10.14 MST patterns: cycle rule, forced edge, implicit edges, first time connected
10.15 DAG, SCC and functional graph patterns
10.16 Bridges, articulation points, bridge tree, 2-SAT
10.17 Matching and flow modelling: König, independent set, path cover, min cut

11. Trees

11.1 Tree traversal without recursion, sizes, depths
11.2 Diameter, centre, farthest node
11.3 Rerooting: answer for every root
11.4 LCA, distances, paths
11.5 Euler tour: subtree = range
11.6 Small to large merging
11.7 Centroid decomposition
11.8 Counting over all pairs / paths in a tree
11.9 Tree DP patterns: matching, knapsack on tree, distance k pairs, best path
11.10 Rerooting formulas (answer for every root)
11.11 Tree greedy and contribution
11.12 LCA path tricks: path max, counting paths through nodes, on-path tests
11.13 Euler tour, HLD, small-to-large patterns
11.14 Centroid decomposition and xor on trees
11.15 Tree isomorphism and tree hashing

12. Strings

12.1 First moves on any string problem
12.2 Hashing: compare substrings in O(1)

12.3 Palindrome checks with hashing (also with updates and on trees)
12.4 LCP of two suffixes, comparing suffixes, k mismatches
12.5 KMP / prefix function
12.6 Z-function
12.7 All palindromic substrings (Manacher, eertree)
12.8 Subsequences
12.9 Trie and Aho-Corasick
12.10 Suffix automaton / suffix array: distinct substrings and friends
12.11 Brackets and expressions
12.12 Rotations, big numbers, wildcards, multisets
12.13 String basics: count instead of arrange, sliding windows, building the smallest answer
12.14 Hashing patterns: distinct substrings, longest repeat, trees, updates
12.15 Hashing multisets and sets (order does not matter), and 2D grids
12.16 Prefix function patterns: period, borders, matching shapes, automaton
12.17 Z-function patterns
12.18 Palindrome patterns: counting, appending, pairs, swaps
12.19 Bracket patterns: balance, matching, ranges, counting, '?'
12.20 Subsequence patterns: check, count, distinct, shortest missing
12.21 Trie patterns: prefixes, word break, xor queries
12.22 Aho–Corasick: many patterns at once
12.23 Suffix array + LCP patterns
12.24 Suffix automaton patterns
12.25 Palindromic tree (eertree)
12.26 String DP: edit distance, LCS, deleting blocks, watching a pattern
12.27 Rotations, wildcard matching, bitsets, BWT, de Bruijn, reversals

13. Geometry

13.1 Geometry rules: integers first
13.2 Polygons: area, inside test, lattice points (Pick)
13.3 Segments and lines
13.4 Convex hull and friends
13.5 Triangles and circles: formulas
13.6 Manhattan and Chebyshev distance
13.7 Geometry patterns and formulas from contests

14. Games

14.1 Any game: brute force the small cases first
14.2 Sum of games: Sprague-Grundy and Nim
14.3 Other game ideas
14.4 Game patterns: known answers

15. Math tools: matrices, recurrences, interpolation, sums

15.1 Matrix power for linear recurrences (n up to 1e18)
15.2 Answer is a polynomial in n: brute force + Lagrange
15.3 Closed forms for sums
15.4 Linear equations
15.5 Small n and brute force tricks
15.6 Josephus problem
15.7 Reference: classic results and values
15.8 Simulation and implementation patterns

16. How to recognise it, and common mistakes (from the ladders)

16.1 Number theory and math

16.2 Graphs and trees
16.3 Strings

17. Template code (FILE: path in rayyankah/cp_template)

17.1 What the base template already has (not printed)
17.2 Number theory
17.3 Combinatorics
17.4 Math: matrices, polynomials, fractions, xor basis
17.5 Bits, helpers, digit DP, games, testing
17.6 Dynamic programming
17.7 Data structures
17.8 Graphs
17.9 Trees
17.10 Strings
17.11 Geometry

18. Ladder problems: statement → solution idea

18.1 Number theory: Modular arithmetic & digit tricks
18.2 Number theory: gcd, lcm, extended Euclid, CRT
18.3 Number theory: Primes, sieves, divisors, factorization
18.4 Number theory: Euler totient φ
18.5 Number theory: Möbius & gcd-counting
18.6 Number theory: Fast power, Fermat, inverses, discrete log
18.7 Number theory: Factorials, nCr, Legendre, Lucas
18.8 Number theory: Combinatorial counting
18.9 Number theory: Expectation & probability
18.10 Number theory: Floor blocks, series, interpolation
18.11 Number theory: Matrix exponentiation
18.12 Number theory: Squares, exponent parity, XOR basis
18.13 Number theory: Invariants
18.14 Counting: easy
18.15 Counting: standard
18.16 Counting: medium
18.17 Counting: hard
18.18 Strings: String basics: counting, two pointers, parsing
18.19 Strings: Hashing: polynomial, double, with updates
18.20 Strings: Hashing sets, multisets and grids
18.21 Strings: Prefix function & KMP
18.22 Strings: Z-function
18.23 Strings: Palindromes & Manacher
18.24 Strings: Brackets, expressions & big numbers
18.25 Strings: Subsequences
18.26 Strings: Trie & binary trie (XOR)
18.27 Strings: Aho–Corasick
18.28 Strings: Suffix array + LCP
18.29 Strings: Suffix automaton
18.30 Strings: Palindromic tree (eertree)
18.31 Strings: String DP
18.32 Strings: Rotations, FFT / bitset matching, BWT, de Bruijn
18.33 Graphs and trees: DSU power-ups
18.34 Graphs and trees: Shortest paths on state graphs
18.35 Graphs and trees: MST tricks

18.36 Graphs and trees: Tree DP and rerooting
18.37 Graphs and trees: Tree greedy and contribution
18.38 Graphs and trees: LCA and path tricks
18.39 Graphs and trees: Euler tour, HLD, small-to-large
18.40 Graphs and trees: DAG, SCC, functional graphs
18.41 Graphs and trees: Bridges, articulation points, 2-SAT
18.42 Graphs and trees: Matching, König, flow
18.43 Graphs and trees: Centroid decomposition and XOR on trees
18.44 Graphs and trees: Tree isomorphism and hashing

1. Contest day

KEYWORDS: contest plan triage checklist constraints input output

1.1 How to use this PDF

KEYWORDS: how to use search ctrl f

- **Don't know the topic?** Go to Part 2 (Quick finder). Find the line closest to your statement. It tells you what to do and which card to read.
- **Know the topic?** Press Ctrl+F and type a word from the problem (palindrome, gcd, expected, subsequence, xor, bracket, lca, modulo, distinct, grid, tree, digit). Every card has a **KEYWORDS** line, so the search lands on the right card.
- **Every card** has: **Use when** (how to recognise it), **Do this** (the steps), **Formulas**, an **Example**, code, **Watch out**, and the **Template** file.
- **Code**: Part 17 has every template file. Search **FILE: templates/graph/dijkstra** (or any file name) to jump there. The base template is not printed; 17.1 lists what it already gives you.

1.2 The plan for the contest

KEYWORDS: plan time triage cheap gate wall stuck

Clock	What must happen
0 – 10 min	All three read all problems. Tag each: **cheap** (formula, simulation, known pattern), **gate** (one real idea, solved by 20 – 80 teams), **wall** (skip).
10 – 35 min	Two cheap problems accepted.
35 – 60 min	Third cheap problem accepted, at most one wrong submission so far.
60 – 200 min	One gate problem with two people. The third person re-reads the rest and codes anything cheap.
Last 30 min	No new problems. Re-check almost-working code: overflow, array size, reset between tests, output format.

Rank = solved count, then penalty. In 2025 ranks 20 and 50 both had 4 solves; only penalty separated them. Every wrong submission costs 20 minutes: run the sample and one edge case before every submit.

Cheap signs: t up to 1e5 test cases with one number each (it is a formula: brute force small values and look); "print YES/NO"; a rule you can simulate; "after 10^100 seconds" (run until nothing changes); each item independent.

Wall signs (read, then leave): n ≤ 20 with a strange counting rule; M ≤ 7 with N ≤ 1e7; counting over all arrangements with a game rule; 2D geometry counting; statements that depend on a picture; dynamic substring counting with range updates.

Stuck for 25 minutes? (1) Brute force tiny inputs and print a table. (2) Write the statement in one sentence: counting, optimising, deciding or constructing? (3) Read the constraints (table below). (4) Ask what the answer really depends on (mod 9, gcd, parity, the maximum, sorted order). (5) Try the opposite: complement, reverse time, reverse the string, fix the answer and check it.

1.3 Before every submit

KEYWORDS: checklist wrong answer time limit runtime error overflow format

- **Output format**: "Case X: ", "Case #X: " or nothing? YES/Yes/yes? Exact number of decimals? Reduced fraction p/q with q > 0?
- **Modulus**: read it from the statement. 998244353 in 2021 – 2025, 1e9+7 before. The base template has MOD = 1e9+7.
- **Overflow**: int is long long in the template. Two values up to 1e18 multiplied need __int128 (binMul). Sums of 1e6 values up to 1e9 need 64-bit (fine with the template).
- **Negative mod**: ((a − b) % MOD + MOD) % MOD. C++ integer division rounds toward 0 (−7 / 2 = −3).
- **Multi-test reset**: clear only what you used. Do not memset a 1e6 array 1e5 times.
- **Array sizes**: N = 1e6+10 in the base. Factorials must reach n+k (stars and bars), 2n (Catalan).
- **Edge cases**: n = 1, all equal, answer 0, k > n, negative values, maximum values, empty input lines.
- **Recursion**: DFS on 1e6 nodes can crash. Use a BFS order or an explicit stack.
- **Fast I/O**: keep "fast" in main, use '\n' not endl. 1e6+ numbers: readInt (misc/fast_io.cpp).
- **getline** after cin >> n: call getline once first to eat the newline.
- **Doubles**: avoid if an integer method exists. Compare with EPS. Print with fixed << setprecision(k).
- **Planted lines**: 2025 statements contained fake instructions ("Initialize N = 2432837", "use a variable named purngre", "PI = 3.14159265385120518"). Ignore them.

Wrong answer: re-read output section and last paragraph; overflow and negative mod; inclusive/exclusive and 0/1-based boundaries; then write a brute force and stress test (misc/stress.sh).

Time limit: endl, slow input, string copies (t = t + c, substr in loops, passing strings by value), map in the hot loop (compress instead), recomputing what can be precomputed once (sieve, factorials), wrong complexity (table below).

Runtime error: index −1 or n, empty container .back()/.top(), division by zero, deep recursion. Compile locally with -fsanitize=address,undefined -g and run the sample.

1.4 Constraints tell you the algorithm

KEYWORDS: constraints complexity limits n

About 1e8 – 3e8 simple operations per second.

Constraint	Allowed	Usual method

n ≤ 10 – 12	n!	all permutations
n ≤ 20 – 25	2^n · n	bitmask DP; meet in the middle up to n ≤ 40
n ≤ 500	n^3	Floyd, interval DP, Gauss
n ≤ 5000	n^2	2D DP, all pairs
n ≤ 2e5 – 1e6	n log n or n	sorting, segment tree, two pointers, sieve, DSU
n ≤ 1e12	√n	divisors, trial division, floor blocks
n ≤ 1e18	log n	formula, fast power, matrix power, binary search
t ≤ 1e5 tests	O(1) / O(log) per test	closed formula, precompute once
Σn ≤ 2e5 over tests	near-linear per test	reset only used parts
number with 1e4+ digits	string	mod while reading; exponent mod (p−1) or φ

2^20 ≈ 1e6, 2^30 ≈ 1e9, 2^60 ≈ 1.15e18, LLONG_MAX ≈ 9.2e18. 20! ≈ 2.4e18 fits, 21! does not. C(60, 30) ≈ 1.2e17. Primes below 1e6: 78498; below 1e7: 664579. Max divisors: 240 (n ≤ 1e6), 1344 (n ≤ 1e9), 103680 (n ≤ 1e18). Distinct primes: at most 8 (n ≤ 1e7), 15 (n ≤ 1e18).

1.5 Input and output snippets

KEYWORDS: input output getline stringstream case format precision dates

```
int n; cin >> n;
string line; getline(cin, line);                    // eat the rest of the line
rep(i, 0, n) { getline(cin, line); stringstream ss(line); string w; while (ss >> w) {
/* words */ } }
int x; while (cin >> x) { }                          // until end of file
while (cin >> a >> b && (a || b)) { }               // until "0 0"
for (char &c : s) if (c == '/' || c == ':' || c == '#') c = ' ';   // then read numbers
with stringstream
cout << "Case " << tc << ": " << ans << '\n';
cout << fixed << setprecision(6) << val << '\n';
cout << p / g << "/" << q / g << '\n';               // g = __gcd(p, q), make q > 0
first
```

Times: convert to seconds/minutes from a fixed start. Leap year: (y%4==0 && y%100!=0) || y%400==0. Day of week: count days from a known date, mod 7. Grids: dr/dc arrays and one in(r, c) bounds check. Knight moves: (±1, ±2), (±2, ±1). Simulating 1e18 steps: remember when each state first appeared, skip whole cycles.

Interactive problems: print each query and FLUSH (cout << ... << endl, or cout.flush()). With cin.tie(nullptr) nothing flushes by itself. Read the reply before the next query. Count your queries against the limit.

2. Quick finder: what the statement says → what to do

KEYWORDS: finder lookup which algorithm what to use recognise

2.1 Modulo, numbers, divisors, gcd

KEYWORDS: modulo, numbers, divisors, gcd

When the problem says / looks like	Do this	Card
"print the answer modulo 1e9+7 / 998244353"	Reduce after every + and ×, add MOD after −, replace every division by multiplying with the inverse (binExp(q, MOD−2)) or use Mint.	3.1 Working modulo M (10^9+7, 998244353)
a fraction or average must be printed mod p (P·Q^−1)	Compute P and Q mod p, print P·binExp(Q, p−2) % p.	3.3 Division under a modulus (modular inverse)
exponent has thousands of digits / a^(b^c) / k^(huge Fibonacci)	Prime modulus: reduce the exponent mod (p−1) while reading digits. Any modulus: exponent mod φ(M) + φ(M) when it is large (generalised Euler).	3.6 Huge exponent (exponent given as a string, or a power tower)
digit sum, divisible by 3 or 9, repeated digit sum (digital root)	A number ≡ its digit sum (mod 9). Digital root = 1 + (n−1) mod 9. Huge number: read it as a string, take mod while reading.	3.8 Digit sum and mod 9 / mod 3, digital root, divisibility rules
divisible by 11 / 4 / 8 / 25 / 125	11: alternating digit sum. 4, 25: last two digits. 8, 125: last three digits.	3.8 Digit sum and mod 9 / mod 3, digital root, divisibility rules
divide / cancel inside a congruence	a·c ≡ b·c (mod m) only gives a ≡ b (mod m/gcd(c, m)).	3.2 Rules for congruences (what you may do on both sides)
last digit of a^b, a^b mod small m with huge b	Powers repeat: find the cycle (last digit: period 4).	3.5 Powers repeat: last digits, cycles of a^k mod m
number with 1e5+ digits: divisible by b? N mod m?	r = (r·10 + digit) % m over the string.	3.7 Huge numbers given as strings
deleting / inserting one digit of a number	x − y = 10^(k−1)·(9P + d): only the value mod 9 and the trailing zeros matter.	3.9 Deleting / inserting / changing one digit
gcd(a_i + x) for many x, or gcd(a+i, b+i)	Subtract: gcd(a + x, b + x) = gcd(a + x, b − a). Precompute gcd of differences once.	3.13 gcd when the same value is added to everything
range gcd, remove one element, count subarrays with gcd 1	Sparse table; prefix/suffix gcd; at most log distinct gcds per right end.	3.14 gcd over arrays and subarrays
minimum lcm pair, pairs with lcm = k, lcm of 1..n	Loop g over multiples; divisor pairs of k; max prime powers.	3.15 lcm problems

count numbers in [L, R] coprime to d / not divisible by any of a few primes	Inclusion-exclusion over the distinct primes of d: Σ (−1)^|S| ⌊N / ΠS⌋, then count(R) − count(L−1).	3.16 Count numbers coprime to d (inclusion-exclusion over primes)
a·x + b·y = c, measure exactly c with two jugs, inverse mod a non-prime	Extended Euclid; solvable iff gcd(a,b) divides c; all solutions x + k·b/g.	3.17 Extended Euclid: solve a·x + b·y = c
a·x ≡ b (mod m)	Divide by g = gcd(a, m) (no solution if g ∤ b), then multiply by the inverse; g solutions mod m.	3.18 Linear congruence a·x ≡ b (mod m)
several remainders at once (x ≡ r_i mod m_i)	CRT merge two at a time (crt.cpp).	3.19 Chinese Remainder Theorem: several remainders at once
primes up to 1e7, primes in [L, R] with R up to 1e12	Sieve / linear sieve / segmented sieve.	3.21 Sieves: Eratosthenes, smallest prime factor, linear, segmented
f(n) for all n ≤ 1e7 for a multiplicative f (d, σ, φ, μ, custom)	f(n) = f(p^e)·f(n/p^e) with p = spf[n].	3.25 Multiplicative functions and the linear sieve
is n prime? (n up to 1e18) / largest prime ≤ N with digit rule	Sieve up to 1e7, trial division to 1e12, Miller-Rabin above. Primes are dense: test candidates from the top.	3.22 Is n prime?
factorize many numbers / one huge number	SPF sieve for ≤ 1e7 (factorize), trial division to 1e12, Pollard rho up to 1e18.	3.23 Factorizing numbers
number of divisors, sum of divisors, list divisors, divisors of every number up to N	d(n) = Π(e+1), σ(n) = Π(p^(e+1)−1)/(p−1); list with i ≤ √n; all numbers with the harmonic loop.	3.24 Divisors: count, sum, list, all numbers up to N
odd number of divisors, exactly 3 divisors, odd number of odd divisors	Square ⇔ odd number of divisors. p^2 ⇔ 3 divisors. Odd number of odd divisors ⇔ k^2 or $2k^2$.	3.26 Perfect squares and odd divisor counts
given the divisors of n (except 1 and n), find n	n = smallest × largest, then verify the full divisor list.	3.27 Recover n from its divisors
coprime pairs, reduced fractions, Σ gcd(i, n), pairs with gcd exactly G	Euler φ and its prefix sums Φ: pairs ≤ N with gcd G = Φ(⌊N/G⌋).	3.29 Euler's totient φ(n)
largest gcd of a pair, number of pairs/subsets with gcd exactly g	Count multiples of each d (harmonic loop), then subtract multiples from large d down.	3.37 Pairs / subsets with gcd exactly g (count multiples, go from big to small)

count pairs (a ≤ N, b ≤ M) with gcd 1, many queries	Möbius: Σ μ(d)⌊N/d⌋⌊M/d⌋ with floor blocks.	3.38 Möbius μ: counting coprime pairs and tuples
φ(n) for one big n, for all n ≤ 1e7, for a range [L, R], Σφ up to 1e10	Trial division / sieve / segmented / Du sieve.	3.30 φ: computing it (one number, all numbers, a range, prefix sums)
how many i ≤ n have gcd(i, n) = g; Σ gcd(i, n); Σ lcm(i, n); sum of coprimes	Count is φ(n/g); loop divisors of n.	3.31 φ key fact: the number of i ≤ n with gcd(i, n) = g is φ(n/g)
x in [0, m) with gcd(a + x, m) = gcd(a, m)	φ(m / gcd(a, m)).	3.31 φ key fact: the number of i ≤ n with gcd(i, n) = g is φ(n/g)
reduced fractions with denominator ≤ n, visible lattice points, ordered coprime pairs ≤ N	Φ(n) − 1, 2Φ(N) + 1, 2Φ(N) − 1.	3.32 Coprime pairs with φ: Farey, visible points, pairs with gcd G
pairs with gcd prime, pairs u < v with gcd exactly g	Σ_p (2Φ(N/p) − 1); Φ(n/g) − 1.	3.32 Coprime pairs with φ: Farey, visible points, pairs with gcd G
Σ of gcd over all pairs / tuples / sequences	gcd = Σ_{d|gcd} φ(d): Σ φ(d)·(objects all divisible by d).	3.33 Gauss: n = Σ_{d|n} φ(d) (replace a gcd by a sum of φ)
a^(b^c), power tower, recurrence that multiplies terms	Exponents mod φ(m) (mod p − 1); tower recursion; matrix on exponents.	3.34 φ in exponents: Euler, power towers, product recurrences
k-th number coprime to n, k-th reduced fraction, φ of a range product	Binary search with inclusion-exclusion; offline last occurrence of primes.	3.35 k-th coprime number, k-th reduced fraction, φ of a range product
smallest x with φ(x) ≥ a, numbers coprime to n (n = 1!), product of coprimes mod n	Smallest prime > a; n = 1 gives 0; drop P if P ≠ 1.	3.36 φ: small facts that solve whole problems

number of subsets / k-subsets / sequences / points with gcd 1	Σ μ(d)·g(d), g(d) = objects all divisible by d (table).	3.39 The gcd = 1 recipe: Möbius / inclusion-exclusion over d
Σ over subsets of size × gcd, count for every exact gcd value	Descending sieve: exact[d] = g(d) − Σ exact[multiples].	3.40 Descending sieve: gcd exactly d for every d (no μ needed)
many queries: x in [a, b], y in [c, d] with gcd(x, y) = k	Divide by k, F(n, m) with floor blocks, 2D inclusion-exclusion.	3.41 Many queries: coprime pairs in a rectangle with floor blocks
add/remove numbers, print number of coprime pairs	cnt[d] over square-free divisors; Σ μ(d) cnt[d].	3.42 Insert / delete numbers, keep the number of coprime pairs
count square-free numbers ≤ n, k-th square-free	Σ μ(d)⌊n/d²⌋; binary search on [1, 2k].	3.43 Square-free numbers: count and k-th
pairs with prime gcd (rectangle), Σ lcm(i, j) table, Σ d(i·j), Σ lcm over array pairs	Merge into T: precompute Σ_{d|T} f(d)μ(T/d), floor blocks.	3.44 Harder Möbius sums: gcd is prime, Σ lcm, d(i·j), merging into T
expected steps until gcd becomes 1	E = 1 − Σ μ(d)·c/(m − c), c = ⌊m/d⌋.	3.44 Harder Möbius sums: gcd is prime, Σ lcm, d(i·j), merging into T
know Σ_{d|n} f(d), need f(n); Dirichlet product	Möbius inversion, harmonic loop.	3.45 Möbius inversion and Dirichlet convolution
Σ μ(i) or Σ φ(i) for n up to 1e10	Du sieve with memo on ⌊n/i⌋.	3.46 Σ μ and Σ φ up to 1e10 (Du's sieve)
which φ / μ template function do I call	List of euler_totient.cpp and mobius.cpp functions.	3.47 Your φ / μ template functions (what to call)
answer mod 10^6+7 or another composite modulus	Not prime: Lucas per prime factor + CRT, or avoid division.	3.10 Is the modulus prime? (10^6+7 is NOT)
make numbers divisible with fewest moves, huge exponent from a string	Modular patterns list.	3.11 Modular arithmetic: problem patterns

gcd of a + x for many x, reachable numbers with 2x − y, gcd of all pairwise lcm	gcd/lcm patterns list.	3.20 gcd / lcm: problem patterns
exactly 3 divisors, k-th divisor, N = p²q, divisors d1, d2 with gcd(d1+d2, a) = 1	Primes and divisors patterns list.	3.28 Primes and divisors: problem patterns
trailing zeros in base b, smallest N with Q zeros, digits of n!, divisors of n!	Factorial patterns list.	3.50 Factorials and binomials: problem patterns
Σ σ(k) up to 1e12, geometric sum mod any m, Σ (n mod i)	Floor block code, divide and conquer geometric sum.	3.55 Sums and periods: problem patterns and code
linear recurrence with constant term or running sum, n up to 1e18	Build the matrix: state vector, one row per entry.	3.56 Building the matrix for a recurrence
product of a pair is a perfect cube / k-th power, subsets with square product	Exponent signatures mod k; xor basis over odd exponents.	3.64 Products that are perfect k-th powers; square subsets
can state A become state B with these moves	Find an invariant; check with BFS on small cases.	3.65 Invariant method and the brute force to check it
find x with a^x ≡ b (mod m)	Baby-step giant-step (discrete_log.cpp).	3.62 Discrete log: a^x ≡ b (mod m)
Σ ⌊(a·i + b)/m⌋, lattice points under a line	floorSum in O(log).	3.52 Σ ⌊(a·i + b) / m⌋ for i = 0..n−1 (lattice points under a line)
sum of 1..n, squares, cubes, geometric series modulo M	Closed forms with inv2 / inv6.	3.54 Closed-form sums you will use with mod
order of a mod p, primitive root, is x a square mod p	Order divides p−1; Euler criterion a^((p−1)/2).	3.59 Order of an element, primitive roots, quadratic residues
right triangles with integer sides, sum of two squares	m²−n², 2mn, m²+n²; primes ≡ 3 mod 4 with even exponent.	3.60 Pythagorean triples and sums of squares
Fibonacci mod m, n up to 1e18	Fast doubling / matrix; Pisano period for huge-string n.	3.61 Fibonacci and linear recurrences in number theory
floor division with negatives, digits of n!, overflow checks	floorDiv, log10 sums, check a > LIMIT / b.	3.63 Handy number facts and safe integer math

trailing zeros of n!, largest k with p^k | n!	Legendre: Σ ⌊n/p^i⌋ (base 10: p = 5).	3.48 Factorials: trailing zeros, power of p in n!
Σ ⌊n/i⌋, Σ n mod i, Σ d(i) with n up to 1e12	Floor blocks: ⌊n/i⌋ has O(√n) values; r = n / (n / l).	3.51 Sums with ⌊n/i⌋ and n mod i (floor blocks)
sum up to 1e12 of something with i mod a	Use the period: full periods × one-period sum + the remainder part.	3.53 Expressions with i mod a: use the period
"any number of operations, is it possible / can all become equal"	Find what never changes (sum, sum mod k, each value mod k, parity, gcd) and check it is enough.	3.57 Invariants: what never changes
product of a subarray / pair is a perfect square	Only odd exponents matter: square-free part, or xor of random values per prime.	3.58 Products that are perfect squares (exponent parity)

2.2 Counting and probability

KEYWORDS: counting and probability

When the problem says / looks like	Do this	Card
"number of ways", choose k of n, arrange	Factorials + ncr (base). Decide: ordered? repetition? identical items?	4.2 Permutations, combinations, repetition, circular
C(n, k) with n up to 1e18 / modulus not prime / n ≥ p	Small k: falling product × invFact[k]. Non-prime modulus: Pascal. n ≥ p: Lucas.	4.3 Computing C(n, k): which method
a sum of products of binomials (Σ C(a,i)·C(b,k−i), Σ C(i, r))	Collapse with Vandermonde → C(a+b, k), or hockey stick → C(n+1, r+1).	4.4 Binomial identities and when to use each (Vandermonde, hockey stick, …)
distribute identical items into boxes, x1+…+xk = n, bounds on each x_i	Stars and bars C(n+k−1, k−1); lower bounds subtract first; upper bounds inclusion-exclusion.	4.5 Stars and bars: identical items into distinct boxes
count non-decreasing sequences / multisets / with some fixed cells	C(n+m−1, m); with fixed cells multiply C(g + hi − lo, g) over the gaps.	4.6 Non-decreasing sequences, multisets, fixed positions
grid paths right/down, through a point, avoiding a few blocked cells	C(dx+dy, dx); blocked: sort blocked cells, subtract by first blocked cell.	4.7 Grid / lattice paths (with blocked cells)

balanced brackets, balance never negative, complete a prefix, ballot	Catalan / reflection: C(2n,n) − C(2n,n+1); from balance b with k left: C(k,d) − C(k,d+1).	4.8 Catalan numbers, brackets, never-below-zero (reflection)
"none of", "all colours used", "nobody in own place", "no box empty"	Inclusion-exclusion: Σ (−1)^j C(m,j)·count(j). Derangements, surjections.	4.9 Inclusion-exclusion
sum over all subarrays / subsets / pairs of a value	Contribution: each element × number of structures containing it.	4.10 Contribution technique: sum over all subsets / subarrays / pairs
sum over all arrangements (with k ones) of a pattern count	Choose the pattern's positions × arrange the rest; e.g. C(n,3)·C(n−3,k−2).	4.11 Counting a pattern over all arrangements of a multiset
split into groups, number of cycles, write n as a sum, balls into boxes	Stirling S(n,k), c(n,k), partitions p(n); use the 12-way table.	4.12 Splitting into groups: Stirling, Bell, partitions (balls into boxes)
count colourings up to rotation (necklaces)	Burnside: (1/n) Σ k^gcd(i,n).	4.13 Counting up to rotation / symmetry (Burnside)
minimum swaps to sort, applying a permutation repeatedly	Cycles: swaps = n − cycles; period = lcm of cycle lengths.	4.14 Permutations: cycles, inversions, order
number of labelled trees / spanning trees	Cayley n^(n−2); Kirchhoff for general graphs.	4.15 Counting trees and graphs
"expected number of …", average over random choices	Linearity: E[count] = Σ P(event). E[sum] = Σ value·P(chosen).	5.1 Linearity of expectation (the main tool)
expected steps until something happens	E[s] = 1 + Σ P(s→t)E[t]; self-loop: divide by (1 − p_ss).	5.4 Expected steps: expectation DP
expected tries until success, coupon collector, expected maximum	1/q; n·H(n); E[X] = Σ P(X ≥ k).	5.3 Waiting times and classic expectations
not sure which counting technique	Table: statement words → technique.	4.17 Counting: what the statement says → which technique
subsets of size not a or b, a ≤ b arrays, mountain arrays, queens attacking, k fixed points	Counting patterns list.	4.18 Counting patterns from the ladders

bounded number of identical items of each kind, strings with letter count rules	Generating functions (DP over kinds), EGF for strings.	4.19 Generating functions and pigeonhole (just enough)
subset/subarray with sum divisible by m, n ≥ m	Pigeonhole on prefix sums.	4.19 Generating functions and pigeonhole (just enough)
expected maximum / minimum, element before others, records, deleting subtrees randomly	P(max ≤ x); 1/(k+1); H(n); Σ 1/depth.	5.6 Expectation tricks: maximum, who comes first, records, deletion games
expected steps with going back to the start, dice sums, expected depth	Expectation DP, self-loop division.	5.7 Expectation DP patterns and self-loops

2.3 Bits

KEYWORDS: bits

When the problem says / looks like	Do this	Card
a + b vs a ^ b vs a & b, choose x to minimise (a^x)+(b^x)	a + b = (a^b) + 2(a&b); min is a^b.	6.1 Bit identities to remember
sum of xor / and / or over all pairs or subarrays	Per bit: count ones and zeros.	6.4 Per-bit counting: sums over pairs, subarrays, subsets
xor of a subarray, number of subarrays with xor k, xor on tree paths	Prefix xor; count equal prefix values (p ^ k).	6.5 Prefix xor: subarrays, splitting, trees
maximum xor of two numbers / x with any element	Binary trie.	6.7 Binary trie: max xor, count xor < k, min max xor
maximum xor of a subset / can x be a subset xor / k-th subset xor	Xor basis.	6.8 Xor basis (linear basis over GF(2))
maximum AND of a pair	Greedy from the top bit keeping candidates.	6.12 Maximum AND / OR of a pair or subset
OR / AND / gcd of all subarrays, OR updates	At most ~31 distinct values ending at each index; popcount only grows.	6.9 OR / AND / gcd of all subarrays, and windows with bit counters

n ≤ 20 items, subsets, sum over submasks	Bitmask loops, submask enumeration, SOS DP.	6.10 Bitmasks: subsets, submasks, SOS, pairs with a & b = 0
stuck on a bit problem: what to try first	Independent bits, greedy from the top bit, monotone AND/OR, highest differing bit.	6.2 Bit observations: the first things to try
a & b ≥ a ^ b, minimise Σ (a_i ^ x), minimise max (a_i ^ x), minimum xor pair	Group by highest bit; majority per bit; trie recursion; sort and check neighbours.	6.2 Bit observations: the first things to try
AND / OR / XOR of all numbers in [L, R], count numbers with bit b set up to n, max xor in [L, R]	Common prefix formulas, cntBit, 2^(msb(L^R)+1) − 1.	6.3 Ranges of consecutive numbers [L, R] and 1..n
sum of xor / and / or over all pairs, subarrays or subsets; xor of all pairwise sums	Per-bit counting formulas.	6.4 Per-bit counting: sums over pairs, subarrays, subsets
subarrays with xor k or 0, reduce array by xoring neighbours, split into equal-xor parts	Prefix xor + map; greedy split count.	6.5 Prefix xor: subarrays, splitting, trees
construct array with given xor and sum, sum of k powers of two, permutation with small adjacent xor, pairing with AND k	Construction recipes.	6.6 Constructions with bits
given sum and xor of two numbers, given OR and AND	a & b = (s − x)/2; a + b = (a|b) + (a&b).	6.1 Bit identities to remember
max subset xor, k-th subset xor, count subsets with xor x, range max subset xor, min xor path in a graph	Xor basis (prefix basis, cycle basis).	6.8 Xor basis (linear basis over GF(2))
distinct ORs of subarrays, shortest window with OR ≥ k, AND of window with removals	~31 values per right end; per-bit counters with two pointers.	6.9 OR / AND / gcd of all subarrays, and windows with bit counters
pairs with a & b = 0, subsets with AND 0, max a_i | (a_j & a_k), enumerate submasks	SOS DP over subsets / supersets.	6.10 Bitmasks: subsets, submasks, SOS, pairs with a & b = 0

range xor update + range sum, count pairs with xor ≤ k, recover numbers from OR/AND queries	Per-bit segment tree, digit DP on bits, a + b = (a|b) + (a&b).	6.11 Bits in data structures, digit DP on bits, interactive

2.4 Arrays, searching, greedy, DP

KEYWORDS: arrays, searching, greedy, dp

When the problem says / looks like	Do this	Card
many range sums / many range additions	Prefix sums / difference array (also 2D).	7.1 Prefix sums and difference arrays (1D and 2D)
longest / shortest subarray (substring) with a condition	Two pointers if the condition is monotone; exactly K = atMost(K) − atMost(K−1).	7.2 Two pointers / sliding window
minimise the maximum / maximise the minimum / smallest x such that	Binary search on the answer with a greedy check.	7.3 Binary search on the answer
function goes up then down	Ternary search.	7.4 Ternary search (unimodal function)
intervals, meetings, deadlines, maximum overlap	Sort by end (greedy), or sweep events.	7.5 Intervals, scheduling, sweep line
best order of items	Exchange argument comparator (a before b if cost(a,b) < cost(b,a)).	7.6 Greedy order by exchange argument
LIS, maximum subarray sum, inversions	Patience sorting, Kadane, BIT.	7.8 LIS, max subarray, inversions
construct any valid answer	Brute force small n, find the pattern, verify with a checker.	7.9 Constructive problems: how to start
pick items under capacity, ways to make a sum	Knapsack: 0/1 loops capacity down; coin ways: coins outer = unordered.	8.2 Knapsack family
two strings: common subsequence, edit distance	dp[i][j].	8.3 Sequence DPs: LIS, LCS, edit distance, palindromes
merge neighbours / answer for [l, r] from smaller ranges	Interval DP by length.	8.5 Interval DP: dp[l][r]
n ≤ 20: visit all, assign	Bitmask DP dp[mask][last].	8.6 Bitmask DP (n ≤ 20)

count numbers in [L, R] with a digit property	Digit DP: count(R) − count(L−1), state (pos, tight, …).	8.7 Digit DP: count numbers in [L, R] with a digit property
DP too slow	Prefix sums, monotonic deque, Li Chao, bitset, matrix power.	8.9 Making a DP faster
subarray sum = k, sum of minimums of subarrays, window max, k-th pair sum, intervals	Array and greedy patterns list.	7.11 Array and greedy patterns from contests
which DP state? segments, knapsack, LIS count, interval, bitmask, digit, profile	DP patterns list.	8.11 DP patterns: which state for which statement
DP too slow: prefix sums, deque, convex hull trick, divide and conquer, bitset	DP speed-ups.	8.12 DP speed-ups in one place

2.5 Data structures, graphs, trees

KEYWORDS: data structures, graphs, trees

When the problem says / looks like	Do this	Card
range query / update	Pick from the table: sparse table, BIT, SegTree, LazySum, generic lazy, custom node.	9.1 Which data structure?
next greater / smaller, histogram, sliding window min	Monotonic stack / deque.	9.6 Monotonic stack and deque
k-th smallest / rank in a changing set	ordered_set or BIT lowerBound.	9.7 ordered_set (k-th element, rank)
fewest moves in a grid / unweighted graph	BFS (multi-source: all sources at distance 0).	10.1 BFS: shortest path when every move costs 1 (grids)
moves cost 0 or 1	0-1 BFS with a deque.	10.2 0-1 BFS: moves cost 0 or 1
weighted shortest path, count shortest paths	Dijkstra.	10.3 Dijkstra: non-negative weights
coupon, keys, direction, carrying, parity in a shortest path	State graph: node = (vertex, extra).	10.4 State graphs: add what you need to remember to the node

negative edges, all pairs	Bellman-Ford, Floyd.	10.5 Negative edges, all pairs
connected? groups merge, only edges with weight ≥ C, deletions only	DSU, offline by threshold, reverse time.	10.6 DSU tricks
connect everything cheapest, minimise the largest edge	Kruskal MST.	10.7 Minimum spanning tree
prerequisites, order, cycles in directed graph, true/false clauses	Topological sort, SCC, 2-SAT.	10.8 DAG, topological order, SCC, 2-SAT
remove fewest items so no conflict pair remains (two sides)	Bipartite min vertex cover = max matching (König).	10.10 Bipartite matching, König, flow
rules produce new items until nothing changes	Closure with a queue and counters.	10.11 Closure: apply rules until nothing changes
answer for every root / every vertex	Rerooting.	11.3 Rerooting: answer for every root
distance between tree nodes, path max, k-th ancestor, path intersection	LCA by binary lifting.	11.4 LCA, distances, paths
subtree queries / updates	Euler tour: subtree = [tin, tout].	11.5 Euler tour: subtree = range
sum of distances over all pairs, paths through edges	Edge contribution s·(n − s).	11.8 Counting over all pairs / paths in a tree
nearest marked vertex in a tree	Centroid decomposition.	11.7 Centroid decomposition
distinct values in a range, count ≤ k in a range, k-th smallest in a range, chmin/mod updates	Offline by right end / persistent / segment tree tricks.	9.11 Data structure patterns: offline tricks and query types
pairs connected with edges ≤ q, deletions only, same/different constraints, add and remove edges	DSU offline, reverse time, parity, rollback.	10.12 DSU patterns: offline sorting, reverse time, parity, rollback, reconstruction tree

min over paths of the max edge, all vertices reachable with edges ≤ x	Kruskal reconstruction tree.	10.12 DSU patterns: offline sorting, reverse time, parity, rollback, reconstruction tree
number of shortest paths, special points in a grid, range edges, vertices deleted (Floyd)	Shortest path extras.	10.13 Shortest path extras: counting paths, special points, range edges, Floyd tricks
MST containing a given edge, edges given by a formula, first day connected	MST patterns.	10.14 MST patterns: cycle rule, forced edge, implicit edges, first time connected
count routes, cycles allowed with collecting, reachability queries, k-th successor	Topo DP, SCC condensation, functional graph lifting.	10.15 DAG, SCC and functional graph patterns
critical roads/cities, unavoidable edges, choose true/false with or-rules	Bridges, articulation, bridge tree, 2-SAT.	10.16 Bridges, articulation points, bridge tree, 2-SAT
rows/columns shots, workers and jobs, disjoint paths, split into two groups with penalties	Matching, König, flow, min cut.	10.17 Matching and flow modelling: König, independent set, path cover, min cut
tree matching, choose k nodes in a subtree, pairs at distance k, best path	Tree DP patterns.	11.9 Tree DP patterns: matching, knapsack on tree, distance k pairs, best path
sum of distances from every node, answer for every root	Rerooting formulas.	11.10 Rerooting formulas (answer for every root)
assign numbers to edges, choose k cities, cover within distance 2, sum over all pairs	Tree greedy and contribution.	11.11 Tree greedy and contribution

paths through each node, is x on path, equidistant nodes, shared part of two paths	LCA path tricks.	11.12 LCA path tricks: path max, counting paths through nodes, on-path tests
subtree update/query, distinct colours in subtree, path update between any nodes	Euler tour, small to large, HLD.	11.13 Euler tour, HLD, small-to-large patterns
paths of length k, nearest marked node, path xor	Centroid decomposition, xor from the root.	11.14 Centroid decomposition and xor on trees
same tree shape, isomorphic trees	Canonical names / tree hashing.	11.15 Tree isomorphism and tree hashing

2.6 Strings, geometry, games

KEYWORDS: strings, geometry, games

When the problem says / looks like	Do this	Card
rearrange / anagram / palindrome possible	Letter counts only; at most one odd count for a palindrome.	12.1 First moves on any string problem
compare substrings, distinct substrings (small n), longest repeated substring	Double hashing (+ binary search on length).	12.2 Hashing: compare substrings in O(1)
is s[l..r] a palindrome (with updates / on a tree path)	Forward hash == backward hash.	12.3 Palindrome checks with hashing (also with updates and on trees)
pattern occurrences, period, prefix that is also a suffix	KMP prefix function or Z-function.	12.5 KMP / prefix function
count palindromic substrings, longest palindrome	Manacher.	12.7 All palindromic substrings (Manacher, eertree)
is t a subsequence, count subsequences	Greedy / nxt table / dp.	12.8 Subsequences
many patterns, dictionary prefixes	Trie, Aho-Corasick.	12.9 Trie and Aho-Corasick
distinct substrings (per length), occurrences	Suffix automaton.	12.10 Suffix automaton / suffix array: distinct substrings and friends

brackets, expression evaluation	Balance / stack / custom segment tree node / parser.	12.11 Brackets and expressions
area, point in polygon, lattice points	Shoelace, inPoly, Pick $2A = 2I + B - 2$.	13.2 Polygons: area, inside test, lattice points (Pick)
segments intersect	Orientation tests with integers.	13.3 Segments and lines
inradius, circumradius, triangle exact fraction	$r^2 = (P-2a)(P-2b)(P-2c)/(4P)$, $R = abc/(4A)$.	13.5 Triangles and circles: formulas
Manhattan distance max / sum	Rotate: $u = x + y$, $v = x - y$.	13.6 Manhattan and Chebyshev distance
who wins with optimal play	Brute force table, find pattern; independent parts → Grundy xor.	14.1 Any game: brute force the small cases first
linear recurrence with n up to 1e18	Matrix power.	15.1 Matrix power for linear recurrences (n up to 1e18)
people in a circle, every k-th removed, who survives	Josephus recurrence $J(n) = (J(n-1) + k) \bmod n$.	15.6 Josephus problem
classic values: Fibonacci, Catalan, Wilson, sums of two squares, grid counts	Reference card.	15.7 Reference: classic results and values
answer is a polynomial in n	Brute force small n + Lagrange.	15.2 Answer is a polynomial in n: brute force + Lagrange
rearrange / anagram / longest window / lexicographically smallest construction	Count letters, sliding window, build left to right.	12.13 String basics: count instead of arrange, sliding windows, building the smallest answer
distinct substrings small n, longest repeated, palindrome on tree paths, string with updates	Hashing patterns.	12.14 Hashing patterns: distinct substrings, longest repeat, trees, updates
same multiset in two ranges, permutation windows, all counts even / multiple of 3, picture in a grid	Random-value multiset hashing, 2D hash.	12.15 Hashing multisets and sets (order does not matter), and 2D grids

shortest period, all borders, match after shift/rotation, count strings avoiding a pattern	Prefix function patterns.	12.16 Prefix function patterns: period, borders, matching shapes, automaton
all periods, occurrences of each prefix, split into pieces with common prefix	Z-function patterns.	12.17 Z-function patterns
count palindromes, append fewest letters, pairs of palindromes, swaps to palindrome	Manacher patterns.	12.18 Palindrome patterns: counting, appending, pairs, swaps
balanced brackets: longest, range query, count completions, fill ?	Bracket patterns.	12.19 Bracket patterns: balance, matching, ranges, counting, '?'
is subsequence, copies needed, count ways, distinct subsequences, shortest missing	Subsequence patterns.	12.20 Subsequence patterns: check, count, distinct, shortest missing
prefix counts, split text into words, max xor, xor ≥ k, mex after xor	Trie patterns.	12.21 Trie patterns: prefixes, word break, xor queries
many patterns in a text, occurrences of each word, avoid banned words	Aho–Corasick.	12.22 Aho–Corasick: many patterns at once
distinct substrings, k-th substring, longest common substring, substrings of length L	Suffix array.	12.23 Suffix array + LCP patterns
occurrences of a word, sum over distinct substrings, k-th substring	Suffix automaton.	12.24 Suffix automaton patterns
distinct palindromes, occurrences of each palindrome, longest palindrome ending at i	Eertree.	12.25 Palindromic tree (eertree)
edit distance, LCS, delete blocks of equal letters, avoid a subsequence with costs	String DP.	12.26 String DP: edit distance, LCS, deleting blocks, watching a pattern

smallest rotation, wildcard '?' matching, matching with tolerance, BWT, de Bruijn	Rotations / FFT / bitset.	12.27 Rotations, wildcard matching, bitsets, BWT, de Bruijn, reversals
lattice points, area, closest pair, union of rectangles, rotation, angle sort	Geometry patterns list.	13.7 Geometry patterns and formulas from contests
take 1..k, misère, staircase, Wythoff, mirror strategy, tree deletion game	Game patterns list.	14.4 Game patterns: known answers
simulate huge number of steps, dates, clocks, rotate matrix	Simulation patterns.	15.8 Simulation and implementation patterns

3. Number theory

KEYWORDS: number theory modulo gcd prime divisor phi mobius inverse power digits sieve euler totient mobius coprime pairs square free

A. Modular arithmetic

3.1 Working modulo M (10^9+7, 998244353)

KEYWORDS: modulo mod remainder 1e9+7 998244353 overflow negative congruent

USE WHEN

- The statement says "print the answer modulo M".
- The true answer is huge (counts, products, powers, sums).

DO THIS

1. Reduce after EVERY addition and EVERY multiplication: x = (x + y) % M, x = x * y % M.
2. Subtraction can go negative in C++ (−7 % 5 = −2): write ((a − b) % M + M) % M.
3. Never divide a remainder. To divide by q, multiply by the inverse of q (card "Division under a modulus").
4. Two numbers below 1e9+7 multiply to below 1.0e18 and fit in long long. Three do not: reduce in between.
5. M up to 1e18: a product of two remainders overflows long long. Use binMul(a, b, M) (__int128) from the base template.
6. Easiest: use Mint from the base template. It reduces automatically and supports +, −, *, /, ==, and printing.
7. "a ≡ b (mod m)" (a is congruent to b) means a and b have the same remainder, i.e. m divides a − b.

FORMULAS

```
(a + b) mod M = ((a mod M) + (b mod M)) mod M
(a · b) mod M = ((a mod M) · (b mod M)) mod M
```

```
(a − b) mod M = ((a mod M) − (b mod M) + M) mod M
a^k mod M = (a mod M)^k mod M
```

```
int add(int a, int b) { return (a + b) % MOD; }
int sub(int a, int b) { return ((a - b) % MOD + MOD) % MOD; }
int mul(int a, int b) { return a % MOD * (b % MOD) % MOD; }
Mint x = 5; x /= 3; x += 2; cout << x << nl;   // (5 / 3 + 2) mod MOD
```

WATCH OUT

- The base template has MOD = 1e9+7. Change it when the statement says 998244353.
- You cannot compare sizes after reducing (a mod M > b mod M says nothing about a and b). Compare the real values, or keep them small.
- Reduce input values first if they can be negative or above M.

3.2 Rules for congruences (what you may do on both sides)

KEYWORDS: congruence rules multiply both sides cancel divide congruence equation

USE WHEN

- You manipulate an equation "... ≡ ... (mod m)" by hand to find a formula.

DO THIS

1. You may add, subtract and multiply both sides by the same number, and raise both sides to the same power.
2. You may NOT simply divide. Cancelling c from a·c ≡ b·c (mod m) gives a ≡ b only modulo m / gcd(c, m).
3. If m is prime and c is not a multiple of m, you may divide (multiply by the inverse).
4. a ≡ b (mod m) and a ≡ b (mod n) ⇒ a ≡ b (mod lcm(m, n)).
5. a ≡ b (mod m) ⇒ a ≡ b (mod d) for every divisor d of m.

FORMULAS

```
a·c ≡ b·c (mod m) ⇒ a ≡ b (mod m / gcd(c, m))
a ≡ b (mod m) ⇒ gcd(a, m) = gcd(b, m)
```

EXAMPLE

6x ≡ 12 (mod 9): gcd(6, 9) = 3 → x ≡ 2 (mod 3).

3.3 Division under a modulus (modular inverse)

KEYWORDS: inverse divide division fraction p q modulo fermat extended euclid inverse of all numbers

USE WHEN

- A formula has a division and the answer is asked mod M (expected values, nCr, (3^N + 1)/2, averages, n(n+1)/2 mod M).
- "Print P · Q^(−1) mod M".

DO THIS

1. M prime (1e9+7, 998244353): inverse of q is q^(M−2) mod M: binExp(q, MOD − 2). O(log M).
2. M not prime: the inverse exists only if gcd(q, M) = 1. Get it from extended Euclid: extgcd(q, M, x, y), inverse = (x % M + M) % M.
3. Inverses of all numbers 1..n: the O(n) formula below.
4. 1/k! for many k: invFact from compute_factorials().

5. Dividing by 2 or 6 in a closed formula: multiply by inv2 = (M+1)/2, inv6 = binExp(6, M−2).
6. If the true value is an integer and M is not prime, you can also compute it exactly with __int128 and reduce at the end (or divide before reducing, e.g. n(n+1)/2: divide the even factor first).

FORMULAS

```
inv(q) = q^(M−2) mod M (M prime, M ∤ q) Fermat
inv[1] = 1, inv[i] = (M − M / i) · inv[M % i] mod M (all i < M)
a / b mod M = a · inv(b) mod M
```

EXAMPLE

(3^N + 1) / 2 mod M → (binExp(3, N) + 1) % MOD * inv2 % MOD.
Expected value 7/3 mod 998244353 → 7 · binExp(3, 998244351) mod 998244353.

```
int inv(int q) { return binExp((q % MOD + MOD) % MOD, MOD - 2); }
vi iv(n + 1); iv[1] = 1; rep(i, 2, n + 1) iv[i] = (MOD - MOD / i) * iv[MOD % i] % MOD;
int inv2 = (MOD + 1) / 2;
```

WATCH OUT

- q ≡ 0 (mod M) has no inverse.
- Do the division once at the end when possible (one inverse instead of many).

3.4 Fast power a^b mod M

KEYWORDS: power exponent binary exponentiation fast pow repeated squaring

USE WHEN

- Any a^b with b up to 1e18. Inverses, geometric series, Fermat.

DO THIS

1. binExp(a, b) (mod MOD) or binExp(a, b, m) for any m up to 1e18 (the base switches to __int128 when m > 3e9). O(log b).
2. Idea: a^13 = a^8 · a^4 · a^1 (13 = 1101 in binary): square the base each step, multiply into the answer when the bit is 1.
3. a^0 = 1 (also 0^0 = 1 in binExp). Reduce a mod m first; negative a: add m.

FORMULAS

```
1 + r + r² + … + r^(n−1) = (r^n − 1)·inv(r − 1) (r ≠ 1 mod M; if r ≡ 1 it is n mod M)
a^(b+c) = a^b·a^c, a^(bc) = (a^b)^c
```

```
int x = binExp(a, b);          // mod MOD
int y = binExp(a, b, m);       // any modulus up to 1e18
```

WATCH OUT

- Geometric series with a non-prime modulus or r ≡ 1: use divide and conquer S(n) = S(n/2)·(1 + r^(n/2)) (+ last term), or a 2×2 matrix [[r,1],[0,1]].

3.5 Powers repeat: last digits, cycles of a^k mod m

KEYWORDS: last digit power cycle period order repeat a^k mod m unit digit

USE WHEN

- "Last digit of a^b", "a^b mod small m" with a huge b, patterns in powers.

DO THIS

1. The sequence a^1, a^2, a^3, ... mod m is eventually periodic. If gcd(a, m) = 1 it is purely periodic and the period (the order of a) divides φ(m).
2. Last digit of a^b: last digit of a decides; cycles: 2→2,4,8,6; 3→3,9,7,1; 7→7,9,3,1; 8→8,4,2,6; 4→4,6; 9→9,1; 0,1,5,6 fixed. Use b mod 4 (if it is 0 use 4).
3. In general for small m: simulate a^k mod m, record the first time each value appears, find the start and length of the cycle, then jump.
4. Last k digits: a^b mod 10^k with binExp.

EXAMPLE

> Last digit of 7^222: 222 mod 4 = 2 → second in (7, 9, 3, 1) → 9.
> 2^100 mod 9: cycle 2, 4, 8, 7, 5, 1 (length 6), 100 mod 6 = 4 → 7.

3.6 Huge exponent (exponent given as a string, or a power tower)

KEYWORDS: huge exponent string digits fermat euler phi power tower 10000 digits generalised euler

USE WHEN

- a^N mod M where N has thousands of digits.
- a^(b^c) mod M, or a^(F_n) with F_n astronomically large.

DO THIS

1. M prime and a not divisible by M: reduce the exponent mod (M − 1) while reading its digits, then binExp. (powStrPrime)
2. M prime and M | a: the answer is 0 for N ≥ 1 (and 1 for N = 0).
3. M not prime, or a shares a factor with M: generalised Euler. Let e be the true exponent. If e ≥ φ(M) use (e mod φ(M)) + φ(M); if e is small use e exactly. (powStrGeneral)
4. Power tower a1^(a2^(a3^...)) mod M: recurse M → φ(M) → φ(φ(M)) ...; this chain reaches 1 after about 2·log2(M) steps. (tower)
5. To know whether e ≥ φ(M), keep a capped exact value of e (stop at a big constant).

FORMULAS

```
a^N ≡ a^(N mod (p−1)) (mod p) p prime, p ∤ a Fermat
a^N ≡ a^(N mod φ(M)) (mod M) gcd(a, M) = 1 Euler
a^N ≡ a^((N mod φ(M)) + φ(M)) (mod M) any a, N ≥ φ(M) generalised Euler
```

EXAMPLE

> 2015 D: (3^N + 1)/2 mod 1e9+7 with N of 10^4 digits: e = N mod (1e9+6) from the string; answer (3^e + 1)·inv2.
> 2016 I: k^(F_(n+2)) mod any M: compute F mod φ(M) with matrix power; if the true F_(n+2) ≥ φ(M) (check with exact Fibonacci numbers capped at 2e18) add φ(M).

```
int e = 0; for (char c : s) e = (e * 10 + (c - '0')) % (MOD - 1);   // exponent mod p-1
int ans = binExp(a, e);
```

Template: number_theory/power_tower.cpp (powStrPrime, powStrGeneral, tower)

3.7 Huge numbers given as strings

KEYWORDS: big number string mod digits divisible big integer 10^5 digits

USE WHEN

- A number with up to 1e6 digits: "is it divisible by b", "N mod m", compare, add.

DO THIS

1. N mod m: r = (r·10 + digit) % m for each digit, left to right. O(length).
2. Divisible by b ⇔ that remainder is 0. A leading '−' does not change divisibility.
3. Comparing two big numbers: compare lengths, then the strings.
4. Real big arithmetic (add, subtract, multiply, divide by small): bigint.cpp.
5. Big N used as an exponent: see the previous card. Big N in ⌊N/k⌋ or N mod k: long division by k digit by digit (bigDivSmall).

```
int bigMod(const string &s, int m) { int r = 0; for (char c : s) if (isdigit(c)) r = (r * 10 + (c - '0')) % m; return r; }
```

Template: strings/bigint.cpp

3.8 Digit sum and mod 9 / mod 3, digital root, divisibility rules

KEYWORDS: digit sum mod 9 divisible by 3 9 11 digital root repeated digit sum last digit divisibility rule

USE WHEN

- Divisibility by 2, 3, 4, 5, 8, 9, 10, 11, 25 of big numbers.
- "Sum the digits again and again until one digit remains" (digital root).
- Rearranging, inserting, deleting digits and asking about divisibility by 3 or 9.

DO THIS

1. A number and the sum of its digits have the SAME remainder mod 9 (and mod 3), because 10 ≡ 1 (mod 9). So replace the number by its digit sum mod 9. The order of the digits does not matter.
2. Digital root (repeated digit sum) of n: 0 if n = 0, else 1 + (n − 1) mod 9 (it gives 9, not 0, for multiples of 9).
3. Mod 11: alternating sum of digits starting from the last digit (+d0 − d1 + d2 − …), because 10 ≡ −1 (mod 11).
4. Mod 2, 5, 10: the last digit. Mod 4, 25, 100: the last two digits. Mod 8, 125, 1000: the last three digits.
5. Mod 7, 13 and others: no shortcut, compute the remainder digit by digit.
6. Divisible by 6: by 2 and 3. By 12: by 3 and 4. By 15: by 3 and 5 (combine coprime rules).

FORMULAS

```
n ≡ digitsum(n) (mod 9) and (mod 3)
dr(n) = 1 + (n − 1) mod 9 (n > 0)
dr(a·b) = dr(dr(a)·dr(b)), dr(a + b) = dr(dr(a) + dr(b))
n mod 11 = (d0 − d1 + d2 − d3 + …) mod 11
```

EXAMPLE

987654321: digit sum 45 → divisible by 9.
2017 I (repeated digit sum of a^b, 50 000-digit a and b): r = a mod 9 from its digit sum. If r = 0: answer 9 (a > 0). If r ∈ {3, 6}: a^b ≡ 0 for b ≥ 2 → 9 (b = 1: dr(a)). Otherwise powers mod 9 repeat every 6: use b mod 6 (0 → 6), answer = dr of r^(that). b = 0 → 1.

```
int digitalRoot(int n) { return n == 0 ? 0 : 1 + (n - 1) % 9; }
```

3.9 Deleting / inserting / changing one digit

KEYWORDS: delete digit remove digit insert digit change number value 10^k

USE WHEN

- "Delete the k-th digit of x", "insert a digit", "change one digit", and what happens to the value.

DO THIS

1. Write x = P·10^k + d·10^(k−1) + Q, where d is the k-th digit from the right, P the digits above it, Q the k−1 digits below. After deleting d: y = P·10^(k−1) + Q.
2. So x − y = 10^(k−1)·(9P + d). The difference is a power of ten times something ≡ d (mod 9).
3. Changing digit d to d' changes the value by (d' − d)·10^(k−1) and the digit sum by d' − d.
4. With point updates on digits: keep the digit sum mod 9 and a std::set of positions of non-zero digits (gives trailing zeros quickly).

FORMULAS

```
x − y = 10^(k−1)·(9P + d)
```

EXAMPLE

2025 A: the answer depends only on z mod 9 and the number of trailing zeros of z: keep both under updates.

3.10 Is the modulus prime? (10^6+7 is NOT)

KEYWORDS: modulus not prime composite 1000007 lucas crt check prime modulus division fails

USE WHEN

- Before using inverses / ncr with factorials, look at the modulus.

DO THIS

1. 1e9+7, 1e9+9, 998244353, 7340033 are prime. 10^6 + 7 = 29 · 34483 is NOT prime (Dhaka used it). 2^64, 10^9, any even number: not prime.
2. Not prime: no Fermat inverse. Options: (1) avoid division (Pascal triangle for nCr, divide-and-conquer geometric sums); (2) extgcd inverse when gcd(x, M) = 1; (3) split M into prime factors, solve mod each (Lucas for nCr), glue with CRT.
3. Quick check of a modulus: isPrime64(M).

EXAMPLE

Kattis Classical Counting: answer mod 10^6+7: compute mod 29 and mod 34483 with Lucas, then CRT.

3.11 Modular arithmetic: problem patterns

KEYWORDS: modulo patterns divisible remainder minimum moves digit huge number classic

FORMULAS

```
fewest +1 moves to make a divisible by b: (b − a % b) % b
make every a_i + x divisible by k: needed x ≡ −a_i (mod k); count equal needs,
answer = largest group handled last
digital root of a^b / a·b with huge a, b: work mod 9 (see digit sum card)
3^N with N given as 10^4 digits: N mod (p−1) from the string
```

```
remainder of a huge string number by small b: r = (r·10 + d) % b
last two digits of a^b: binExp(a, b, 100)
(a^b) mod m with b ≥ 0 and m = 1: answer 0 (binExp handles 1 % m)
a^(b^c) mod p (p prime): e = binExp(b, c, p−1); answer binExp(a, e, p); careful a
% p == 0 → 0 unless b^c == 0
```

B. gcd and lcm

3.12 gcd and lcm: rules

KEYWORDS: gcd lcm greatest common divisor least common multiple euclid properties

USE WHEN

- Anything with gcd / lcm / common divisor / tiling / "same ratio" / "reduce the fraction".

DO THIS

1. gcd: __gcd(a, b) (works for long long). gcd(0, x) = x. gcd of an array: fold left. gcd of negatives: use absolute values.
2. lcm(a, b) = a / gcd(a, b) · b. Divide FIRST. lcm of many numbers grows very fast: cap it (stop when it exceeds the limit) or use __int128.
3. Prime view: gcd takes the SMALLER exponent of every prime, lcm the LARGER.
4. gcd(a, b) = gcd(a, b − a) = gcd(a, b mod a). Consecutive numbers are coprime: gcd(n, n+1) = 1.
5. Reduce a fraction p/q: divide both by gcd(p, q), make q > 0.

FORMULAS

```
gcd(a, b)·lcm(a, b) = a·b
gcd(a·k, b·k) = k·gcd(a, b)
gcd(a, b) = 1 and a | b·c ⇒ a | c
gcd(2^a − 1, 2^b − 1) = 2^gcd(a,b) − 1 gcd(F_a, F_b) = F_gcd(a,b)
number of steps of Euclid ≤ 2·log2(min) (≈ 90 for 1e18)
```

EXAMPLE

12 = 2²·3, 18 = 2·3²: gcd = 2·3 = 6, lcm = 2²·3² = 36, 6·36 = 12·18.

3.13 gcd when the same value is added to everything

KEYWORDS: gcd shift a+x b+x add same value differences coprime gcd(a+i,b+i) row gcd

USE WHEN

- Queries gcd(a1 + x, a2 + x, ..., an + x) for many x.
- "Count i in [0, m] with gcd(a + i, b + i) = 1".
- "Longest subarray whose elements are all equal mod some m ≥ 2".

DO THIS

1. Subtract: gcd(a + x, b + x) = gcd(a + x, b − a). The difference does not depend on x.
2. Array: G = gcd(a2 − a1, a3 − a1, ..., an − a1) once; each query is gcd(a1 + x, G).
3. gcd(a + i, b + i) = 1 ⇔ a + i is coprime to D = |b − a|: count numbers in [a, a + m] coprime to D (next card). D = 0: gcd = a + i, which is 1 only when a + i = 1. D = 1: all are coprime.
4. All equal mod m inside a subarray ⇔ m divides all adjacent differences ⇔ gcd of the differences ≥ 2: longest subarray of the difference array with gcd ≥ 2 (two pointers +

sparse table).

FORMULAS

```
gcd(a1 + x, …, an + x) = gcd(a1 + x, gcd(a2 − a1, …, an − a1))
gcd(a + i, b + i) = gcd(a + i, |b − a|)
```

EXAMPLE

2020 F (Co Primes). CF 1458A Row GCD. CF 1548B Integers Have Friends.

3.14 gcd over arrays and subarrays

KEYWORDS: gcd subarray range gcd remove one element prefix suffix count subarrays gcd distinct gcd values

USE WHEN

- Range gcd queries, "remove one element to maximise the gcd", "count subarrays with gcd 1 / gcd = k", "longest subarray with gcd > 1".

DO THIS

1. Range gcd with no updates: sparse table (gcd is idempotent), O(1) per query. With updates: segment tree.
2. Remove one element: prefix gcd pre[i] and suffix gcd suf[i]; without a[i] = gcd(pre[i−1], suf[i+1]).
3. For a fixed right end r, gcd(a[l..r]) over all l takes at most log2(max) + 1 different values (each change at least halves). Keep the list of (gcd value, how many l give it) for the current r; build the list for r+1 from it. Count subarrays with gcd exactly k by summing counts.
4. Gcd of a subarray only decreases as it grows: two pointers for "longest subarray with gcd > 1" (with a sparse table to test).

```
vector<pi> cur;                        // (gcd value, number of left ends) for subarrays
ending at r
for (int x : a) {
    vector<pi> nxt = {{x, 1}};
    for (auto &p : cur) { int g = __gcd(p.first, x);
        if (g == nxt.back().first) nxt.back().second += p.second; else nxt.pb({g,
p.second}); }
    cur = nxt;
    for (auto &p : cur) if (p.first == 1) cnt1 += p.second;   // subarrays with gcd 1
ending here
}
```

```
Template: data_structures/sparse_table.cpp (op = __gcd),
number_theory/sparse_gcd.cpp
```

3.15 lcm problems

KEYWORDS: lcm minimum lcm pair lcm equals k count pairs lcm of range lcm of all

USE WHEN

- "Pair with minimum lcm", "count pairs with lcm = k", "smallest number divisible by 1..n".

DO THIS

1. Minimum lcm of a pair (values ≤ 1e6): for each g from 1 to MX, take the two smallest array values that are multiples of g; candidate lcm = x·y/g. The best over all g is the

answer (when g is their real gcd it is exact, otherwise it is larger).
2. Pairs (x, y) with lcm(x, y) = k: both must divide k; loop over divisor pairs of k ($d(k)^2 \le 1.8e6$ for $k \le 1e9$).
3. lcm(1..n): Π over primes p ≤ n of the largest p^e ≤ n.
4. lcm of a sequence mod M: keep the max exponent of each prime (factorize each element), multiply p^max at the end.

FORMULAS

```
lcm(a, b) = Π p^max(ea, eb)
Σ_{i=1..n} lcm(i, n) = n/2 · (1 + Σ_{d|n} d·φ(d))
```

3.16 Count numbers coprime to d (inclusion-exclusion over primes)

KEYWORDS: coprime count range inclusion exclusion primes of d not divisible by any

USE WHEN

- Count x in [L, R] with gcd(x, d) = 1.
- Count x in [1, N] divisible by at least one / none of a few given numbers.

DO THIS

1. Factor d and keep its DISTINCT primes p1..pk (k ≤ 15 for d ≤ 1e18). Trial division up to √d (or Pollard for d up to 1e18).
2. count(N) = Σ over subsets S of the primes of $(-1)^{|S|} \cdot \lfloor N / \text{product}(S) \rfloor$. Skip subsets whose product exceeds N.
3. Answer for [L, R] = count(R) − count(L − 1).
4. "Divisible by at least one of d1..dk" (not necessarily primes): Σ over non-empty subsets $(-1)^{(|S|+1)} \lfloor N / \text{lcm}(S) \rfloor$, cap the lcm at N + 1.
5. Many queries with the same d: precompute the subset products once.

FORMULAS

```
#{x ≤ N : gcd(x, d) = 1} = Σ_{S ⊆ primes(d)} (−1)^|S| ⌊N / Π S⌋
For N = d this equals φ(d).
```

EXAMPLE

Coprime to 12 (primes 2, 3) in [1, 20]: 20 − 10 − 6 + 3 = 7 (1, 5, 7, 11, 13, 17, 19).

```
Template: number_theory/floor_blocks.cpp (distinctPrimes, coprimeCount);
combinatorics.cpp countDivisibleByAny
```

3.17 Extended Euclid: solve a·x + b·y = c

KEYWORDS: extended euclid bezout diophantine linear equation ax+by=c integer solutions count solutions range

USE WHEN

- Integer solutions of a·x + b·y = c. "Measure exactly c litres with a and b litre jugs". Inverse modulo a non-prime. Count solutions with x, y in ranges.

DO THIS

1. g = gcd(a, b). No solution if c is not a multiple of g.
2. extgcd(a, b, x0, y0) gives a·x0 + b·y0 = g. Scale: x = x0·(c/g), y = y0·(c/g).
3. All solutions: x + k·(b/g), y − k·(a/g) for every integer k.

4. x in [x1, x2] gives a range of k; y in [y1, y2] gives another range of k; the answer is the size of their intersection (careful with ceil/floor of negative numbers).
5. Smallest non-negative x: x mod (b/g), made non-negative.
6. Coprime a, b ≥ 1 with x, y ≥ 0: every c ≥ (a−1)(b−1) is reachable; the largest unreachable is a·b − a − b; there are (a−1)(b−1)/2 unreachable values.

FORMULAS

```
a·x + b·y = c solvable ⇔ gcd(a, b) | c
x = x0·c/g + k·b/g, y = y0·c/g − k·a/g
```

```
int x, y, g = extgcd(a, b, x, y);         // base template: a*x + b*y = g
if (c % g) { /* no solution */ }
__int128 X = (__int128)x * (c / g), Y = (__int128)y * (c / g);
int step = b / g; int xmin = ((X % step) + step) % step;   // smallest x >= 0
```

WATCH OUT

- x0·(c/g) overflows easily: use __int128.
- Negative a or b: solve with |a|, |b| and flip the sign of x or y.
- floor of a negative division in C++ rounds toward 0: write your own floorDiv.

3.18 Linear congruence a·x ≡ b (mod m)

KEYWORDS: congruence ax = b mod m solve linear modular equation number of solutions

USE WHEN

- Find x with a·x ≡ b (mod m) (m not necessarily prime).

DO THIS

1. g = gcd(a, m). No solution if g does not divide b.
2. Divide everything by g: a' = a/g, b' = b/g, m' = m/g. Then x ≡ b'·inv(a') (mod m'), inverse from extended Euclid.
3. There are exactly g solutions modulo m: x0, x0 + m', x0 + 2m', ...

EXAMPLE

3x ≡ 6 (mod 12): g = 3, x ≡ 2 (mod 4) → x = 2, 6, 10 (mod 12).

3.19 Chinese Remainder Theorem: several remainders at once

KEYWORDS: crt chinese remainder theorem system of congruences remainders combine moduli

USE WHEN

- x ≡ r1 (mod m1), x ≡ r2 (mod m2), ...: find x. "A number leaves remainder 2 when divided by 3 and 3 when divided by 5".
- An answer is needed modulo a composite number: solve modulo each prime power, then glue with CRT.

DO THIS

1. Merge two conditions at a time. Solvable ⇔ gcd(m1, m2) divides r2 − r1. The merged condition is modulo lcm(m1, m2).
2. With pairwise coprime moduli there is always exactly one answer modulo their product.
3. crt.cpp handles non-coprime moduli and uses __int128.

EXAMPLE

x ≡ 2 (mod 3), x ≡ 3 (mod 5): 2, 5, 8 → 8 ≡ 3 (mod 5) → x ≡ 8 (mod 15).

Template: number_theory/crt.cpp

3.20 gcd / lcm: problem patterns

KEYWORDS: gcd lcm patterns classic problems common divisors football season reachable board orac fadi strange function

FORMULAS

```
largest gcd of a pair in an array: count multiples cnt[d] (harmonic), largest d
with cnt[d] ≥ 2 (CSES Common Divisors)
gcd of (a_i + x) for many x: gcd(a_1 + x, G), G = gcd of differences (CF 1458A)
x·w + y·d = p with x + y ≤ n, x, y ≥ 0: loop y from 0 to w−1 (y ≥ w can be traded
for d more x's), check (p − y·d) % w (CF 1244C)
starting numbers, operation 2x − y: you can reach k ⇔ (k − x_1) is divisible by
gcd of all (x_i − x_1) (CF 1478D)
numbers you can add as gcd of present numbers: d is reachable ⇔ gcd of all present
multiples of d equals d (CF 1627D)
gcd of all pairwise lcm: ans = gcd over i of lcm(a_i, gcd(a_{i+1..n})) (CF 1349A)
min max(a, b) with lcm(a, b) = X: try divisors a of X with gcd(a, X/a) = 1 (CF
1285C)
Σ_{i=1..n} (smallest x not dividing i) = n + Σ_{k ≥ 1} ⌊n / lcm(1..k)⌋ (CF 1542C)
gcd of a subarray ≥ 2 for "all equal mod m": gcd of adjacent differences (CF
1548B)
two arithmetic progressions a1·k + b1 = a2·l + b2 in a range: CRT for x ≡ b1 (a1),
x ≡ b2 (a2) (CF 710D)
```

EXAMPLE

Checked by brute force: Orac, Strange Function, Divide Both formulas.

C. Primes, sieves, factorization, divisors

3.21 Sieves: Eratosthenes, smallest prime factor, linear, segmented

KEYWORDS: sieve eratosthenes spf smallest prime factor linear sieve segmented sieve primes in range count primes

USE WHEN

- Primes / factorizations of many numbers up to 1e7; primes in a window [L, R] with R up to 1e12.

DO THIS

1. Base template sieve(): spf, isPrime, primes for all numbers below N (1e6+10; raise N to 1e7 if needed, memory ≈ 80 MB as long long, use int32 arrays for more).
2. Plain Eratosthenes: for each prime p, cross out p^2, $p^2 + p$, ...; O(n log log n).
3. Linear sieve (linear_sieve.cpp): also gives φ and μ for every number in O(n).
4. Segmented sieve for [L, R], R ≤ 1e12, R − L ≤ 1e6: get primes up to √R, for each prime cross out its multiples inside the window starting at max(p^2, ⌈L/p⌉·p). Remember 1 is not prime.
5. Counting primes ≤ n for n ≤ 1e7: prefix count over isPrime.

FORMULAS

```
π(1e6) = 78498, π(1e7) = 664579, π(1e9) = 50847534; π(n) ≈ n / ln n
```

```
// segmented sieve: primes in [L, R]
vector<bool> comp(R - L + 1, false);
for (int p : primes) { if (p * p > R) break;
    for (int x = max(p * p, (L + p - 1) / p * p); x <= R; x += p) comp[x - L] = true; }
for (int x = max(L, 2LL); x <= R; x++) if (!comp[x - L]) { /* x is prime */ }
```

Template: base sieve(); number_theory/linear_sieve.cpp

3.22 Is n prime?

KEYWORDS: prime primality test miller rabin trial division is prime large prime

USE WHEN

- Testing one or a few numbers; up to 1e18.

DO THIS

1. n ≤ 1e7 with a sieve: isPrime[n].
2. One n ≤ 1e12: trial division by d = 2, 3, ... while d·d ≤ n (√1e12 = 1e6 steps).
3. n ≤ 1e18: isPrime64(n) (Miller-Rabin, deterministic with 12 bases) in pollard_rho.cpp.
4. "Largest prime ≤ N with some property": primes are dense (gaps below 1e18 are under ~1500), so try candidates from the top and test each with Miller-Rabin (2018 D: digits only 2, 3, 5, 7: DFS over digits from the largest).

FORMULAS

> Bertrand: there is always a prime between n and 2n.
> Goldbach (checked far beyond contest limits): every even number ≥ 4 is a sum of two primes; every odd number ≥ 7 is a sum of three primes.

Template: number_theory/pollard_rho.cpp

3.23 Factorizing numbers

KEYWORDS: factorize prime factorization spf smallest prime factor pollard rho trial division prime factors

USE WHEN

- You need the prime factors of one number or of many numbers.

DO THIS

1. Many numbers below N: sieve() once, then factorize(x) repeatedly divides by spf[x]: O(log x).
2. One number ≤ 1e12 (or ≤ 1e14 a few times): trial division up to √n (prime_factor_root_n).
3. Up to 1e18: factor(n) in pollard_rho.cpp (Miller-Rabin + Pollard rho) returns the sorted prime factors with repetition.
4. Trick without Pollard: divide out every prime ≤ 1e6 (= n^(1/3) for 1e18). What remains is 1, a prime, p^2 (check: perfect square and its root prime), or p·q with two different primes.
5. Number of distinct primes: at most 8 for n ≤ 1e7, 9 for ≤ 1e9, 15 for ≤ 1e18.

```
vpi f = factorize(x);      // base, after sieve(): {prime, exponent}
vi pf = factor(n);         // pollard_rho.cpp: sorted primes with repetition
```

Template: base factorize; number_theory/prime_factor_root_n; number_theory/pollard_rho.cpp

3.24 Divisors: count, sum, list, all numbers up to N

KEYWORDS: divisors number of divisors sum of divisors list divisors harmonic d(n) sigma tau

USE WHEN

- Number or sum of divisors, list all divisors, "for every number up to N, its divisors".

DO THIS

1. From n = Π p^e: number of divisors d(n) = Π (e + 1); sum σ(n) = Π (1 + p + ... + p^e).
2. List divisors of one n: for i with i·i ≤ n, if n % i == 0 take i and n / i (avoid taking √n twice). O(√n).
3. List from the factorization: start with {1}, for each p^e multiply the current list by p, p², ..., p^e (getDivisors in pollard_rho.cpp for n up to 1e18).
4. Divisors (or a divisor function) of every number up to N: for d in 1..N, for m = d, 2d, 3d, ...: add d to m. Total N·(1 + 1/2 + ... + 1/N) ≈ N ln N.
5. Looping over all divisors is cheap: max d(n) is 240 (n ≤ 1e6), 1344 (n ≤ 1e9), 103680 (n ≤ 1e18).

FORMULAS

```
n = Π p^e: d(n) = Π (e + 1), σ(n) = Π (p^(e+1) − 1)/(p − 1), product of divisors = n^(d(n)/2)
Σ_{i ≤ N} d(i) = Σ_{k ≤ N} ⌊N/k⌋ Σ_{i ≤ N} σ(i) = Σ_{k ≤ N} k·⌊N/k⌋
```

EXAMPLE

720 = $2^4 \cdot 3^2 \cdot 5$ → d = 5·3·2 = 30, σ = 31·13·6 = 2418.
10! = $2^8 \cdot 3^4 \cdot 5^2 \cdot 7$ → d = 9·5·3·2 = 270.

```
vvi divs(MX + 1);
for (int d = 1; d <= MX; d++) for (int m = d; m <= MX; m += d) divs[m].pb(d);
```

3.25 Multiplicative functions and the linear sieve

KEYWORDS: multiplicative function linear sieve compute f for all n phi mu d sigma prime power

USE WHEN

- You need f(n) for every n ≤ 1e7 where f(a·b) = f(a)·f(b) for coprime a, b (d, σ, φ, μ, and many custom ones).

DO THIS

1. f is decided by its values on prime powers. Compute with a sieve that knows, for every i, its smallest prime p and its exponent.
2. Linear sieve: each composite is visited once as i·p with p = smallest prime of i·p. If p divides i, then f(i·p) needs the power of p: keep cnt[i] (exponent of the smallest prime) and rest[i] (i without that prime power).
3. Simplest safe way: f(n) = f(p^e)·f(n / p^e), with p = spf[n] and p^e the full power of p in n.

FORMULAS

```
d(p^e) = e + 1 σ(p^e) = (p^(e+1) − 1)/(p − 1) φ(p^e) = p^e − p^(e−1) μ(p) = −1, μ(p^e) = 0 for e ≥ 2
```

```
// f for all n < MX from f(p^e); uses spf from the base sieve()
vi f(MX); f[1] = 1;
rep(n, 2, MX) { int p = spf[n], m = n, pe = 1, e = 0;
```

```
    while (m % p == 0) m /= p, pe *= p, e++;
    f[n] = fPrimePower(p, e, pe) * f[m]; }      // e.g. d: e + 1; phi: pe - pe / p; mu:
e == 1 ? -1 : 0
```

Template: number_theory/linear_sieve.cpp

3.26 Perfect squares and odd divisor counts

KEYWORDS: perfect square odd number of divisors odd divisors square twice square exactly three divisors lamps toggle

USE WHEN

- "Odd number of divisors", "exactly 3 divisors", "number of odd divisors is odd", lamps toggled by every divisor.

DO THIS

1. d(n) is odd ⇔ n is a perfect square (divisors pair up as d, n/d except √n). Lamp i toggled by every divisor of i ends ON ⇔ i is a square.
2. Exactly 3 divisors ⇔ n = p^2 with p prime. Exactly 4 divisors ⇔ n = p^3 or p·q.
3. Odd divisors of n = divisors of its odd part (remove all 2s). Number of odd divisors is odd ⇔ the odd part is a square ⇔ n = k^2 or n = 2·k^2.
4. Count in [L, R]: f(R) − f(L − 1) with an EXACT integer square root.

FORMULAS

```
#squares in [1, X] = isqrt(X)
#{n ≤ X : n = k² or 2k²} = isqrt(X) + isqrt(X / 2)
#{n ≤ X : n = p², p prime} = π(isqrt(X))
```

EXAMPLE

2018 C: f(X) = isqrt(X) + isqrt(X/2), answer f(R) − f(L − 1).

```
int isqrt(int x) { int r = sqrtl((long double)x); while (r * r > x) r--; while ((r + 1)
* (r + 1) <= x) r++; return r; }
```

3.27 Recover n from its divisors

KEYWORDS: recover number from divisors smallest largest divisor list missing

USE WHEN

- All divisors of n except 1 and n are given (or all divisors) and you must find n, or say it is impossible.

DO THIS

1. Sort them. n = smallest · largest (divisors pair up as d and n/d). Only one divisor d given: n = d^2.
2. Then VERIFY: generate all divisors of n (√n loop) and compare with the given list exactly; otherwise print −1.

EXAMPLE

Given 2, 3, 4, 6: n = 2·6 = 12; divisors of 12 except 1 and 12 are 2, 3, 4, 6 → correct. (2019 J, CF 1165D)

3.28 Primes and divisors: problem patterns

KEYWORDS: prime divisor patterns t primes kth divisor almost prime divisor analysis prime multiples two divisors moves

FORMULAS

```
exactly 3 divisors (T-primes): x = p², check isqrt(x)² == x and isqrt prime (CF 230B)
k-th smallest divisor of n: list i ≤ √n, then n/i in reverse (CF 762A)
numbers with exactly two distinct prime divisors up to n: sieve count distinct primes (CF 26A)
number of divisors of every x ≤ 1e6 queries: sieve cnt (CSES 1713)
n given as p^e list (huge): count Π(e+1); sum Π(p^(e+1)−1)/(p−1); product of divisors n^(d(n)/2): exponent of p is e·(e+1)/2 · Π_{q≠p}(e_q+1), keep exponents mod (MOD − 1) (CSES 2182)
numbers ≤ n divisible by at least one of k ≤ 20 primes: inclusion-exclusion over subsets (CSES 2185)
primes in [a, b] with b ≤ 2^31, b − a ≤ 1e5: segmented sieve (LightOJ 1197)
largest prime ≤ N (1e18) with digits only 2,3,5,7: DFS digits from the top + Miller-Rabin (2018 D)
N = p²·q with N ≤ 9e18: try d up to N^(1/3): if d² | N then p = d; else if d | N then q = d, p = isqrt(N/d)
two divisors d1, d2 > 1 of a with gcd(d1 + d2, a) = 1: d1 = full power of the smallest prime, d2 = a/d1 (−1 if d2 = 1) (CF 1366D)
make a = b in exactly k moves (divide by a divisor > 1): max = Ω(a) + Ω(b) (prime factors with repetition); min = 0 if a = b, 1 if one divides the other, else 2; k = 1 needs a ≠ b and one dividing the other (CF 1538D)
number of divisors of n!: Π over primes p ≤ n of (v_p(n!) + 1)
```

D. Euler φ (totient) and counting by gcd

3.29 Euler's totient φ(n)

KEYWORDS: phi totient coprime count reduced fractions euler function prefix sum pairs gcd

USE WHEN

- "How many numbers ≤ n are coprime to n", fractions in lowest terms, Σ gcd(i, n), pairs with gcd exactly G, reducing exponents (Euler's theorem).

DO THIS

1. Single n: r = n; for each distinct prime p of n: r −= r / p. O(√n) (phiSqrt in power_tower.cpp).
2. All n ≤ 1e7: φ sieve: phi[i] = i; for each prime p: for multiples m: phi[m] −= phi[m] / p. Or the linear sieve.
3. Pairs 1 ≤ x ≤ y ≤ N with gcd(x, y) = 1: Φ(N) = φ(1) + … + φ(N) (the pair (1,1) counted once).
4. Pairs with gcd exactly G: Φ(⌊N/G⌋) (divide both numbers by G).
5. Ordered pairs in [1, N]² with gcd 1: 2·Φ(N) − 1.

FORMULAS

```
φ(n) = n·Π (1 − 1/p); φ(p^k) = p^k − p^(k−1); φ(p) = p − 1; φ multiplicative
Σ_{d|n} φ(d) = n
```

```
Σ_{i=1..n} gcd(i, n) = Σ_{d|n} d·φ(n/d)
Σ_{i=1..n} [gcd(i, n) = 1]·i = n·φ(n)/2 (n ≥ 2)
reduced fractions p/q with 0 < p < q ≤ N: Φ(N) − 1
```

EXAMPLE

φ(12) = 12·(1/2)·(2/3) = 4 (1, 5, 7, 11).
2019 G: largest G with at least P pairs of gcd G among numbers ≤ N: prefix Φ up to 1e7; the count Φ(⌊N/G⌋) only grows as G shrinks; find the smallest m with Φ(m) ≥ P (binary search), then G = ⌊N/m⌋; −1 if Φ(N) < P.

```
vi phi(MX + 1); iota(all(phi), 0);
for (int p = 2; p <= MX; p++) if (phi[p] == p) for (int m = p; m <= MX; m += p) phi[m]
-= phi[m] / p;
```

Template: number_theory/linear_sieve.cpp, number_theory/euler_totient.cpp

3.30 φ: computing it (one number, all numbers, a range, prefix sums)

KEYWORDS: phi compute single sieve segment prefix sum Phi totient template phi_sieve phi_single

DO THIS

1. One n ≤ 1e12: r = n; for each distinct prime p of n: r = r / p · (p − 1). O(√n). (phi_single in euler_totient.cpp, phiSqrt in power_tower.cpp)
2. All n ≤ 1e7: ph[i] = i; for each p with ph[p] == p (prime): for multiples m: ph[m] −= ph[m] / p. Or the linear sieve (phi_sieve, ~130 MB at 1e7; use int32 arrays).
3. φ for x in [L, R] with R ≤ 1e12, R − L ≤ 1e6: segmented: start with val = x, rest = x; for each prime p ≤ √R divide p out of rest for its multiples in the window and apply ×(1 − 1/p); leftover rest > 1 is a prime (phi_segment).
4. Prefix sums Φ(n) = φ(1) + … + φ(n) in long long (Φ(1e7) ≈ 3·10^13).
5. Φ(n) for n up to 1e10–1e11: Du's sieve Φ(n) = n(n+1)/2 − Σ_{i≥2} Φ(⌊n/i⌋), memoised with floor blocks (Phi_du).

FORMULAS

```
φ(n) = n · Π_{p | n} (1 − 1/p) φ(p) = p − 1 φ(p^k) = p^k − p^(k−1)
φ(ab) = φ(a)φ(b) if gcd(a, b) = 1; φ(n) is even for n ≥ 3; φ(n) = n − 1 ⇔ n prime
Φ(n) ≈ 3n²/π² ≈ 0.304·n²
```

```
vi ph(MX + 1); iota(all(ph), 0);
rep(p, 2, MX + 1) if (ph[p] == p) for (int m = p; m <= MX; m += p) ph[m] -= ph[m] / p;
vi Phi(MX + 1, 0); rep(i, 1, MX + 1) Phi[i] = Phi[i - 1] + ph[i];
```

Template: number_theory/euler_totient.cpp (phi_sieve, phi_single, phi_segment, Phi_du)

3.31 φ key fact: the number of i ≤ n with gcd(i, n) = g is φ(n/g)

KEYWORDS: gcd group by gcd value count i with gcd i n equal g same gcds sum gcd(i,n)

USE WHEN

- A sum or count over i = 1..n where only gcd(i, n) matters.

DO THIS

1. Numbers i ≤ n with gcd(i, n) = g are i = g·k with k ≤ n/g and gcd(k, n/g) = 1: exactly φ(n/g) of them (g must divide n).

2. So Σ_{i=1..n} F(gcd(i, n)) = Σ_{g | n} F(g)·φ(n/g). Loop over divisors only.
3. Shifted version: x in [0, m) with gcd(a + x, m) = gcd(a, m): answer φ(m / gcd(a, m)) (a + x runs over all residues mod m).

FORMULAS

```
#{i ≤ n : gcd(i, n) = g} = φ(n/g)
Σ_{i=1..n} gcd(i, n) = Σ_{g|n} g·φ(n/g) (Pillai)
Σ_{i=1..n} lcm(i, n) = n/2 · (1 + Σ_{d|n} d·φ(d))
Σ_{i ≤ n, gcd(i,n)=1} i = n·φ(n)/2 (n ≥ 2)
```

EXAMPLE

n = 12, g = 4: i ∈ {4, 8}: φ(3) = 2 ✓. Σ gcd(i, 6) = 1·2 + 2·2 + 3·1 + 6·1 = 15 ✓.
CF 1295D Same GCDs: φ(m / gcd(a, m)). SPOJ LCMSUM (check: n = 12 → 468).

`Template: euler_totient.cpp build_gcd_lcm_sums, lcm_sum, sum_coprime`

3.32 Coprime pairs with φ: Farey, visible points, pairs with gcd G

KEYWORDS: coprime pairs count phi prefix farey fractions visible lattice points gcd exactly g pairs gcd prime send a table

USE WHEN

- Count pairs (x, y) in [1, N]² (square, same bound on both) with gcd = 1, = G, or prime.

DO THIS

1. Pairs x ≤ y ≤ N with gcd 1: Φ(N) (for each y, the φ(y) values x ≤ y coprime to y; (1,1) once).
2. Ordered pairs in [1, N]²: 2Φ(N) − 1. Unordered x < y: Φ(N) − 1.
3. gcd exactly G: replace N by ⌊N/G⌋.
4. gcd is a prime: Σ over primes p ≤ N of (2Φ(⌊N/p⌋) − 1) (ordered).
5. Different bounds N ≠ M (rectangle): use Möbius instead (next cards).

FORMULAS

```
reduced fractions a/b with 0 < a < b ≤ n: Φ(n) − 1 (POJ 2478)
ordered coprime pairs in [1, N]²: 2Φ(N) − 1 (UVa 10820, check N = 2 → 3)
visible points (x, y) in [0, N]² except origin: 2Φ(N) + 1 (POJ 3090, check N = 2 → 5)
pairs u < v ≤ n with gcd exactly g: Φ(⌊n/g⌋) − 1 (CF 1731E)
pairs x ≤ y ≤ N with gcd exactly G: Φ(⌊N/G⌋) (2019 G)
```

EXAMPLE

2019 G: Φ(⌊N/G⌋) grows as G shrinks: binary search the smallest m with Φ(m) ≥ P, answer G = ⌊N/m⌋; −1 if Φ(N) < P.

3.33 Gauss: n = Σ_{d|n} φ(d) (replace a gcd by a sum of φ)

KEYWORDS: gauss identity sum phi divisors equals n gcd replace sum of gcd tuples holmes children

USE WHEN

- You need Σ of gcd(...) over many objects (pairs, tuples, subsets), or g(n) = Σ_{d|n} φ(d) appears.

DO THIS

1. Write gcd = Σ_{d | gcd} φ(d). Swap the sums: φ(d) is added once for every object whose elements are ALL divisible by d. So Σ gcd = Σ_d φ(d)·(number of objects with every element divisible by d).
2. Σ_{d|n} φ(d) = n means "apply φ then sum over divisors" does nothing (CF 776E: the answer is φ applied ⌈k/2⌉ times; stop when it reaches 1, it at least halves every two steps).

FORMULAS

```
Σ_{i,j ≤ N} gcd(i, j) = Σ_d φ(d)·⌊N/d⌋² rectangle: Σ_d φ(d)·⌊N/d⌋·⌊M/d⌋ (floor
blocks + prefix φ)
Σ_{i<j ≤ N} gcd(i, j) = Σ_{n ≤ N} (Σ_{g|n} g·φ(n/g) − n) (UVa 11426 GCD Extreme)
Σ over all K^N sequences of gcd = Σ_{d=1..K} φ(d)·⌊K/d⌋^N (ABC162 E)
Σ_{i<j} gcd(a_i, a_j) over an array = Σ_d φ(d)·C(cnt[d], 2), cnt[d] = number of
multiples of d
```

```
Template: euler_totient.cpp sum_gcd_rect, sum_gcd_i_lt_j; mobius.cpp
sum_gcd_all_pairs
```

3.34 φ in exponents: Euler, power towers, product recurrences

KEYWORDS: euler theorem exponent reduce phi tower huge mods product recurrence exponent mod p-1

DO THIS

1. gcd(a, m) = 1: a^e ≡ a^(e mod φ(m)) (mod m). Prime p: exponent mod p − 1.
2. Any a, e ≥ φ(m) (in fact e ≥ log2 m is enough): a^e ≡ a^((e mod φ(m)) + φ(m)) (mod m).
3. Tower a1^(a2^(a3^...)) mod m: recurse with m → φ(m); keep "value mod m, and whether the true value ≥ m" (MOD trick: x < m ? x : x % m + m). Template tower(a, 0, m) % m, test: 2^2^2^2 mod 1000 = 536.
4. Recurrence that MULTIPLIES (f_x = c^(2x−6)·f_(x−1)·f_(x−2)·f_(x−3)): write f_n = f1^A·f2^B·f3^C·c^D, the exponents follow a linear recurrence; compute them with matrix power mod (p − 1), then fast powers mod p (CF 1182E).

```
Template: euler_totient.cpp (power_m, tower, pow_huge, inv_mod_phi);
number_theory/power_tower.cpp
```

3.35 k-th coprime number, k-th reduced fraction, φ of a range product

KEYWORDS: kth coprime number kth fraction farey enumerate rational phi of product range queries distinct primes

DO THIS

1. k-th positive integer coprime to n: binary search x with count_coprime(x, n) ≥ k (inclusion-exclusion over primes of n) (kth_coprime).
2. k-th fraction in the list 0/1, 1/1, 1/2, 1/3, 2/3, 1/4, 3/4, ...: terms with denominator ≤ b: 1 + Φ(b); binary search b, then the r-th numerator coprime to b (UVa 11327, check k = 5 → 2/3).
3. φ of the product of a range: φ(P) = P·Π (p − 1)/p over DISTINCT primes p | P. Prefix products for P; for "each prime once in [l, r]": offline by r, put the factor (p−1)/p at the last occurrence of p and cancel it at the previous one (Fenwick of products) (CF 594D).

4. Values ≤ 300 with range multiply: 62 primes fit in a 64-bit mask; segment tree of (product, mask) (CF 1114F).

`Template: euler_totient.cpp count_coprime, kth_coprime`

3.36 φ: small facts that solve whole problems

KEYWORDS: phi facts minimum x with phi at least a smallest prime product of coprimes inverse phi

FORMULAS

```
smallest x with φ(x) ≥ a → the smallest prime > a (φ(x) ≤ x − 1, equality only for
primes) (LightOJ 1370)
numbers below n coprime to n: φ(n) for n ≥ 2, but the answer is 0 for n = 1 (φ(1)
= 1) (UVa 10299)
product of all numbers in [1, n−1] coprime to n, mod n: P; if P ≠ 1 drop the
number P itself (CF 1514C)
iterating φ: φ(φ(…φ(n))) reaches 1 in O(log n) steps
all n with φ(n) = k: inverse_phi(k) in euler_totient.cpp
f(n) = #{x + y = n, gcd(x, y) = 1} = φ(n) (since gcd(x, y) = gcd(x, n))
```

`Template: number_theory/euler_totient.cpp`

E. Möbius, inclusion-exclusion over divisors, gcd = 1 counting

3.37 Pairs / subsets with gcd exactly g (count multiples, go from big to small)

KEYWORDS: gcd exactly g pairs count multiples harmonic descending sieve largest gcd pair subsets gcd

USE WHEN

- "Largest gcd among all pairs", "number of pairs / subsets / k-subsets with gcd exactly g", values ≤ 1e6.

DO THIS

1. cnt[v] = how many times v appears. c[d] = cnt[d] + cnt[2d] + cnt[3d] + … for every d (harmonic loop, V ln V).
2. Largest pair gcd: the largest d with c[d] ≥ 2.
3. Pairs with gcd EXACTLY d: f[d] = C(c[d], 2) − f[2d] − f[3d] − …, computed from the largest d down to 1.
4. Subsets with gcd exactly d: same with 2^c[d] − 1. k-subsets: C(c[d], k).
5. Subset with gcd exactly K exists ⇔ the gcd of ALL multiples of K in the array equals K (2018 G).
6. Smallest subset with gcd 1 has at most 7 elements for values ≤ 3e5: loop k = 1..7 with the formula above.

FORMULAS

```
f[d] = (number of structures using only multiples of d) − Σ_{k ≥ 2} f[k·d]
```

```
for (int d = MX; d >= 1; d--) { int c = 0; for (int m = d; m <= MX; m += d) c +=
cnt[m];
    f[d] = c * (c - 1) / 2; for (int m = 2 * d; m <= MX; m += d) f[d] -= f[m]; }
```

3.38 Möbius μ: counting coprime pairs and tuples

KEYWORDS: mobius mu gcd 1 coprime pairs count rectangle sum gcd inclusion exclusion divisors visible lattice points

USE WHEN

- Count pairs (a, b) with a ≤ N, b ≤ M and gcd(a, b) = 1 (or = k), many queries.
- Count coprime subsets; "visible lattice points" (gcd(x, y) = 1).

DO THIS

1. μ(1) = 1; μ(n) = 0 if a square of a prime divides n; otherwise (−1)^(number of primes). Sieve it (linear_sieve.cpp muL, mobius.cpp).
2. The key fact Σ_{d|n} μ(d) = [n = 1] turns "gcd = 1" into a sum over common divisors d: Σ_d μ(d)·(number of pairs where d divides both).
3. Pairs in [1,N]×[1,M] with gcd 1: Σ_d μ(d)·⌊N/d⌋·⌊M/d⌋. Many queries: prefix sums of μ + floor blocks (both ⌊N/d⌋ and ⌊M/d⌋ constant on a block: r = min(N/(N/l), M/(M/l))). O(√N) per query.
4. gcd exactly k: same with N/k and M/k.
5. Coprime subsets of an array: Σ_d μ(d)·(2^c[d] − 1), c[d] = number of multiples of d.
6. Without μ: the descending sieve of the previous card gives the same answers.

FORMULAS

```
Σ_{d|n} μ(d) = [n = 1]
#{a ≤ N, b ≤ M, gcd(a, b) = 1} = Σ_d μ(d)·⌊N/d⌋·⌊M/d⌋
Σ_{a ≤ N, b ≤ N} gcd(a, b) = Σ_d φ(d)·⌊N/d⌋²
Σ_{i<j} gcd(a_i, a_j) = Σ_d φ(d)·C(c[d], 2)
Möbius inversion: g(n) = Σ_{d|n} f(d) ⇔ f(n) = Σ_{d|n} μ(d)·g(n/d)
```

```
// pairs (a, b), a <= N, b <= M, gcd = 1; pre[i] = mu[1] + ... + mu[i]
int res = 0;
for (int l = 1, r; l <= min(N, M); l = r + 1) {
    r = min(N / (N / l), M / (M / l));
    res += (pre[r] - pre[l - 1]) * (N / l) * (M / l);
}
```

Template: number_theory/mobius.cpp, number_theory/linear_sieve.cpp

3.39 The gcd = 1 recipe: Möbius / inclusion-exclusion over d

KEYWORDS: mobius recipe gcd 1 count objects all divisible by d g(d) mu sum coprime

USE WHEN

- Count (or sum over) objects whose gcd is 1 (or exactly k): pairs, tuples, subsets, sequences, points.

DO THIS

1. Step 1: find g(d) = the number of objects whose elements are ALL divisible by d. This is always easy (table below).
2. Step 2: answer for gcd = 1 is Σ_d μ(d)·g(d). For gcd = k: use g restricted to multiples of k, or substitute (divide everything by k).
3. Alternative without μ (safer): exact[d] = g(d) − Σ_{m = 2d, 3d, ...} exact[m], from the largest d down. exact[1] is the gcd-1 answer, and every exact[d] is available (descending sieve).

4. μ(n): 1 for n = 1; 0 if a prime square divides n; else (−1)^(number of distinct primes). Sieve: mobius_sieve / linear_sieve; one big n: mobius(n).

FORMULAS

```
objects g(d) = all elements divisible by d
pairs i<j from an array C(cnt[d], 2) (cnt[d] = number of multiples of d)
non-empty subsets of an array 2^cnt[d] − 1
k-subsets C(cnt[d], k)
sequences of length N with values 1..K ⌊K/d⌋^N
pairs in [1,N]×[1,M] ⌊N/d⌋·⌊M/d⌋
pairs in [L,R]² (⌊R/d⌋ − ⌊(L−1)/d⌋)²
points in [0,N]³ except origin (⌊N/d⌋ + 1)³ − 1
sequences of positive ints with sum s 2^(s/d − 1) if d | s, else 0
total size of all subsets cnt[d]·2^(cnt[d] − 1)
```

EXAMPLE

Coprime subsequences (CF 803F): Σ μ(d)(2^cnt[d] − 1). Visible points 3D (SPOJ VLATTICE): Σ μ(d)((⌊N/d⌋+1)³ − 1), N = 1 → 7, N = 2 → 19.
Unusual sequences (CF 900D): gcd x, sum y: if x ∤ y → 0; s = y/x; Σ_{d|s} μ(d)·2^(s/d − 1).

```
// pairs i<j with gcd(a_i, a_j) = 1   (CSES 2417)
vi cnt = count_multiples(a, MX);           // mobius.cpp
int ans = 0; rep(d, 1, MX + 1) ans += mu[d] * cnt[d] * (cnt[d] - 1) / 2;
```

Template: number_theory/mobius.cpp (mobius_sieve, count_multiples, descending sieve, coprime_pairs)

3.40 Descending sieve: gcd exactly d for every d (no μ needed)

KEYWORDS: descending sieve exactly gcd d subtract multiples winter is here divide both sum size times gcd

USE WHEN

- You need the count for EVERY exact gcd value (e.g. Σ g × (number with gcd exactly g)), or you prefer no signs.

DO THIS

1. exact[d] = g(d) − exact[2d] − exact[3d] − … for d from MX down to 1. O(MX log MX). Take % MOD after each subtraction (add MOD).
2. Then combine: Σ_d d·exact[d], or Σ_{d ≥ 2} …, etc.

FORMULAS

```
Σ over subsets with gcd > 1 of (size × gcd): g(d) = cnt·2^(cnt−1), answer Σ_{d≥2}
d·exact[d] (CF 839D)
ordered (x, y) in [L, R] with g = gcd ≠ 1, x ≠ g, y ≠ g: Σ_{g≥2} exact[g] −
Σ_{g=max(2,L)..R} (2·c(g) − 1), c(g) = multiples of g in [L, R] (ABC206 E)
smallest subset with gcd 1: answer ≤ 7 (values ≤ 3e5); first k with Σ
μ(d)·C(cnt[d], k) ≠ 0 (mod a prime) (CF 1043F)
```

```
for (int d = MX; d >= 1; d--) {
    ex[d] = g[d];
    for (int m = 2 * d; m <= MX; m += d) ex[d] = (ex[d] - ex[m] + MOD) % MOD;
}
```

Template: mobius.cpp (descending sieve function)

3.41 Many queries: coprime pairs in a rectangle with floor blocks

KEYWORDS: zap queries coprime pairs rectangle range floor blocks prefix mu gcd equals d ranges a b c d

USE WHEN

- Up to 5e4 queries: count x ≤ a, y ≤ b with gcd(x, y) = d; or x ∈ [a, b], y ∈ [c, d].

DO THIS

1. F(n, m) = #{x ≤ n, y ≤ m, gcd 1} = Σ_t μ(t)·⌊n/t⌋·⌊m/t⌋. Walk blocks where both ⌊n/t⌋ and ⌊m/t⌋ are constant: r = min(n/(n/l), m/(m/l)), add (preMu[r] − preMu[l−1])·(n/l)·(m/l). O(√n + √m).
2. gcd = d: F(⌊a/d⌋, ⌊b/d⌋).
3. Ranges not starting at 1 (2D inclusion-exclusion): F(b, d) − F(a−1, d) − F(b, c−1) + F(a−1, c−1).

```
int F(int n, int m) {                    // needs preMu = prefix sums of mu
    int res = 0;
    for (int l = 1, r; l <= min(n, m); l = r + 1) {
        r = min(n / (n / l), m / (m / l));
        res += (preMu[r] - preMu[l - 1]) * (n / l) * (m / l);
    }
    return res;
}
```

Template: mobius.cpp coprime_pairs_range

3.42 Insert / delete numbers, keep the number of coprime pairs

KEYWORDS: mike and foam dynamic insert remove coprime count squarefree divisors cnt shelf

USE WHEN

- Elements are added and removed; after each change print the number of coprime pairs (or count elements coprime to x).

DO THIS

1. Keep cnt[d] = number of current elements divisible by d (only square-free d matter).
2. #elements coprime to v = Σ μ(d)·cnt[d] over square-free divisors d of v (≤ 64 for v ≤ 5e5; ≤ 128 for v ≤ 1e6). Get them with sqf_divisors(v).
3. Insert v: ans += coprime(v), then cnt[d]++ for each square-free d | v. Remove v: cnt[d] −− first, then ans −= coprime(v).

Template: mobius.cpp sqf_divisors (CF 547C Mike and Foam)

3.43 Square-free numbers: count and k-th

KEYWORDS: square free count numbers not divisible by square mu n d squared kth squarefree binary search

DO THIS

1. Count of square-free x ≤ n = Σ_{d ≤ √n} μ(d)·⌊n/d²⌋ (count_squarefree; sieve μ up to √n; n = 1e14 → 1e7).
2. k-th square-free number: binary search n in [1, 2k] (about 61% of numbers are square-free).
3. Is x square-free: no prime appears twice (factorize).

FORMULAS

> #{square-free ≤ n} = Σ_d μ(d)·⌊n/d²⌋ ≈ 6n/π²

Template: mobius.cpp count_squarefree

3.44 Harder Möbius sums: gcd is prime, Σ lcm, d(i·j), merging into T

KEYWORDS: gcd prime pairs pgcd sum lcm table crash d(ij) divisor count merge T dirichlet

DO THIS

1. Pairs $x \le a$, $y \le b$ with gcd prime: $\Sigma_T \lfloor a/T \rfloor \lfloor b/T \rfloor \cdot h(T)$, $h(T) = \Sigma_{\text{prime } p \mid T} \mu(T/p)$. Precompute h by "for each prime p, for each k: h[p·k] += μ[k]", prefix sums, then floor blocks (SPOJ PGCD).
2. $\Sigma_{i \le n, j \le m} \operatorname{lcm}(i, j) = \Sigma_g g \cdot \Sigma_d \mu(d) \cdot d^2 \cdot S(\lfloor n/(gd) \rfloor) \cdot S(\lfloor m/(gd) \rfloor)$, $S(k) = k(k+1)/2$; two nested block loops (Luogu P1829).
3. $d(i \cdot j) = \#\{x \mid i, y \mid j, \gcd(x, y) = 1\} \Rightarrow \Sigma_{i \le n, j \le m} d(ij) = \Sigma_t \mu(t) \cdot D(\lfloor n/t \rfloor) \cdot D(\lfloor m/t \rfloor)$, $D(k) = \Sigma_{x \le k} \lfloor k/x \rfloor$ (Luogu P3327).
4. $\Sigma_{i<j} \operatorname{lcm}(a_i, a_j)$ over an array: $1/\gcd = \Sigma_{d \mid \gcd} w(d)$ with $w(n) = \Sigma_{d \mid n} \mu(n/d) \cdot \text{inv}(d)$; answer $\Sigma_d w(d) \cdot (S[d]^2 - Q[d])/2$, S/Q = sums of multiples of d and of their squares (AGC038 C).
5. Expected length until gcd becomes 1, random values 1..m: $E = 1 - \Sigma_{d=2..m} \mu(d) \cdot c/(m - c)$, $c = \lfloor m/d \rfloor$ (CF 1139D, m = 2 → 2).

FORMULAS

```
general shape: Σ_T (product of floors in T)·(Σ_{d | T} f(d)·μ(T/d)) — precompute
the inner sum with a harmonic loop, then floor blocks
```

3.45 Möbius inversion and Dirichlet convolution

KEYWORDS: mobius inversion dirichlet convolution multiplicative identities phi mu id divisor sum transform

USE WHEN

- You know $g(n) = \Sigma_{d \mid n} f(d)$ for all n and need f (or the reverse).

DO THIS

1. Inversion: $g(n) = \Sigma_{d \mid n} f(d) \Leftrightarrow f(n) = \Sigma_{d \mid n} \mu(n/d) \cdot g(d)$.
2. Compute any $(f * g)(n) = \Sigma_{d \mid n} f(d) \cdot g(n/d)$ for all $n \le M$ with a harmonic loop, O(M log M) (dirichlet in mobius.cpp).
3. Sum over divisors for all n (g from f): for d, for multiples m of d: g[m] += f[d]. Back to f: for n from 1 up: f[n] = g[n] − Σ_{d|n, d<n} f[d] (same harmonic loop).

FORMULAS

```
1 * μ = ε (ε(1) = 1, else 0) φ * 1 = id μ * id = φ
1 * 1 = d (number of divisors) id * 1 = σ φ(n)/n = Σ_{d|n} μ(d)/d
```

Template: mobius.cpp dirichlet

3.46 Σ μ and Σ φ up to 1e10 (Du's sieve)

KEYWORDS: mertens prefix sum mu phi up to 1e10 du sieve

DO THIS

1. Mertens $M(n) = \Sigma_{i \le n} \mu(i) = 1 - \Sigma_{i \ge 2} M(\lfloor n/i \rfloor)$. $\Phi(n) = n(n+1)/2 - \Sigma_{i \ge 2} \Phi(\lfloor n/i \rfloor)$.
2. Sieve small values up to ~n^(2/3), memoise the rest in a map, loop i in floor blocks. O(n^(2/3)).

Template: mobius.cpp mertens, euler_totient.cpp Phi_du

3.47 Your φ / μ template functions (what to call)

KEYWORDS: template functions euler_totient mobius list phi_sieve coprime_pairs_range count_squarefree tower necklaces

FORMULAS

```
euler_totient.cpp:
phi_sieve(LIM) → phi[], Phi[] (prefix), primes | phi_single(n) n ≤ 1e12 |
phi_segment(L, R)
power128(a, e, m) | inv_mod_phi(a, m) inverse for any m (gcd 1) | pow_huge(a,
"string", m) | tower(a, 0, m) % m
primitive_root(p) | build_gcd_lcm_sums(n) → pillai[n] = Σ gcd(i,n), lcmG |
lcm_sum(n)
sum_gcd_rect(n, m) = Σ_{i≤n,j≤m} gcd | sum_gcd_i_lt_j(n) | coprime_pairs_square(n)
= 2Φ(n) − 1
pairs_gcd_exact(n, g) | pairs_gcd_prime(n) | count_coprime(X, n) | kth_coprime(k,
n) | sum_coprime(n)
necklaces(n, k) | Phi_du(n) up to 1e11 | inverse_phi(k) = all n with φ(n) = k
mobius.cpp:
mobius_sieve(LIM) → mu[], phi[], preMu[] | mobius(n) one n ≤ 1e12 |
sqf_divisors(x) → {d, μ(d)}
count_multiples(a, M) → cnt[d] | descending sieve (at least → exactly) |
coprime_pairs(a)
coprime_pairs_range(n, m) | count_squarefree(n) | sum_gcd_all_pairs(n) |
dirichlet(f, g) | mertens(n)
```

WATCH OUT

- euler_totient.cpp and mobius.cpp both define phi / primes: paste only one of them (or rename).

F. Factorials, binomials and primes

3.48 Factorials: trailing zeros, power of p in n!

KEYWORDS: legendre factorial trailing zeros power of prime n! divides kummer base b zeros

USE WHEN

- "Trailing zeros of n!", "largest k with p^k dividing n!", zeros of n! in base b, power of p in C(n, k), "does m divide n!".

DO THIS

1. Exponent of a prime p in n!: Σ ⌊n / p^i⌋ (stop when p^i > n). O(log n).
2. Trailing zeros in base 10: the exponent of 5 (there are always more 2s).
3. Trailing zeros in base b: factor b = Π q^e; answer = min over q of ⌊(exponent of q in n!) / e⌋.
4. Does m divide n!: for every p^e in m check v_p(n!) ≥ e.
5. Exponent of p in C(n, k) = v_p(n!) − v_p(k!) − v_p((n−k)!) = number of carries when adding k and n − k in base p (Kummer).
6. Exponent of p in 1!·2!·…·n!: Σ_{i=1..n} v_p(i!).

FORMULAS

```
v_p(n!) = ⌊n/p⌋ + ⌊n/p²⌋ + ⌊n/p³⌋ + …
v_p(n!) = (n − (sum of the base-p digits of n)) / (p − 1)
```

EXAMPLE

Zeros of 100!: 20 + 4 = 24.

3.49 C(n, k) modulo a prime (and when n is big)

KEYWORDS: ncr binomial modulo prime lucas factorial mod p wilson n factorial mod p

USE WHEN

- Binomial coefficients appear inside a number theory problem.

DO THIS

1. Standard: compute_factorials() and ncr(n, k) for n < 1e6, prime modulus > n. (See the Counting part for all methods.)
2. n ≥ p (small prime p): Lucas: C(n, k) ≡ Π C(n_i, k_i) over the base-p digits; 0 as soon as some k_i > n_i.
3. n! mod p for n ≥ p is 0. Wilson: (p − 1)! ≡ −1 (mod p).
4. C(n, k) is odd ⇔ (n & k) == k. Number of odd entries in row n: 2^popcount(n).

```
Template: combinatorics/combinatorics.cpp (ncrBig, Lucas)
```

3.50 Factorials and binomials: problem patterns

KEYWORDS: factorial patterns trailing zeros smallest n digits of factorial k divides n factorial modular stability

FORMULAS

```
trailing zeros of n! in base b: factor b = Π q^e; min over q of ⌊v_q(n!) / e⌋
(LightOJ 1045-type; 1000! in base 12 → 497)
smallest N whose factorial has exactly Q trailing zeros: binary search N on
v_5(N!) (it only grows); print "impossible" if it skips Q
smallest N with K | N!: for each p^e in K find the smallest N with v_p(N!) ≥ e
(binary search), take the max
number of digits of n! in base b: ⌊Σ_{i≤n} log(i) / log(b)⌋ + 1 (prefix sums of
logs, add 1e-9 before floor)
number of divisors of n!: Π over primes p ≤ n of (v_p(n!) + 1)
exponent of p in 1!·2!·…·n!: Σ_{i ≤ n} v_p(i!)
increasing arrays a1 < … < ak ≤ n where every element is a multiple of a1: Σ_d
C(⌊n/d⌋ − 1, k − 1) (CF 1359E)
C(n, k) mod a composite like 10^6+7 = 29·34483: Lucas mod each prime + CRT
```

G. Sums, floors, periods, matrices

3.51 Sums with ⌊n/i⌋ and n mod i (floor blocks)

KEYWORDS: floor division sum n/i blocks sqrt decomposition n mod i sum of divisors up to n harmonic lemma

USE WHEN

- Σ_{i=1..n} f(⌊n/i⌋) or Σ (n mod i) with n up to 1e12; sums of d(i) or σ(i) up to n.

DO THIS

1. ⌊n/i⌋ takes at most 2√n different values. Walk the blocks: start l, q = n / l, the block ends at r = n / q (every i in [l, r] has the same quotient).
2. Add (r − l + 1)·g(q), or (prefix of f over [l, r])·g(q).
3. n mod i = n − i·⌊n/i⌋, so Σ n mod i = n·n − Σ i·⌊n/i⌋; the sum of i over a block is (l + r)(r − l + 1)/2.
4. Two numbers N, M at once: r = min(N / (N / l), M / (M / l)).
5. Ceil version ⌈n/i⌉ = ⌊(n − 1)/i⌋ + 1.

FORMULAS

```
Σ_{i≤n} ⌊n/i⌋ = Σ_{i≤n} d(i) Σ_{i≤n} i·⌊n/i⌋ = Σ_{i≤n} σ(i)
```

```
for (int l = 1, r; l <= n; l = r + 1) {
    int q = n / l; r = n / q;              // every i in [l, r] has n / i == q
    ans += (r - l + 1) * q;                // here: sum of n / i  (= sum of d(i), i <=
n)
}
```

Template: number_theory/floor_blocks.cpp

3.52 Σ ⌊(a·i + b) / m⌋ for i = 0..n−1 (lattice points under a line)

KEYWORDS: floor sum lattice points under line sum floor a i + b over m

USE WHEN

- Sum of the floor of a linear function; lattice points under a line segment; n up to 1e9.

DO THIS

1. floorSum(n, m, a, b) in O(log). Needs a, b ≥ 0: if b is negative, shift it (b + k·m adds k·n to the sum).

Template: number_theory/floor_blocks.cpp floorSum

3.53 Expressions with i mod a: use the period

KEYWORDS: periodic pattern mod repeat cycle period sum i mod a simulate many steps

USE WHEN

- A sum over i up to 1e12 of something with i mod a, i mod b; any pattern that repeats; simulate 1e18 steps.

DO THIS

1. Values repeat every P = a (or lcm of all moduli). Sum one period S; answer = (N / P)·S + (sum of the first N mod P terms).
2. Period too long: split i = q·a + r and treat each q as a block with a formula inside (2023 B: Σ |i mod A − i mod (A−1)|, with i mod (A−1) = (q + r) mod (A − 1)).
3. State machines: store the step when each state was first seen; at the first repeat, jump whole cycles.

FORMULAS

```
Σ_{i=0}^{N−1} (i mod a) = (N / a)·a(a−1)/2 + (N mod a)(N mod a − 1)/2
```

3.54 Closed-form sums you will use with mod

KEYWORDS: sum formulas arithmetic geometric squares cubes series mod inverse 2 6

DO THIS

1. With a modulus, reduce n first (n % MOD) and multiply by inv2 / inv6 instead of dividing.
2. Without a modulus, divide the even factor before multiplying to avoid overflow (n(n+1)/2: divide whichever is even).

FORMULAS

```
Σ_{i=1..n} i = n(n+1)/2 Σ i² = n(n+1)(2n+1)/6 Σ i³ = (n(n+1)/2)²
Σ_{i=l..r} i = (l + r)(r − l + 1)/2
geometric: Σ_{i=0}^{n−1} r^i = (r^n − 1)/(r − 1)
Σ i·r^i (i = 0..n−1) = (r − n·r^n + (n−1)·r^(n+1)) / (1 − r)²
Σ_{i=1..n} i^k is a polynomial of degree k + 1 (Lagrange interpolation for big k)
```

3.55 Sums and periods: problem patterns and code

KEYWORDS: sum of divisors up to n floor block code sum sigma geometric divide conquer periodic count x mod a mod b

FORMULAS

```
Σ_{i ≤ n} (n mod i) = n² − Σ i·⌊n/i⌋
count x in [l, r] with (x mod a) mod b ≠ (x mod b) mod a: period a·b, prefix
counts of one period
pairs a ≤ x, b ≤ y with ⌊a/b⌋ = a mod b: a = k(b + 1); Σ over k of #b (CF 1485C)
Σ_{i ≤ n} i^k for k up to 1e6: Lagrange on k + 2 values
values of ⌊n/i⌋ and ⌊m/i⌋ together: block end min(n/(n/l), m/(m/l))
```

```
// Σ_{k ≤ n} σ(k) = Σ_i i·⌊n/i⌋ mod MOD  (n up to 1e12)
int ans = 0, inv2 = (MOD + 1) / 2;
for (int i = 1, j; i <= n; i = j + 1) { int q = n / i; j = n / q;
    int blk = (j - i + 1) % MOD * ((i + j) % MOD) % MOD * inv2 % MOD;   // i + ... + j
    ans = (ans + q % MOD * blk) % MOD; }
// geometric 1 + a + ... + a^(n-1) mod ANY m: {sum, a^n}
pi geo(int a, int n, int m) {
    if (n == 0) return {0, 1 % m};
    if (n & 1) { pi r = geo(a, n - 1, m); return {(r.first * a + 1) % m, r.second * a %
m}; }
    pi h = geo(a, n / 2, m); return {h.first * (1 + h.second) % m, h.second * h.second
% m};
}
```

3.56 Building the matrix for a recurrence

KEYWORDS: matrix recurrence build state constant term running sum walks automaton count strings length n

DO THIS

1. Put everything the next step needs into a state vector; each new entry is a fixed linear combination of old entries = one row of the matrix.
2. Constant term a(n) = 2a(n−1) + 3a(n−2) + 5: state (a(n), a(n−1), 1), rows (2,3,5), (1,0,0), (0,0,1).
3. Running sum S(n): add S to the state, row S(n) = S(n−1) + (row of a(n)).
4. Dice sums (order matters): ways(n) = ways(n−1) + … + ways(n−6): 6×6, first row all 1, shift rows below.

5. Walks of length k from i to j: (A^k)[i][j]. Strings of length n avoiding a pattern: walks in the KMP automaton.
6. Cost: $k^3 \cdot \log n$; k = 100 is fine for one query. Many queries: precompute M^(2^i), multiply a vector (k^2 per step).

EXAMPLE

> a(n) = a(n−1) + a(n−3): state (a(n), a(n−1), a(n−2)), rows (1,0,1), (1,0,0), (0,1,0).

```
Template: math/matrix.cpp
```

H. Structure tricks: squares, invariants, special numbers

3.57 Invariants: what never changes

KEYWORDS: invariant possible reach operations any number of moves parity sum glasses equalise

USE WHEN

- "Any number of operations", "is it possible to reach", "can all become equal", "what is the final value".

DO THIS

1. Find a quantity the operation never changes: total sum, sum mod k, each element mod k, parity, gcd, set of primes present, sorted multiset, xor of all.
2. That gives a necessary condition. Check with a brute force on small cases that it is also sufficient (often it is, maybe with one extra condition).
3. Total preserved and everything must become equal ⇒ total divisible by the count, target = total / count.

EXAMPLE

> 2021 A: move 1 unit between 3 glasses: possible ⇔ (A + B + C) % 3 == 0.
> 2021 B: move K units: total % 3 == 0 and every glass ≡ total/3 (mod K).
> 2022 D: as 2021 B, plus (all already equal, or at least two glasses hold ≥ K).
> 2022 B: Alice can only use primes already present in A; Bob scales everything by a common divisor, so A_i / gcd(A) never changes.

3.58 Products that are perfect squares (exponent parity)

KEYWORDS: perfect square product square free parity exponent xor hash subset square

USE WHEN

- "Product of the pair / subarray / subset is a perfect square".

DO THIS

1. Only the parity of each prime exponent matters. Replace x by its square-free part (product of its primes with odd exponent), computed with spf.
2. Two numbers multiply to a square ⇔ equal square-free parts: group and count pairs C(cnt, 2).
3. Subarray product is a square: give every prime a random 64-bit value; h(x) = xor over primes with odd exponent; prefix xor equal ⇔ square.
4. Subset with a square product: vectors over GF(2) (odd exponents), xor basis; non-empty such subsets = 2^(n − rank) − 1.

EXAMPLE

> $12 = 2^2 \cdot 3 \to$ part 3; $27 = 3^3 \to$ part 3 $\to 12 \cdot 27 = 324 = 18^2$.

3.59 Order of an element, primitive roots, quadratic residues

KEYWORDS: order of element primitive root generator quadratic residue euler criterion legendre symbol

USE WHEN

- "Smallest k with a^k ≡ 1 (mod p)", cyclic structure of powers, "is x a square mod p".

DO THIS

1. Order of a modulo p (gcd(a, p) = 1): it divides p − 1. Start with t = p − 1, and for each prime q of p − 1: while t % q == 0 and a^(t/q) ≡ 1, set t /= q.
2. g is a primitive root mod p ⇔ g^((p−1)/q) ≢ 1 for every prime q of p − 1. 3 is a primitive root of 998244353 and 7340033.
3. Euler's criterion: a (not a multiple of p) is a square mod p ⇔ a^((p−1)/2) ≡ 1; otherwise it is ≡ −1.
4. Half of the non-zero residues mod an odd prime are squares.

3.60 Pythagorean triples and sums of squares

KEYWORDS: pythagorean triples right triangle integer sides sum of two squares

DO THIS

1. All primitive triples: $a = m^2 - n^2$, $b = 2mn$, $c = m^2 + n^2$, with $m > n > 0$, gcd(m, n) = 1, m − n odd. All triples: multiply by k.
2. n is a sum of two squares ⇔ every prime ≡ 3 (mod 4) has an even exponent in n.
3. Lattice points on the circle $x^2 + y^2 = n$: $4 \cdot (d1(n) - d3(n))$, d1 / d3 = divisors ≡ 1 / 3 (mod 4).

EXAMPLE

> m = 2, n = 1 → (3, 4, 5); m = 3, n = 2 → (5, 12, 13).

3.61 Fibonacci and linear recurrences in number theory

KEYWORDS: fibonacci mod m fast doubling pisano period gcd fibonacci zeckendorf

DO THIS

1. F(n) mod m for n up to 1e18: fast doubling or 2×2 matrix power.
2. Pisano period π(m) (Fibonacci mod m repeats): π(m) ≤ 6m; π(10) = 60. Use it when n is given as a huge string and m is small.
3. Every n has a unique representation as a sum of non-adjacent Fibonacci numbers (Zeckendorf): take the largest F ≤ n greedily.
4. Binary strings of length n with no two adjacent 1s: F(n + 2) (2016 I).

FORMULAS

```
F(2k) = F(k)·(2F(k+1) − F(k)), F(2k+1) = F(k)² + F(k+1)²
gcd(F_a, F_b) = F_gcd(a, b); F_a divides F_b ⇔ a divides b (a ≥ 3)
```

```
pi fib(int n) {                          // {F(n), F(n+1)} mod MOD
    if (n == 0) return {0, 1};
    pi t = fib(n >> 1); int a = t.first, b = t.second;
    int c = a * ((2 * b - a + MOD) % MOD) % MOD, d = (a * a + b * b) % MOD;
```

```
    return n & 1 ? pi{d, (c + d) % MOD} : pi{c, d};
}
```

Template: math/matrix.cpp

3.62 Discrete log: a^x ≡ b (mod m)

KEYWORDS: discrete logarithm baby step giant step find exponent a^x = b

USE WHEN

- Find the smallest x ≥ 0 with a^x ≡ b (mod m).

DO THIS

1. Baby-step giant-step in O(√m): discreteLog(a, b, m) works for any m and returns −1 if there is no x.

Template: number_theory/discrete_log.cpp

3.63 Handy number facts and safe integer math

KEYWORDS: isqrt cbrt number of digits log10 first digits overflow int128 base conversion floor division negative

DO THIS

1. Number of digits of n: to_string(n).size(). Digits of n!: ⌊Σ log10(i)⌋ + 1.
2. First k digits of a^b: x = b·log10(a); take ⌊10^(x − ⌊x⌋ + k − 1)⌋ (long double).
3. Exact square / cube root: sqrtl / cbrtl, then adjust ±1 with integer checks (isqrt in the squares card).
4. Floor division with negatives: C++ rounds toward 0. floorDiv(a, b) = a / b − ((a % b != 0) && ((a < 0) != (b < 0))).
5. __int128 for products up to 1e36; print with str128 (misc/common.cpp).
6. Base k digits: while (n) { d = n % k; n /= k; } (least significant first).
7. Overflow check before multiplying: if (a > LIMIT / b) it overflows.

```
int floorDiv(int a, int b) { return a / b - ((a % b != 0) && ((a < 0) != (b < 0))); }
int ceilDiv(int a, int b) { return -floorDiv(-a, b); }
```

3.64 Products that are perfect k-th powers; square subsets

KEYWORDS: perfect kth power signature exponents mod k pairs product cube square subsets xor basis count

DO THIS

1. a·b is a perfect k-th power ⇔ for every prime the exponents add to a multiple of k. Signature of x: Π p^(e mod k). Partner signature: Π p^((k − e) mod k). Count pairs with a map (CF 1225D).
2. Subsets with a square product (values ≤ 70): 19 primes → 19-bit vectors of odd exponents; build a xor basis of rank r; answer 2^(n − r) − 1 (CF 895C).
3. σ(n) is odd ⇔ n is a square or twice a square. Number of i ≤ n with even σ(i): n − isqrt(n) − isqrt(n/2).

```
vi basis(20, 0); int rnk = 0;
for (int x : a) { int v = maskOf(x);          // bit i = exponent of prime #i is odd
    repb(b, 19, 0) { if (!(v >> b & 1)) continue; if (!basis[b]) { basis[b] = v; rnk++;
break; } v ^= basis[b]; } }
int answer = (binExp(2, n - rnk) - 1 + MOD) % MOD;
```

3.65 Invariant method and the brute force to check it

KEYWORDS: invariant method necessary sufficient bfs brute force check reachable states coloring parity gcd monovariant

DO THIS

1. 1. Write exactly what one move does. 2. Find something that never changes (sum, sum mod k, parity, gcd, colour count) or only moves one way. 3. Different at start and target ⇒ "no". 4. Check sufficiency on small cases with a BFS over states.
2. Common invariants: total (and total mod k); parity of a permutation (each swap flips it); gcd (x → x − y keeps gcd(x, y)); chessboard colouring (a domino covers one black, one white); a non-negative quantity that strictly decreases (the process ends).
3. To construct a solution: bring both start and target to a normal form (sorted, everything in one pile); if moves are reversible, start → normal → target.

EXAMPLE

> (a, b) → (a + b, b) or (a, a + b) keeps the gcd: from (3, 5) you can never reach (6, 10).

```
set<vi> seen; queue<vi> q; q.push(start); seen.insert(start);
while (!q.empty()) { vi s = q.front(); q.pop();
    for (vi t : moves(s)) if (!seen.count(t)) { seen.insert(t); q.push(t); } }
// compare seen.count(target) with your formula on many small inputs
```

4. Counting (combinatorics)

KEYWORDS: count number of ways combinatorics binomial ncr arrangements distribute paths

4.1 Basic rules: add, multiply, complement, fix one

KEYWORDS: sum rule product rule complement at least one fix one element pairs count

USE WHEN

- Every counting problem starts here.

DO THIS

1. Cases that cannot overlap: ADD their counts.
2. Independent steps (step 2 always has the same NUMBER of choices whatever step 1 chose): MULTIPLY.
3. "At least one", "not all equal", "contains": count everything, subtract the ones with none (complement).
4. Pairs / triples with a property: fix one member (usually the right one or the middle one), count partners with a map or prefix counts.
5. Pairs with a_i + a_j ≤ X: sort, two pointers from both ends (for l, move r down while a[l] + a[r] > X, add r − l).
6. Triples i < j < k: fix the middle j, multiply (count on the left) × (count on the right).
7. Ordered or unordered? Swap two chosen items: if it gives a different answer, order matters.

EXAMPLE

> 4-digit PINs with at least one 7: 10^4 − 9^4 = 3439.
> Pairs i < j with a_i + a_j = 10: walk j, add cnt[10 − a_j], then cnt[a_j]++.

4.2 Permutations, combinations, repetition, circular

KEYWORDS: permutation combination arrange order repeat multinomial circular round table necklace identical letters

USE WHEN

- Arranging or choosing items.

DO THIS

1. Use the table. With identical items divide by the factorial of each group size (multinomial).
2. Round table (only neighbours matter): fix one person, $(n-1)!$. If flipping also counts as the same: $(n-1)!/2$ for $n \geq 3$.
3. Two people must sit together: glue them into one block, count, multiply by 2.
4. Two people must NOT sit together: total − together.

FORMULAS

```
n distinct in a row: n!
choose k, order matters: P(n,k) = n!/(n−k)!
choose k, order does not matter: C(n,k) = n!/(k!(n−k)!)
k from n types with repetition, order matters: n^k
k from n types with repetition, order does not matter (multiset): C(n+k−1, k)
arrangements of a multiset (BANANA): n!/(a!·b!·…) = 6!/(3!·2!·1!) = 60
```

```
int multinomial(const vi &cnt);   // combinatorics.cpp: (Σc)! / Π c!
int npr(int n, int r);           // combinatorics.cpp
```

4.3 Computing C(n, k): which method

KEYWORDS: ncr binomial coefficient compute factorial inverse lucas pascal big n small k modulo

USE WHEN

- Any binomial coefficient under a modulus.

DO THIS

1. n ≤ 1e6 and prime modulus > n: compute_factorials() once, then ncr(n, k) is O(1). (Base template. Factorials up to N = 1e6+10.)
2. n ≤ 5000 or modulus NOT prime: Pascal triangle C[i][j] = C[i−1][j−1] + C[i−1][j] (only additions).
3. n up to 1e18 but k small (≤ 1e6): multiply n·(n−1)·…·(n−k+1) (reduce each factor mod p first) times invFact[k]. ncrBig in combinatorics.cpp.
4. n ≥ p with a small prime p: Lucas: write n, k in base p, multiply C(n_i, k_i). Struct Lucas in combinatorics.cpp.
5. Exact value (no modulus) for n ≤ 60: Pascal in long long (C(60,30) ≈ 1.2e17 fits; C(67,33) overflows).

FORMULAS

```
C(n, k) = fact[n] · invFact[k] · invFact[n−k]
C(n, k) = 0 when k < 0 or k > n (your ncr must return 0)
C(n, k) is odd ⇔ (n & k) == k
```

WATCH OUT

- Factorial table too short: stars and bars needs n+k, Catalan needs 2n, grid paths need rows+cols.

- Using fact when n ≥ MOD gives 0 silently: use Lucas.

```
Template: base compute_factorials/ncr; combinatorics/combinatorics.cpp (ncrBig, Lucas)
```

4.4 Binomial identities and when to use each (Vandermonde, hockey stick, …)

KEYWORDS: vandermonde hockey stick pascal symmetry binomial theorem identity sum of binomials collapse

USE WHEN

- A sum of binomials appears in your formula and you want to collapse it to one binomial (O(1) instead of a loop).

DO THIS

1. Σ_i C(a, i)·C(b, k − i) (two groups, choose k in total): VANDERMONDE → C(a + b, k).
2. Σ_i C(a, i)·C(b, i): write C(b, i) = C(b, b − i), then Vandermonde → C(a + b, b) = C(a + b, a).
3. Σ_{i=m..n} C(i, r) (fixed bottom r): HOCKEY STICK → C(n + 1, r + 1) − C(m, r + 1).
4. Σ of C(n, k) over all k: 2^n. Over even k: 2^(n−1) (n ≥ 1).
5. Σ k·C(n, k): n·2^(n−1). Σ k(k−1)·C(n,k): n(n−1)·2^(n−2). Use k·C(n,k) = n·C(n−1,k−1) to remove a factor k.
6. Σ C(n, k)·x^k: (1 + x)^n (binomial theorem).
7. C(n, k) = C(n, n−k). C(n, k) = C(n−1, k−1) + C(n−1, k) (Pascal).
8. C(n, k)·C(k, j) = C(n, j)·C(n−j, k−j) (choose a team, then a subteam = choose the subteam first).

FORMULAS

```
Vandermonde: Σ_i C(m, i)·C(n, k−i) = C(m+n, k)
Special case: Σ_i C(n, i)² = C(2n, n)
Hockey stick: C(r, r) + C(r+1, r) + … + C(n, r) = C(n+1, r+1)
Diagonal: C(n, 0) + C(n+1, 1) + … + C(n+k, k) = C(n+k+1, k)
Row sum: Σ_k C(n, k) = 2^n, alternating Σ (−1)^k C(n,k) = 0 (n ≥ 1)
Leader trick: Σ_k k·C(n, k) = n·2^(n−1)
Binomial: (x + y)^n = Σ_k C(n, k) x^k y^(n−k)
```

EXAMPLE

Boys m = 2, girls n = 3, choose 2: Σ C(2,i)C(3,2−i) = 3 + 6 + 1 = 10 = C(5,2).
Hockey: C(2,2)+C(3,2)+C(4,2) = 1+3+6 = 10 = C(5,3).

4.5 Stars and bars: identical items into distinct boxes

KEYWORDS: stars and bars distribute identical candies boxes x1+x2+...=n non negative solutions upper bound

USE WHEN

- Split n identical things among k people / boxes.
- Number of solutions of x1 + … + xk = n.
- Number of multisets, number of non-decreasing sequences.

DO THIS

1. x_i ≥ 0: C(n + k − 1, k − 1).

2. x_i ≥ 1: C(n − 1, k − 1).
3. x_i ≥ l_i: subtract all minimums from n first.
4. x_i ≤ u (upper bound): inclusion-exclusion over j boxes that overflow: Σ_j (−1)^j C(k, j) C(n − j(u+1) + k − 1, k − 1), stop when n − j(u+1) < 0.
5. Sum ≤ n instead of = n: add a slack variable (k + 1 variables, sum exactly n).
6. Write M as an ordered product of N numbers: for every prime power p^e of M, C(e + N − 1, N − 1), multiply.

FORMULAS

```
#{x1+…+xk = n, xi ≥ 0} = C(n+k−1, k−1)
#{x1+…+xk = n, xi ≥ 1} = C(n−1, k−1)
#{x1+…+xk = n, 0 ≤ xi ≤ u} = Σ_j (−1)^j C(k,j) C(n−j(u+1)+k−1, k−1)
```

EXAMPLE

> 4 candies, 3 kids: C(6,2) = 15. x1+x2+x3 = 10 with xi ≤ 4: C(12,2) − 3C(7,2) + 3C(2,2) = 66 − 63 + 3 = 6.

`Template: combinatorics.cpp starsBars, starsBarsPos, starsBarsUpper`

4.6 Non-decreasing sequences, multisets, fixed positions

KEYWORDS: non decreasing sequence multiset count sorted sequence values range fixed cells gaps

USE WHEN

- Count non-decreasing arrays of length m with values in [1, n], possibly with some positions already fixed.

DO THIS

1. No fixed cells: C(n + m − 1, m) (a multiset of size m from n values).
2. Strictly increasing: C(n, m).
3. Fixed cells: split into gaps. A gap of g free cells between fixed values lo ≤ hi has C(g + (hi − lo), g) choices (values from lo to hi, hi − lo + 1 kinds). Multiply all gaps. If a fixed value decreases: 0. Ends: lo = 1 at the start, hi = n at the end.

EXAMPLE

> 2017 B (Trends).

4.7 Grid / lattice paths (with blocked cells)

KEYWORDS: grid paths lattice right down up blocked cells obstacles number of paths

USE WHEN

- Moving only right/down (or right/up) in a grid. Huge grid with a few blocked cells.

DO THIS

1. From (0,0) to (a, b): C(a + b, a).
2. Through a point P: paths(A → P) · paths(P → B).
3. Small grid with any obstacles: dp[r][c] = dp[r−1][c] + dp[r][c−1].
4. Huge grid, k ≤ 3000 blocked cells: sort blocked cells by (row, col), put the target last. f[i] = paths(start → i) − Σ_{j before i, j up-left of i} f[j]·paths(j → i). Answer f[target]. Every bad path is subtracted exactly once (by its FIRST blocked cell).
5. Allowed to step on at most B blocked cells: f[i][t] = ways to reach blocked cell i having used exactly t blocked cells; same subtraction with one more index (2018 E).

FORMULAS

```
paths((r1,c1) → (r2,c2)) = C((r2−r1) + (c2−c1), r2−r1) (0 if r2 < r1 or c2 < c1)
```

```
Template: combinatorics.cpp paths, gridPathsBlocked
```

4.8 Catalan numbers, brackets, never-below-zero (reflection)

KEYWORDS: catalan bracket balanced parentheses dyck path never below zero reflection ballot binary trees

USE WHEN

- Balanced brackets, a balance that never goes negative, binary trees with n nodes, triangulations, non-crossing pairings, stack-sortable permutations.

DO THIS

1. Catalan(n) counts balanced sequences with n pairs.
2. Paths with n steps up and m steps down (n ≥ m) never going below 0: C(n+m, m) − C(n+m, m−1) (reflection).
3. Complete a given prefix of a bracket sequence: if the prefix ever goes negative → 0. Let b = balance, k = characters left. Need d = (k + b)/2 closing brackets: answer C(k, d) − C(k, d + 1); 0 if k + b is odd or b > k.
4. Ballot: A has a votes, B has b, a > b, A always strictly ahead: probability (a − b)/(a + b).

FORMULAS

```
Cat(n) = C(2n, n)/(n+1) = C(2n, n) − C(2n, n+1)
Cat: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796
Cat(n+1) = Σ_{i=0..n} Cat(i)·Cat(n−i)
```

EXAMPLE

Length 6 starting with "((": k = 4, b = 2, d = 3 → C(4,3) − C(4,4) = 3.

```
Template: combinatorics.cpp catalan, dyckFrom; combinatorics/catalan.cpp
```

4.9 Inclusion-exclusion

KEYWORDS: inclusion exclusion at least none all present every colour used derangement surjection onto

USE WHEN

- "None of the bad things happen", "every box non-empty", "all colours used", "nobody in their own place".
- Counting "these k things are forced" is easy, counting "exactly" is hard.

DO THIS

1. good = Σ over sets S of bad events (−1)^|S| · (number of objects where every event in S happens).
2. If the count depends only on |S| = j: good = Σ_j (−1)^j · C(m, j) · count(j).
3. Up to ~20 different events: loop over all bitmasks, sign by popcount.
4. Mod p: add MOD after subtracting.

FORMULAS

```
Derangements: D(n) = Σ_j (−1)^j C(n,j)(n−j)! = (n−1)(D(n−1) + D(n−2)): 1, 0, 1, 2,
9, 44, 265, 1854
Onto maps n → k (no empty box): Σ_j (−1)^j C(k,j)(k−j)^n = k!·S(n,k)
Strings of length n using all k letters: same formula
|A ∪ B ∪ C| = |A|+|B|+|C| − |A∩B| − |A∩C| − |B∩C| + |A∩B∩C|
```

EXAMPLE

> Length 5 over {a,b,c} using all 3: 243 − 3·32 + 3·1 = 150.

`Template: combinatorics.cpp derangements, surjections, countDivisibleByAny`

4.10 Contribution technique: sum over all subsets / subarrays / pairs

KEYWORDS: contribution sum over all subarrays subsets pairs each element counted how many times

USE WHEN

- "Sum over all subarrays / subsets / pairs / paths / permutations of something". Too many structures to list.

DO THIS

1. Flip the sum: for each element (or pair, or edge), count how many structures it appears in and how much it adds. Answer = Σ value × count.
2. Ties (for max/min): break by index so every structure has exactly one max (strict on one side, non-strict on the other).

FORMULAS

```
subarrays containing index i (0-based, length n): (i+1)·(n−i)
sum of all subarray sums = Σ a_i·(i+1)·(n−i)
sorted ascending, a_i is the max of 2^i subsets, of C(i, k−1) subsets of size k
Σ_{i<j} |a_i − a_j|: sort, Σ_j (a_j·j − prefix_j)
Σ_{i<j} a_i·a_j = (S² − Σa_i²)/2
sum of subarray minimums: a_i·(i − prevLess_i)·(nextLessEq_i − i)
tree edge splitting the tree into s and n−s vertices is on s·(n−s) paths
max over subarrays: Σ_t (number of subarrays containing some value ≥ t)
```

EXAMPLE

> Max of every subset of {1,2,3}: 1·1 + 2·2 + 3·4 = 17.

`Template: data_structures/monotonic_stack.cpp sumSubarrayMin/Max`

4.11 Counting a pattern over all arrangements of a multiset

KEYWORDS: pattern count over all permutations binary strings with k ones subsequence 101 sum over arrangements

USE WHEN

- "Sum over all distinct arrangements (with k ones, …) of the number of occurrences of a small pattern / of a statistic".

DO THIS

1. First simplify the statistic to "count occurrences of a fixed small pattern" (often after reading the definition carefully).
2. By linearity: Σ over arrangements of #occurrences = Σ over position-sets of the pattern of (number of arrangements that place the pattern there) = (ways to choose positions) × (arrangements of the remaining items).

EXAMPLE

> 2025 F: the statistic equals the number of "1 0 1" subsequences. Over all binary strings of length n with k ones: choose 3 positions C(n,3), they hold 1,0,1, the other n−3 positions hold k−2 ones: C(n,3)·C(n−3, k−2).

4.12 Splitting into groups: Stirling, Bell, partitions (balls into boxes)

KEYWORDS: stirling bell partition groups teams unlabelled boxes identical balls cycles

USE WHEN

- "Split into k groups", "number of cycles", "write n as a sum", balls into boxes.

DO THIS

1. Use the 12-way table below. Decide: are the items distinct? are the boxes distinct? can boxes be empty?
2. Stirling 2nd kind S(n, k): distinct items into k unnamed non-empty groups.
3. Stirling 1st kind c(n, k): permutations of n with exactly k cycles.
4. Partitions p(n): n as a sum, order ignored. Compositions: order matters, 2^(n−1).

FORMULAS

```
S(n,k) = k·S(n−1,k) + S(n−1,k−1) c(n,k) = (n−1)·c(n−1,k) + c(n−1,k−1)
Bell B(n) = Σ_k S(n,k): 1, 1, 2, 5, 15, 52, 203, 877, 4140
p(n): 1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42, 56, 77, 101
p_k(n) (exactly k parts) = p_{k−1}(n−1) + p_k(n−k)
distinct → distinct, empty ok: k^n no empty: k!·S(n,k)
distinct → identical, empty ok: Σ_{j≤k} S(n,j) no empty: S(n,k)
identical → distinct, empty ok: C(n+k−1,k−1) no empty: C(n−1,k−1)
identical → identical: partitions of n into ≤ k parts / exactly k parts
```

```
// partitions p(n): parts outer loop = order ignored
vi dp(n + 1, 0); dp[0] = 1;
rep(part, 1, n + 1) rep(s, part, n + 1) dp[s] = (dp[s] + dp[s - part]) % MOD;
```

```
Template: combinatorics.cpp stirling2Table, stirling1Table, bell, partitions,
partitionsExactK
```

4.13 Counting up to rotation / symmetry (Burnside)

KEYWORDS: burnside necklace bracelet rotation symmetry distinct colourings up to rotation

USE WHEN

- Colourings that are "the same if you rotate (or flip)".

DO THIS

1. Average over all symmetries the number of colourings each symmetry leaves unchanged.
2. Rotation by i positions of n beads splits them into gcd(i, n) cycles: k^gcd(i,n) fixed colourings.
3. With flips (bracelets) add the n reflections and divide by 2n.

FORMULAS

```
necklaces(n, k) = (1/n) Σ_{i=0..n−1} k^gcd(i,n) = (1/n) Σ_{d|n} φ(d)·k^(n/d)
bracelets: n odd: (necklaces·n + n·k^((n+1)/2)) / (2n); n even: (necklaces·n +
(n/2)(k^(n/2+1) + k^(n/2))) / (2n)
```

```
Template: combinatorics.cpp necklaces, bracelets
```

4.14 Permutations: cycles, inversions, order

KEYWORDS: permutation cycles inversions sort swaps lcm order of permutation

USE WHEN

- A permutation is applied repeatedly, minimum swaps to sort, counting inversions.

DO THIS

1. Split into cycles (follow i → p[i]). Minimum swaps (any two elements) to sort = n − (#cycles).
2. Applying p again and again returns to the start after lcm(cycle lengths) steps. Big lcm: keep max exponent of each prime, multiply mod M (2018 Tile Game).
3. Minimum adjacent swaps to sort = number of inversions (BIT, O(n log n)).
4. Permutations of n with exactly k inversions: dp with prefix sums (inversionCounts in combinatorics.cpp).
5. Random permutation: expected cycles H(n), expected inversions n(n−1)/4, expected fixed points 1.

4.15 Counting trees and graphs

KEYWORDS: cayley labelled trees prufer number of spanning trees forests

FORMULAS

```
labelled trees on n vertices: n^(n−2) (Cayley)
labelled trees with degrees d_i: (n−2)! / Π (d_i − 1)!
forests on n vertices with k trees whose roots are fixed vertices: k·n^(n−k−1)
labelled graphs on n vertices: 2^(n(n−1)/2)
spanning trees of any graph: Kirchhoff (determinant of Laplacian minor, gauss.cpp)
```

```
Template: combinatorics.cpp cayley, forests, treesWithDegrees, pruferEncode
```

4.16 When there is no formula: counting DP

KEYWORDS: counting dp number of sequences prefix sums transition distinct subsequences pigeonhole

USE WHEN

- The constraints allow O(n·state) and no formula appears.

DO THIS

1. dp over positions with a SMALL state (last value, balance, remainder, count used, last letter).
2. If a transition sums a range of previous states, keep prefix sums of dp.
3. Count distinct subsequences: dp[i] = 2·dp[i−1] − dp[last[c] − 1] (dp counts the empty one).
4. Pigeonhole shortcuts: among n prefix sums mod n two are equal; if n > m then the product of all pairwise differences mod m is 0.

4.17 Counting: what the statement says → which technique

KEYWORDS: counting table which technique recognise pattern complement stars bars derangement reflection burnside prufer vandermonde lucas

FORMULAS

```
at least one / not all the same / contains → complement (total − none)
identical items into distinct boxes, non-decreasing → stars and bars C(n+k−1, k−1)
upper bounds on parts, every value appears, ≤ 20 rules → inclusion–exclusion
(bitmask)
no element in its own place → derangements D(n) = (n−1)(D(n−1)+D(n−2))
```

```
huge grid, right/down, few blocked cells → sort blocked cells, subtract by first blocked
never below zero, brackets, change in a queue → reflection, Catalan, ballot (a−b)/(a+b)
split into unnamed groups / count cycles → Stirling 2nd / 1st kind
sum over all subsets / subarrays / pairs → contribution of each element
same up to rotation / reflection → Burnside
labelled trees, degrees given → Prüfer, Cayley n^(n−2)
Σ C(a,i)·C(b,k−i) or Σ C(a,i)·C(b,i) → Vandermonde C(a+b, k) / C(a+b, b)
Σ_x C(n,x)·x^k, n huge, k small → x^k = Σ S(k,j)·x(x−1)…(x−j+1)
n ≥ p with a small prime modulus → Lucas
huge n, small r → falling product × invFact[r]
count x in [L, R] with a digit rule → digit DP, f(R) − f(L−1)
a sequence of answers 1, 2, 5, 14 / 1, 3, 9, 27 / … → brute force small n, look it up (reference card)
```

4.18 Counting patterns from the ladders

KEYWORDS: counting patterns bouquet two arrays count the arrays chess queen glue reverse non decreasing exactly one pair permutations fixed points inversions k cycles

FORMULAS

```
non-empty subsets of size neither a nor b, n ≤ 1e9: 2^n − 1 − C(n,a) − C(n,b), falling product (ABC156 D)
a non-decreasing, b non-increasing, a_i ≤ b_i: glue a + reverse(b) → one non-decreasing array of length 2m: C(n+2m−1, 2m) (CF 1288C)
mountain array with exactly one equal pair: C(m, n−1)·(n−2)·2^(n−3) (CF 1312D)
two queens attacking on n×m (n ≤ m): rows nm(m−1) + columns nm(n−1) + diagonals 2·(2·Σ_{L=1..n−1} L(L−1) + (m−n+1)·n(n−1)) (UVa 11538)
permutations with exactly k fixed points: C(n,k)·D(n−k)
permutations with exactly K inversions: dp[i][k] = Σ_{t=0..i−1} dp[i−1][k−t] with prefix sums
min swaps (any two) to sort: n − #cycles; min adjacent swaps: #inversions
apply a permutation until it returns: lcm of cycle lengths
strings using all of k letters: Σ (−1)^j C(k,j)(k−j)^n
x1+…+xk = n with 0 ≤ xi ≤ u: Σ_j (−1)^j C(k,j) C(n − j(u+1) + k−1, k−1)
product of N numbers equal to M: factor M, for each p^e: C(e+N−1, N−1), multiply
round table: (n−1)!; necklace you can flip: (n−1)!/2 (n ≥ 3); two people together: glue, ×2
number of subarrays containing index i (0-based): (i+1)(n−i)
Σ_{k} k·C(n,k) = n·2^(n−1); Σ k²C(n,k) = n(n+1)2^(n−2); C(n,k) odd ⇔ (n & k) == k
```

4.19 Generating functions and pigeonhole (just enough)

KEYWORDS: generating function polynomial multiply coefficient bounded items exponential strings ntt pigeonhole prefix sum divisible subset sum mod m

DO THIS

1. Ordinary GF for identical items: kind with 0..u copies → 1 + x + … + x^u; any number → 1/(1 − x). Multiply the brackets (it is a DP over kinds; NTT if large). Answer = coefficient of x^n.

2. Exponential GF for arrangements (positions matter): each letter → Σ x^c / c! over allowed counts c; multiply; answer = coefficient · n!.
3. Pigeonhole: n + 1 prefix sums mod n ⇒ two equal ⇒ a block with sum divisible by n. "Subset with sum divisible by m": if n ≥ m answer YES, else O(m²) DP (CF 577B).
4. Any process with S states repeats within S + 1 steps → find the cycle and skip.

```
Template: math/ntt.cpp, math/fft.cpp
```

5. Probability and expected value

KEYWORDS: probability expected value expectation average random mod p linearity

5.1 Linearity of expectation (the main tool)

KEYWORDS: expected value linearity indicator sum of probabilities average count

USE WHEN

- "Expected number of ...", "average over all ...", random choices.

DO THIS

1. E[X + Y] = E[X] + E[Y] ALWAYS, even if X and Y depend on each other.
2. Write the quantity as a sum of 0/1 indicators (one per event). E[count] = Σ P(event i). Each P(event i) is usually easy.
3. Expected sum of chosen values = Σ value_i · P(i is chosen).

FORMULAS

```
E[X] = Σ value · P(value)
E[Σ X_i] = Σ E[X_i]
E[#events] = Σ P(event_i)
```

EXAMPLE

Shuffle n cards: card i in place with probability 1/n → expected fixed points 1.
2024 D: choose K of N boxes, random apple of two from each: E = Σ_i (K/N)·(r_i + g_i)/2 = K·S/(2N).

5.2 Printing a probability / expectation mod p

KEYWORDS: probability modulo p q inverse fraction print modular expected value mint

DO THIS

1. Do every step in Mint (or mod arithmetic). Replace "divide by q" by "multiply by binExp(q, MOD−2)".
2. Never compute with doubles and convert at the end.
3. Probability 1/2 mod 998244353 = 499122177.

EXAMPLE

Expected value 3/2 → 3·inv(2) = 499122178 (mod 998244353).

5.3 Waiting times and classic expectations

KEYWORDS: expected tries until success geometric coupon collector expected maximum dice

FORMULAS

```
success with probability q per try: expected tries 1/q
coupon collector (n types): n·(1 + 1/2 + … + 1/n)
non-negative integer X: E[X] = Σ_{k≥1} P(X ≥ k)
max of n independent uniform values in 1..m: E = Σ_{k=1..m} (1 − ((k−1)/m)^n)
random permutation: E[inversions] = n(n−1)/4, E[cycles] = 1 + 1/2 + … + 1/n,
E[fixed points] = 1
two dice: E[sum] = 7
```

EXAMPLE

Roll until a 6: 6 rolls on average.

5.4 Expected steps: expectation DP

KEYWORDS: expected number of steps moves until markov dp self loop absorbing state

USE WHEN

- "Expected number of steps / days until the process ends" with a random transition.

DO THIS

1. E[s] = expected steps from state s to the end. End state: 0.
2. E[s] = 1 + Σ_t P(s → t)·E[t].
3. If s can stay in s with probability p: E[s] = (1 + Σ_{t≠s} P(s→t)·E[t]) / (1 − p).
4. Compute states in an order where each uses only finished states (e.g. the state only grows: go from the largest down).
5. States with cycles between them: write the linear equations and use Gaussian elimination (gauss.cpp).

EXAMPLE

Coin until heads: E = 1 + (1/2)·E → E = 2.
2022 C: E[i] = (1 + Σ_{j>i} P(i,j)E[j]) / (1 − P(i,i)).

5.5 Probability DP (forward)

KEYWORDS: probability dp distribution sum of dice forward

DO THIS

1. P[next state] += P[current] · (transition probability). Go forward in time.
2. Distribution of a sum of independent things: DP over the sum (convolution).
3. Conditional probability: P(A | B) = P(A and B) / P(B).

5.6 Expectation tricks: maximum, who comes first, records, deletion games

KEYWORDS: expected maximum at most x min kth largest who comes first random permutation records harmonic deleting subtree game on tree expected picks

DO THIS

1. Expected maximum of n values uniform in 1..k: P(max ≤ x) = (x/k)^n, so P(max = x) = (x/k)^n − ((x−1)/k)^n; E = Σ x·P(max = x). Same for min and k-th largest (CF 453A).
2. For X ≥ 0 integer: E[X] = Σ_{x≥1} P(X ≥ x).
3. Random permutation: element a comes before all of b1..bk with probability 1/(k+1).
4. Expected number of records (prefix maxima): H(n) = 1 + 1/2 + … + 1/n.

5. Delete a random remaining node with its whole subtree until empty: expected picks Σ_v 1/depth(v) (root depth 1) (CF 280C).
6. Expected inversions of a random permutation: n(n−1)/4. Fixed points: 1.
7. Choose K of N boxes at random, each gives (r + g)/2 on average: E = K·S/(2N) (Dhaka 2024 D).
8. Coupon collector: n·H(n). Geometric waiting: 1/p tries.

```
Template: templates/base (Mint)
```

5.7 Expectation DP patterns and self-loops

KEYWORDS: expected steps self loop beautiful mirrors dice probability candy lottery journey tree expected length escalator

DO THIS

1. E[s] = 1 + Σ P(s→t)·E[t]; if s can stay in s with prob q: E[s] = (1 + Σ_{t≠s} P·E[t]) / (1 − q).
2. Chain where failure sends you back to the start (Beautiful Mirrors, mirror i says yes with p_i): E = (1 + p1 + p1p2 + ... + p1...p(n−1)) / (p1...pn) (CF 1265E).
3. Probability of each sum of n dice (CSES Dice Probability): dp over dice with doubles.
4. Expected path length in a tree from the root (random child each step): E[v] = 1 + avg E[child] (CF 839C Journey).
5. Distribution over time (Ilya and Escalator): dp[t][j] = P(j boarded after t seconds); answer Σ j·dp[T][j].
6. When the answer is a fraction mod p, every division becomes × modInv; never compare modded values.

6. Bits and XOR

KEYWORDS: bitwise xor and or bits popcount mask trie basis

A. Identities and basics

6.1 Bit identities to remember

KEYWORDS: bit identities xor and or sum relation power of two lowest bit popcount builtin clz ctz parity complement

FORMULAS

```
a + b = (a ^ b) + 2·(a & b) = (a | b) + (a & b)
a ^ b = (a | b) − (a & b) a | b = a + b − (a & b)
a + b == (a ^ b) ⇔ (a & b) == 0 (no carries)
a ^ b ≤ a + b, a & b ≤ min(a, b) ≤ max(a, b) ≤ a | b
x ^ x = 0, x ^ 0 = x, a ^ b = c ⇔ a ^ c = b (xor is its own undo)
popcount(a ^ b) ≡ popcount(a) + popcount(b) (mod 2)
~x = −x − 1; x mod 2^k = x & ((1 << k) − 1); x / 2^k = x >> k (x ≥ 0)
lowest set bit x & −x (= largest power of 2 dividing x); remove it x & (x − 1)
x & (x + 1): clear trailing ones; x | (x + 1): set the lowest 0; x ^ (x + 1) =
2^(t+1) − 1, t = trailing ones of x
power of two ⇔ x > 0 && (x & (x − 1)) == 0; sum of two powers of two ⇔ popcount ≤
2 (or x = 2^k + 2^k)
```

```
min over d of (a ^ d) + (b ^ d) = a ^ b (d = a & b)
OR and AND of two unknown numbers given: possible ⇔ (and & ~or) == 0; then xor =
or ^ and, sum = or + and
__builtin_popcountll(x), __builtin_parityll(x), 63 - __builtin_clzll(x) (highest
bit, x > 0), __builtin_ctzll(x) (x > 0)
```

EXAMPLE

2025 G: one cell change xors row value r and column value c by the same d; best new r + c is r ^ c; gain r + c − (r ^ c) = 2(r & c). Answer Total − 2·max(r_i & c_j).

WATCH OUT

- 1 << 40 overflows: write 1LL << k. Shifting by ≥ 64 is undefined. Precedence: a & b == c means a & (b == c): use brackets. clz/ctz of 0 is undefined.

B. Observations that solve problems

6.2 Bit observations: the first things to try

KEYWORDS: observation highest bit msb independent bits greedy from top bit and decreases or increases monotone xor compare highest differing bit majority

DO THIS

1. Bits are independent in &, |, ^: solve each bit separately and add count·2^b.
2. Higher bit beats all lower bits together (2^b > 2^b − 1): decide the answer GREEDILY from the top bit down ("can I keep this bit?").
3. Extending a range: AND only loses bits, OR only gains bits ⇒ monotone ⇒ two pointers / binary search; and at most ~log(V) different values for a fixed end.
4. Compare a ^ x with b ^ x: only the highest bit where a and b differ matters; a ^ x < b ^ x ⇔ x has the same bit there as a. So "choose x so the array becomes sorted after xor" gives one constraint per adjacent pair on one bit of x.
5. a & b ≥ a ^ b ⇔ a and b have the same highest bit (a, b > 0) → group by highest bit, count pairs inside groups (CF 1420B).
6. Minimise Σ (a_i ^ x): for each bit set x's bit if more than half of the a_i have it.
7. Minimise max_i (a_i ^ x): recursion on the top bit: if all numbers agree on the bit, x copies it; else answer 2^b + min(solve(group0), solve(group1)) (CF 1285D, trie).
8. Minimum xor pair = minimum over ADJACENT elements after sorting.
9. Many operations "a_i = a_i & a_j": you can reach the AND of everything; the answer of "minimise the max" is the AND of all (CF 1559A).
10. Operations that only add bits (|=) or only remove bits (&=) can change each number at most ~60 times → amortised brute force.
11. i·j − k·(a_i | a_j) with a_i ≤ n, k small: the best pair uses large indices, only check the last ~2k (CF 1554B).
12. When stuck: print a table of small cases in binary and look at it.

EXAMPLE

a = [5 (101), 6 (110), 2 (010)]: sorted 2, 5, 6 → adjacent xors 7, 3 → min xor pair = 3 (5 ^ 6).

6.3 Ranges of consecutive numbers [L, R] and 1..n

KEYWORDS: and of range l to r or of range consecutive integers xor of 1..n count numbers with bit set in 0..n max xor in range and then there were k sum of changed bits

FORMULAS

```
xor of 0..n: n%4 = 0 → n, 1 → 1, 2 → n + 1, 3 → 0; xor of L..R = f(R) ^ f(L−1)
AND of L, L+1, …, R = common binary prefix of L and R, then zeros
OR of L, L+1, …, R = common binary prefix of L and R, then all ones
max (x ^ y) with L ≤ x, y ≤ R: 2^(msb(L ^ R) + 1) − 1 (0 if L = R)
numbers in [0, n] with bit b set: ⌊n / 2^(b+1)⌋·2^b + max(0, n mod 2^(b+1) − 2^b + 1)
Σ popcount(0..n) = Σ over b of the line above
fewest deletions so that AND of [l..r] is non-zero: (r − l + 1) − max over b of (count with bit b) (CF 1615B, prefix counts per bit)
largest k with n & (n−1) & … & k = 0: 2^msb(n) − 1 (CF 1527A)
Σ_{i=1..n} popcount(i ^ (i−1)) (total changed bits when counting) = 2n − popcount(n) (CF 1362C)
count x in [0, n] with n + x = n ^ x: 2^(number of zero bits of n below its highest bit)
ordered pairs (x, y) with x | y = n: 3^popcount(n); with x | y = n and x & y = 0: 2^popcount(n)
```

```
int cntBit(int n, int b) {                  // numbers in [0, n] with bit b set
    int p = 1LL << b, full = n / (2 * p) * p;
    return full + max(0LL, n % (2 * p) - p + 1);
}
int xorUpTo(int n) { int r[4] = {n, 1, n + 1, 0}; return r[n % 4]; }
```

C. Counting sums of xor / and / or

6.4 Per-bit counting: sums over pairs, subarrays, subsets

KEYWORDS: sum of xor of all pairs sum of and or over subarrays subsets per bit count ones zeros runs prefix parity xor of all pairwise sums present

DO THIS

1. For bit b let ones = numbers having it, zeros = n − ones. Add (count)·2^b over all bits.

FORMULAS

```
Σ over pairs i < j of (a_i ^ a_j): ones·zeros·2^b
Σ over pairs of (a_i & a_j): C(ones, 2)·2^b Σ over pairs of (a_i | a_j): (C(n,2) − C(zeros,2))·2^b
Σ over subarrays of AND: for each run of L consecutive ones: L(L+1)/2 ·2^b
Σ over subarrays of OR: (n(n+1)/2 − Σ over runs of zeros L(L+1)/2)·2^b
Σ over subarrays of XOR: (number of pairs of prefix parities that differ)·2^b = even·odd·2^b (prefix parities incl. p0)
Σ over non-empty subsets of XOR = (OR of all)·2^(n−1)
Σ over non-empty subsets of AND: (2^ones − 1)·2^b of OR: (2^n − 2^zeros)·2^b
expected value versions: divide by the number of objects (mod p: × inverse)
XOR of all pairwise sums a_i + a_j (CF 1322B): for each bit k take b = a mod 2^(k+1), sort; count pairs with b_i + b_j in [2^k, 2^(k+1) − 1] or [3·2^k, 2^(k+2) − 2] (binary search / two pointers); bit k is set iff that count is odd
```

```
int ans = 0;                                // Σ over all subarrays of XOR
rep(b, 0, 30) {
    int cnt[2] = {1, 0}, p = 0, pairs = 0;
    rep(i, 0, n) { p ^= a[i] >> b & 1; pairs += cnt[p ^ 1]; cnt[p]++; }
    ans += pairs << b;
}
```

Template: misc/bit_tricks.cpp sumPairXor

D. Prefix xor

6.5 Prefix xor: subarrays, splitting, trees

KEYWORDS: prefix xor subarray xor equal k count zero xor split into segments equal xor agagaga reduce array adjacent xor tree path xor

DO THIS

1. p[0] = 0, p[i] = p[i−1] ^ a[i]; xor(l..r) = p[r] ^ p[l−1].
2. Count subarrays with xor = k: walk r, add cnt[p[r] ^ k], then cnt[p[r]]++ (cnt[0] = 1 first). xor 0 ⇔ equal prefix values.
3. Subarray xor 0 with even/odd length restrictions: keep separate maps for even and odd indices (CF 1109A: count p[l−1] = p[r] with same parity of index).
4. Replace adjacent pairs by their xor until ≥ 2 equal elements remain (CF 1516B): total T = 0 ⇒ yes; else yes ⇔ greedy splitting into pieces with xor T gives ≥ 3 pieces.
5. Split into k parts that all have xor v: the total must be v if k is odd and 0 if k is even.
6. Tree: x(u) = xor from the root; path xor(u, v) = x(u) ^ x(v); max path xor → trie on x; count paths with xor k → map.
7. Xor of a range of an array with range "xor-assign" updates: segment tree with per-bit counts (next card group).

EXAMPLE

> a = [3, 1, 2]: p = 0, 3, 2, 0 → p0 = p3 → whole array has xor 0.

E. Constructions

6.6 Constructions with bits

KEYWORDS: construct array given xor and sum ehab xorcist sum of k powers of two permutation minimum maximum adjacent xor roof construction pairing and equal k complement distinct numbers xor zero even odd xor

FORMULAS

```
shortest array with xor u and sum v: impossible if u > v or (v − u) odd; u = v → [u] (empty if 0); x = (v − u)/2: if (x & u) == 0 → [u + x, x], else [u, x, x] (CF 1325D)
n as a sum of exactly k powers of two: possible ⇔ popcount(n) ≤ k ≤ n; start from the binary digits and split the largest power until there are k (CF 1095C)
permutation of 0..n−1 minimising the max adjacent xor: answer 2^msb(n−1); write 2^k − 1, …, 1, 0, 2^k, 2^k + 1, …, n − 1 (CF 1632B)
pair 0..n−1 (n a power of two) so that Σ (x & y) = k, with c(x) = n − 1 − x (x & c(x) = 0): k = 0 → all (x, c(x)); 0 < k < n−1 → (k, n−1), (0, c(k)), rest (x, c(x)); k = n−1 → (n−1, n−2), (1, n−3), (0, 2), rest (x, c(x)), impossible for n =
```

```
4 (CF 1630A)
n distinct numbers with XOR of even positions = XOR of odd positions (⇔ total XOR
0): 1, 2, …, n−3, then 2^29, 2^30, and last = (1 ^ … ^ (n−3)) ^ 2^29 ^ 2^30 (it
has both high bits, so it is new) (CF 1722G)
two numbers with given sum s and xor x: a & b = (s − x)/2 must be an integer with
(a & b) & x = 0; then a = (s − x)/2 + x, b = (s − x)/2
x ⊆ y (submask) ⇔ (x & y) == x ⇔ (x | y) == y
build answer bit by bit from the top: try setting the bit, check feasibility with
a mask of "bits fixed so far" (typical for max AND / max OR subset questions)
```

F. Binary trie and xor basis

6.7 Binary trie: max xor, count xor < k, min max xor

KEYWORDS: binary trie max xor pair query xor with set count xor less than k insert delete subarray max xor

USE WHEN

- "Max a_i ^ a_j", "for each query x, max x ^ a_i", "subarray with max xor" (prefix xors), with insertions/deletions; "count pairs with xor < k".

DO THIS

1. Insert numbers from the highest bit. To maximise x ^ y: at each bit go to the opposite bit if that child is non-empty.
2. Count y with x ^ y < k: walk bits of k; where k has 1, all y in the child that makes the xor bit 0 are smaller — add its count, continue in the other child.
3. Deletion: keep counts on nodes, decrement along the path.
4. Small values (≤ 2^20): an array of "exists" over all values and brute force over masks may be simpler.

```
BinTrie t(30); t.insert(0);
int best = 0, p = 0;
for (int x : a) { p ^= x; best = max(best, t.maxXor(p)); t.insert(p); }   // max
subarray xor
```

```
Template: data_structures/binary_trie.cpp
```

6.8 Xor basis (linear basis over GF(2))

KEYWORDS: xor basis linear basis subset xor maximum kth smallest count subsets with xor x prefix linear basis range query ivan and burgers graph path xor cycles gauss mod 2

USE WHEN

- "Max xor of a subset", "can x be made", "how many distinct subset xors", "k-th smallest subset xor", "number of subsets with xor x", range versions, max xor path in a graph.

DO THIS

1. Insert every number (≤ 60 basis vectors). max subset xor: greedy from the top with the basis. canMake(x): reduce x to 0.
2. Distinct reachable values: 2^rank. Subsets (including empty) giving any reachable value: 2^(n − rank).
3. k-th smallest: reduce the basis (each leading bit appears in one vector only), sort by leading bit; the bits of k−1 choose vectors (watch whether 0 is reachable: it is iff rank < n).

4. Max subset xor in a[l..r], many queries: prefix basis keeping, for each bit, the vector with the LARGEST index (swap on insert); query r uses vectors with index ≥ l (CF 1100F).
5. Max/min xor path 1 → n in a graph: d[v] = xor along a DFS tree; each non-tree edge gives a cycle value d[u] ^ d[v] ^ w; insert into a basis; answer = d[n] reduced by the basis (min: greedy to decrease) (CF 845G).
6. Lights-out / switches toggling sets: Gaussian elimination mod 2 with bitset rows; solutions = 2^(free variables).

```
int bas[61];                                        // insert
bool ins(int x) { repb(b, 60, 0) if (x >> b & 1) { if (!bas[b]) { bas[b] = x; return
true; } x ^= bas[b]; } return false; }
int maxX(int x = 0) { repb(b, 60, 0) x = max(x, x ^ bas[b]); return x; }   // max of x
^ (subset)
```

Template: math/xor_basis.cpp, math/gauss.cpp

G. AND / OR over ranges and subarrays

6.9 OR / AND / gcd of all subarrays, and windows with bit counters

KEYWORDS: or of subarrays distinct values and of subarrays count subarrays with or at least k two pointers bit counter cnt per bit window and

DO THIS

1. For a fixed right end r, OR of a[l..r] over all l has at most ~31 different values (bits are only added). Same for AND and gcd.
2. Keep a list of (value, count) for subarrays ending at r; next list = {a[r+1]} ∪ {v | a[r+1]}, merge equal values. Distinct ORs over all subarrays ≤ 31·n (CF 243A).
3. Window OR/AND with removals: keep cnt[b] = how many numbers in the window have bit b; OR = bits with cnt > 0, AND = bits with cnt = window length. Then two pointers ("shortest subarray with OR ≥ k", "count subarrays with AND ≥ x").
4. Range AND/OR queries: sparse table (idempotent) — O(1).
5. Longest subarray with AND > 0: per bit, longest run of ones.
6. Smallest OR of a subarray of length ≥ … / largest AND of length k: binary search + sparse table.

```
vector<pi> cur;                         // (OR value, number of l) for subarrays ending
at r
for (int x : a) {
    vector<pi> nx = {{x, 1}};
    for (auto &p : cur) { int v = p.first | x; if (nx.back().first == v)
nx.back().second += p.second; else nx.push_back({v, p.second}); }
    cur = nx;                                 // use cur here
}
```

Template: data_structures/sparse_table.cpp

H. Masks, SOS and bit DP

6.10 Bitmasks: subsets, submasks, SOS, pairs with a & b = 0

KEYWORDS: bitmask subsets submask enumeration 3^n sos dp sum over subsets supersets count pairs and zero or equals compatible numbers bits and pieces gosper next combination gray code

DO THIS

1. All subsets: for m in [0, 2^n). Submasks of m: for (s = m; s; s = (s − 1) & m) (+ s = 0 separately); all masks together 3^n.
2. SOS: f[m] = Σ over submasks — for b, for m: if (m >> b & 1) f[m] += f[m ^ (1 << b)]. Supersets: if !(m >> b & 1) f[m] += f[m | 1 << b]. O(n·2^n). Same loops with max/min or "any index".
3. Pairs with a_i & a_j = 0 (a_i + a_j = a_i ^ a_j): for each a, count numbers that are submasks of the complement ~a & (2^B − 1) (SOS count) (CF 165E: find one partner — store an index instead).
4. Number of subsets with AND exactly 0: SOS of counts over supersets → g(m) = 2^cnt_sup(m) − 1 subsets with AND ⊇ m; inclusion–exclusion Σ (−1)^popcount(m) g(m).
5. max a_i | (a_j & a_k) with i < j < k: for each mask keep the two largest indices of supersets (SOS), greedy bits from the top for each i (CF 1208F).
6. Gray code g = i ^ (i >> 1): consecutive codes differ in one bit (enumerate subsets changing one element at a time).
7. Next mask with the same popcount (Gosper): c = x & −x; r = x + c; x = (((r ^ x) >> 2) / c) | r.
8. Subset sums with n ≤ 40: meet in the middle (two halves of 2^20).
9. bitset<N> for subset-sum reachability (b |= b << a) and for transitive closure: 64× faster.

```
rep(b, 0, B) rep(m, 0, 1LL << B) if (m >> b & 1) f[m] += f[m ^ (1LL << b)];   // f[m] = Σ f[sub ⊆ m]
for (int s = m; ; s = (s - 1) & m) { /* use s */ if (s == 0) break; }      // includes 0
```

```
Template: misc/common.cpp (sosSubsets, sosSupersets), math/fwht.cpp
```

6.11 Bits in data structures, digit DP on bits, interactive

KEYWORDS: segment tree per bit xor update range sum count bits lazy flip digit dp on bits count pairs a xor b at most n interactive and or queries recover numbers fwht xor convolution

DO THIS

1. Range "xor with x" + range sum: segment tree storing cnt[b] (ones of bit b) per node; lazy xor flips cnt[b] = len − cnt[b] for bits of x (CF 242E). Range AND/OR assign: same per-bit idea.
2. Digit DP over bits (count x ≤ N, or pairs (a, b) ≤ N with a & b = 0, a ^ b ≤ k ...): go from the top bit, state = tight flags for each bounded number (2 or 4 states) + what you need.
3. Count pairs with (a_i ^ a_j) ≤ k or ≥ k: trie (count xor < k) or digit DP.
4. Interactive: ask a | b and a & b for pairs; a + b = (a | b) + (a & b); with three numbers: sums of the three pairs give each number (CF 1556D). Ask a ^ b for all b vs one fixed a: n − 1 queries recover everything once one value is known.
5. XOR convolution (count pairs with a_i ^ a_j = v for all v): FWHT in O(V log V). AND/OR convolution: SOS on both, multiply, inverse.

```
Template: math/fwht.cpp, misc/digit_dp.cpp,
data_structures/segment_tree/custom_node.cpp
```

6.12 Maximum AND / OR of a pair or subset

KEYWORDS: maximum and pair two arrays max and subset of size k max or pair greedy top bit candidates

DO THIS

1. Max AND of a pair: from the top bit try ans | bit; keep only numbers containing all chosen bits; keep the bit if ≥ 2 candidates (or ≥ 1 in each array for two arrays).
2. Max AND of a subset of size k: same with "≥ k candidates".
3. Max OR of a pair: for each a_i the best partner covers its missing bits; with values ≤ 2^20 use SOS "exists a superset of the wanted mask" and decide bits greedily from the top. (The largest element is NOT always in the best pair: 12, 10, 5 → 10 | 5 = 15.)
4. Max (a_i & a_j) over i < j inside many sets: sort / bucket by highest bits.

```
Template: misc/bit_tricks.cpp maxAndPair, maxAndPairOne
```

7. Arrays, searching, greedy, constructive

KEYWORDS: array prefix sum two pointers sliding window binary search greedy sort sweep intervals constructive

7.1 Prefix sums and difference arrays (1D and 2D)

KEYWORDS: prefix sum range sum difference array range add 2d prefix rectangle sum

USE WHEN

- Many range-sum queries (no updates): prefix sums. Many range-add updates, read at the end: difference array.

DO THIS

1. Subarrays with sum k: count earlier prefix sums equal to p − k (map).
2. Subarray sum divisible by m: two equal prefix sums mod m.
3. Count subarrays not all equal: total n(n+1)/2 minus Σ over maximal equal runs L(L+1)/2 (2023 E).

```
vi p(n + 1, 0); rep(i, 0, n) p[i + 1] = p[i] + a[i];     // sum a[l..r] = p[r+1] - p[l]
vi d(n + 1, 0); d[l] += v; d[r + 1] -= v;                  // add v to a[l..r]; then
a[i] = prefix sum of d
// 2D: P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] - P[i][j]
// sum (r1,c1)..(r2,c2) = P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]
// 2D difference: D[r1][c1]+=v, D[r1][c2+1]-=v, D[r2+1][c1]-=v, D[r2+1][c2+1]+=v, then
2D prefix sums
```

7.2 Two pointers / sliding window

KEYWORDS: two pointers sliding window longest shortest subarray at most k distinct sum at most

USE WHEN

- "Longest / shortest subarray (substring) such that ..." and the condition is monotone (shrinking a good window keeps it good, or growing keeps it good). Values non-negative for sums.

DO THIS

1. Longest: move r forward; while the window is bad, move l forward; record r − l + 1.

2. Shortest: move r until good; then move l while still good, recording lengths.
3. Count subarrays with at most K (distinct values / sum / bad items): for each r add r − l + 1.
4. EXACTLY K = atMost(K) − atMost(K − 1).

```
for (int l = 0, r = 0; r < n; r++) {
    add(a[r]);
    while (bad()) remove(a[l++]);
    best = max(best, r - l + 1);    // or: cnt += r - l + 1
}
```

WATCH OUT

- With negative numbers sums are not monotone: use prefix sums + map / sorted structure instead.

7.3 Binary search on the answer

KEYWORDS: binary search answer minimise maximum maximise minimum feasible check monotone kth smallest

USE WHEN

- "Minimise the maximum", "maximise the minimum", "smallest x such that it is possible", "k-th smallest value of something you can count".

DO THIS

1. Write ok(x) = "can we do it with answer x" (usually a greedy pass). It must be monotone (false…false true…true).
2. Binary search the first x with ok(x) = true.
3. k-th smallest: ok(v) = (count of items ≤ v) ≥ k.

```
int lo = 0, hi = 2e18;                  // ok(hi) must be true
while (lo < hi) { int mid = lo + (hi - lo) / 2; if (ok(mid)) hi = mid; else lo = mid +
1; }
// doubles: rep(it, 0, 100) { double m = (lo + hi) / 2; if (ok(m)) hi = m; else lo = m;
}
```

WATCH OUT

- Inside ok() products can overflow 1e18: use __int128 or stop early when the sum exceeds the target.

```
Template: misc/common.cpp firstTrue
```

7.4 Ternary search (unimodal function)

KEYWORDS: ternary search unimodal maximum minimum convex function

DO THIS

1. Function increases then decreases (or the opposite): compare f(m1) and f(m2), drop the worse third. Integers: stop when hi − lo ≤ 2 and check all. Doubles: 100 iterations.

```
Template: misc/common.cpp ternaryMax
```

7.5 Intervals, scheduling, sweep line

KEYWORDS: intervals meetings schedule overlap sweep events non overlapping merge minimum points deadline

DO THIS

1. Most non-overlapping intervals: sort by END, take each interval that starts after the last taken end.
2. Fewest points that hit all intervals: sort by end, place a point at the end when the interval is not hit yet.
3. Maximum overlap at one time: events (start, +1), (end, −1), sort (put −1 before +1 at the same time if touching does not overlap).
4. Merge overlapping intervals: sort by start, extend the current end.
5. Deadlines with profits (unit tasks): sort by deadline, push profit into a min-heap, if heap size > deadline pop the smallest.
6. Minimum rooms / machines: sort by start, min-heap of end times.

7.6 Greedy order by exchange argument

KEYWORDS: greedy sort comparator order exchange argument swap adjacent

USE WHEN

- Choose an order of items to minimise a total that depends on the order.

DO THIS

1. Compare two neighbours a then b vs b then a. Put a first when cost(a, b) < cost(b, a). Sort with that comparator.
2. Write the comparator with products (a.t·b.w < b.t·a.w), not divisions. It must be a strict ordering (no ≤).

EXAMPLE

> Jobs with time t and weight w, minimise Σ w·(finish time): sort by t/w ascending (a.t·b.w < b.t·a.w).

7.7 Other greedy patterns

KEYWORDS: greedy priority queue largest smallest pair huffman lexicographically smallest

DO THIS

1. Take the k best: sort or nth_element. Repeatedly take the best available: priority_queue.
2. Merge cost (merge two piles, pay their sum): always merge the two smallest (min-heap).
3. Pair to minimise the maximum pair sum: sort, pair smallest with largest.
4. Lexicographically smallest result: decide position by position, try the smallest choice that can still be completed.
5. Sum of $|x_i - c|$ is minimised at c = median. Sum of $(x_i - c)^2$ at c = mean.
6. Not sure greedy is right? Brute force small random cases and compare.

7.8 LIS, max subarray, inversions

KEYWORDS: longest increasing subsequence kadane maximum subarray sum inversions count

DO THIS

1. Min number of non-increasing subsequences to cover = LIS (Dilworth). Longest chain in 2D: sort by x, LIS on y (equal x: sort y descending).

2. Max subarray after reversing one subarray: best sum of at most two disjoint segments (2020 E).

```
// LIS strictly increasing: lower_bound; non-decreasing: upper_bound
vi t; for (int x : a) { auto it = lower_bound(all(t), x); if (it == t.end()) t.pb(x);
else *it = x; }  // t.size()
// Kadane (non-empty)
int best = LLONG_MIN, cur = 0; for (int x : a) { cur = max(x, cur + x); best =
max(best, cur); }
// inversions: BIT over compressed values, for i from left: inv += i - bit.sum(a[i])
```

7.9 Constructive problems: how to start

KEYWORDS: construct any valid answer output construction pattern small cases special judge

DO THIS

1. Write a brute force for n = 1..8 that finds all valid answers. Print them. Look for a pattern: sorted, reversed, alternating, pairs (i, n+1−i), powers of two, blocks.
2. Find a necessary condition (parity, sum, count). Try to build an answer whenever it holds.
3. Keep a checker function and test your construction for all n up to 1000 locally.
4. Special judge: any valid answer, but the exact format.

EXAMPLE

> 2016 F: rebuild an array from range-minimum answers: a[i] = max answer over queries covering i (others 1e9), then verify all queries.
> 2018 J: orient tree edges so the longest directed path is 1: colour by depth parity, orient from colour 0 to 1.

7.10 Coordinate compression and sorting tricks

KEYWORDS: coordinate compression rank big values index sort pairs

DO THIS

1. Sort indices by value: vi id(n); iota(all(id), 0); sort(all(id), [&](int i, int j){ return a[i] < a[j]; });
2. nth_element for the k-th smallest in O(n).

```
vi v = a; sort(all(v)); v.erase(unique(all(v)), v.end());
for (int &x : a) x = lower_bound(all(v), x) - v.begin();       // ranks 0..k-1
```

```
Template: misc/common.cpp compress
```

7.11 Array and greedy patterns from contests

KEYWORDS: array patterns subarray sum equals k count prefix map sum of minimums of subarrays median binary search kth minimum maximum average stack of next greater merge intervals meeting rooms

FORMULAS

```
subarrays with sum k: prefix sums + map of counts (also sum divisible by m: map of
prefix mod m)
longest subarray with sum ≤ S (positives): two pointers; with negatives: prefix
sums + monotonic stack / sorted prefix
sum of min (max) over all subarrays: previous/next smaller with a monotonic stack:
a_i·(i − L)·(R − i), use < on one side and ≤ on the other
```

```
max of every window of size k: monotonic deque
max average subarray with length ≥ k: binary search the average, subtract it,
check prefix minimum
k-th smallest pair sum / distance: binary search the value + two pointers counting
median ≥ x check: replace by +1 / −1, look for a subarray with positive sum
largest rectangle in a histogram: monotonic stack; all-1 submatrix: histogram per
row
minimum rooms for intervals: sort start (+1) and end (−1) events; max overlap
choose max number of non-overlapping intervals: sort by end, greedy
min number of points to stab all intervals: sort by end, place at the end when not
stabbed
jobs with deadlines and profits: sort by profit, put each in the latest free slot
(DSU) — or by deadline with a min-heap
minimise Σ completion times: shortest job first; with weights: sort by t/w
(exchange argument)
two sorted arrays merging / k-way: priority queue
next permutation / k-th permutation: factorial number system
make the array non-decreasing with fewest "+1 to a suffix" ops: Σ max(0, a_i −
a_{i+1})
```

8. Dynamic programming

KEYWORDS: dp dynamic programming state transition knapsack lis interval bitmask digit tree memo

8.1 How to build a DP in 4 questions

KEYWORDS: dp design state transition base case order memoization

DO THIS

1. STATE: what must I remember about what I already did to decide the rest? (index, last value, count used, sum so far, remainder, mask, balance, tight flag)
2. TRANSITION: what is the LAST decision? dp[state] = best / sum over that decision of dp[smaller state].
3. ORDER and BASE: loop so smaller states are done first. Write the base case. Know where the answer is.
4. SIZE: states × transitions ≤ ~1e8. Too big? Drop a dimension that follows from others; swap value and index (dp[value] = min index); speed up the transition (prefix sums, monotonic deque, bitset).
5. Recursion with memo is fine for small states; for n ≥ 1e5 loop iteratively.

WATCH OUT

- Initialise "impossible" states to −INF / INF, not 0.
- Reset arrays between test cases (only the used part).

8.2 Knapsack family

KEYWORDS: knapsack 0/1 unbounded coin change number of ways subset sum bounded items weight value

USE WHEN

- Pick items with weights under a capacity; count ways to make a sum; can we reach sum S.

DO THIS

1. 0/1 (each item once): for each item, for w from W DOWN to wt: dp[w] = max(dp[w], dp[w − wt] + val).
2. Unbounded (unlimited copies): same, w goes UP.
3. Ways to make sum S with coins, ORDER DOES NOT MATTER: coins outer loop, sum inner loop going up.
4. Ways, ORDER MATTERS (sequences): sum outer loop, coins inner loop.
5. Reachable sums (yes/no): bitset b; b[0]=1; for x: b |= b << x. O(n·S/64).
6. Big weights (1e9) but small values: dp[value] = min weight.
7. Bounded copies (c_i copies): split into 1, 2, 4, ..., rest (binary splitting), then 0/1.

```
vi dp(W + 1, 0);
rep(i, 0, n) repb(w, W, wt[i]) dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);   // 0/1
knapsack
```

```
Template: dp/dp_patterns.cpp knapsack, subsetSums, countSubsetSums
```

8.3 Sequence DPs: LIS, LCS, edit distance, palindromes

KEYWORDS: lcs longest common subsequence edit distance lis palindrome subsequence string dp

FORMULAS

```
LCS: dp[i][j] = a[i−1]==b[j−1] ? dp[i−1][j−1]+1 : max(dp[i−1][j], dp[i][j−1])
Edit distance: dp[i][j] = min(dp[i−1][j]+1, dp[i][j−1]+1, dp[i−1][j−1] +
(a[i−1]!=b[j−1]))
Longest palindromic subsequence = LCS(s, reverse(s)); min insertions to make a
palindrome = n − LPS
Count palindromic substrings: Manacher, or dp[l][r] = s[l]==s[r] && dp[l+1][r−1]
```

```
Template: strings/string_dp.cpp, dp/dp_patterns.cpp lcs
```

8.4 Grid DP

KEYWORDS: grid dp paths maximum sum path obstacles right down

DO THIS

1. dp[r][c] from dp[r−1][c] and dp[r][c−1]. Obstacles: dp = 0 there.
2. Two paths at once (go and return): walk both together, state (step, r1, r2) since c = step − r.
3. Rows ≤ 10, columns huge: state = mask of the current column (profile DP), transition column by column; matrix power if columns ≤ 1e18.

8.5 Interval DP: dp[l][r]

KEYWORDS: interval dp merge stones remove brackets palindrome split range dp l r

USE WHEN

- Merging neighbours, removing segments, matching brackets, cutting a stick — the answer for [l, r] comes from smaller ranges.

DO THIS

1. Loop by length from small to large. dp[l][r] = best over split point k of dp[l][k] + dp[k+1][r] + cost(l, r), or by matching s[l] with some s[k].
2. $O(n^3)$ for n ≤ 400–500.

```
rep(len, 2, n + 1) rep(l, 0, n - len + 1) { int r = l + len - 1; dp[l][r] = INF;
    rep(k, l, r) dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + sum(l, r)); }
```

```
Template: dp/dp_patterns.cpp mergeCost
```

8.6 Bitmask DP (n ≤ 20)

KEYWORDS: bitmask dp tsp travelling salesman assignment subsets visit all set cover

USE WHEN

- n ≤ 20 (or ≤ 23): visit all, assign each person a task, cover a set, order matters among few items.

DO THIS

1. Visit all and end at v: dp[mask][v] = min over u in mask of dp[mask without v][u] + cost(u, v). O(2^n·n²).
2. Assignment (person i = popcount(mask) gets one task): dp[mask] over tasks used. O(2^n·n).
3. Set cover with few elements: dp[mask] = min sets to cover mask, dp[mask | s] = min(dp[mask] + 1).
4. Many special points on a grid: BFS from each special point to get pairwise distances, then bitmask DP.

```
Template: dp/dp_patterns.cpp tsp
```

8.7 Digit DP: count numbers in [L, R] with a digit property

KEYWORDS: digit dp count numbers range digit sum divisible tight leading zeros between l and r

USE WHEN

- "How many numbers in [L, R] have (digit sum = S / divisible by k / no two equal adjacent digits / at most 3 non-zero digits ...)", with R up to 1e18.

DO THIS

1. Answer = count(R) − count(L − 1), where count(N) counts x in [0, N].
2. Walk the digits of N from the left. State: pos (which digit), tight (are we still equal to N's prefix? then the next digit can go only up to N's digit), plus what the property needs: digit sum so far, remainder mod k, last digit, number of non-zero digits, started (have we placed a non-zero digit yet — needed when leading zeros would break the property, like "last digit" or "count of digits").
3. At the end (pos == length) return 1 if the property holds, else 0.
4. Memoize only states with tight = false (tight states are visited once).
5. Clear the memo when N changes (or include N's digits nowhere in the memo: with tight=false the memo does not depend on N, so you can keep it between calls if the property is the same).

EXAMPLE

> K = 7, N = 25: numbers 0..25 with digit sum divisible by 7 are 0, 7, 16, 25 → countUpTo(25) = 4 (0 included).

```
string D; int K;                            // count x in [0, N] whose digit sum is
divisible by K
int memo[20][200]; bool vis[20][200];
int go(int pos, int rem, bool tight) {
```

```
    if (pos == (int)D.size()) return rem == 0;
    if (!tight && vis[pos][rem]) return memo[pos][rem];
    int lim = tight ? D[pos] - '0' : 9, res = 0;
    rep(d, 0, lim + 1) res += go(pos + 1, (rem + d) % K, tight && d == lim);
    if (!tight) vis[pos][rem] = 1, memo[pos][rem] = res;
    return res;
}
int countUpTo(int N) { if (N < 0) return 0; D = to_string(N); memset(vis, 0, sizeof
vis); return go(0, 0, true); }
// answer = countUpTo(R) - countUpTo(L - 1)
// leading zeros matter (e.g. "no two adjacent equal digits"): add bool started; while
!started, digit 0 keeps started false
// and does not count as the "last digit".
```

WATCH OUT

- 0 is counted by count(N) (digit sum 0). Subtract it if the problem wants positive numbers.
- Large state (sum up to 9·18 = 162, remainder up to k): array sizes must fit.

`Template: misc/digit_dp.cpp`

8.8 DP on trees

KEYWORDS: tree dp subtree independent set matching path counting choose k nodes

DO THIS

1. dp[v] from dp[children]. Process vertices in reverse BFS order (no recursion problems).
2. Max independent set: in[v] = val + Σ out[c], out[v] = Σ max(in[c], out[c]).
3. Choose exactly k vertices (knapsack on tree): merge child tables; loop sizes only up to subtree sizes → $O(n^2)$ total.
4. Count paths with a small property (exactly 2 marked nodes): dp[v][t] = number of downward paths from v with t marks; at v combine pairs of children (2016 D).
5. Answer for EVERY root: rerooting (Trees part).

`Template: trees/rerooting.cpp`

8.9 Making a DP faster

KEYWORDS: dp optimization prefix sum monotonic queue deque convex hull trick li chao bitset matrix

DO THIS

1. Transition sums a range of previous dp values: keep prefix sums of dp.
2. Transition takes min/max over a sliding window of previous dp: monotonic deque.
3. dp[i] = min_j (dp[j] + b_j·x_i): each j is a line, query at x_i: Li Chao tree (li_chao.cpp).
4. Boolean dp over sums: bitset.
5. Same transition many times (n up to 1e18): matrix power.
6. Split into k contiguous groups with cost(l, r): dp[j][i] = min_p dp[j−1][p] + cost(p+1, i); cost of a group with the median: Σ|x − median| via prefix sums (2022 G).
7. Memory: keep only the previous row.

8.10 Expectation / probability DP

KEYWORDS: expected value dp probability dp

DO THIS

1. See the Probability part: E[s] = 1 + Σ P(s→t)·E[t], computed from the end backwards; probabilities forward.

8.11 DP patterns: which state for which statement

KEYWORDS: dp patterns state choose which dp lis count paths partition into segments k segments palindrome partition subset sum bitset coin change ways profile

FORMULAS

```
split the array into segments, cost per segment: dp[i] = best over last segment
start j (O(n²), or optimise)
exactly k segments: dp[k][i]; cost monotone → divide and conquer / Knuth
take or skip with a limit (knapsack): dp[w]; items once → w downward; unlimited →
w upward
counting subsets with sum s, n·s ≤ 1e8: bitset<S> b; b |= b << a (existence only)
number of ways, order matters (compositions): for s outer, for coin inner; order
does not matter: coin outer
grid paths with obstacles: dp[i][j] = up + left; max path sum; two walkers at
once: dp[step][r1][r2]
LIS in O(n log n): tails array with lower_bound (upper_bound for non-decreasing)
count increasing subsequences: BIT over compressed values, dp[i] = 1 + Σ dp[j] for
a_j < a_i
interval dp[l][r]: merging stones, palindromes, removing blocks, matrix chain —
fill by length
bitmask dp[mask] (n ≤ 20): assignment, TSP dp[mask][last], partition into groups
(submask enumeration 3^n)
digit DP: (pos, tight, started, extra state); answer f(R) − f(L−1)
tree DP: children before parent; rerooting for every root
dp over subsets of primes ≤ 30 (only 10 primes): mask of used primes
profile DP for tilings: dp[column][mask of filled cells]
when values are big but n small: swap the roles (dp[value] = minimum weight
instead of dp[weight] = max value)
probability DP forward, expectation DP backward
```

8.12 DP speed-ups in one place

KEYWORDS: dp optimisation prefix sums monotonic queue convex hull trick li chao divide and conquer knuth bitset slope

DO THIS

1. Range of previous row summed: prefix sums → O(1) per cell.
2. min over a sliding window of the previous row: monotonic deque.
3. dp[i] = min_j (dp[j] + b_j·x_i) (lines): convex hull trick / Li Chao tree (li_chao.cpp).
4. dp[k][i] = min_j dp[k−1][j] + cost(j, i) with opt(i) monotone: divide and conquer, O(k n log n).
5. Booleans: bitset (64× faster).
6. Too many states: drop a dimension that can be computed from the others (e.g. sum of positions).

```
Template: data_structures/li_chao.cpp, dp/dp_patterns.cpp
```

9. Data structures

KEYWORDS: segment tree lazy fenwick bit sparse table dsu monotonic stack ordered set mo trie

9.1 Which data structure?

KEYWORDS: which data structure range query update choose

FORMULAS

```
static range sum: prefix sums static range min/max/gcd: sparse table O(1)
point update + range sum/min/max/gcd: SegTree (point_update.cpp) or BIT (sum)
range add + range sum: LazySum range add + max: range_add_max / generic lazy
range assign + range query: generic lazy segment tree, or ODT map if there are few
runs
non-commutative merge (brackets, max subarray, hash): custom_node.cpp
k-th smallest in a dynamic set / count less than x: ordered_set, or BIT over
values + lowerBound
k-th smallest in a range (static): persistent segment tree
offline queries on ranges with add/remove: Mo
connectivity with merges: DSU
next greater / smaller, histogram: monotonic stack
sliding window min/max: monotonic deque
```

9.2 Segment tree: how to use your templates

KEYWORDS: segment tree point update range query lazy propagation sum min max

DO THIS

1. Point update, range query of any associative op: SegTree st(n) / st(a); st.set(i, v); st.query(l, r). Change op and ID inside (sum: 0; max: LLONG_MIN; min: LLONG_MAX; gcd: 0).
2. Range add + range sum: LazySum st(n); st.add(l, r, v); st.sum(l, r). All 0-based, inclusive.
3. Anything else with range updates: lazy_segment_tree_generic.cpp, change only node, identity(), makeLeaf(), merge(), applyNode().
4. Find first index i ≥ l with a[i] ≥ x: walk down: go left if the left child's max ≥ x, else right.

```
Template: data_structures/segment_tree/*
```

9.3 Segment tree node ideas (custom merge)

KEYWORDS: segment tree node merge max subarray brackets count of minimum hash palindrome

FORMULAS

```
max subarray sum: {sum, pre, suf, best}; merge: sum = L.sum+R.sum, pre =
max(L.pre, L.sum+R.pre), suf = max(R.suf, R.sum+L.suf), best = max(L.best, R.best,
L.suf+R.pre)
brackets: {match, open, close}; t = min(L.open, R.close); {L.match+R.match+t,
L.open+R.open−t, L.close+R.close−t}
min and how many times: {mn, cnt}; equal mins add counts
hash of a range: {h, len}; h = L.h·p^(R.len) + R.h (dynamic_hash.cpp)
longest run of equal / 1s: {len, pre, suf, best}
```

```
Template: data_structures/segment_tree/custom_node.cpp
```

9.4 Fenwick tree (BIT)

KEYWORDS: fenwick bit binary indexed tree prefix sum inversions count less

DO THIS

1. BIT b(n); b.add(i, v); b.sum(r) = a[0..r]; b.sum(l, r). b.lowerBound(k) = first index with prefix ≥ k.
2. Count inversions: go left to right, inv += (number already inserted) − b.sum(a[i]); b.add(a[i], 1) (compress values).
3. Count distinct values in ranges offline: sort queries by r; when you reach i, add 1 at i and remove 1 at the previous occurrence of a[i]; answer = b.sum(l, r).

```
Template: data_structures/fenwick.cpp
```

9.5 DSU (union-find)

KEYWORDS: dsu union find connected components merge groups size

DO THIS

1. DSU d(n); d.unite(a, b) (false if already together); d.find(x); d.size(x); d.comps.
2. Offline by threshold, reverse deletions, parity, rollback, small-to-large: see Graphs part, card "DSU tricks".

```
Template: data_structures/dsu.cpp, dsu_parity.cpp, dsu_rollback.cpp
```

9.6 Monotonic stack and deque

KEYWORDS: monotonic stack next greater element previous smaller histogram largest rectangle sliding window minimum

USE WHEN

- "Next greater / previous smaller element", "largest rectangle in histogram", "for each element, how far can it extend as the minimum", sliding window min/max.

DO THIS

1. Keep a stack of indices with increasing values. Before pushing i, pop all with value ≥ a[i]: for each popped j, i is its next smaller-or-equal. The top after popping is i's previous smaller.
2. Sum of min over all subarrays: Σ a_i·(i − prevLess)·(nextLessEq − i).
3. Sliding window min: deque of indices with increasing values, pop the front when it leaves the window.

```
Template: data_structures/monotonic_stack.cpp
```

9.7 ordered_set (k-th element, rank)

KEYWORDS: ordered set pbds order of key find by order kth element rank multiset

DO THIS

1. Uncomment the pb_ds includes in the base ABOVE "#define int ll", then paste the two lines from ordered_set.cpp.
2. os.order_of_key(x) = number of elements < x. *os.find_by_order(k) = k-th smallest (0-based).
3. For a multiset store pairs (value, unique id).

```
Template: data_structures/ordered_set.cpp
```

9.8 STL tricks that save time

KEYWORDS: set multiset map lower bound priority queue erase one deque

DO THIS

1. multiset: erase ONE copy with ms.erase(ms.find(x)) (ms.erase(x) removes all).
2. set/map lower_bound(x) = first ≥ x, upper_bound(x) = first > x; prev(it) for the last < x.
3. priority_queue is a max-heap; min-heap: priority_queue<int, vi, greater<int>>.
4. Map with pair key: map<pi, int>. Hash map safe from hacks: umap<K,V> (base).
5. Sort descending: sort(a.rbegin(), a.rend()).

9.9 Mo's algorithm (offline range queries)

KEYWORDS: mo algorithm offline range queries distinct count add remove sqrt

DO THIS

1. Queries (l, r) offline whose answer you can update when adding/removing one element: sort by block of l, then r; move pointers. O((n + q)√n).

```
Template: data_structures/mo.cpp
```

9.10 Range assign with few runs (ODT / map of runs)

KEYWORDS: range assign set range equal value runs chtholly odt

DO THIS

1. Keep the array as runs [start, value] in a map. assign(l, r, v) erases the runs inside and inserts one. Queries walk the runs inside [l, r].
2. Fast when assignments dominate (each assign removes runs).

```
Template: data_structures/interval_assign.cpp
```

9.11 Data structure patterns: offline tricks and query types

KEYWORDS: offline queries sort by right end distinct values in range count of elements less than k in range kth smallest in range persistent merge sort tree last occurrence inversion count sweep

FORMULAS

```
number of distinct values in [l, r]: offline by r; keep 1 at the last occurrence
of each value (seg tree / BIT); or Mo
count of a_i ≤ k in [l, r]: offline sort queries by k and values by a, insert; or
merge sort tree; or persistent seg tree
k-th smallest in [l, r]: persistent segment tree over values
(persistent_segment_tree.cpp)
inversions: BIT over compressed values (count greater seen before)
range add + range sum/max: lazy segment tree (lazy_segment_tree_generic.cpp)
range assign + query: lazy with an "assigned" tag, or ODT if assignments are
random/few runs
range chmin / "set a_i = a_i mod x": segment tree with max; only descend where max
≥ x (each value halves)
range sqrt / divide: each value becomes 1 quickly — keep a set of positions still
> 1
next greater element / nearest smaller: monotonic stack
```

```
first index ≥ x in a range / first position with prefix sum ≥ s: descend the segment tree
2D points, rectangle counts: sweep x + BIT on y (offline)
earliest time something becomes true: parallel binary search
max subarray sum with updates: node (sum, best prefix, best suffix, best)
hashing a range for equality with updates: segment tree of polynomial hashes
order statistics, rank: ordered_set (pb_ds) with pairs for duplicates
```

Template: data_structures/ (segment_tree/*, persistent_segment_tree.cpp, mo.cpp, ordered_set.cpp, sparse_table.cpp)

10. Graphs

KEYWORDS: graph bfs dfs shortest path dijkstra dsu mst topological scc bridges matching flow cycle bipartite

10.1 BFS: shortest path when every move costs 1 (grids)

KEYWORDS: bfs shortest path grid unweighted multi source moves minimum steps knight

USE WHEN

- Fewest moves, every move costs the same. Grids, knight moves, word ladders.

DO THIS

1. Queue, dist = −1 for unvisited. Multi-source (distance to the nearest of many sources): push all sources with distance 0 first.
2. Grid: dr = {1,−1,0,0}, dc = {0,0,1,−1}; check bounds and walls.
3. Store parent to print the path.

```
vi d(n, -1); queue<int> q; d[s] = 0; q.push(s);
while (!q.empty()) { int v = q.front(); q.pop();
    for (int u : g[v]) if (d[u] == -1) d[u] = d[v] + 1, q.push(u); }
```

Template: graph/bfs_variants.cpp gridBfs

10.2 0-1 BFS: moves cost 0 or 1

KEYWORDS: 0-1 bfs deque zero one weights free moves

DO THIS

1. Deque: a 0-cost edge pushes to the FRONT, a 1-cost edge to the BACK. O(V + E).
2. Typical: "count only left moves", "turning costs 1, going straight is free", "breaking a wall costs 1".

Template: graph/bfs_variants.cpp bfs01

10.3 Dijkstra: non-negative weights

KEYWORDS: dijkstra shortest path weighted non negative priority queue count shortest paths

DO THIS

1. Priority queue of (dist, v). Pop; if dist != d[v] skip (stale entry). Relax neighbours.
2. Count shortest paths: shorter found → cnt[u] = cnt[v]; equal → cnt[u] += cnt[v] (mod).
3. Edge (u, v, w) lies on some shortest s→t path ⇔ ds[u] + w + dt[v] == ds[t] (run from s and from t on the reversed graph).
4. Only k special nodes matter: run Dijkstra from each of them.

```
vi d = dijkstra(g, s);   // g[u] = {v, w}; LLONG_MAX = unreachable
```

WATCH OUT

- Forgetting the stale-entry skip gives TLE. Negative edges break Dijkstra.

```
Template: graph/dijkstra.cpp
```

10.4 State graphs: add what you need to remember to the node

KEYWORDS: state graph coupon discount keys direction sliding queen carrying item parity time mod

USE WHEN

- The best move depends on more than the current vertex: a coupon used or not, keys collected, current direction, carrying something, parity, time mod k.

DO THIS

1. Make node = (vertex, extra). Build edges between these states. Run BFS / 0-1 BFS / Dijkstra on them.
2. Check that (#vertices × #extra values) fits in memory/time.
3. Encode the state as one integer: id = v·K + extra.

FORMULAS

```
one coupon (half price once): (v, used∈{0,1}); edges (v,0)→(u,0) cost c,
(v,0)→(u,1) cost c/2, (v,1)→(u,1) cost c
sliding piece, cost per slide: (cell, direction); same direction 0, new slide C_d,
stop 0 (2021 F)
keys ≤ 10: (cell, mask); carrying an object: (cell, carrying)
path cost = sum − max + min: (v, removed max?, added min again?) → 4 states
forbidden turns a→b→c: state = last edge (prev, cur)
```

```
vector<vpi> G(2 * n);                        // node id = v * 2 + used
for (auto &e : E) { int a = e[0], b = e[1], c = e[2];
    G[a*2].pb({b*2, c}); G[a*2].pb({b*2+1, c / 2}); G[a*2+1].pb({b*2+1, c}); }
vi d = dijkstra(G, s * 2); int ans = min(d[t*2], d[t*2+1]);
```

10.5 Negative edges, all pairs

KEYWORDS: bellman ford negative cycle floyd warshall all pairs shortest paths

DO THIS

1. Negative weights or negative cycle detection: Bellman-Ford O(nm) (bellman_ford.cpp gives negInf vertices).
2. All pairs with n ≤ 500: Floyd, k is the OUTER loop: d[i][j] = min(d[i][j], d[i][k] + d[k][j]).
3. Vertices added one by one and you need all-pairs after each: Floyd with the added vertex as k (reverse deletions).
4. Shortest paths using only some vertices as middle points (e.g. sorted by a value): add them in that order as k (2019 Interplanetary).

```
Template: graph/bellman_ford.cpp, graph/floyd_warshall_directed.cpp
```

10.6 DSU tricks

KEYWORDS: dsu offline queries threshold sort edges reverse deletions parity bipartite small to large kruskal tree

USE WHEN

- Connectivity that only grows; "using only edges with weight ≥ C"; deletions; same/different constraints.

DO THIS

1. Offline threshold: sort edges by weight, sort queries by threshold; add edges up to the threshold, answer (2017 D: size of node 1's component).
2. Merging groups of sizes a and b creates a·b new connected pairs.
3. Only deletions: reverse time, add instead of delete, store answers, print reversed.
4. "x and y are the same / different": DSU with parity; contradiction ⇔ parity mismatch. Bipartite check while adding edges = unite(a, b, 1).
5. Groups carry sets: merge the smaller set into the larger (each element moves O(log n) times).
6. Edges added and removed in any order: rollback DSU + segment tree over time.

```
Template: data_structures/dsu.cpp, dsu_parity.cpp, dsu_rollback.cpp,
dsu_offline_edge_deletion.cpp
```

10.7 Minimum spanning tree

KEYWORDS: mst minimum spanning tree kruskal cheapest connect all bottleneck minimise maximum edge

DO THIS

1. Kruskal: sort edges, take an edge when it joins two components. Total weight is minimal AND the largest edge on every tree path is as small as possible (bottleneck).
2. MST that must contain edge e: W + w(e) − (max edge on the tree path between its ends) (binary lifting max).
3. Complete graph with cost a_x + a_y: connect every vertex to the one with the smallest a.
4. Cost |x_i − x_j| on points of a line: only neighbours after sorting matter.

```
Template: graph/kruskal.cpp
```

10.8 DAG, topological order, SCC, 2-SAT

KEYWORDS: topological sort prerequisites dag dp longest path cycle scc strongly connected 2sat

DO THIS

1. Prerequisites / order: Kahn's topological sort. If fewer than n vertices come out, there is a cycle. Smallest lexicographic order: min-heap.
2. DP on a DAG (longest path, number of paths): process in topological order.
3. Directed graph with cycles: SCC (Tarjan), condense into a DAG, then DP.
4. Each variable true/false with clauses (a OR b): 2-SAT (two_sat.cpp).
5. Each node has exactly one outgoing edge (functional graph): every component is one cycle plus trees. k-th next node: binary lifting.

```
Template: graph/topo_sort.cpp, graph/scc_tarjan.cpp, graph/two_sat.cpp
```

10.9 Cycles, bipartite check, Euler path, bridges

KEYWORDS: find cycle bipartite two coloring odd cycle euler path circuit bridge articulation point

DO THIS

1. Bipartite ⇔ no odd cycle: BFS colouring (bipartite_coloring.cpp).

2. Find any cycle (directed or not): finding_simple_cycle.cpp.
3. Euler circuit (undirected): connected and all degrees even. Path: exactly 0 or 2 odd vertices. Directed: in = out everywhere, or one vertex out−in = 1 and one in−out = 1.
4. Bridge: edge whose removal disconnects. Articulation point: vertex whose removal disconnects. Use bridges.cpp (handles multi-edges).

```
Template: graph/euler_path_hierholzer.cpp, graph/bridges.cpp
```

10.10 Bipartite matching, König, flow

KEYWORDS: bipartite matching kuhn hopcroft karp vertex cover independent set konig max flow min cut assignment

DO THIS

1. Max matching: Kuhn (small) or Hopcroft-Karp (V, E up to 1e5).
2. König: minimum vertex cover = maximum matching. Max independent set = V − matching. Print the cover with konig_vertex_cover.cpp.
3. "Remove the fewest items so that no conflicting pair remains" with two sides (each conflict is between side A and side B) = minimum vertex cover (2024 B).
4. Min-cost assignment: Hungarian. Max flow / min cut: Dinic.
5. Min cut models: choose for each item one of two options with costs, plus penalties for pairs that choose differently → edges S→i, i→T, i↔j.

```
Template: graph/hopcroft_karp.cpp, graph/konig_vertex_cover.cpp,
graph/hungarian.cpp, graph/dinic.cpp
```

10.11 Closure: apply rules until nothing changes

KEYWORDS: closure rules until nothing changes fixpoint chemical reaction worklist produce new elements

USE WHEN

- "After infinitely long", "repeat while possible", rules that produce new items from existing ones (2025 H).

DO THIS

1. Queue of present items. For each rule keep how many of its inputs are still missing. When an item becomes present, decrease the counters of the rules using it; when a counter reaches 0, the output becomes present (push it if new).
2. Compress big values with a map first.

```
queue<int> q; for (int x : init) if (!have[x]) have[x] = 1, q.push(x);
while (!q.empty()) { int x = q.front(); q.pop();
    for (int r : usedBy[x]) if (--need[r] == 0 && !have[out[r]]) have[out[r]] = 1,
q.push(out[r]); }
```

More graph patterns

10.12 DSU patterns: offline sorting, reverse time, parity, rollback, reconstruction tree

KEYWORDS: dsu offline queries sorted by weight pairs count edges at most q reverse time deletions destroying array parity liars bipartite rollback dynamic connectivity kruskal reconstruction tree

DO THIS

1. Pairs connected using edges of weight ≤ q, many q: sort edges and queries; add edges in order; a merge of sizes a and b adds a·b pairs; answer = running total.
2. Things only get deleted: reverse time, add them back; merge with existing neighbours; record answers, print reversed (Destroying Array: max block sum).
3. Components after a gcd/sharing rule: connect each number to its prime "rooms" (first index containing prime p), not to each other.
4. "Same / different" constraints (a says b is liar, bipartite with added edges): DSU with a parity bit; contradiction when pa ^ pb != d. Max liars: per component take the larger of parity-0 / parity-1 counts.
5. Edges added AND removed (offline): each edge lives on a time interval; segment tree over time, DSU with rollback (no path compression), DFS the segment tree.
6. Merge sets carried by components: small into large (each element moves O(log n) times).
7. Kruskal reconstruction tree: each merge creates a node with the edge weight, children = the two leaders. min over paths of (max edge) between u, v = weight at LCA(u, v). "All vertices reachable from v using edges ≤ x" = subtree of the highest ancestor with weight ≤ x (Euler tour range).
8. Two forests, add as many edges as possible to both (Mocha and Diana): try to connect everything to vertex 1 in both, then pair leftover components.

EXAMPLE

> Tree edges 1–2 (3), 2–3 (1), 3–4 (2): pairs with all edges ≤ 2: merge 2–3 (+1), 3–4 (+2) → 3.

```
Template: data_structures/dsu.cpp, dsu_parity.cpp, dsu_rollback.cpp,
dsu_offline_edge_deletion.cpp
```

10.13 Shortest path extras: counting paths, special points, range edges, Floyd tricks

KEYWORDS: number of shortest paths count min edges dijkstra extra info special points candies bitmask tsp floyd reverse deletion negative cycle print segment tree graph range edges shortest path tree tree plus few edges

DO THIS

1. Count shortest paths: on strict improvement cnt[u] = cnt[v]; on equal distance cnt[u] += cnt[v]. Same for min/max number of edges among shortest paths (CSES Investigation).
2. Few special points (≤ 20) in a big grid: BFS from each, then bitmask DP dp[mask][i] over the special points.
3. All pairs with n ≤ 500: Floyd, k outermost. Vertices deleted one by one, sum of distances before each deletion: add vertices back in reverse order as the next k; sum over added vertices (CF 295B).
4. Negative cycle: Bellman–Ford, relaxation in round n ⇒ cycle; walk parents n times from that vertex to land inside it, then collect.
5. Edges from a vertex to a whole range [l, r] (or range to vertex): two segment trees of extra nodes ("down" tree with 0 edges to children, "up" tree with 0 edges to parents), Dijkstra on ~4n nodes (CF 786B Legacy).

6. Keep k edges so that most vertices keep their distance: edges of the shortest path tree in Dijkstra finalisation order (CF 1076D).
7. Tree + few extra edges ($m - n \le 20$): answer = min(tree distance via LCA, min over endpoints x of extra edges of d_x[u] + d_x[v]) (CF 1051F).
8. Path cost = sum − max + min: state (vertex, dropped one edge?, doubled one edge?) — 4 states (CF 1473E).
9. Forbidden triples a→b→c: state = last edge used.

```
Template: graph/dijkstra.cpp, bellman_ford.cpp, floyd_warshall_directed.cpp,
bfs_variants.cpp
```

10.14 MST patterns: cycle rule, forced edge, implicit edges, first time connected

KEYWORDS: mst each edge must contain edge max edge on path bottleneck implicit edges formula cost a_x + a_y special offers gcd mst first day connected parallel binary search

DO THIS

1. Cut rule: the cheapest edge across any cut is in some MST. Cycle rule: the heaviest edge on a cycle is never needed.
2. Cheapest spanning tree that must contain edge e = (u, v, w): W + w − (max edge on the MST path u–v) (binary lifting or reconstruction tree) (CF 609E).
3. Edge costs given by a formula (n^2 edges): prove only O(n) matter. Cost a_x + a_y: only edges to the vertex with the smallest a, plus the special offers (CF 1095F).
4. gcd(a[i..j]) = min(a[i..j]) costs: process values from small to large, spread left/right over multiples, stop at connected neighbours; use p-edges when $v \ge p$ (CF 1513D).
5. First time (day) u and v are connected: MST in day order; answer = max edge on the MST path (or parallel binary search over days).
6. Minimise the maximum edge (bottleneck): MST path maximum; or binary search + DSU.
7. Prim ($O(n^2)$) for dense graphs given as a matrix.

```
Template: graph/kruskal.cpp, trees/binary_lifting_lca.cpp
```

10.15 DAG, SCC and functional graph patterns

KEYWORDS: topological order count routes longest path dag scc condensation coin collector checkposts reachability queries bitset functional graph successor k steps planets queries cycle tail

DO THIS

1. Kahn: queue of in-degree 0; if fewer than n come out, there is a cycle (answer may be "infinite"/"IMPOSSIBLE").
2. DP in topological order: number of routes, longest path, most frequent letter on a path (run once per letter).
3. Cycles you can walk around freely: SCC (Kosaraju/Tarjan), condense into a DAG, DP there (Coin Collector: sum of coins per SCC).
4. One guard per SCC: cost = Σ min cost in each SCC; ways = Π (number of minimum-cost vertices in each) (CF 427C).
5. Reachability for many queries: process the condensation in reverse topological order, bitset of reachable targets (in chunks if memory is tight).

6. Make the graph strongly connected with fewest added edges: max(#sources, #sinks) of the condensation (0 if already one SCC).
7. Functional graph (every node has one next): a cycle with trees hanging in. k-th successor: binary lifting. Distance a → b: if b is on a tail, check jumping dist[a] − dist[b] steps lands on b; if on a cycle, go to the cycle and add (pos[b] − pos[e] + len) mod len.
8. Permutation applied k times: cycles; each element moves k mod (cycle length).

```
Template: graph/topo_sort.cpp, graph/scc_tarjan.cpp
```

10.16 Bridges, articulation points, bridge tree, 2-SAT

KEYWORDS: bridge critical road articulation point cut vertex low link tin low bridge tree 2-edge-connected diameter artifacts 2-sat implications pizza toppings

DO THIS

1. DFS times tin, low[v] = min(tin[v], tin of back-edge targets, low of children).
2. Edge p–c is a bridge ⇔ low[c] > tin[p]. Vertex p is an articulation point ⇔ some child c has low[c] ≥ tin[p] (root: ⇔ 2+ DFS children).
3. Skip the parent edge by EDGE ID (parallel edges are real).
4. Bridge tree: remove bridges, label 2-edge-connected pieces (BFS not crossing bridges), connect pieces by bridges → a tree.
5. Most unavoidable edges between some s, t: diameter of the bridge tree (CF 1000E). Edges collapse after use, pick an artifact on the way: mark pieces with an artifact edge, check the bridge-tree path (CF 652E).
6. 2-SAT: each clause (a ∨ b) gives ¬a → b and ¬b → a. Unsatisfiable ⇔ x and ¬x in one SCC. With Kosaraju numbering (topological), set x = true ⇔ comp[x] > comp[¬x]. "x must be true": (x ∨ x). "Exactly one": (x ∨ y) and (¬x ∨ ¬y).

```
Template: graph/bridges.cpp, articulation_points.cpp, biconnected_components.cpp,
two_sat.cpp
```

10.17 Matching and flow modelling: König, independent set, path cover, min cut

KEYWORDS: bipartite matching kuhn hopcroft karp konig vertex cover rows columns minimum independent set path cover dag max flow dinic min cut capacities edge disjoint paths project selection

DO THIS

1. Max matching: Kuhn O(V·E) for ~ few thousand vertices; Hopcroft–Karp for 1e5.
2. König: min vertex cover = max matching (bipartite). Targets cleared by row/column shots: rows left, columns right, target = edge. Recover the cover: from unmatched left vertices alternate (any edge → right, matching edge → left); cover = unvisited left + visited right (konig_vertex_cover.cpp).
3. Max independent set = V − matching. Min edge cover = V − matching (no isolated vertices).
4. Min path cover of a DAG (vertex-disjoint) = n − matching on split graph (v_out → u_in). If paths may share vertices: take the transitive closure first.
5. Capacities ("worker takes up to 5 jobs"): source → worker cap 5, worker → job cap 1, job → sink cap 1; Dinic.
6. Edge-disjoint paths s → t = max flow with unit capacities. Vertex-disjoint: split each vertex into in → out with capacity 1.

7. Min cut = max flow. Two-groups-with-penalties problems: s side = choice A, t side = choice B; edge s → v (cost of choosing B), v → t (cost of choosing A), u ↔ v (penalty if different) (Landscaping SWERC 2015). After flow, source side = vertices with lvl ≥ 0.
8. Project selection: profit p > 0 → edge s → v cap p; cost c → v → t cap c; "v needs u" → v → u cap ∞; answer Σ positive − maxflow.

```
Template: graph/kuhn_matching.cpp, hopcroft_karp.cpp, konig_vertex_cover.cpp,
dinic.cpp, min_cost_flow.cpp, hungarian.cpp
```

11. Trees

KEYWORDS: tree lca subtree rerooting diameter path centroid euler tour dfs order

11.1 Tree traversal without recursion, sizes, depths

KEYWORDS: tree bfs order parent depth subtree size iterative dfs stack overflow

DO THIS

1. Recursion on 1e6 nodes can crash: get a BFS order from the root; going backwards through that order visits children before parents.
2. Bipartite colour = depth parity. (−1)^dist(u, v) = +1 ⇔ u, v have the same colour (2021 G).

```
vi par(n, -1), order = {root}, dep(n, 0); par[root] = root;
rep(i, 0, n) { int v = order[i]; for (int u : g[v]) if (u != par[v]) par[u] = v, dep[u]
= dep[v] + 1, order.pb(u); }
par[root] = -1;
vi sz(n, 1); repb(i, n - 1, 1) sz[par[order[i]]] += sz[order[i]];
```

11.2 Diameter, centre, farthest node

KEYWORDS: tree diameter longest path farthest node center eccentricity

DO THIS

1. BFS from any node to the farthest node a; BFS from a to the farthest b. dist(a, b) is the diameter.
2. Farthest node from every v: max(dist(v, a), dist(v, b)).
3. Centre: middle vertex (or two) of the a–b path.

```
Template: trees/tree_diameter.cpp
```

11.3 Rerooting: answer for every root

KEYWORDS: rerooting all roots every vertex sum of distances to all nodes

USE WHEN

- "For every vertex v, compute something as if v were the root" (sum of distances, best value, ...).

DO THIS

1. Step 1: root at 0, compute down[v] (answer inside v's subtree) bottom-up.
2. Step 2: top-down, compute ans[child] from ans[parent] by moving the root across one edge.
3. Sum of distances: ans[c] = ans[p] − sz[c] + (n − sz[c]) (sz[c] nodes get closer, the rest farther).

4. For max-type values you need "best of the other children": keep the top two child values (or prefix/suffix maxima).

```
Template: trees/rerooting.cpp
```

11.4 LCA, distances, paths

KEYWORDS: lca lowest common ancestor distance kth ancestor path maximum binary lifting path intersection

DO THIS

1. Binary lifting: up[k][v]. lca(u, v). dist(u, v) = dep[u] + dep[v] − 2·dep[lca].
2. k-th ancestor: jump by bits of k. Path max/min: store it in a second lifting table.
3. x is on the path u–v ⇔ dist(u, x) + dist(x, v) == dist(u, v).
4. Common part of paths a–b and c–d: take the two deepest of lca(a,c), lca(a,d), lca(b,c), lca(b,d); if both lie on both paths, answer dist + 1 vertices, else empty.
5. Add 1 to every vertex on path u–v (many paths, read at the end): +1 at u and v, −1 at lca, −1 at parent(lca), then subtree sums.

```
Template: trees/binary_lifting_lca.cpp
```

11.5 Euler tour: subtree = range

KEYWORDS: euler tour tin tout subtree query update flatten tree ancestor check

DO THIS

1. DFS order: tin[v] when entering, tout[v] = last tin in its subtree. Subtree of v = positions [tin[v], tout[v]] → range queries with a segment tree / BIT.
2. u is an ancestor of v ⇔ tin[u] ≤ tin[v] ≤ tout[u].
3. Path queries with updates: HLD (hld.cpp).

```
Template: trees/hld.cpp (pos[v], sz[v] give the subtree range too)
```

11.6 Small to large merging

KEYWORDS: small to large merge sets subtree distinct colors dsu on tree

DO THIS

1. For each node you need a set/map of its subtree (distinct colours, most frequent value). Keep the set of the largest child, insert the other children's elements into it. Each element moves O(log n) times.

11.7 Centroid decomposition

KEYWORDS: centroid decomposition nearest marked node count paths length k distance queries

DO THIS

1. Every path goes through the highest centroid on it; each vertex has O(log n) centroid ancestors.
2. Nearest marked vertex: best[c] = min distance from c to a marked vertex in its part; query v = min over centroid ancestors c of best[c] + dist(v, c). With deletions / ties use a multiset per centroid (2022 H).

```
Template: trees/centroid_decomposition.cpp
```

11.8 Counting over all pairs / paths in a tree

KEYWORDS: tree all pairs sum of distances paths through edge contribution

DO THIS

1. Edge with s vertices below it is used by s·(n − s) paths. Sum of all distances = Σ s·(n − s).
2. Pairs (u, w) whose path contains v: C(n, 2) − Σ over the parts left after deleting v of C(part, 2) (parts = child subtrees and n − sz[v]).
3. Root-to-node strings / values: carry the information down from the parent (hash, xor, count).

More tree patterns

11.9 Tree DP patterns: matching, knapsack on tree, distance k pairs, best path

KEYWORDS: tree dp matching maximum independent set knapsack on tree choose k nodes subtree size merge distance k pairs best path two chains fair nut

DO THIS

1. Process children before the parent: BFS order, then loop backwards (no recursion on 2e5 chains).
2. Max matching: dp0[v] = Σ max(dp0[c], dp1[c]); dp1[v] = dp0[v] + max over c of (1 + dp0[c] − max(dp0[c], dp1[c])). (Greedy also works: match a leaf with its parent, deepest first.)
3. Max independent set: in[v] = val + Σ out[c]; out[v] = Σ max(in[c], out[c]).
4. Knapsack on a tree (choose k nodes / delete subtrees): dp[v][j]; merge children with loops bounded by min(size, k): total O(n·k).
5. Pairs at distance exactly k (k ≤ 500): cnt[v][d]; when merging a child count pairs a + b + 1 = k first, then merge. (Or centroid decomposition.)
6. Best path value (vertex values − edge lengths): best down-chain from each v; path topped at v uses the two largest child chains.
7. Number of ways to colour / count independent sets: product over children of (sum over allowed child states).

```
Template: trees/rerooting.cpp, dp/dp_patterns.cpp
```

11.10 Rerooting formulas (answer for every root)

KEYWORDS: rerooting sum of distances from every node tree distances 2 max distance every node maximum white subtree prefix suffix products

DO THIS

1. Sum of distances from every node: ans[root] = Σ depth; moving the root to child c: ans[c] = ans[v] − sz[c] + (n − sz[c]).
2. Weighted version Σ dist·a_i: same with sz replaced by "sum of a in the subtree".
3. Farthest node from every v: max(dist(a, v), dist(b, v)) where a, b are diameter ends (3 BFS).
4. Non-sum combine (best connected piece containing v, white +1 black −1): down[v] = col(v) + Σ max(0, down[c]); full[c] = down[c] + max(0, full[v] − max(0, down[c])).

5. Combine is a product/max without inverse: prefix and suffix over each node's children list ("all children except c").
6. Count of nodes at distance ≤ d from every node etc.: same up/down pattern.

EXAMPLE

> Path 1–2–3–4: ans[1] = 6, sz[2] = 3 → ans[2] = 6 − 3 + 1 = 4 ✓.

```
Template: trees/rerooting.cpp, trees/sum_of_all_pair_distances.cpp
```

11.11 Tree greedy and contribution

KEYWORDS: edge contribution s(n-s) maximum distributed tree assign values to edges linova kingdom choose k nodes depth minus size deepest first cover guards distance 2 orient edges

DO THIS

1. Edge used by s·(n − s) paths (s = subtree size below it). Σ all pairwise distances = Σ s(n − s).
2. Assign given numbers to edges to maximise Σ over pairs: sort edges by s(n − s) (exact integers!), biggest numbers to the most used edges; extra primes multiply onto the top edge (CF 1401D).
3. Choose k nodes to maximise a sum where choices interact: compute a value that already includes the interaction (Linova: depth[v] − (sz[v] − 1)), sort, take top k (CF 1336A).
4. Cover all nodes within distance d of the root with fewest new edges: repeatedly take the deepest uncovered node, connect its PARENT to the root and mark the parent's neighbours (CF 1029E). Guards with radius R: binary search R, deepest first, put the guard R levels up.
5. Distances after adding edges between nodes at distance 2: Σ ⌈d/2⌉ = (Σ d + #pairs with odd d)/2; odd d ⇔ different depth parity.
6. Orient edges to minimise the longest directed path: colour by depth parity, all edges 0 → 1, answer 1.
7. Always draw small trees by hand first.

```
Template: trees/sum_of_all_pair_distances.cpp
```

11.12 LCA path tricks: path max, counting paths through nodes, on-path tests

KEYWORDS: lca path maximum min edge binary lifting kth ancestor difference on tree paths through node count is x on path k nodes on one path shared stations equidistant nodes

DO THIS

1. Binary lifting can carry max/min/sum/gcd: mx[v][j] = max(mx[v][j−1], mx[up[v][j−1]][j−1]); take max over all jumps during the LCA search.
2. How many paths cover each node: +1 at a, +1 at b, −1 at lca, −1 at parent(lca); answer = subtree sums. For edges: +1, +1, −2 at lca, value at v = edge (v, parent).
3. x on path a–b ⇔ dist(a, x) + dist(x, b) = dist(a, b).
4. k nodes on one simple path: u = deepest, v = farthest from u; check every node is on u–v.
5. Shared stations of paths a→c and b→c: (dist(a, c) + dist(b, c) − dist(a, b))/2 + 1 (try all 3 choices of c and take max for CF 832D).

6. Nodes equally far from x and y: 0 if dist odd; find the middle m with kth ancestor; if depths equal: n − sz(child of m toward x) − sz(child toward y); else sz[m] − sz(child of m toward the deeper one).
7. A root path through or next to all given nodes: replace each by its parent, sort by depth, check each is an ancestor of the next (tin/tout).
8. 10^6 queries: Euler tour + sparse table LCA in O(1).

```
Template: trees/binary_lifting_lca.cpp
```

11.13 Euler tour, HLD, small-to-large patterns

KEYWORDS: euler tour subtree query root path sum range add point query colours bitmask subtree paint distinct colours depth queries per depth list dsu on tree sack lomsat heavy light path update

DO THIS

1. Subtree of v = positions tin[v]..tout[v]. Subtree sum with point updates: BIT/segment tree on positions.
2. Root-to-v path sums: update u ⇒ range add on [tin[u], tout[u]], query v ⇒ point value at tin[v].
3. ≤ 60 colours, paint subtree / count distinct in subtree: lazy segment tree of 64-bit masks (OR), popcount (CF 620E).
4. Nodes in v's subtree at depth h: sorted tin lists per depth, two binary searches (CF 208E-type, 570D: palindrome possible ⇔ ≤ 1 odd letter count, xor of letter masks).
5. For every node a statistic of its subtree (most frequent colour): small to large / keep the big child's map (CF 600E). Depth-indexed: long-path decomposition O(n).
6. Path update/query between any a and b: HLD, O($\log^2 n$) with a segment tree; values on edges → store at the lower endpoint, skip the LCA.
7. Root-path string palindromes during DFS: push/pop forward and backward hashes (palindrome_tracker.cpp).

```
Template: trees/hld.cpp, data_structures/segment_tree/*,
strings/hashing/palindrome_tracker.cpp
```

11.14 Centroid decomposition and xor on trees

KEYWORDS: centroid decomposition count paths length k at most k range nearest marked node xenia xor distance path xor maximum xor path count pairs xor k

DO THIS

1. Count paths of length exactly k: for centroid c, cnt[0] = 1; for each child branch: first count pairs cnt[k − d] for every depth d in the branch, then add the branch's depths; undo afterwards (do not clear the whole array). O(n log n).
2. Length in [k1, k2]: Fenwick over depth, range sum per node: O(n $\log^2 n$).
3. Nearest marked node with updates: each node stores its ≤ log n centroid ancestors and distances; mark v: best[c] = min(best[c], dist(v, c)); query v: min over ancestors of best[c] + dist(v, c) (Dhaka 2022 H).
4. Path xor = x[u] ^ x[v] (x = xor from the root). Max path xor: binary trie over x. Pairs with xor exactly k: hash map of x values.
5. Path weights where the path value is a sum: x[u] + x[v] − 2·x[lca].

```
Template: trees/centroid_decomposition.cpp, data_structures/binary_trie.cpp
```

11.15 Tree isomorphism and tree hashing

KEYWORDS: tree isomorphism same shape rooted unrooted canonical name hash of subtree count distinct subtrees

DO THIS

1. Rooted: name leaves 0; a node's name = id of the sorted list of its children's names in a shared map<vector<int>, int>. Same root name ⇔ isomorphic. O(n log² n).
2. Unrooted: root at the centroid(s) (one or two); try both centroids of the second tree.
3. Hash version: hash(v) = 1 + Σ f(hash(c)) with a random mixing f (e.g. splitmix64), unsigned overflow. Number of distinct subtree shapes = distinct hashes.

```
Template: (write inline)
```

12. Strings

KEYWORDS: string substring hashing kmp z function palindrome trie suffix automaton subsequence bracket anagram

12.1 First moves on any string problem

KEYWORDS: anagram rearrange letter count palindrome possible lexicographic smallest reading lines

DO THIS

1. Rearranging allowed ("anagram", "can we make"): only letter counts matter. cnt[c − 'a']++.
2. Palindrome by rearranging: at most one letter with an odd count.
3. Longest / shortest substring with a condition: sliding window (two pointers).
4. Lexicographically smallest: build left to right, smallest letter that keeps the rest possible.
5. s.size() is unsigned: write (int)s.size() − 1. t += c, not t = t + c. Pass const string &.
6. Read lines with spaces: getline (after cin >> n, call getline once to eat the newline).

12.2 Hashing: compare substrings in O(1)

KEYWORDS: hash double hashing substring equal compare distinct substrings collision

USE WHEN

- Many questions "is s[l1..r1] equal to s[l2..r2]?", count distinct substrings (n ≤ 5000), longest repeated substring, compare across two strings.

DO THIS

1. Use DOUBLE hashing (two moduli). One modulus 1e9 fails with ~1e5+ hashes (birthday paradox). preCompute(maxLen) once, then Hash h(s); h.getHash(l, r) (0-based, inclusive) returns a pair.
2. Compare only pieces of the same length (or put the length in the key).
3. Distinct substrings of a small string: put (hash, length) of every substring into a vector, sort, unique.
4. Longest substring that appears twice: binary search the length L, set of hashes of all length-L pieces.
5. Strings from two different inputs: build both Hash objects after the same preCompute.

FORMULAS

```
H[i+1] = H[i]·p + s[i] (mod M)
hash(l..r) = H[r+1] − H[l]·p^(r−l+1) (mod M)
```

```
preCompute(n + 1);
Hash h(s);
if (h.getHash(l1, r1) == h.getHash(l2, r2)) { /* equal */ }
```

WATCH OUT

- Never use overflow mod 2^64 (Thue-Morse strings break it). Characters above 127: cast to unsigned char (template does).

`Template: strings/hashing/double_hash.cpp`

12.3 Palindrome checks with hashing (also with updates and on trees)

KEYWORDS: palindrome check substring hash forward backward updates tree path root

DO THIS

1. s[l..r] is a palindrome ⇔ its forward hash equals its backward hash. PalHash ph(s); ph.isPal(l, r).
2. With point updates: DynPal p(s); p.set(i, c); p.isPal(l, r) (segment trees of hashes).
3. Can s[l..r] become a palindrome by deleting one character? Find the first mismatch from both ends (binary search on the matched length with hashes), then test deleting the left or the right mismatching char (2020 C).
4. Root-to-node string on a tree: carry F(v) = F(par)·p + c and B(v) = B(par) + c·p^depth; palindrome ⇔ F == B (2021 D).

`Template: strings/hashing/palindrome_hash.cpp, strings/hashing/dynamic_hash.cpp`

12.4 LCP of two suffixes, comparing suffixes, k mismatches

KEYWORDS: lcp longest common prefix compare suffixes binary search hash mismatches hamming distance k

DO THIS

1. lcp(ha, i, hb, j): binary search the largest k with equal hashes of length k. O(log n).
2. Compare a[i..] and b[j..] lexicographically: k = lcp, then compare a[i+k] and b[j+k] (shorter one first if it ended).
3. Count windows that differ from P in at most K positions (K ≤ 4): at each alignment jump over the equal part with lcp, skip one mismatch, repeat at most K + 1 times (2019 A).

`Template: strings/hashing/lcp_double_hash.cpp`

12.5 KMP / prefix function

KEYWORDS: kmp prefix function pattern matching occurrences period border shortest period

USE WHEN

- Find all occurrences of a pattern; shortest period; borders (prefix = suffix); counting strings that avoid a pattern.

DO THIS

1. pi[i] = length of the longest proper prefix of s[0..i] that is also a suffix of it.
2. Occurrences of P in T: prefixFunction(P + '#' + T); positions with value |P|. Or find(text, pat) in kmp.cpp.

3. Shortest period p = n − pi[n−1]. s is a repetition of a block ⇔ n % p == 0.
4. All borders: pi[n−1], pi[pi[n−1] − 1], … .
5. Occurrences of every prefix: cnt[pi[i]]++ for all i; then for j = n down to 1: cnt[pi[j−1]] += cnt[j]; then add 1 for each prefix itself.
6. Strings of length n that do not contain P: DP over KMP automaton states (matrix power when n is huge).

```
Template: strings/kmp.cpp
```

12.6 Z-function

KEYWORDS: z function z array prefix match occurrences compare with prefix

DO THIS

1. z[i] = length of the longest common prefix of s and s[i..].
2. Occurrences of P in T: z of P + '#' + T, positions with z == |P|.
3. For each position of T, longest prefix of P starting there: z of P + '#' + T.
4. Period: smallest p with p + z[p] == n.

```
Template: strings/z_function.cpp
```

12.7 All palindromic substrings (Manacher, eertree)

KEYWORDS: manacher palindromic substrings count longest palindrome eertree distinct palindromes

DO THIS

1. Manacher in O(n): d1[i] = number of odd palindromes centered at i, d2[i] = even ones centered between i−1 and i.
2. Number of palindromic substrings = Σ d1 + Σ d2. Longest = max(2·d1 − 1, 2·d2).
3. Distinct palindromes / occurrences of each: palindromic tree (eertree): number of distinct = nodes − 2.
4. Min cuts into palindromes: dp over positions with isPal (n^2 with Manacher or a dp table).

```
Template: strings/manacher.cpp, strings/palindromic_tree.cpp
```

12.8 Subsequences

KEYWORDS: subsequence is subsequence count occurrences distinct subsequences greedy next

DO THIS

1. Is t a subsequence of s: two pointers, greedy. Many t's: nxt[i][c] = first position ≥ i with letter c (subsequence_automaton.cpp).
2. Number of times t occurs as a subsequence of s: dp[j] += dp[j−1] when s[i] == t[j−1] (j from |t| down to 1), dp[0] = 1.
3. Number of distinct subsequences: dp[i] = 2·dp[i−1] − dp[last[c] − 1].
4. Maximum number of disjoint copies of a short pattern (like ^_^): binary search k, use the k leftmost and k rightmost suitable characters, check the middle part greedily (2015 E).

```
Template: strings/subsequence_automaton.cpp
```

12.9 Trie and Aho-Corasick

KEYWORDS: trie prefix dictionary words aho corasick many patterns multiple patterns text dictionary dp

DO THIS

1. Trie: prefix counts, LCP of a word with a dictionary, k-th word, sorted traversal. trie.cpp.
2. Best "LCP + bonus if anagram" (2018 I): one trie for LCP, plus words grouped by their sorted letters.
3. Many patterns in one text: Aho-Corasick; after build, walk the text, out[v] = number of patterns ending here.
4. Fewest pieces from a dictionary to type S: dp[i] = min over patterns ending at i of dp[i − len] + 1 (walk dict links in Aho-Corasick, 2020 G).

```
Template: data_structures/trie.cpp, strings/aho_corasick.cpp
```

12.10 Suffix automaton / suffix array: distinct substrings and friends

KEYWORDS: suffix automaton suffix array distinct substrings occurrences longest common substring kth substring

DO THIS

1. Distinct substrings: SAM Σ_v (len[v] − len[link[v]]), or SA n(n+1)/2 − Σ lcp.
2. Distinct substrings of EACH length L: every SAM state v covers lengths (len[link[v]], len[v]]: +1 on that range with a difference array (2018 F: strings of length L not in S = 26^L − distinct(L)).
3. Occurrences of a substring: SAM countOcc().
4. Longest common substring of s and t: walk t through the SAM of s keeping the current length.
5. Substrings shared by several strings: generalized SAM.

```
Template: strings/suffix_automaton.cpp, strings/suffix_array.cpp,
strings/generalized_sam.cpp
```

12.11 Brackets and expressions

KEYWORDS: bracket sequence balanced parentheses regular stack minimum changes expression evaluate parse

DO THIS

1. Balanced ⇔ balance never negative and ends at 0. Mixed ()[]{}: stack.
2. Longest balanced substring: stack of indices (push −1 first).
3. Minimum flips to balance (even length): cancel matched pairs; left with c ')' then o '(' → ⌈c/2⌉ + ⌈o/2⌉.
4. Range queries with updates: segment tree node {matched, open, close} (custom_node.cpp).
5. Evaluate an expression with + − * / and parentheses: expression_parser.cpp.

```
Template: strings/brackets.cpp, strings/expression_parser.cpp
```

12.12 Rotations, big numbers, wildcards, multisets

KEYWORDS: rotation cyclic minimal rotation big integer wildcard multiset permutation equal xor hash

DO THIS

1. Smallest rotation: min_rotation.cpp. All rotations: work on s + s.
2. Big integer add / multiply / compare as strings: bigint.cpp.
3. Pattern with '?' wildcards: FFT (wildcard_match.cpp).

4. Are two ranges permutations of each other / same multiset: random 64-bit value per value, compare sums (xor_hash.cpp).
5. 2D pattern in a grid: hash_2d.cpp.

```
Template: strings/min_rotation.cpp, strings/bigint.cpp,
strings/wildcard_match.cpp, strings/hashing/xor_hash.cpp
```

Patterns lesson by lesson

12.13 String basics: count instead of arrange, sliding windows, building the smallest answer

KEYWORDS: rearrange anagram palindrome reorder sliding window longest shortest substring lexicographically smallest recursive c-good

USE WHEN

- "Rearrange", "anagram", "can we make": the order does not matter, only letter counts.
- "Longest / shortest substring such that ..." with a condition that is monotone.
- "Lexicographically smallest" answer.

DO THIS

1. Ask first: if I shuffle the string, does the answer change? If not, count letters (cnt[s[i] − 'a']).
2. Palindrome from letters: possible ⇔ at most one letter has an odd count. Build: half of each letter, the odd letter in the middle, mirror.
3. Longest window with at most k bad things: move r forward; while the window is bad move l forward; answer max(r − l + 1). Works because a smaller window of a good one is good.
4. Shortest window containing everything: grow r until good, then shrink l while still good, record the length. Keep cnt[] and "distinct letters inside" (enters at 0→1, leaves at 1→0).
5. Lexicographically smallest: fill from left to right, try the smallest letter first, keep it only if the rest can still be completed (write that check carefully; e.g. no letter more than ⌈r/2⌉ of r remaining letters for "no two neighbours equal").
6. Recursive definition (a string of length 2^k is c-good if one half is all c and the other half is (c+1)-good): solve(piece, c) = min of the two choices, O(n log n).

EXAMPLE

> Change at most k letters of an a/b string to get the longest block of equal letters: longest window with at most k 'b' (and again for 'a'). "abbaab", k = 1 → 3.

WATCH OUT

- s.size() − 1 on an empty string wraps around (unsigned): write (int)s.size() − 1.
- t = t + c in a loop is O(n²): use t += c. substr inside a loop copies. Pass strings as const string &.

```
Template: templates/strings/ (no special file)
```

12.14 Hashing patterns: distinct substrings, longest repeat, trees, updates

KEYWORDS: hash distinct substrings count longest repeated substring binary search tree path palindrome updates dynamic hash range assign anti hash

DO THIS

1. Compare two pieces: Hash h(s); h.getHash(l1, r1) == h.getHash(l2, r2) (0-based, inclusive, same length!). Call preCompute(maxLen) once.
2. Count distinct substrings with a property, n ≤ 1500–3000: put (hash pair, length) of every good substring in a vector, sort, unique. (CF 271D)
3. Longest substring appearing twice: binary search the length L (if L works, L − 1 works); check with a set of hashes of all length-L pieces. O(n log² n).
4. LCP of two suffixes: binary search on the length with hash compares (lcp_double_hash.cpp). The next characters after the LCP tell which suffix is smaller → you can sort suffixes / compare substrings.
5. At most K mismatches in a window: jump over the matching part with LCP, skip one mismatch, repeat K + 1 times. (Dhaka 2019 A)
6. Updates s[i] = c: segment tree of (hash, length); merge: L·p^lenR + R (dynamic_hash.cpp). Range assign of one letter c: hash = c·(1 + p + ... + p^(len−1)), precompute these sums.
7. Root-to-node string on a tree: F(x) = F(par)·p + c, B(x) = B(par) + c·p^depth(par...); palindrome ⇔ F = B. Use BFS order, not recursion (10^6 nodes). (Dhaka 2021 D)
8. Delete one char to make a palindrome, with updates: forward + backward segment trees; binary search the first mismatch from both ends; try deleting each of the two mismatching characters. (Dhaka 2020 C)

FORMULAS

```
hash(s[l..r]) = H[r+1] − H[l]·p^(r−l+1) (mod M)
collision becomes likely after about √M values (birthday paradox) → ALWAYS double hash when hashes go into a set/map
break a single 31-bit hash (Dhaka 2017 E): random changes on two halves of ~40 positions, meet in the middle on the hash difference
```

WATCH OUT

- mod 2^64 overflow hashing is broken by the Thue–Morse string. Characters above 127: cast to unsigned char.

```
Template: strings/hashing/double_hash.cpp, lcp_double_hash.cpp, dynamic_hash.cpp, palindrome_hash.cpp, palindrome_tracker.cpp
```

12.15 Hashing multisets and sets (order does not matter), and 2D grids

KEYWORDS: multiset hash same multiset rearrangement permutation of 1..k random values xor hash parity every value even times multiple of 3 equal letter counts 2d hash matrix matcher

USE WHEN

- "Can a[l..r] be rearranged into b[L..R]", "same set of values", "window is a permutation of 1..k", "every value appears an even number of times / exactly 3 times", "all letters appear equally often".

DO THIS

1. Give every value x a random 64-bit number val(x) (xor_hash.cpp keeps them in a map). Hash of a multiset = Σ val (unsigned overflow is fine). Prefix sums → any range in O(1).
2. Sets (repeats do not count): add val(x) only the first time x appears in the prefix.
3. "Every count even": prefix XOR of val; subarray good ⇔ X[r] == X[l−1]; count equal prefix values with a map. XOR cannot tell {2,2} from {3,3}: use the SUM for multiset equality.
4. "Every count a multiple of 3": key = Σ val(x)·(cnt(x) mod 3) (subtract 3·val when a count hits 3). "Exactly 3": also keep a window where no value occurs more than 3 times.
5. Window is a permutation of 1..k: perm[k] = val(1) + ... + val(k); window of length k good ⇔ its sum = perm[k]. Counting all such windows: the window contains a 1 and its length equals its maximum; walk right from each 1 keeping the max m, check the single window of length m ending at r; repeat on the reversed array. (CF 1175F)
6. All letters equally frequent in a substring: prefix key = Σ val(c)·(cnt(c) − cnt('a')); reading 'a' subtracts Σ_{c ≠ a} val(c), reading c adds val(c); equal keys = good substring.
7. Groups of leaves in two trees: node hash = sum of its leaves' val. Compare sets of node hashes.
8. 2D: hash rows with base p1, then columns of row-hashes with base p2; rectangle = 4 prefix values (hash_2d.cpp, mod 2^61 − 1).

EXAMPLE

> a = [1,2,2,1,3,3]: prefix xors 0,5,12,5,0,14,0 → 4 subarrays with all counts even.

WATCH OUT

- Never use val(x) = x (attackable). Never use xor when you need multiset equality.

```
Template: strings/hashing/xor_hash.cpp, strings/hashing/hash_2d.cpp
```

12.16 Prefix function patterns: period, borders, matching shapes, automaton

KEYWORDS: prefix function pi border period shortest period all borders rotation cyclic shift difference array kmp automaton count strings avoiding pattern

DO THIS

1. π[i] = length of the longest border (prefix that is also a suffix, not the whole) of s[0..i].
2. Find p in t: π of p + '#' + t; π == |p| at text index i ⇒ match ending there.
3. Shortest period: n − π[n−1]. The string is a full repetition (like "abcabc") ⇔ that period divides n; then the number of blocks is n / period.
4. All borders: π[n−1], π[π[n−1] − 1], ... until 0.
5. Count occurrences of every prefix: cnt[π[i]]++ for all i, then for i from n down: cnt[π[i−1]] += cnt[i]; finally +1 each (the prefix itself).
6. Match after shifting all values (walls raised/lowered): KMP on the difference arrays. Rotation check: search b in a + a. Clock hands: KMP on gaps between consecutive angles, cyclically.
7. Count strings of length n avoiding / containing pattern p: DP over (length, automaton state); aut[state][letter] from kmpAutomaton(p); state |p| is forbidden (or absorbing).

8. Fill '?' to maximise occurrences: dp[i][state] with the automaton (CF 808G).
9. Shortest period of every substring s[j..i]: run π on every suffix s[j..]: O(n²) (CF 825F string compression).
10. Border tree: parent of prefix length j is π[j−1]; all prefixes of P ending at a position = path to the root (Dhaka 2023 D).

EXAMPLE

> s = "cabcabca": π[7] = 5 → period 3 ("cab").

WATCH OUT

- Do not name an array pi (macro). π[n−1], not π[n]. Forgetting the separator.

```
Template: strings/kmp.cpp (prefixFunction, kmpAutomaton)
```

12.17 Z-function patterns

KEYWORDS: z function z array periods prefix occurrences match from both ends common prefix of pieces harmonic

DO THIS

1. z[i] = how many characters from position i agree with the start of the string.
2. All periods: p is a period ⇔ p + z[p] = n (and n itself).
3. Pattern search: z of p + '#' + t; z == |p| ⇒ occurrence starts there.
4. Occurrences of every prefix length L: number of i with z[i] ≥ L (suffix sums of a count array; z[0] = n counts the prefix itself).
5. Word w = (piece of s) + (later piece of s): z of w + '#' + s gives, for each prefix length, the earliest end; same on reversed strings gives the latest start; try all splits (CF 149E).
6. Split into k pieces with the longest common prefix L: greedy jumps over positions with z ≥ L, each jump ≥ L, so all L together cost O(n log n) (harmonic sum). Answer for each k by binary search or by trying every L (CF 1968G).

WATCH OUT

- Template sets z[0] = n; some problems (CSES) want 0.

```
Template: strings/z_function.cpp
```

12.18 Palindrome patterns: counting, appending, pairs, swaps

KEYWORDS: palindrome count palindromic substrings longest palindrome append make palindrome pairs of palindromes non overlapping swaps every window palindrome manacher d1 d2

DO THIS

1. n ≤ 5000 and many queries: pal[l][r] = (s[l] == s[r]) && pal[l+1][r−1], fill by length.
2. Manacher: d1[i] = number of odd palindromes centred at i (longest 2·d1[i] − 1); d2[i] = number of even ones centred at the gap before i (longest 2·d2[i]). isPal(l, r) in O(1).
3. Number of palindromic substrings = Σ d1 + Σ d2.
4. Fewest characters to append at the end to make a palindrome: n − (longest palindromic suffix). At the front: longest palindromic prefix. (UVa 11475)
5. Longest palindrome = prefix of s + suffix of s with a palindrome in the middle (CF 1326D2): take the longest matching outer part a…reverse(a), then the longest palindromic prefix or suffix of the middle.
6. Pairs of palindromes, one completely left of the other: ends[i] and starts[i] via difference arrays (each centre adds +1 on a range of end positions); answer Σ (ends at

≤ i) · starts[i+1] (CF 17E).

7. All windows of length k palindromes: k = 2 → all equal letters; k = 3 → alternating; generally check small cases (CF 1943B).
8. Minimum swaps (any positions) to make a palindrome, K ≤ 5 letters: mismatched mirror pairs are edges between letters; answer = (#mismatched pairs) − (#cycles), take 2-cycles first, then 3-cycles, … (Dhaka 2019 E).
9. Minimum ADJACENT swaps to make a palindrome: greedy from the outside: match the leftmost char with the nearest equal from the right; count inversions with a BIT for n large.
10. Different palindromes / occurrences of each / longest ending at each position: eertree card.

EXAMPLE

> "aaa": d1 = 1 2 1, d2 = 0 1 1 → 6 palindromic substrings.

```
Template: strings/manacher.cpp
```

12.19 Bracket patterns: balance, matching, ranges, counting, '?'

KEYWORDS: bracket balanced regular sequence longest balanced substring range query segment tree count sequences reflection question marks fill expression

DO THIS

1. Balanced ⇔ balance (+1 for '(', −1 for ')') never below 0 and ends at 0. Several kinds: a stack, the popped opener must be the same kind.
2. Matching pairs: stack of indices of '('.
3. Longest balanced substring: stack starting with −1; on ')' pop; if empty push this index (a wall), else length = i − top. Count how many times the max occurs too (CF 5C).
4. Range query "longest balanced subsequence in [l, r]": segment tree node (pairs, open, close); merge: t = min(L.open, R.close); pairs = L.pairs + R.pairs + t; open = L.open + R.open − t; close = L.close + R.close − t; answer 2·pairs (CF 380C).
5. Minimum insertions to balance: scan; when balance would go negative, add one '(' (count it) and keep 0; at the end add `balance` ')'.
6. Count completions: prefix with height b, rem characters left: need up = (rem − b)/2 more '('; ways = C(rem, up) − C(rem, up + b + 1) (0 if parity wrong or prefix ever negative).
7. Fill '?': exactly n/2 '(' in total; put the missing '(' at the earliest '?' positions, then check. Unique filling? Try swapping the last '(' placed with the first ')' placed and check again.
8. Balanced subsequence of length k from a balanced string: take '(' while opened < k/2, take ')' while closed < opened.
9. Minimum replacements with several kinds: shape must be balanced (check with a stack of any kind); each matched pair of different kinds costs 1.
10. Expressions: recursive descent (expr = term ± term, term = atom * atom, atom = number | −atom | (expr)) or shunting-yard to postfix.

```
Template: strings/brackets.cpp, strings/expression_parser.cpp, strings/bigint.cpp
```

12.20 Subsequence patterns: check, count, distinct, shortest missing

KEYWORDS: subsequence check is subsequence next occurrence table copies needed count occurrences of pattern distinct subsequences shortest non subsequence remove substring keep subsequence emoticons

DO THIS

1. Is t a subsequence of s: one pass, match each letter of t at its earliest possible position.
2. Many t: nxt[i][c] = first position ≥ i with letter c (build from the right); each check is |t| jumps (subsequence_automaton.cpp).
3. Copies of s needed to get t: walk t with nxt; when a letter is not found, start a new copy (−1 if a letter never appears).
4. Number of ways pattern p appears as a subsequence: dp[j] = ways to match p[0..j−1]; for each char c of s, for j from |p| down to 1: if p[j−1] == c, dp[j] += dp[j−1].
5. With '?' (any of a, b, c) counting "abc" over all strings: keep cnt (number of strings), a, ab, abc; a '?' does: abc = 3·abc + ab; ab = 3·ab + a; a = 3·a + cnt; cnt = 3·cnt (CF 1426F).
6. Number of distinct subsequences (with empty): dp = 1; for each char c: new = 2·dp − last[c]; last[c] = dp (the value before this step); dp = new.
7. Distinct subsequences of each length: dp[i][len] with the same "subtract at the previous occurrence" trick (CF 1183H).
8. Shortest string (over alphabet A) that is NOT a subsequence: cut s greedily into blocks that contain every letter; answer length = blocks + 1; letters = the letter completing each block, then a letter missing from the tail (CSES 1087).
9. Delete the longest substring keeping t a subsequence: pre[i] = earliest end of t[0..i), suf[i] = latest start of t[i..); answer max over i of suf[i] − pre[i] (CF 1203D2).
10. Max number of disjoint copies of a pattern like "^_^": binary search k; take the first k '^' as left eyes and the last k as right eyes, greedily give each pair a '_' between (Dhaka 2015 E).

WATCH OUT

- Subtraction under mod: add MOD. In counting DPs loop j downwards (do not reuse a value updated in this step).

```
Template: strings/subsequence_automaton.cpp, strings/string_dp.cpp
(distinctSubsequences)
```

12.21 Trie patterns: prefixes, word break, xor queries

KEYWORDS: trie prefix count phone list word break word combinations dictionary split binary trie max xor subarray xor count xor at least k mex after xor

DO THIS

1. Count words with a given prefix: counter on every node (+1 when a word passes).
2. "Is some word a prefix of another" (phone list): while inserting, passing an end mark or ending at a node with children ⇒ yes.
3. Split a text into dictionary words (count ways): ways[n] = 1; for i from n−1 down: walk the trie from i, at every word end at length L add ways[i + L]. O(n · max word length).
4. Max x xor y over a set: binary trie, from the top bit try the opposite bit. Max xor subarray: insert prefix xors one by one, query each.
5. Count subarrays with xor ≥ k (or < k): walk the trie along bits of k and pre[r]; where k has 0, the branch that makes the xor bit 1 is entirely bigger: add its count (CF 665E).
6. Xor every element with X (cumulative), then mex: walk the trie of distinct values with X; at each bit go to X's bit child unless that subtree is full (size = 2^bits), then go the other way and set the answer bit (CF 842D).

7. Choose x to minimise inversions of a_i xor x: count, per bit level, inversions inside each trie node between its 0-child and 1-child both ways; choose each bit of x independently (CF 1416C).
8. Best LCP with any dictionary word: deepest node reached by walking A. Anagram groups: key = sorted letters (Dhaka 2018 I).
9. Delete from a binary trie: decrease counters along the path; count 0 = missing.

WATCH OUT

- Enough bits: values up to 1e12 need 40 bits. Use int32 arrays for 1e6 · 30 nodes.

```
Template: data_structures/trie.cpp, data_structures/binary_trie.cpp
```

12.22 Aho–Corasick: many patterns at once

KEYWORDS: aho corasick many patterns occurrences of each word dictionary filter banned words dp avoid typewriter suffix link tree

USE WHEN

- Many patterns (total length L) and one or more texts; "for each word count occurrences", "avoid all banned words", "split the text into dictionary words fast".

DO THIS

1. Build the trie of all words, then build() computes suffix links and full transitions go[v][c]; walk the text: v = go[v][c].
2. Words ending here: out[v] += out[link[v]] in BFS order gives "number of words ending at this position" in O(1).
3. List the words ending here: follow dict links (only nodes where a word ends). Cost = output size.
4. Occurrences of each word in the text: visit count per node, then push counts up the suffix links in reverse BFS order; answer = count at the word's node.
5. DP avoiding banned words: state = automaton node; forbidden nodes are those with out > 0. dp over (position, node).
6. Text split DP with keys/words: dp[i] = min over words ending at i of dp[i − len] + 1, listing words by dict links. Distinct word lengths ≤ √(2L), so this is fast (Dhaka 2020 G).
7. Words switched on/off: Euler tour on the suffix-link tree; turning word p on adds +1 on its subtree; a position's count is a point query (CF 163E).

WATCH OUT

- Call build() after inserting all words. Equal words share a node: store a count/list. Characters beyond a–z: map them or use bigger alphabet.

```
Template: strings/aho_corasick.cpp
```

12.23 Suffix array + LCP patterns

KEYWORDS: suffix array lcp distinct substrings longest repeated substring kth substring longest common substring two strings substring of length L count not substrings

DO THIS

1. sa = sorted suffix starts, lcp[i] = common prefix of sa[i] and sa[i+1] (Kasai, O(n)).
2. Distinct substrings: n(n+1)/2 − Σ lcp.
3. Longest repeated substring: max lcp. Repeated at least k times: max over windows of k−1 consecutive lcp values of their minimum.

4. k-th smallest distinct substring: go down sa; suffix i adds lengths lcp(prev) + 1 … its length, in order.
5. LCP of any two suffixes: min of lcp between their ranks (sparse table) → compare any two substrings in O(1).
6. Longest common substring of a and b: SA of a + '#' + b; best lcp between neighbours from different strings. Of m strings: separators, binary search length, sliding window over sa covering all strings.
7. Distinct substrings of each length L: suffix i adds +1 on lengths [lcp + 1, its length] → difference array. Count strings of length in [N, M] that are NOT substrings: Σ 26^L − (distinct of length L) (Dhaka 2018 F).
8. Number of occurrences of pattern p: binary search the range of suffixes starting with p.

WATCH OUT

- Separator must not appear in the strings (and must be smaller than letters in the template: \x01). lcp[i] is between places i and i+1.

```
Template: strings/suffix_array.cpp
```

12.24 Suffix automaton patterns

KEYWORDS: suffix automaton sam occurrences count distinct substrings kth substring longest common substring endpos sum over substrings generalized

DO THIS

1. Walk a word from state 0: missing edge ⇒ not a substring. O(|word|).
2. Distinct substrings: Σ over states (len[v] − len[link[v]]).
3. Occurrences: cnt = 1 for every prefix state (not clones), then push cnt to link in decreasing len order; a word's count = cnt of the state where its walk ends.
4. k-th distinct substring: paths[v] = Σ over edges (1 + paths[to]); walk from 0 choosing letters in order (SPOJ SUBLEX). For k-th with repeats, use cnt instead of 1.
5. Longest common substring of s and t: SAM of s, walk t; on a missing edge follow links (match length = len of the state) (SPOJ LCS). Several strings: keep per-state minimum of best matches.
6. Sum over all distinct substrings of f(occurrences): Σ_v (len[v] − len[link[v]]) · f(cnt[v]) (CF 123D: f(c) = c(c+1)/2).
7. Forbidden end positions: give those prefix states cnt 0 (CF 873F: max |t|·occ).
8. Several strings: generalized SAM; each prefix state gets the weight of its string (CF 616F).
9. Count pairs (substring of X, substring of Y) forming a palindrome: walk X through the SAM of reverse(Y), sum cnt; multiply by palindromes starting at each position (Dhaka 2021 E).

WATCH OUT

- Clones start with cnt 0. Push counts from longest to shortest (counting sort by len). Memory 2n · 26 ints.

```
Template: strings/suffix_automaton.cpp, strings/generalized_sam.cpp
```

12.25 Palindromic tree (eertree)

KEYWORDS: eertree palindromic tree distinct palindromes count occurrences of each palindrome longest palindrome ending at each position palindrome partition

DO THIS

1. Nodes = distinct palindromes (at most n), two roots (len −1 and 0). add(c) returns the node of the longest palindrome ending at the new character.
2. Distinct palindromes: number of nodes − 2.
3. Longest palindrome ending at each position: len of the node returned by add.
4. Occurrences of each palindrome: +1 to the returned node at each step, then push counts along suffix links, newest nodes first. "Max length × occurrences" is then a loop (APIO 2014).
5. Number of palindromes ending at position i: depth of the node in the suffix-link tree (store cntEnd[v] = cntEnd[link] + 1).
6. Fewest palindromes to split s into: series links (lengths form O(log n) arithmetic progressions), O(n log n) (CF 932G). For n ≤ 5000 just use the O(n²) DP with the pal table.

```
Template: strings/palindromic_tree.cpp
```

12.26 String DP: edit distance, LCS, deleting blocks, watching a pattern

KEYWORDS: edit distance lcs longest common subsequence print lcs interval dp delete blocks clear the string watch pattern hard cost delete

DO THIS

1. Edit distance: d[i][0] = i, d[0][j] = j; d[i][j] = d[i−1][j−1] if a[i−1] == b[j−1], else 1 + min(d[i−1][j], d[i][j−1], d[i−1][j−1]).
2. LCS: match → 1 + d[i−1][j−1], else max(d[i−1][j], d[i][j−1]). Print it by walking back from the corner.
3. Delete blocks of equal letters (fewest operations): dp[l][r] = min(1 + dp[l+1][r], min over m with s[m] == s[l] of dp[l+1][m−1] + dp[m][r]); fill by length (CF 1132F).
4. Delete letters (with costs) so that "hard" is not a subsequence: dp[j] = min cost having matched j letters of "hard" (j ≤ 3); a letter equal to pattern[j] either is deleted (pay) or moves j to j + 1 (CF 1096D).
5. Pattern must not appear as a SUBSTRING: state = KMP automaton state.
6. Count ways to empty a string by removing equal adjacent pairs: interval DP with a binomial for interleaving the two parts' operations (CSES Empty String).
7. Say the state in one sentence before writing the recurrence.

WATCH OUT

- 5000 × 5000 long long = 200 MB: keep two rows or use int.

```
Template: strings/string_dp.cpp
```

12.27 Rotations, wildcard matching, bitsets, BWT, de Bruijn, reversals

KEYWORDS: minimum rotation cyclic necklace canonical lyndon wildcard question mark fft matching bitset matching tolerance bwt burrows wheeler de bruijn lock code substring reversals treap

DO THIS

1. Smallest rotation in O(n): two candidates i, j; compare k chars; the loser jumps past k + 1 (min_rotation.cpp). Canonical form of a necklace = its smallest rotation; distinct necklaces = distinct canonical forms.
2. Lyndon factorization (Duval): splits into non-increasing Lyndon words; also gives the smallest rotation (lyndon.cpp).

3. Pattern with '?' (matches anything): Σ p·t·(p − t)² = 0 at a position ⇔ match; expand into 3 convolutions with FFT (wildcard_match.cpp). Also "for every shift count something" is one convolution.
4. Approximate matching (letter may be within distance k): bitset per letter of available positions; ok &= near[p[j]] >> j (CF 528D). n·m/64 brute force often fits.
5. Inverse BWT: the k-th occurrence of a letter in the last column = the k-th in the first column (bwt.cpp, CSES String Transform).
6. De Bruijn: every length-n word over m letters once: length m^n + n − 1 (non-cyclic) (de_bruijn.cpp).
7. Reverse/cut/paste pieces: implicit treap with a reverse flag (treap.cpp).
8. Cyclic shifts that are palindromes with '?': pairs p + q ≡ σ (mod n); count conflicts and '?'-'?' pairs with cyclic self-convolutions (Dhaka 2022 E).

```
Template: strings/min_rotation.cpp, lyndon.cpp, wildcard_match.cpp, bwt.cpp;
combinatorics/de_bruijn.cpp; data_structures/treap.cpp
```

13. Geometry

KEYWORDS: geometry cross product area polygon convex hull distance circle triangle lattice points pick

13.1 Geometry rules: integers first

KEYWORDS: cross product dot orientation integer precision eps long double

DO THIS

1. Keep integer inputs integer: cross product, dot product, squared distances. |coords| ≤ 1e9: cross fits in long long. Bigger or products of crosses: __int128.
2. cross(b − a, c − a) > 0: c is to the LEFT of a→b (counter-clockwise). = 0: collinear. < 0: right.
3. Doubles: use long double, EPS = 1e-9; compare a < b − EPS; clamp acos/asin arguments to [−1, 1].
4. Never compare doubles with ==. Print with fixed and setprecision.

FORMULAS

```
cross(a, b) = a.x·b.y − a.y·b.x dot(a, b) = a.x·b.x + a.y·b.y
distance² = dx² + dy² angle = atan2l(y, x)
rotate by θ: (x cosθ − y sinθ, x sinθ + y cosθ); by 90°: (−y, x)
```

```
Template: geometry/geometry_2d.cpp
```

13.2 Polygons: area, inside test, lattice points (Pick)

KEYWORDS: polygon area shoelace point in polygon lattice points pick theorem boundary interior

DO THIS

1. Area: shoelace. area2 = Σ cross(p_i, p_{i+1}), area = |area2| / 2. Positive area2 = counter-clockwise order.
2. Point in polygon (any simple polygon): inPoly → 0 outside, 1 inside, 2 on boundary. Convex polygon: O(log n) inside().
3. Lattice points on a segment between two lattice points: gcd(|dx|, |dy|) + 1.
4. Pick: 2A = 2I + B − 2 (A area, I interior lattice points, B boundary lattice points = Σ gcd over edges). Use it to get I or A.

5. Scaling a lattice polygon by k: B → k·B, A → k²·A (2020 I).

FORMULAS

```
2A = 2I + B − 2
I = (2A − B + 2) / 2
```

`Template: geometry/geometry_2d.cpp area2, inPoly, boundaryPoints`

13.3 Segments and lines

KEYWORDS: segment intersection lines intersect point distance point to line touching collinear

DO THIS

1. Segments intersect (touching counts): segInter (orientation tests + collinear overlap check). Exact with integers.
2. Intersection point of two lines: lineInter (long double). Of two segments when it is a single point: segInterPoint.
3. Distance point–line: |cross(b − a, p − a)| / |b − a|. Point–segment: distPointSeg.
4. Many segments: decide pairwise with cross products; conflicts between two sides → bipartite graph (2024 B).

`Template: geometry/geometry_2d.cpp`

13.4 Convex hull and friends

KEYWORDS: convex hull monotone chain farthest pair rotating calipers closest pair minkowski

DO THIS

1. Convex hull: monotone chain O(n log n) (convex_hull.cpp, counter-clockwise, no collinear points).
2. Union of all triangles of a point set = its convex hull: "is q inside some triangle" = "is q inside the hull".
3. Farthest pair: rotating calipers on the hull (diameter2). Closest pair: closestPair2.
4. Minkowski sum for "a + b over two convex sets": minkowski_sum.cpp.

`Template: geometry/convex_hull.cpp`

13.5 Triangles and circles: formulas

KEYWORDS: triangle area heron inradius circumradius incenter law of cosines circle intersection exact fraction

DO THIS

1. Exact fraction output: Frac (math/fraction.cpp), reduce by gcd, print p/q.
2. Circle–line and circle–circle intersections: circleLine, circleCircle in geometry_2d.cpp.

FORMULAS

```
area from points: |cross(b − a, c − a)| / 2
Heron: 16·Area² = (a+b+c)(−a+b+c)(a−b+c)(a+b−c)
inradius r = 2·Area / (a+b+c); r² = (P−2a)(P−2b)(P−2c) / (4P) with P = a+b+c
(integer sides → exact fraction)
circumradius R = a·b·c / (4·Area)
law of cosines: c² = a² + b² − 2ab·cos C; law of sines: a / sin A = 2R
incenter = (a·A + b·B + c·C) / (a+b+c) (a = side opposite A)
the point at equal distance from all three sides is the incenter (2024 F)
```

```
circle: area πr², chord length 2r·sin(θ/2), segment area r²(θ − sinθ)/2
regular n-gon with side s: area = n·s² / (4·tan(π/n))
```

EXAMPLE

2024 F: sides 57, 13, 60 → P = 130, r² = 16·104·10 / 520 = 32 → "32/1".

13.6 Manhattan and Chebyshev distance

KEYWORDS: manhattan distance chebyshev rotate 45 degrees x+y x-y max distance

DO THIS

1. |x1−x2| + |y1−y2| = max(|u1−u2|, |v1−v2|) with u = x + y, v = x − y.
2. Max Manhattan distance among points: max(max u − min u, max v − min v).
3. Sum of Manhattan distances over all pairs: handle x and y separately (sort + prefix sums).
4. Intersection of all squares containing the points: bounded by min/max of x, y, x+y, x−y (an octagon, 2020 A).

13.7 Geometry patterns and formulas from contests

KEYWORDS: geometry patterns closest pair angle sort polar sweep circle line intersection point segment distance area of union rectangles lattice points on segment gcd rotation

FORMULAS

```
lattice points on segment (x1,y1)−(x2,y2) including ends: gcd(|dx|, |dy|) + 1
Pick: Area = I + B/2 − 1 → interior points I = A − B/2 + 1 (use 2A as an integer)
twice the polygon area (shoelace): Σ (x_i·y_{i+1} − x_{i+1}·y_i); sign =
orientation
point in polygon: ray casting (or winding); on an edge: cross = 0 and dot ≤ 0
distance point−line: |cross(B−A, P−A)| / |B−A|; point−segment: project, clamp t to
[0, 1]
rotate (x, y) by θ: (x cosθ − y sinθ, x sinθ + y cosθ); by 90°: (−y, x)
sort by angle: atan2, or half-plane split + cross (exact)
three points collinear: cross = 0; triangle area = |cross|/2
circle through 3 points: perpendicular bisectors; incircle r = A/s; circumcircle R
= abc/(4A)
Heron: A = √(s(s−a)(s−b)(s−c)); law of cosines c² = a² + b² − 2ab cos C
closest pair of points: sort by x, divide and conquer or sweep with a set by y
max points on a line: for each point, map of reduced direction (dx/g, dy/g) with a
sign rule
union area of rectangles: sweep x + segment tree of covered length (coordinate
compressed)
Manhattan → Chebyshev: (x + y, x − y); max Manhattan distance = max of 4 sign
combos
convex polygon: point inside in O(log n) with binary search on the fan
3D: cross product for normals, distances in 3d_distances.cpp
```

WATCH OUT

- Integers whenever possible (cross products in long long / __int128). EPS = 1e-9 only for doubles. Print with enough precision.

```
Template: geometry/geometry_2d.cpp, convex_hull.cpp, minkowski_sum.cpp,
3d_distances.cpp
```

14. Games

KEYWORDS: game theory win lose two players optimal play nim grundy sprague mex

14.1 Any game: brute force the small cases first

KEYWORDS: game who wins table pattern brute force small n optimal

USE WHEN

- Two players, optimal play, "who wins", N up to 1e16.

DO THIS

1. Write win[n] = true if some move leads to a state with win = false. Compute for n = 0..50 and PRINT the table.
2. Find the pattern: period (n mod k), parity, powers of two, depends on who starts. Then code the formula (2024 E).
3. Template grundy_brute.cpp prints the table.

```
Template: misc/grundy_brute.cpp
```

14.2 Sum of games: Sprague-Grundy and Nim

KEYWORDS: grundy sprague nim xor mex independent games piles split

DO THIS

1. Game splits into independent parts (piles, segments, components): compute g(part), xor them. First player wins ⇔ xor ≠ 0.
2. g(state) = mex { g(next state) } (smallest non-negative integer not in the set).
3. A move that splits a part into two parts: next value is g(left) ^ g(right).
4. Nim: g(pile) = size. Take 1..k stones: g(n) = n mod (k+1). Take any divisor / special moves: brute force g and look for a pattern.

FORMULAS

```
mex{0, 1, 3} = 2
Dawson's Kayles (remove two adjacent vertices of a path, 2022 A): g(m) = mex over
a + b = m − 2 of g(a) ^ g(b), g(0) = g(1) = 0
Misère Nim (last mover loses): all piles of size 1 → first wins ⇔ number of piles
even; otherwise same as normal Nim
```

14.3 Other game ideas

KEYWORDS: symmetry mirror strategy parity game adversary hangman

DO THIS

1. Symmetric position: the second player copies the first player's move (mirror strategy).
2. Game on a tree / grid with parity: count moves left; the winner is decided by parity.
3. "Can the hider always survive?": the adversary keeps the largest set of hidden states still consistent with all answers (2025 B).
4. Game DP on intervals: dp[l][r] = best score difference for the player to move.

14.4 Game patterns: known answers

KEYWORDS: game patterns nim misere bash game take 1..k subtraction staircase nim wythoff green hackenbush mirror strategy parity last move

FORMULAS

```
take 1..k stones, last move wins: lose ⇔ n % (k+1) == 0
Nim: xor of piles; zero ⇒ first player loses
misère Nim: if all piles ≤ 1: first wins ⇔ number of piles is even; else same as normal Nim
staircase Nim (move coins one step down): xor of piles on odd steps
Wythoff (take from one pile or equal from both): losing (⌊kφ⌋, ⌊kφ²⌋), φ = (1+√5)/2
subtraction game with a set S: Grundy by DP, look for the period
divisor game / "replace n by n − d": brute force small n, print the win/lose table and look
symmetric board / two equal piles: mirror strategy (second player copies)
moves that always change parity: answer depends on parity of the total moves
Grundy of a sum = xor; Grundy of a position = mex of the moves
tree game where you delete a subtree (Green Hackenbush on trees): g(v) = xor over children of (g(c) + 1)
when n is huge: compute Grundy for small n with grundy_brute.cpp and find the pattern
```

Template: misc/grundy_brute.cpp

15. Math tools: matrices, recurrences, interpolation, sums

KEYWORDS: matrix exponentiation recurrence fibonacci lagrange interpolation polynomial sums gauss

15.1 Matrix power for linear recurrences (n up to 1e18)

KEYWORDS: matrix exponentiation linear recurrence fibonacci tiling count walks length n

USE WHEN

- f(n) = c1·f(n−1) + … + ck·f(n−k) with huge n. Number of walks of length n. DP with a small state and the same transition every step.

DO THIS

1. Build the k×k transition matrix, raise it to the power n (matPow), multiply by the first values. O(k³ log n).
2. Walks of length n from u to v = (A^n)[u][v].

FORMULAS

```
[f(n), f(n−1)]ᵀ = [[1,1],[1,0]] · [f(n−1), f(n−2)]ᵀ (Fibonacci)
F(2k) = F(k)·(2F(k+1) − F(k)), F(2k+1) = F(k)² + F(k+1)²
binary strings of length n without two adjacent 1s: F(n+2)
```

Template: math/matrix.cpp

15.2 Answer is a polynomial in n: brute force + Lagrange

KEYWORDS: lagrange interpolation polynomial sum of powers brute force small n fit

USE WHEN

- n up to 1e9+ and the answer looks like a polynomial in n (sum of i^k, counting placements on an n×n board, 2020 B).

DO THIS

1. Compute f(0), f(1), ..., f(d) with a brute force (d = degree; try d up to ~10 and check extra points).
2. lagrange(y, x) returns f(x) mod MOD in O(d).
3. Check degree: if the d-th differences are constant, degree is d.

FORMULAS

```
Σ_{i=1..n} i^k is a polynomial of degree k+1
```

```
Template: math/lagrange.cpp
```

15.3 Closed forms for sums

KEYWORDS: sum formula 1 to n squares cubes arithmetic geometric series

FORMULAS

```
Σ i = n(n+1)/2 Σ i² = n(n+1)(2n+1)/6 Σ i³ = (n(n+1)/2)²
arithmetic: (first + last)·count / 2
geometric: a·(r^n − 1)/(r − 1)
Σ_{i=1..n} i·r^i: differentiate, or matrix power
mod p: multiply by inv(2), inv(6) instead of dividing
harmonic: Σ_{i≤n} 1/i ≈ ln n + 0.5772
```

15.4 Linear equations

KEYWORDS: gaussian elimination linear system equations mod p 2x2

DO THIS

1. Solve A·x = b mod prime p: gauss.cpp (0 = none, 1 = unique, 2 = many).
2. 2×2 integer system: linear_equations_2x2.cpp (Cramer).
3. Expectation with cyclic states: write E equations, solve with Gauss.

```
Template: math/gauss.cpp, math/linear_equations_2x2.cpp
```

15.5 Small n and brute force tricks

KEYWORDS: meet in the middle permutations brute force pattern stress test random

DO THIS

1. n ≤ 10: all permutations with next_permutation (sort first).
2. n ≤ 40, subset sums: meet in the middle (2^20 per half, sort one half, binary search).
3. Guess a formula: brute force n = 1..10, look at differences, ratios, known sequences (2^n, n!, Fibonacci, Catalan, Bell, derangements, partitions).
4. Stress test a wrong solution: misc/stress.sh compares sol.cpp with brute.cpp on random tests from gen.cpp.

```
Template: misc/stress.sh, misc/gen_example.cpp
```

15.6 Josephus problem

KEYWORDS: josephus circle every k-th person eliminated last survivor

FORMULAS

```
0-indexed survivor: J(1) = 0, J(n) = (J(n−1) + k) mod n (O(n))
k = 2: write n = 2^m + L (0 ≤ L < 2^m); survivor (1-indexed) = 2L + 1
```

15.7 Reference: classic results and values

KEYWORDS: wilson fermat sum of two squares fibonacci pisano zeckendorf grid squares rectangles handshake euler formula values

FORMULAS

```
Fermat: a^(p−1) ≡ 1 (mod p), p prime, p ∤ a. Wilson: (p−1)! ≡ −1 (mod p).
n is a sum of two squares ⇔ every prime ≡ 3 (mod 4) has an even exponent.
gcd(F_a, F_b) = F_gcd(a,b). Every n is a unique sum of non-adjacent Fibonacci
numbers (Zeckendorf, greedy).
Pisano period (Fibonacci mod m) ≤ 6m; mod 10: 60; mod 1e9+7: 2·10^9+16.
Squares in an n×m grid of cells: Σ_{k≥1} (n−k+1)(m−k+1). Rectangles: C(n+1,
2)·C(m+1, 2).
Handshake: Σ degrees = 2·edges. Planar: V − E + F = 2. Tree: E = V − 1.
998244353 = 119·2^23 + 1 (root 3); 7340033 = 7·2^20 + 1 (root 3). 1e9+7 and 1e9+9
are prime.
Catalan 1 1 2 5 14 42 132 429 1430 4862 | Bell 1 1 2 5 15 52 203 877 4140 |
Derangements 1 0 1 2 9 44 265 1854
Partitions 1 1 2 3 5 7 11 15 22 30 42 56 77 101 | Fibonacci 0 1 1 2 3 5 8 13 21 34
55 89 144
Factorials: 10! = 3628800, 12! = 4.8e8, 13! = 6.2e9, 15! = 1.3e12, 20! = 2.4e18
```

15.8 Simulation and implementation patterns

KEYWORDS: simulation implementation cycle detection huge steps calendar date day of week clock parse format spiral rotate matrix

FORMULAS

```
simulate 1e18 steps: store the first time each state was seen; when a state
repeats, skip whole cycles
dates: count days from a fixed day; leap year (y%4==0 && y%100!=0) || y%400==0;
day of week = days mod 7
Zeller / Sakamoto: dow(y,m,d) with t = {0,3,2,5,0,3,5,1,4,6,2,4}; if m < 3 y−−; (y
+ y/4 − y/100 + y/400 + t[m−1] + d) % 7 (0 = Sunday)
times: convert to seconds; clock hands angle at h:m: |30h − 5.5m|, take min(a, 360
− a)
rotate a matrix 90° clockwise: b[j][n−1−i] = a[i][j]; spiral order with four
bounds
grid directions: dr = {−1,0,1,0}, dc = {0,1,0,−1}; knight moves (±1, ±2), (±2, ±1)
roman numerals, base conversion, big numbers as strings (bigint.cpp)
read until EOF: while (cin >> x); lines: getline after cin.ignore()
```

16. How to recognise it, and common mistakes (from the ladders)

KEYWORDS: mistakes traps recognise signals

16.1 Number theory and math

KEYWORDS: number theory and math

Modular arithmetic & digit tricks

Recognise: Any "print modulo 10^9+7", any number given as a long string, and anything about digit sums or divisibility by 3, 9 or 11.

gcd, lcm, extended Euclid, CRT

Recognise: "The same value is added to every element", "measure exactly", "can you reach x using steps of a and b", "satisfies several remainders at once", or "largest gcd / gcd exactly k among pairs".

Primes, sieves, divisors, factorization

Recognise: Divisor counts and sums, "product is a perfect square", "coprime to x", "smallest/largest prime factor", and big inputs, where the limit tells you whether to sieve (up to 10^7) or use Miller–Rabin/Pollard (up to 10^{18}).

Euler totient φ

Recognise: "Count fractions in lowest terms", "Σ gcd(i, n)", "pairs with gcd equal to G", power towers, or an exponent too big to store.

Möbius & gcd-counting

Recognise: "Count pairs, tuples or subsets with gcd 1 (or gcd exactly k)", "Σ gcd over all pairs", "visible lattice points" (a point is visible from the origin exactly when its coordinates are coprime).

Fast power, Fermat, inverses, discrete log

Recognise: Any division under a prime modulus, probabilities printed mod p, exponents too big to loop, and power towers (combine with the φ lesson).

Factorials, nCr, Legendre, Lucas

Recognise: "Trailing zeros", "largest k such that p^k divides …", any nCr in a counting problem, and nCr with an unusual modulus. Always check whether the modulus is prime.

Combinatorial counting

Recognise: "Number of ways", "distribute", "arrange", "paths in a grid", "at least one of each", "nobody in their own place". Before writing a DP, check whether a picture turns it into one binomial.

Expectation & probability

Recognise: The words "expected" or "average", random choices or permutations, and "print modulo p" next to a probability.

Floor blocks, series, interpolation

Recognise: A sum that runs to 10^{10} or more with $\lfloor n/i \rfloor$ or n mod i inside, "sum of divisor counts up to n", or a pattern with an obvious period. The giveaway is that n is too big for O(n) but O(√n) fits.

Matrix exponentiation

Recognise: A linear recurrence, or "number of ways of length n" with a small state, and n up to 10^{18}. If n were 10^6, you'd just run the DP.

Squares, exponent parity, XOR basis

Recognise: "Product is a perfect square", "odd number of divisors", toggling, or anything where only the parity of exponents matters.

Invariants

Recognise: "Any number of operations", "is it possible to reach", "what is the final value", or games where you must prove someone can't win. In a preliminary set, these are often the problems with surprisingly short solutions.

16.2 Graphs and trees

KEYWORDS: graphs and trees

DSU power-ups

Recognise: "Are a and b connected?", components that only grow, "using edges with weight at most x", deletions that you can replay backwards, "same / different" constraints, bipartiteness under edge additions, or "after merging, what's the biggest/smallest in the group?".

Shortest paths on state graphs

Recognise: "Fewest moves/cost", grids, flights, keys, coupons, limited turns. The question to ask yourself every time: what does a node need to remember? Then: are all costs equal (BFS), 0 or 1 (deque), non-negative (Dijkstra), negative (Bellman–Ford), or do I need all pairs on a small graph (Floyd)?

MST tricks

Recognise: "Connect everything as cheaply as possible", "minimise the largest edge used", "the first time two things become connected", or a graph defined by a formula on pairs. In the last case, look for the argument that throws away almost all edges.

Tree DP and rerooting

Recognise: A tree with up to $2 \cdot 10^5$ nodes and an answer built from subtrees. "For every node / every root" means rerooting. A "choose k" or "exactly k" with small k means knapsack merging. A "best path" means combining the top two chains.

Tree greedy and contribution

Recognise: "Sum over all pairs or all paths" (use contributions), "choose k nodes or edges" (find the per-item value, then sort), "cover / guard / within distance" (deepest-first greedy, often inside a binary search).

LCA and path tricks

Recognise: Anything about paths between two nodes on a static tree: distances, the k-th node on a path, path maximums, counting paths through nodes or edges, "where do these paths meet?".

Euler tour, HLD, small-to-large

Recognise: "Update/query a subtree" (Euler tour + Fenwick or segment tree). "Root-to-node path" (Euler tour with range add). "For every node, something about its subtree" (small to large). "Update/query any path" (HLD). Depth-limited subtree questions (Euler tour + per-depth lists).

DAG, SCC, functional graphs

Recognise: Prerequisites, "one-way", "levels", "count routes" (topological order + DP). Cycles in a directed graph that you can go around freely (SCC + condensation). "Each node points to one node", teleporters, permutations (functional graph).

Bridges, articulation points, 2-SAT

Recognise: "Critical roads/cities", "removing one edge/vertex disconnects", "unavoidable roads" (bridges, articulation points, bridge tree). Choices between two options for each item with pairwise constraints (2-SAT).

Tree isomorphism and hashing

Recognise: "Same shape", "isomorphic", "how many different subtrees", "group the subtrees by structure".

16.3 Strings

KEYWORDS: strings

String basics: counting, two pointers, parsing

Recognise: "Rearrange", "anagram" or "can we make" usually means counting. "Longest or shortest substring such that..." means a sliding window, if the condition is monotone. "Lexicographically smallest" means building from the left with a check.

- s.size() - 1 on an empty string: size() is unsigned and wraps to a huge number. Write (int)s.size() - 1 .
- Building strings with t = t + c in a loop.
- Assuming lowercase letters when the statement allows other characters.

Hashing: polynomial, double, with updates

Recognise: Many comparisons of pieces of a string; "is this piece equal to that one"; counting different substrings with small n; "longest substring such that..." with binary search; string problems with updates.

- A single modulus when many hashes go into a set or map.
- Off-by-one: getHash(l, r) includes both ends.
- preCompute called for a shorter length than a string you hash later.
- Recursion over a tree with 10^6 nodes. Process nodes in BFS order instead.

Hashing sets, multisets and grids

Recognise: "rearranged into", "same multiset", "permutation of 1..k", "each value appears exactly / an even number of times", "same groups", and matching a picture in a grid.

- Using val(x) = x or another fixed formula. It can be attacked.
- Using xor when you need multiset equality: xor only detects even or odd counts.
- For sets, adding val each time instead of only on the first appearance.

Prefix function & KMP

Recognise: "Occurrences of a pattern", "shortest period", "borders", "prefix that is also a suffix", "match after a shift or rotation", and counting strings that must avoid or contain a pattern.

- Forgetting the separator between pattern and text.

- Using π[n] instead of π[n−1].
- Naming an array pi : your template defines it as a macro.
- Treating "period" as "divides n" when partial repetitions are allowed, or the other way round.

Z-function

Recognise: "How much does the string from position i agree with its beginning", periods, counting prefix occurrences, splitting into pieces with a common prefix.

- Restarting from 0 inside the box: still correct, but $O(n^2)$ on "aaaa…".
- z[0]: the template sets it to n, CSES expects 0. Read the statement.
- Forgetting the separator when concatenating.

Palindromes & Manacher

Recognise: "Palindrome", "reads the same backwards", "make it symmetric", counting symmetric pieces. For different palindromes, or palindromes ending at each position, see the eertree lesson.

- Forgetting even-length palindromes.
- Mixing up which gap d2[i] refers to (the one before i).
- Using the n^2 table when n is 10^5.

Brackets, expressions & big numbers

Recognise: "Regular / balanced / correct bracket sequence", "parentheses", "expression", "evaluate", or anything that opens and closes.

- Checking only that the totals are equal, and not that the balance never goes negative.
- Accepting "([)]" when bracket kinds matter.
- Deep recursion in a parser on very nested input: use the stack-based method instead.

Subsequences

Recognise: "Subsequence", "delete some characters", "keeping the order", "how many ways to pick positions", "can t be obtained from s by erasing".

- Mixing up substring (contiguous) and subsequence (any positions, same order).
- In counting DPs, using a value already updated in the same step.
- Negative numbers after subtracting under a modulo: add MOD before taking %.

Trie & binary trie (XOR)

Recognise: Many words and questions about beginnings; splitting text into dictionary words; "maximum / minimum xor", "subarray xor", "xor everything with x".

- Not enough bits: values up to 10^{12} need 40 (BinTrie t(40)).
- Memory: 10^6 numbers × 30 bits of long long children is too much. Use int32 arrays.
- Deleting: decrease the counters along the path and treat count 0 as missing.

Aho–Corasick

Recognise: Many patterns at once; "for each word, count occurrences"; "avoid all forbidden words"; building strings that must not contain any word from a list.

- Forgetting to call build() after adding the words.
- Equal words share one node: store a list of word ids (or a count) per node.
- Characters other than a–z: map digits to 'a'..'j', or use 128 children for full ASCII.

- Memory: nodes × 26 children; use int32.

Suffix array + LCP

Recognise: "Different substrings", "k-th smallest substring", "longest repeated / common substring", many questions about shared beginnings of suffixes.

- A separator that also appears in the strings.
- Off-by-one: here lcp[i] is between the suffixes at places i and i+1.
- Forgetting that two non-neighbouring suffixes share the minimum of the LCPs between them.

Suffix automaton

Recognise: "How many times does this word occur", "over all different substrings", "k-th substring", "substring of t that occurs in s", several strings sharing substrings.

- Memory: 2n states × 26 edges. Use int32 (the template does).
- Passing counts in the wrong order (must be from the longest states to the shortest).
- Giving clones a count of 1.

Palindromic tree (eertree)

Recognise: "Different palindromes", "occurrences of each palindrome", "longest palindrome ending here", splitting into palindromes.

- Passing counts in the wrong order (newest nodes first).
- Forgetting that the −1 root makes the "character before" check compare the letter with itself, which is why it always works.
- Memory with 26 children per node: use int32.

String DP

Recognise: Two strings with "fewest changes" or "longest common"; "delete blocks" or "merge pieces"; "fill '?' to maximise / count"; a string with n ≤ 5000.

- Memory: a 5000 × 5000 table of long long is 200 MB. Keep two rows, or use smaller types.
- Interval DP filled in the wrong order (it must go by increasing length).
- Mixing up "the first i letters" with "the letter at index i".

Rotations, FFT / bitset matching, BWT, de Bruijn

Recognise: "Rotation", "cyclic", "necklace"; '?' wildcards or matching with tolerance; "for every shift" or "for every k, count…"; "shortest string containing every word of length n".

17. Template code (FILE: path in rayyankah/cp_template)

KEYWORDS: template code file snippet

17.1 What the base template already has (not printed)

KEYWORDS: base template macros binexp ncr mint sieve factorize

File: templates/base/cp_template.cpp (repo commit 3934a6e). Write your code under "Code starts here". It already gives:

- Macros: `int` = long long, `vi vvi pi vpi si mii pb all rep(i,a,b) repb(i,a,b) myCeil(a,b) precise(x) nl fast`. Never name variables pi, pb, nl, N, all.
- `MOD` = 1e9+7 (change to 998244353 when needed), `N` = 1e6+10, `rnd()`, `rndRange(l, r)`, debug printing `debug(...)`, `debugArr(a, n)` with -DDEBUG.
- `binMul(a, b, m)` (__int128), `binExp(a, b, m = MOD)`, `extgcd(a, b, x, y)`.
- `fact, invFact, compute_factorials(), ncr(n, r)` (n < N).
- `Mint` (modular int with + - * / and ==, <<).
- `sieve()`: spf, isPrime, primes below N. `factorize(x)` for x < N. `factorizebig(n)` trial division.
- `custom_hash`, `umap<K,V>`, `uset<T>` (anti-hack hash maps). `preprocessing()` calls sieve and factorials.
- `solve(int tc)` with tc = test number; uncomment `cin >> t` in main for multiple tests.

17.2 Number theory

KEYWORDS: number theory template code

FILE: templates/number_theory/pollard_rho.cpp

```
// miller rabin (deterministic for 64-bit) + pollard rho factorization, ~n^(1/4) per
factor
// isPrime64(n); factor(n) -> sorted list of prime factors with repetition
int mulmod(int a, int b, int m) { return (__int128)a * b % m; }
int powmod(int a, int e, int m)
{
    int r = 1 % m;
    a %= m;
    while (e) { if (e & 1) r = mulmod(r, a, m); a = mulmod(a, a, m); e >>= 1; }
    return r;
}
bool isPrime64(int n)
{
    if (n < 2) return false;
    for (int p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) if (n % p == 0) return n
== p;
    int d = n - 1, s = 0;
    while (d % 2 == 0) d /= 2, s++;
    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
    {
        int x = powmod(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool comp = true;
        rep(r, 1, s) { x = mulmod(x, x, n); if (x == n - 1) { comp = false; break; } }
        if (comp) return false;
    }
    return true;
}
int pollard(int n)
{
    if (n % 2 == 0) return 2;
    static mt19937_64 rg(12345);
    while (true)
    {
        int x = rg() % (n - 2) + 2, y = x, c = rg() % (n - 1) + 1, d = 1;
        auto f = [&](int v) { return (mulmod(v, v, n) + c) % n; };
        while (d == 1)
```

```
        {
            int prod = 1, sx = x, sy = y; // batch 64 gcds into one; remember the batch
start
            rep(i, 0, 64)
            {
                x = f(x), y = f(f(y));
                int t = mulmod(prod, llabs(x - y), n);
                if (t == 0) { prod = 0; break; } // collided mod every factor: gcd(0,
n) = n sends us to the fallback
                prod = t;
            }
            d = __gcd(prod, n);
            if (prod == 1 || d == n) // fall back to step-by-step, replaying this batch
from its start
            {
                x = sx, y = sy, d = 1;
                while (d == 1) x = f(x), y = f(f(y)), d = __gcd(llabs(x - y), n);
            }
        }
        if (d != n) return d;
    }
}
void factorRec(int n, vi &out)
{
    if (n == 1) return;
    if (isPrime64(n)) { out.pb(n); return; }
    for (int p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
        if (n % p == 0) { out.pb(p); factorRec(n / p, out); return; }
    int d = pollard(n);
    factorRec(d, out), factorRec(n / d, out);
}
vi factor(int n) { vi f; factorRec(n, f); sort(all(f)); return f; }

vi getDivisors(int n)
{
    vi pf = factor(n);
    vi divs = {1};
    for (int i = 0; i < (int)pf.size(); )
    {
        int p = pf[i], cnt = 0;
        while (i < (int)pf.size() && pf[i] == p) cnt++, i++;
        int sz = divs.size();
        int pw = 1;
        rep(c, 0, cnt)
        {
            pw *= p;
            rep(j, 0, sz) divs.pb(divs[j] * pw);
        }
    }
    sort(all(divs)); // Optional: remove if you don't need them sorted
    return divs;
}

int phi(int n)
{
    vi pf = factor(n);
    pf.erase(unique(all(pf)), pf.end()); // Keep only distinct prime factors
    for (int p : pf) n = n / p * (p - 1);
    return n;
}
```

FILE: templates/number_theory/prime_factor_root_n

```
vpi prime_factorize(ll n) {
    vpi factors;
    for (ll d = 2; d * d <= n; d += (d == 2 ? 1 : 2)) {
        if (n % d == 0) {
            int cnt = 0;
            while (n % d == 0) {
                n /= d;
                cnt++;
            }
            factors.emplace_back(d, cnt);
        }
    }
    if (n > 1) {
        factors.emplace_back(n, 1);
    }
    return factors;
}
```

FILE: templates/number_theory/linear_sieve.cpp

```
// linear sieve up to n: primes, smallest prime factor, mobius mu, euler phi in O(n)
vi lp, primesL, muL, phiL;
void linearSieve(int n)
{
    lp.assign(n + 1, 0), muL.assign(n + 1, 0), phiL.assign(n + 1, 0), primesL.clear();
    muL[1] = 1, phiL[1] = 1;
    rep(i, 2, n + 1)
    {
        if (!lp[i]) lp[i] = i, primesL.pb(i), muL[i] = -1, phiL[i] = i - 1;
        for (int p : primesL)
        {
            if (p > lp[i] || i * p > n) break;
            lp[i * p] = p;
            if (p == lp[i]) { muL[i * p] = 0, phiL[i * p] = phiL[i] * p; }
            else { muL[i * p] = -muL[i], phiL[i * p] = phiL[i] * (p - 1); }
        }
    }
}
// segmented sieve: primes in [L, R], R up to ~1e12 with R - L up to ~1e7
vi segmentedSieve(int L, int R)
{
    int lim = sqrtl((long double)R) + 1;
    vector<char> small(lim + 1, 1), mark(R - L + 1, 1);
    vi res;
    for (int i = 2; i <= lim; i++)
    {
        if (!small[i]) continue;
        for (int j = i * i; j <= lim; j += i) small[j] = 0;
        for (int j = max(i * i, (L + i - 1) / i * i); j <= R; j += i) mark[j - L] = 0;
    }
    rep(i, max(L, 2LL), R + 1) if (mark[i - L]) res.pb(i);
    return res;
}
```

FILE: templates/number_theory/floor_blocks.cpp

```
// floor blocks: floor(n / i) takes only O(sqrt n) different values. loop over blocks
[l, r] with the same q
// sum_{i=1..n} f(i) * g(n / i): for (l, r, q) in blocks add (F(r) - F(l-1)) * g(q), F
= prefix sum of f
// example: sum_{i=1..n} n / i  in O(sqrt n)
int sumFloor(int n)
{
    int s = 0;
    for (int l = 1, r; l <= n; l = r + 1)
    {
        int q = n / l;
        r = n / q;            // last i with n / i == q
        s += (r - l + 1) * q; // all i in [l, r] give the same q
    }
    return s;
}
// ceil blocks: ceil(n / i) for i = 1..n-1: q = (n + l - 1) / l, r = (n - 1) / (q - 1)
when q > 1
// count of x in [1, R] coprime to d: inclusion-exclusion over distinct primes of d (d
up to 1e12+: get
// them with pollard_rho.cpp factor(), or trial division up to sqrt(d)). at most 15
primes for d <= 1e18
int coprimeCount(int R, const vi &ps) // ps = distinct primes of d
{
    int k = ps.size(), res = 0;
    rep(mask, 0, 1LL << k)
    {
        int prod = 1, bits = 0;
        bool over = false;
        rep(j, 0, k) if (mask >> j & 1) { bits++; if (prod > R / ps[j]) { over = true;
break; } prod *= ps[j]; }
        if (over) continue;
        res += (bits % 2 ? -1 : 1) * (R / prod);
    }
    return res; // count in [L, R] = coprimeCount(R) - coprimeCount(L - 1)
}
vi distinctPrimes(int d) { vi ps; for (int p = 2; p * p <= d; p++) if (d % p == 0) {
ps.pb(p); while (d % p == 0) d /= p; } if (d > 1) ps.pb(d); return ps; }
// floor sum: sum_{i=0}^{n-1} floor((a*i + b) / m), n >= 0, m >= 1, a, b >= 0, O(log).
lattice points under a line
int floorSum(int n, int m, int a, int b)
{
    int ans = 0;
    while (true)
    {
        if (a >= m) ans += (n - 1) * n / 2 * (a / m), a %= m;
        if (b >= m) ans += n * (b / m), b %= m;
        int y = a * n + b;
        if (y < m) break;
        n = y / m, b = y % m, swap(m, a);
    }
    return ans;
}
```

FILE: templates/number_theory/power_tower.cpp

```
// (1) a^e mod m when e is a HUGE decimal string (10^4+ digits):
//     m prime and gcd(a, m) = 1: reduce e mod (m - 1) (fermat).  any m: use
powStrGeneral (generalised euler)
// (2) generalised euler: for any a, m and e >= log2(m):  a^e = a^(e mod phi(m) +
phi(m))  (mod m)
// (3) power tower a1^(a2^(a3^...)) mod m by recursion m -> phi(m) -> phi(phi(m)) ->
... (depth O(log m))
int phiSqrt(int m) { int r = m; for (int p = 2; p * p <= m; p++) if (m % p == 0) {
while (m % p == 0) m /= p; r -= r / p; } if (m > 1) r -= r / m; return r; }
int powStrPrime(int a, const string &e, int m) // m prime, a % m != 0
{
    int r = 0;
    for (char c : e) r = (r * 10 + (c - '0')) % (m - 1);
    return binExp(a % m, r, m);
}
int powStrGeneral(int a, const string &e, int m) // any a, any m >= 1
{
    int ph = phiSqrt(m), r = 0;
    bool big = false; // is e >= ph ?
    for (char c : e) { r = r * 10 + (c - '0'); if (r >= ph) big = true, r %= ph; }
    return binExp(a % m, big ? r + ph : r, m) % m;
}
// tower: a[0]^(a[1]^(a[2]^...)) mod m, any m >= 1, any a[i] >= 0 (0^0 = 1). O(len *
sqrt(m)) for phi
const int CAPV = (int)4e18;
int powCap(int a, int e) // min(a^e, CAPV), exact below CAPV
{
    if (a == 0) return e == 0 ? 1 : 0;
    if (a == 1 || e == 0) return 1;
    __int128 v = 1;
    rep(i, 0, e) { v *= a; if (v >= CAPV) return CAPV; }
    return (int)v;
}
int towerRec(const vi &a, const vi &capv, int i, int m)
{
    if (m == 1) return 0;
    if (i == (int)a.size() - 1) return a[i] % m;
    int E = capv[i + 1], ph = phiSqrt(m);
    int ex = (E >= ph) ? towerRec(a, capv, i + 1, ph) + ph : E; // generalised euler
when the exponent is big
    return binExp(a[i] % m, ex, m) % m;
}
int tower(const vi &a, int m)
{
    int n = a.size();
    vi capv(n);
    capv[n - 1] = min(a[n - 1], CAPV);
    repb(i, n - 2, 0) capv[i] = powCap(a[i], capv[i + 1]);
    return towerRec(a, capv, 0, m);
}
```

FILE: templates/number_theory/crt.cpp

Needs: base extgcd

```
// chinese remainder theorem for any moduli (not necessarily coprime)
// crt({r1, m1}, {r2, m2}, ...) -> {r, lcm} or {-1, -1} if no solution. needs extgcd
(base template)
```

```
pi crtMerge(int r1, int m1, int r2, int m2)
{
    int x, y, g = extgcd(m1, m2, x, y);
    if ((r2 - r1) % g != 0) return {-1, -1};
    int l = m1 / g * m2;
    int k = (__int128)((r2 - r1) / g) * x % (m2 / g);
    int r = ((r1 + (__int128)m1 * k) % l + l) % l;
    return {r, l};
}
pi crt(const vpi &eqs) // eqs[i] = {remainder, modulus}
{
    pi cur = {0, 1};
    for (auto &e : eqs)
    {
        cur = crtMerge(cur.first, cur.second, ((e.first % e.second) + e.second) %
e.second, e.second);
        if (cur.first == -1) return cur;
    }
    return cur;
}
// inverse of a mod m for any m (gcd(a, m) must be 1), -1 otherwise
int invGeneral(int a, int m)
{
    int x, y, g = extgcd(((a % m) + m) % m, m, x, y);
    if (g != 1) return -1;
    return ((x % m) + m) % m;
}
```

FILE: templates/number_theory/discrete_log.cpp

```
// discrete log (baby step giant step): smallest x >= 0 with a^x = b (mod m), any m, -1
if none. O(sqrt m)
int discreteLog(int a, int b, int m)
{
    a %= m, b %= m;
    int k = 1 % m, add = 0, g;
    while ((g = __gcd(a, m)) > 1) // make a and m coprime
    {
        if (b == k) return add;
        if (b % g) return -1;
        b /= g, m /= g, add++;
        k = k * (a / g) % m;
    }
    int n = sqrtl((long double)m) + 1, an = 1;
    rep(i, 0, n) an = an * a % m;
    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; q++) vals[cur] = q, cur = cur * a % m;
    for (int p = 1, cur = k; p <= n; p++)
    {
        cur = cur * an % m;
        if (vals.count(cur)) return n * p - vals[cur] + add;
    }
    return -1;
}
```

FILE: templates/number_theory/sparse_gcd.cpp

```
// sparse table for range gcd: O(n log n) build, O(1) query rangeGCD(l, r) on 0-indexed
[l, r]
```

```
const int MAXN = 200005, MAXLOG = 18; // MAXLOG > log2(MAXN)
int n, q;
int a[MAXN];
int st[MAXN][MAXLOG];

void buildSparseTable() {
    for (int i = 0; i < n; i++)
        st[i][0] = a[i];
    for (int j = 1; (1 << j) <= n; j++)
        for (int i = 0; i + (1 << j) <= n; i++)
            st[i][j] = __gcd(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
}

int rangeGCD(int l, int r) {
    int k = log2(r - l + 1);
    return __gcd(st[l][k], st[r - (1 << k) + 1][k]);
}
```

FILE: templates/number_theory/euler_totient.cpp

Only the part of this file below the base template is printed.

Needs: base template (Mint, umap). Do not paste together with mobius.cpp section

```
// ================ Euler Totient ================
// Uses: int(=ll), vi, rep, pb, all, MOD, Mint, umap. __int128 needs GCC/Clang.
// Don't paste together with the Mobius section (both define phi / primes).

vector<int32_t> phi; // phi[i], i <= LIM (int32_t to save memory)
vi Phi, primes;      // Phi[i] = phi[1] + ... + phi[i]
void phi_sieve(int LIM) // linear sieve, O(LIM). LIM = 1e7 uses ~130 MB
{
    phi.assign(LIM + 1, 0), Phi.assign(LIM + 1, 0), primes.clear();
    vector<bool> comp(LIM + 1, false);
    if (LIM >= 1) phi[1] = 1;
    rep(i, 2, LIM + 1)
    {
        if (!comp[i]) primes.pb(i), phi[i] = i - 1;
        for (int p : primes)
        {
            if (i * p > LIM) break;
            comp[i * p] = true;
            if (i % p == 0) { phi[i * p] = phi[i] * p; break; } // p already in i
            phi[i * p] = phi[i] * (p - 1);                      // p is new
        }
    }
    rep(i, 1, LIM + 1) Phi[i] = Phi[i - 1] + phi[i];
}

int phi_single(int n) // one n up to ~1e12, O(sqrt n)
{
    int res = n;
    for (int p = 2; p * p <= n; p++)
        if (n % p == 0)
        {
            while (n % p == 0) n /= p;
            res -= res / p;
        }
    if (n > 1) res -= res / n;
    return res;
}
```

```
vi phi_segment(int L, int R) // phi(x) for x in [L, R], R up to 1e12, R-L ~ 1e6. needs
phi_sieve(>= sqrt R)
{
    int len = R - L + 1;
    vi rem(len), res(len);
    rep(i, 0, len) rem[i] = res[i] = L + i;
    for (int p : primes)
    {
        if (p * p > R) break;
        for (int x = (L + p - 1) / p * p; x <= R; x += p)
        {
            int i = x - L;
            res[i] -= res[i] / p;
            while (rem[i] % p == 0) rem[i] /= p;
        }
    }
    rep(i, 0, len) if (rem[i] > 1) res[i] -= res[i] / rem[i]; // one big prime left
    return res;
}

// ---- Euler's theorem ----
int power128(int a, int e, int m) // a^e mod m, m up to ~1e18
{
    int r = 1 % m;
    a %= m; if (a < 0) a += m;
    while (e > 0)
    {
        if (e & 1) r = (__int128)r * a % m;
        a = (__int128)a * a % m;
        e >>= 1;
    }
    return r;
}
int inv_mod_phi(int a, int m) { return power128(a, phi_single(m) - 1, m); } // needs
gcd(a, m) = 1, m any

int str_mod(const string &s, int m)
{
    int r = 0;
    for (char c : s) r = ((__int128)r * 10 + (c - '0')) % m;
    return r;
}
int pow_huge(int a, const string &b, int m) // a^b mod m, b = huge decimal string, any
a, m
{
    if (b.size() <= 2) return power128(a, stoll(b), m);
    int ph = phi_single(m);
    return power128(a, str_mod(b, ph) + ph, m); // extended Euler
}

int MODc(__int128 x, int m) { return x < m ? (int)x : (int)(x % m) + m; } // keeps "was
it >= m"
int power_m(int a, int e, int m)
{
    int r = 1;
    while (e > 0)
    {
        if (e & 1) r = MODc((__int128)r * a, m);
        a = MODc((__int128)a * a, m);
```

```
        e >>= 1;
    }
    return r;
}
// a[i]^(a[i+1]^(...)), all a >= 1. final answer = tower(a, 0, m) % m
int tower(const vi &a, int i, int m)
{
    if (m == 1 || a[i] == 1 || i + 1 == (int)a.size()) return MODc(a[i], m);
    return power_m(a[i], tower(a, i + 1, phi_single(m)), m);
}

int primitive_root(int p) // p prime, smallest generator
{
    int ph = p - 1, n = ph;
    vi fac;
    for (int q = 2; q * q <= n; q++)
        if (n % q == 0) { fac.pb(q); while (n % q == 0) n /= q; }
    if (n > 1) fac.pb(n);
    for (int g = 2; g < p; g++)
    {
        bool ok = true;
        for (int q : fac) if (power128(g, ph / q, p) == 1) { ok = false; break; }
        if (ok) return g;
    }
    return 1; // p = 2
}

// ---- gcd / lcm sums ----
vi pillai, lcmG; // pillai[n] = sum_{i<=n} gcd(i, n), lcmG[n] = sum_{d|n} d*phi(d)
void build_gcd_lcm_sums(int n) // O(n log n), needs phi_sieve(n)
{
    pillai.assign(n + 1, 0), lcmG.assign(n + 1, 0);
    rep(d, 1, n + 1) for (int m = d; m <= n; m += d)
        pillai[m] += d * phi[m / d], lcmG[m] += d * phi[d];
}
int lcm_sum(int n) { return n * (lcmG[n] + 1) / 2; } // sum_{i<=n} lcm(i, n), exact for
n <= 1e6

template <class F> int sum_f_gcd(int n, F f) // sum_{k=1..n} f(gcd(k, n)) for one n
{
    int res = 0;
    for (int d = 1; d * d <= n; d++)
        if (n % d == 0)
        {
            res += f(d) * phi_single(n / d);
            if (d != n / d) res += f(n / d) * phi_single(d);
        }
    return res;
}

int sum_gcd_rect(int n, int m) // sum_{i<=n, j<=m} gcd(i, j), O(sqrt), needs
phi_sieve(>= min(n, m))
{
    int ans = 0;
    for (int l = 1, r, lim = min(n, m); l <= lim; l = r + 1)
    {
        r = min(n / (n / l), m / (m / l));
        ans += (Phi[r] - Phi[l - 1]) * (n / l) * (m / l);
    }
    return ans;
```

```
}
int sum_gcd_i_lt_j(int n) { return (sum_gcd_rect(n, n) - n * (n + 1) / 2) / 2; } // sum
over i < j

// ---- coprime counting (square [1, n]^2 only; for rectangles use Mobius) ----
int coprime_pairs_square(int n) { return n ? 2 * Phi[n] - 1 : 0; } // ordered (a, b)
int pairs_gcd_exact(int n, int g) { return coprime_pairs_square(n / g); }
int pairs_gcd_prime(int n)
{
    int ans = 0;
    for (int p : primes) { if (p > n) break; ans += coprime_pairs_square(n / p); }
    return ans;
}

int count_coprime(int X, int n) // #x in [1, X] with gcd(x, n) = 1
{
    vi ps;
    for (int p = 2; p * p <= n; p++)
        if (n % p == 0) { ps.pb(p); while (n % p == 0) n /= p; }
    if (n > 1) ps.pb(n);
    int k = ps.size(), res = 0;
    rep(mask, 0, 1LL << k)
    {
        int prod = 1, bits = 0;
        rep(i, 0, k) if (mask >> i & 1) prod *= ps[i], bits++;
        res += (bits & 1 ? -1 : 1) * (X / prod);
    }
    return res;
}
int kth_coprime(int k, int n) // k-th positive integer coprime to n
{
    int ph = phi_single(n), full = (k - 1) / ph, r = (k - 1) % ph + 1;
    int lo = 1, hi = n;
    while (lo < hi)
    {
        int mid = (lo + hi) / 2;
        if (count_coprime(mid, n) >= r) hi = mid; else lo = mid + 1;
    }
    return full * n + lo;
}

Mint sum_coprime(int n) // sum of k <= n with gcd(k, n) = 1, mod MOD
{
    if (n == 1) return 1;
    int ph = phi_single(n);
    return n % 2 == 0 ? Mint(n / 2) * Mint(ph) : Mint(n) * Mint(ph / 2);
}

Mint necklaces(int n, int k) // n beads, k colours, rotations count once. n < MOD
{
    Mint s = 0;
    for (int d = 1; d * d <= n; d++)
        if (n % d == 0)
        {
            s += Mint(phi_single(d)) * mexp(Mint(k), n / d);
            if (d != n / d) s += Mint(phi_single(n / d)) * mexp(Mint(k), d);
        }
    return s / Mint(n);
}
```

```
// ---- advanced ----
umap<int, Mint> memoPhi;
Mint tri_mod(int n) // n(n+1)/2 mod MOD
{
    int a = n, b = n + 1;
    if (a % 2 == 0) a /= 2; else b /= 2;
    return Mint(a % MOD) * Mint(b % MOD);
}
Mint Phi_du(int n) // (phi(1)+...+phi(n)) mod MOD, n up to ~1e11. needs phi_sieve(LIM ~
n^(2/3))
{
    if (n < (int)Phi.size()) return Mint(Phi[n] % MOD);
    auto it = memoPhi.find(n);
    if (it != memoPhi.end()) return it->second;
    Mint res = tri_mod(n);
    for (int l = 2, r; l <= n; l = r + 1)
    {
        r = n / (n / l);
        res -= Mint((r - l + 1) % MOD) * Phi_du(n / l);
    }
    return memoPhi[n] = res;
}

bool is_prime_slow(int x)
{
    if (x < 2) return false;
    for (int d = 2; d * d <= x; d++) if (x % d == 0) return false;
    return true;
}
vi invCand, invSols;
void dfs_inv(int i, int rem, int cur)
{
    if (rem == 1) invSols.pb(cur);
    rep(j, i, (int)invCand.size())
    {
        int p = invCand[j];
        if (rem % (p - 1) != 0) continue;
        int r = rem / (p - 1), c = cur * p;
        while (true)
        {
            dfs_inv(j + 1, r, c);
            if (r % p != 0) break;
            r /= p, c *= p;
        }
    }
}
vi inverse_phi(int k) // all n with phi(n) = k, sorted
{
    invCand.clear(), invSols.clear();
    for (int d = 1; d * d <= k; d++)
        if (k % d == 0)
        {
            if (is_prime_slow(d + 1)) invCand.pb(d + 1);
            if (d != k / d && is_prime_slow(k / d + 1)) invCand.pb(k / d + 1);
        }
    sort(all(invCand));
    dfs_inv(0, k, 1);
    sort(all(invSols));
    return invSols;
}
```

FILE: templates/number_theory/mobius.cpp

Only the part of this file below the base template is printed.

Needs: base template sieve (spf) for sqf_divisors

```
// ================ Mobius ================
// Paste below the "Sieve" section. Uses: int(=ll), vi, vpi, rep, repb, pb, all, umap,
spf (only sqf_divisors).
// mu / phi / preMu are int32_t on purpose: sieving to 1e7 with long long would cost
~240 MB.

vector<int32_t> mu, phi, preMu; // preMu[i] = mu[1] + ... + mu[i]
vi primes;
void mobius_sieve(int LIM) // linear sieve, O(LIM). call once in main with LIM = max
value needed
{
    mu.assign(LIM + 1, 0), phi.assign(LIM + 1, 0), preMu.assign(LIM + 1, 0);
    vector<bool> comp(LIM + 1, false);
    primes.clear();
    mu[1] = phi[1] = 1;
    rep(i, 2, LIM + 1)
    {
        if (!comp[i]) primes.pb(i), mu[i] = -1, phi[i] = i - 1;
        for (int p : primes)
        {
            if (i * p > LIM) break;
            comp[i * p] = true;
            if (i % p == 0) { mu[i * p] = 0, phi[i * p] = phi[i] * p; break; } // p
repeats
            mu[i * p] = -mu[i], phi[i * p] = phi[i] * (p - 1);               // new
prime
        }
    }
    rep(i, 1, LIM + 1) preMu[i] = preMu[i - 1] + mu[i];
}

int mobius(int n) // mu of ONE big n (up to ~1e12), O(sqrt n), no sieve needed
{
    int res = 1;
    for (int p = 2; p * p <= n; p++)
        if (n % p == 0)
        {
            n /= p;
            if (n % p == 0) return 0;
            res = -res;
        }
    if (n > 1) res = -res;
    return res;
}

// squarefree divisors of x with their mu: {d, mu(d)}. at most 2^7 = 128 of them for x
<= 1e6. needs sieve()
// use: #elements coprime to x in a multiset = sum over these of mu(d) * cnt[d]
vpi sqf_divisors(int x)
{
    vpi res = {{1, 1}};
    while (x > 1)
    {
        int p = spf[x];
        while (x % p == 0) x /= p;
```

```
        int s = res.size();
        rep(i, 0, s) res.pb({res[i].first * p, -res[i].second});
    }
    return res;
}

// cnt[d] = how many a[i] are divisible by d, d = 1..M. O(M log M)
vi count_multiples(const vi &a, int M)
{
    vi freq(M + 1, 0), cnt(M + 1, 0);
    for (int x : a) freq[x]++;
    rep(d, 1, M + 1) for (int v = d; v <= M; v += d) cnt[d] += freq[v];
    return cnt;
}

// "at least" -> "exactly": input g[d] = #objects with d | gcd, output f[d] = #objects
with gcd == d
// descending sieve, O(M log M). works for int or Mint
template <class T> vector<T> exact_from_atleast(vector<T> g)
{
    int M = (int)g.size() - 1;
    repb(d, M, 1) for (int n = 2 * d; n <= M; n += d) g[d] -= g[n];
    return g;
}

// #pairs i<j with gcd(a[i], a[j]) == 1. needs mobius_sieve(>= max a)
int coprime_pairs(const vi &a)
{
    int M = *max_element(all(a));
    vi cnt = count_multiples(a, M);
    int ans = 0;
    rep(d, 1, M + 1) ans += mu[d] * (cnt[d] * (cnt[d] - 1) / 2);
    return ans;
}

// #pairs (x, y), 1<=x<=n, 1<=y<=m, gcd(x,y) == 1. divisor blocks: O(sqrt n + sqrt m)
per query
// needs mobius_sieve(>= min(n, m)). for gcd == k call coprime_pairs_range(n / k, m /
k)
int coprime_pairs_range(int n, int m)
{
    if (n > m) swap(n, m);
    int ans = 0;
    for (int l = 1, r; l <= n; l = r + 1)
    {
        r = min(n / (n / l), m / (m / l)); // n/d and m/d stay constant for d in [l, r]
        ans += (int)(preMu[r] - preMu[l - 1]) * (n / l) * (m / l);
    }
    return ans;
}

// #squarefree numbers in [1, n]. needs mobius_sieve(>= sqrt n)
int count_squarefree(int n)
{
    int ans = 0;
    for (int d = 1; d * d <= n; d++) ans += mu[d] * (n / (d * d));
    return ans;
}

// sum of gcd(i, j) over ALL ordered pairs 1<=i,j<=n. needs mobius_sieve(n)
```

```
// for i<j only: (sum_gcd_all_pairs(n) - n*(n+1)/2) / 2
int sum_gcd_all_pairs(int n)
{
    int ans = 0;
    rep(d, 1, n + 1) ans += (int)phi[d] * (n / d) * (n / d);
    return ans;
}

// Dirichlet convolution h[n] = sum_{d | n} f[d] * g[n / d], indices 1..M. O(M log M).
int or Mint
template <class T> vector<T> dirichlet(const vector<T> &f, const vector<T> &g)
{
    int M = (int)f.size() - 1;
    vector<T> h(M + 1, T(0));
    rep(i, 1, M + 1) for (int j = 1; i * j <= M; j++) h[i * j] += f[i] * g[j];
    return h;
}

// Mertens M(n) = mu[1] + ... + mu[n] for n up to ~1e11 (Du's sieve)
// needs mobius_sieve(LIM) with LIM about n^(2/3) (1e6 for n = 1e9, 5e6 for n = 1e10)
umap<int, int> memoMertens;
int mertens(int n)
{
    if (n < (int)preMu.size()) return preMu[n];
    auto it = memoMertens.find(n);
    if (it != memoMertens.end()) return it->second;
    int res = 1; // sum_{i=1..n} M(n / i) = 1
    for (int l = 2, r; l <= n; l = r + 1)
    {
        r = n / (n / l);
        res -= (r - l + 1) * mertens(n / l);
    }
    return memoMertens[n] = res;
}
```

17.3 Combinatorics

KEYWORDS: combinatorics template code

FILE: templates/combinatorics/combinatorics.cpp

Only the part of this file below the base template is printed.

Needs: base template factorials (call compute_factorials())

```
// ================ Combinatorics ================
// Call compute_factorials() first. Everything is mod MOD (must be prime) unless a
modulus is passed.
// Every function returns 0 for impossible arguments (r < 0, r > n, negative totals).

// ---- basics ----
int modInv(int a) { return binExp((a % MOD + MOD) % MOD, MOD - 2); } // a must be non-
zero mod MOD
int invNum(int n) { return invFact[n] * fact[n - 1] % MOD; }           // 1/n for 1 <=
n < N, O(1)
int npr(int n, int r) { return (r < 0 || r > n) ? 0 : fact[n] * invFact[n - r] % MOD; }
int multinomial(const vi &cnt)                                          // (sum c)! /
prod c!
{
    int s = 0, res = 1;
    for (int c : cnt) s += c, res = res * invFact[c] % MOD;
```

```
    return fact[s] * res % MOD;
}
int ncrBig(int n, int r)                                                    // n up to
1e18, r < N and r < MOD
{
    if (r < 0 || r > n) return 0;
    int num = 1;
    rep(i, 0, r) num = num * ((n - i) % MOD) % MOD;
    return num * invFact[r] % MOD;
}
vector<vector<int32_t>> pascal(int K, int m)                               // C[n][r] for
n <= K, ANY modulus m
{                                                                           // int32
storage: K = 5000 is ~100 MB
    vector<vector<int32_t>> C(K + 1, vector<int32_t>(K + 1, 0));
    rep(i, 0, K + 1)
    {
        C[i][0] = 1 % m;
        rep(j, 1, i + 1) C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % m;
    }
    return C;
}
struct Lucas                                                                // C(n, r) mod
small prime p, n, r up to 1e18
{
    int p; vi f, fi;
    Lucas(int p_) : p(p_), f(p_), fi(p_)
    {
        f[0] = 1;
        rep(i, 1, p) f[i] = f[i - 1] * i % p;
        fi[p - 1] = binExp(f[p - 1], p - 2, p);
        repb(i, p - 1, 1) fi[i - 1] = fi[i] * i % p;
    }
    int small(int n, int r) { return r > n ? 0 : f[n] * fi[r] % p * fi[n - r] % p; }
    int C(int n, int r)
    {
        if (r < 0 || r > n) return 0;
        int res = 1;
        while ((n || r) && res) res = res * small(n % p, r % p) % p, n /= p, r /= p;
        return res;
    }
};

// ---- stars and bars ----
int starsBars(int n, int k)                                                 // x1+..+xk =
n, xi >= 0
{
    if (n < 0 || k < 0) return 0;
    if (k == 0) return n == 0;
    return ncr(n + k - 1, k - 1);
}
int starsBarsPos(int n, int k) { return starsBars(n - k, k); }             // xi >= 1
int starsBarsUpper(int n, int k, int u)                                     // 0 <= xi <=
u (inclusion-exclusion)
{
    int res = 0;
    for (int j = 0; j <= k && j * (u + 1) <= n; j++)
    {
        int term = ncr(k, j) * starsBars(n - j * (u + 1), k) % MOD;
        res = (j & 1) ? (res - term + MOD) % MOD : (res + term) % MOD;
```

```
    }
    return res;
}

// ---- lattice paths, Catalan ----
int paths(int dx, int dy) { return (dx < 0 || dy < 0) ? 0 : ncr(dx + dy, dx); } //
right dx, up dy
int catalan(int n) { return (ncr(2 * n, n) - ncr(2 * n, n + 1) + MOD) % MOD; }
int dyckFrom(int k, int b)                                                  // k steps of
+-1 from height b to 0, never below 0
{                                                                           // =
completions of a bracket prefix with balance b
    if (b < 0 || b > k || (k + b) % 2) return 0;
    int d = (k + b) / 2;
    return (ncr(k, d) - ncr(k, d + 1) + MOD) % MOD;
}
int gridPathsBlocked(int h, int w, vpi blocked)                             // (1,1) ->
(h,w) right/down, avoid blocked; O(k^2)
{
    sort(all(blocked));
    blocked.pb({h, w});
    int k = blocked.size();
    vi f(k);
    rep(i, 0, k)
    {
        f[i] = paths(blocked[i].first - 1, blocked[i].second - 1);
        rep(j, 0, i) if (blocked[j].second <= blocked[i].second)
            f[i] = (f[i] - f[j] * paths(blocked[i].first - blocked[j].first,
blocked[i].second - blocked[j].second) % MOD + MOD) % MOD;
    }
    return f[k - 1];
}

// ---- inclusion-exclusion families ----
vi derangements(int n)                                                      // D[0..n]
{
    vi D(max(n + 1, 2LL));
    D[0] = 1, D[1] = 0;
    rep(i, 2, n + 1) D[i] = (i - 1) % MOD * ((D[i - 1] + D[i - 2]) % MOD) % MOD;
    return D;
}
int surjections(int n, int k)                                               // onto maps n
-> k = k! * S2(n, k), O(k log n)
{
    int res = 0;
    rep(j, 0, k + 1)
    {
        int term = ncr(k, j) * binExp(k - j, n) % MOD;
        res = (j & 1) ? (res - term + MOD) % MOD : (res + term) % MOD;
    }
    return res;
}
int stirling2(int n, int k) { return (k < 0 || k > n) ? 0 : surjections(n, k) *
invFact[k] % MOD; } // single value
int countDivisibleByAny(int n, const vi &d)                                 // #x in [1,
n] divisible by some d[i]; |d| <= ~20, exact
{
    int k = d.size(), total = 0;
    rep(mask, 1, 1LL << k)
    {
```

```
        int l = 1;
        rep(i, 0, k) if (mask >> i & 1)
        {
            l /= __gcd(l, d[i]);
            if (l > n / d[i]) { l = n + 1; break; }                     // lcm too
large: term is 0
            l *= d[i];
        }
        total += (__builtin_popcountll(mask) & 1 ? 1 : -1) * (n / l);
    }
    return total;
}

// ---- Stirling, Bell, partitions (O(n^2) tables) ----
vvi stirling2Table(int n)                                               // S[i][j]: i
labelled items into j unlabelled non-empty blocks
{
    vvi S(n + 1, vi(n + 1, 0));
    S[0][0] = 1;
    rep(i, 1, n + 1) rep(j, 1, i + 1) S[i][j] = (j * S[i - 1][j] + S[i - 1][j - 1]) %
MOD;
    return S;
}
vvi stirling1Table(int n)                                               // c[i][j]:
permutations of i with j cycles (unsigned)
{
    vvi S(n + 1, vi(n + 1, 0));
    S[0][0] = 1;
    rep(i, 1, n + 1) rep(j, 1, i + 1) S[i][j] = ((i - 1) * S[i - 1][j] + S[i - 1][j -
1]) % MOD;
    return S;
}
vi bell(int n)                                                          // B[0..n] via
Bell triangle, O(n^2)
{
    vi B(n + 1), row(1, 1);
    B[0] = 1;
    rep(i, 1, n + 1)
    {
        vi nr(i + 1);
        nr[0] = row[i - 1];
        rep(j, 1, i + 1) nr[j] = (nr[j - 1] + row[j - 1]) % MOD;
        B[i] = nr[0];
        row = nr;
    }
    return B;
}
vi partitions(int n)                                                    // p[0..n],
order of parts ignored
{
    vi p(n + 1, 0);
    p[0] = 1;
    rep(part, 1, n + 1) rep(s, part, n + 1) p[s] = (p[s] + p[s - part]) % MOD;
    return p;
}
vvi partitionsExactK(int n)                                             // q[k][s]:
partitions of s into exactly k parts
{                                                                       // (into at
most k parts = q[k][s + k])
    vvi q(n + 1, vi(n + 1, 0));
```

```
    q[0][0] = 1;
    rep(k, 1, n + 1) rep(s, k, n + 1) q[k][s] = (q[k - 1][s - 1] + q[k][s - k]) % MOD;
    return q;
}

// ---- permutations ----
vi inversionCounts(int n, int K)                                        // dp[k] =
#perms of n with k inversions, O(nK)
{
    vi dp(K + 1, 0);
    dp[0] = 1;
    rep(i, 1, n + 1)
    {
        vi pre(K + 2, 0), nd(K + 1);
        rep(k, 0, K + 1) pre[k + 1] = (pre[k] + dp[k]) % MOD;
        rep(k, 0, K + 1) nd[k] = (pre[k + 1] - pre[max(0LL, k - i + 1)] + MOD) % MOD;
        dp = nd;
    }
    return dp;
}
int countCycles(const vi &p)                                            // p is a 0-
indexed permutation
{
    int n = p.size(), c = 0;
    vector<bool> seen(n, false);
    rep(i, 0, n) if (!seen[i]) { c++; for (int j = i; !seen[j]; j = p[j]) seen[j] =
true; }
    return c;
}

// ---- Burnside ----
int phiOf(int n)                                                        // Euler phi,
O(sqrt n)
{
    int r = n;
    for (int p = 2; p * p <= n; p++) if (n % p == 0) { while (n % p == 0) n /= p; r -=
r / p; }
    if (n > 1) r -= r / n;
    return r;
}
int necklaces(int n, int k)                                             // k colours,
rotations equal; n up to 1e12
{
    int s = 0;
    for (int d = 1; d * d <= n; d++) if (n % d == 0)
    {
        s = (s + phiOf(n / d) % MOD * binExp(k, d)) % MOD;
        if (d * d != n) s = (s + phiOf(d) % MOD * binExp(k, n / d)) % MOD;
    }
    return s * modInv(n) % MOD;
}
int bracelets(int n, int k)                                             // rotations +
reflections equal
{
    int refl = (n & 1) ? n % MOD * binExp(k, (n + 1) / 2) % MOD
                       : (n / 2) % MOD * ((binExp(k, n / 2 + 1) + binExp(k, n / 2)) %
MOD) % MOD;
    int rot = necklaces(n, k) * (n % MOD) % MOD;
    return (rot + refl) % MOD * modInv(2 * n) % MOD;
}
```

```
// ---- trees (vertices 0..n-1) ----
int cayley(int n) { return n <= 2 ? 1 : binExp(n, n - 2); }             // labelled
trees
int forests(int n, int k) { return k == n ? 1 : k % MOD * binExp(n, n - k - 1) % MOD; }
// k trees, vertices 0..k-1 in different trees
int treesWithDegrees(const vi &deg)                                      // (n-2)! /
prod (d_i - 1)!
{
    int n = deg.size(), s = 0, res = fact[max(0LL, n - 2)];
    if (n == 1) return deg[0] == 0;
    for (int d : deg) { if (d < 1) return 0; s += d - 1; res = res * invFact[d - 1] %
MOD; }
    return s == n - 2 ? res : 0;
}
vpi pruferDecode(const vi &code)                                         // code length
n-2 -> n-1 edges, O(n)
{
    int n = code.size() + 2, ptr = 0;
    vi deg(n, 1);
    for (int x : code) deg[x]++;
    while (deg[ptr] != 1) ptr++;
    int leaf = ptr;
    vpi edges;
    for (int v : code)
    {
        edges.pb({leaf, v});
        if (--deg[v] == 1 && v < ptr) leaf = v;
        else { ptr++; while (deg[ptr] != 1) ptr++; leaf = ptr; }
    }
    edges.pb({leaf, n - 1});
    return edges;
}
vi pruferEncode(int n, const vpi &edges)                                 // tree ->
code of length n-2, O(n)
{
    vvi adj(n);
    for (auto &e : edges) adj[e.first].pb(e.second), adj[e.second].pb(e.first);
    vi par(n, -1), st(1, n - 1);
    par[n - 1] = n - 1;
    while (!st.empty())
    {
        int u = st.back(); st.pop_back();
        for (int v : adj[u]) if (par[v] == -1) par[v] = u, st.pb(v);
    }
    vi deg(n), code;
    rep(i, 0, n) deg[i] = adj[i].size();
    int ptr = 0;
    while (ptr < n && deg[ptr] != 1) ptr++;
    int leaf = ptr;
    rep(i, 0, n - 2)
    {
        int nxt = par[leaf];
        code.pb(nxt);
        if (--deg[nxt] == 1 && nxt < ptr) leaf = nxt;
        else { ptr++; while (deg[ptr] != 1) ptr++; leaf = ptr; }
    }
    return code;
}
```

```
// ---- number theory for counting ----
vi mobius(int n)                                                    // mu[0..n],
linear sieve
{
    vi mu(n + 1, 0), primes;
    vector<bool> comp(n + 1, false);
    if (n >= 1) mu[1] = 1;
    rep(i, 2, n + 1)
    {
        if (!comp[i]) primes.pb(i), mu[i] = -1;
        for (int p : primes)
        {
            if (i * p > n) break;
            comp[i * p] = true;
            if (i % p == 0) { mu[i * p] = 0; break; }
            mu[i * p] = -mu[i];
        }
    }
    return mu;
}

// ---- polynomials / generating functions ----
void ntt(vi &a, bool invert)                                        // MOD must be
998244353 (primitive root 3)
{
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++)
    {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1)
    {
        int w = binExp(3, (MOD - 1) / len);
        if (invert) w = binExp(w, MOD - 2);
        for (int i = 0; i < n; i += len)
        {
            int wn = 1;
            rep(j, 0, len / 2)
            {
                int u = a[i + j], v = a[i + j + len / 2] * wn % MOD;
                a[i + j] = u + v < MOD ? u + v : u + v - MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
                wn = wn * w % MOD;
            }
        }
    }
    if (invert) { int ni = binExp(n, MOD - 2); for (int &x : a) x = x * ni % MOD; }
}
vi multiply(vi a, vi b)                                             // polynomial
product; naive if small or MOD != 998244353
{
    if (a.empty() || b.empty()) return vi();
    int need = a.size() + b.size() - 1;
    if (min(a.size(), b.size()) <= 32 || MOD != 998244353)
    {
        vi c(need, 0);
        rep(i, 0, (int)a.size()) rep(j, 0, (int)b.size()) c[i + j] = (c[i + j] + a[i] *
```

```
b[j]) % MOD;
        return c;
    }
    int sz = 1;
    while (sz < need) sz <<= 1;
    a.resize(sz), b.resize(sz);
    ntt(a, false), ntt(b, false);
    rep(i, 0, sz) a[i] = a[i] * b[i] % MOD;
    ntt(a, true);
    a.resize(need);
    return a;
}
```

FILE: templates/combinatorics/catalan.cpp

```
// catalan numbers C_n = C(2n, n) / (n + 1), O(1) each after compute_factorials()
(needs 2n < N)
int catalanNum(int n) { return ncr(2 * n, n) * binExp(n + 1, MOD - 2) % MOD; }
// table for small n with any modulus (no inverses), O(n^2)
vi catalanTable(int n, int m = MOD)
{
    vi c(n + 1, 0);
    c[0] = 1;
    rep(i, 1, n + 1) rep(j, 0, i) c[i] = (c[i] + c[j] * c[i - 1 - j]) % m;
    return c;
}
```

FILE: templates/combinatorics/de_bruijn.cpp

```
// de bruijn sequence over m symbols, windows of length n (fkm algorithm)
// set n, m, need, a.assign(n + 1, 0), call db(1, 1); seq has the cyclic sequence
(stops after need symbols)
// de bruijn sequence (fkm algorithm), stops early once 'need' symbols are generated
int n, m, need;
vi a, seq;
void db(int t, int p)
{
    if ((int)seq.size() >= need) return;
    if (t > n)
    {
        if (n % p == 0) rep(j, 1, p + 1) seq.pb(a[j]);
        return;
    }
    a[t] = a[t - p];
    db(t + 1, p);
    rep(j, a[t - p] + 1, m)
    {
        a[t] = j;
        db(t + 1, t);
    }
}
```

17.4 Math: matrices, polynomials, fractions, xor basis

KEYWORDS: math: matrices, polynomials, fractions, xor basis template code

FILE: templates/math/matrix.cpp

```
// matrix power mod MOD: linear recurrences with n up to 1e18. Mat = vector<vi>
typedef vector<vi> Mat;
Mat matMul(const Mat &A, const Mat &B, int m = MOD)
{
    int n = A.size(), k = B.size(), p = B[0].size();
    Mat C(n, vi(p, 0));
    rep(i, 0, n) rep(t, 0, k) if (A[i][t])
        rep(j, 0, p) C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % m;
    return C;
}
Mat matPow(Mat A, int e, int m = MOD)
{
    int n = A.size();
    Mat R(n, vi(n, 0));
    rep(i, 0, n) R[i][i] = 1 % m;
    while (e) { if (e & 1) R = matMul(R, A, m); A = matMul(A, A, m); e >>= 1; }
    return R;
}
// fibonacci: F(n) = matPow({{1, 1}, {1, 0}}, n)[0][1]
```

FILE: templates/math/lagrange.cpp

```
// lagrange interpolation: y[i] = f(i) for i = 0..k (f has degree <= k), returns f(x)
mod MOD in O(k)
// needs fact / invFact (compute_factorials) with k < N. example: sum of i^p for i <= n
is a degree p+1 polynomial
int lagrange(const vi &y, int x)
{
    int k = y.size() - 1;
    x %= MOD;
    if (x >= 0 && x <= k) return y[x];
    vi pre(k + 2, 1), suf(k + 2, 1);
    rep(i, 0, k + 1) pre[i + 1] = pre[i] * ((x - i) % MOD + MOD) % MOD;
    repb(i, k, 0) suf[i] = suf[i + 1] * ((x - i) % MOD + MOD) % MOD;
    int res = 0;
    rep(i, 0, k + 1)
    {
        int t = y[i] * pre[i] % MOD * suf[i + 1] % MOD * invFact[i] % MOD * invFact[k -
i] % MOD;
        res = ((k - i) & 1) ? (res - t + MOD) % MOD : (res + t) % MOD;
    }
    return res;
}
```

FILE: templates/math/gauss.cpp

Needs: base binExp

```
// gaussian elimination mod a prime p: solve A x = b. returns 0 (no solution), 1
(unique), 2 (infinitely many)
// x gets one solution. needs binExp
int gaussMod(vector<vi> a, vi b, vi &x, int p = MOD)
{
    int n = a.size(), m = a[0].size();
    rep(i, 0, n) a[i].pb(b[i]);
    vi where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; col++)
    {
```

```
        int sel = -1;
        rep(i, row, n) if (a[i][col]) { sel = i; break; }
        if (sel == -1) continue;
        swap(a[sel], a[row]);
        where[col] = row;
        int inv = binExp(a[row][col], p - 2, p);
        rep(j, 0, m + 1) a[row][j] = a[row][j] * inv % p;
        rep(i, 0, n) if (i != row && a[i][col])
        {
            int f = a[i][col];
            rep(j, 0, m + 1) a[i][j] = ((a[i][j] - f * a[row][j]) % p + p) % p;
        }
        row++;
    }
    x.assign(m, 0);
    rep(j, 0, m) if (where[j] != -1) x[j] = a[where[j]][m];
    rep(i, 0, n)
    {
        int s = 0;
        rep(j, 0, m) s = (s + a[i][j] * x[j]) % p;
        if (s != a[i][m]) return 0;
    }
    rep(j, 0, m) if (where[j] == -1) return 2;
    return 1;
}
// gaussian elimination with doubles: same idea, pick the largest pivot (partial
pivoting), EPS = 1e-9
// xor basis (linear algebra over bits): insert numbers, max xor of a subset, check
representability
struct XorBasis
{
    int b[63] = {0}, rnk = 0;
    bool insert(int x)
    {
        repb(i, 62, 0) if (x >> i & 1)
        {
            if (!b[i]) { b[i] = x, rnk++; return true; }
            x ^= b[i];
        }
        return false; // x was already representable
    }
    bool canMake(int x) { repb(i, 62, 0) if (x >> i & 1) { if (!b[i]) return false; x
^= b[i]; } return true; }
    int maxXor(int x = 0) { repb(i, 62, 0) x = max(x, x ^ b[i]); return x; }
};
```

FILE: templates/math/xor_basis.cpp

```
// xor basis (linear basis over GF(2)) for numbers < 2^B
// insert(x): true if x was independent. canMake(x): is x a xor of some subset.
maxXor(): best subset xor
// number of distinct subset xors = 2^rank. k-th smallest subset xor: reduce the basis,
then use bits of k
struct XorBasis
{
    static const int B = 61;
    int b[B] = {0}, rank = 0;
    bool insert(int x)
    {
        repb(i, B - 1, 0) if (x >> i & 1)
```

```
        {
            if (!b[i]) { b[i] = x, rank++; return true; }
            x ^= b[i];
        }
        return false;
    }
    bool canMake(int x) { repb(i, B - 1, 0) if (x >> i & 1) { if (!b[i]) return false;
x ^= b[i]; } return true; }
    int maxXor(int x = 0) { repb(i, B - 1, 0) x = max(x, x ^ b[i]); return x; } // max
of x ^ (subset xor)
    int minXor(int x) { repb(i, B - 1, 0) x = min(x, x ^ b[i]); return x; }
    int kth(int k) // k-th smallest (0-based) among the 2^rank distinct subset xors (0
included), k < 2^rank
    {
        vi v;
        repb(i, B - 1, 0) if (b[i]) rep(j, i + 1, B) if (b[j] >> i & 1) b[j] ^= b[i];
// reduce
        rep(i, 0, B) if (b[i]) v.pb(b[i]);
        int r = 0;
        rep(i, 0, (int)v.size()) if (k >> i & 1) r ^= v[i];
        return r;
    }
};
```

FILE: templates/math/fraction.cpp

Needs: base binExp (only for modval)

```
// exact fraction p/q, always reduced, q > 0. __int128 inside so products of 1e18-size
values don't overflow
// Frac a(3, 4), b(1, 6); a + b, a - b, a * b, a / b, a < b, a == b; cout << a  prints
"p/q"
// for "p/q" output also when q = 1, keep print as is; if the judge wants just "p" when
q = 1, change print
struct Frac
{
    __int128 p, q;
    static __int128 g(__int128 a, __int128 b) { if (a < 0) a = -a; if (b < 0) b = -b;
while (b) { __int128 t = a % b; a = b, b = t; } return a; }
    Frac(__int128 a = 0, __int128 b = 1)
    {
        if (b < 0) a = -a, b = -b;
        __int128 d = g(a, b);
        if (d == 0) d = 1;
        p = a / d, q = b / d;
    }
    Frac operator+(const Frac &o) const { return Frac(p * o.q + o.p * q, q * o.q); }
    Frac operator-(const Frac &o) const { return Frac(p * o.q - o.p * q, q * o.q); }
    Frac operator*(const Frac &o) const { return Frac(p * o.p, q * o.q); }
    Frac operator/(const Frac &o) const { return Frac(p * o.q, q * o.p); } // o must
not be 0
    bool operator<(const Frac &o) const { return p * o.q < o.p * q; }
    bool operator==(const Frac &o) const { return p == o.p && q == o.q; }
    long double val() const { return (long double)p / (long double)q; }
    int modval(int m = MOD) const { return (int)(((p % m) + m) % m) * binExp((int)(q %
m), m - 2, m) % m; } // p * q^-1 mod prime m
};
string i128(__int128 x) { if (x == 0) return "0"; bool neg = x < 0; if (neg) x = -x;
string s; while (x) s += char('0' + x % 10), x /= 10; if (neg) s += '-';
reverse(all(s)); return s; }
```

```
ostream &operator<<(ostream &os, const Frac &f) { return os << i128(f.p) << "/" <<
i128(f.q); }
```

FILE: templates/math/linear_equations_2x2.cpp

```
pi solveEq(int a1, int b1, int n1, int a2, int b2, int n2)
{
    // a1x + b1y = n1
    // a2x + b2y = n2
    int det = a1 * b2 - a2 * b1;
    if (det == 0)
    {
        // check if they are consistent
        if (a1 * n2 == a2 * n1)
        {
            // inf solutions
            return {0, 0};
        }
        else
        {
            // no solution
            return {-1, -1};
        }
    }
    else
    {
        if((n1*b2 - n2*b1) % det != 0 || (a1*n2 - a2*n1) % det != 0)
            return {-1,-1};
        int x = (n1 * b2 - n2 * b1) / det;
        int y = (a1 * n2 - a2 * n1) / det;
        if(x<0 || y<0)
            return {-1,-1};
        return {x, y};
    }
}
```

FILE: templates/math/fft.cpp

```
// fft with complex doubles: exact integer convolution while results stay below ~1e15
(n up to ~1e6 with small values)
// multiplying huge numbers: put digits in a, b, convolve, then carry
typedef complex<double> cd;
void fft(vector<cd> &a, bool inv)
{
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++)
    {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1)
    {
        double ang = 2 * acos(-1.0) / len * (inv ? -1 : 1);
        cd wl(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len)
        {
            cd w(1);
```

```
            rep(j, 0, len / 2)
            {
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v, a[i + j + len / 2] = u - v;
                w *= wl;
            }
        }
    }
    if (inv) for (auto &x : a) x /= n;
}
vi convolve(const vi &a, const vi &b)
{
    if (a.empty() || b.empty()) return {};
    int n = 1;
    while (n < (int)(a.size() + b.size())) n <<= 1;
    vector<cd> fa(all(a)), fb(all(b));
    fa.resize(n), fb.resize(n);
    fft(fa, false), fft(fb, false);
    rep(i, 0, n) fa[i] *= fb[i];
    fft(fa, true);
    vi res(a.size() + b.size() - 1);
    rep(i, 0, (int)res.size()) res[i] = llround(fa[i].real());
    return res;
}
```

FILE: templates/math/ntt.cpp

```
// ntt mod 998244353: multiply(a, b) returns the product polynomial
// example use: 103119A Accelerator (product of (1 + a_i x) by divide and conquer)
const int md = 998244353, ntt_root = 3;
void ntt(vi &a, bool inv)
{
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++)
    {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1)
    {
        int w = binExp(ntt_root, (md - 1) / len, md);
        if (inv) w = binExp(w, md - 2, md);
        for (int i = 0; i < n; i += len)
        {
            int wn = 1;
            rep(j, 0, len / 2)
            {
                int u = a[i + j], v = a[i + j + len / 2] * wn % md;
                a[i + j] = (u + v) % md, a[i + j + len / 2] = (u - v + md) % md;
                wn = wn * w % md;
            }
        }
    }
    if (inv)
    {
        int ninv = binExp(n, md - 2, md);
        for (auto &x : a) x = x * ninv % md;
    }
```

```
}
vi multiply(vi a, vi b)
{
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <<= 1;
    if (min(a.size(), b.size()) < 32) // small: plain multiplication
    {
        vi c(need, 0);
        rep(i, 0, (int)a.size()) rep(j, 0, (int)b.size()) c[i + j] = (c[i + j] + a[i] *
b[j]) % md;
        return c;
    }
    a.resize(n), b.resize(n);
    ntt(a, false), ntt(b, false);
    rep(i, 0, n) a[i] = a[i] * b[i] % md;
    ntt(a, true);
    a.resize(need);
    return a;
}
```

FILE: templates/math/exact_conv.cpp

```
// ntt for any ntt-friendly prime (root 3) + exact convolution of non-negative values
using two primes and crt
// conv(a, b) is exact while every result is below about 4e17
void ntt(vi &a, bool inv, int md)
{
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++)
    {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1)
    {
        int w = binExp(3, (md - 1) / len, md);
        if (inv) w = binExp(w, md - 2, md);
        for (int i = 0; i < n; i += len)
        {
            int wn = 1;
            rep(j, 0, len / 2)
            {
                int u = a[i + j], v = a[i + j + len / 2] * wn % md;
                a[i + j] = (u + v) % md, a[i + j + len / 2] = (u - v + md) % md;
                wn = wn * w % md;
            }
        }
    }
    if (inv)
    {
        int ninv = binExp(n, md - 2, md);
        for (auto &x : a) x = x * ninv % md;
    }
}
vi conv(vi a, vi b)
{
    const int m1 = 998244353, m2 = 469762049;
    int need = a.size() + b.size() - 1, n = 1;
```

```
    while (n < need) n <<= 1;
    vi r1, r2;
    for (int md : {m1, m2})
    {
        vi x = a, y = b;
        x.resize(n), y.resize(n);
        ntt(x, false, md), ntt(y, false, md);
        rep(i, 0, n) x[i] = x[i] * y[i] % md;
        ntt(x, true, md);
        (md == m1 ? r1 : r2) = x;
    }
    vi res(need);
    int inv = binExp(m1 % m2, m2 - 2, m2);
    rep(i, 0, need) res[i] = r1[i] + (r2[i] - r1[i] % m2 + m2) % m2 * inv % m2 * m1;
    return res;
}
```

FILE: templates/math/fwht.cpp

```
// fast walsh hadamard transform (xor convolution)
// c = a xor-conv b: fwht(a,0), fwht(b,0), c[i] = a[i]*b[i], fwht(c,1)
void fwht(vi &a, bool inv)
{
    int n = a.size();
    for (int len = 1; len < n; len <<= 1)
        for (int i = 0; i < n; i += 2 * len)
            rep(j, i, i + len)
            {
                int u = a[j], v = a[j + len];
                a[j] = u + v, a[j + len] = u - v;
            }
    if (inv) for (auto &x : a) x /= n;
}
```

17.5 Bits, helpers, digit DP, games, testing

KEYWORDS: bits, helpers, digit dp, games, testing template code

FILE: templates/misc/bit_tricks.cpp

```
// bit identities:  a + b = (a ^ b) + 2 (a & b)      a + b = (a | b) + (a & b)      a ^
b = (a | b) - (a & b)
// min over d of (a ^ d) + (b ^ d) = a ^ b              popcount:
__builtin_popcountll(x)
// highest bit: 63 - __builtin_clzll(x) (x > 0)        lowest set bit: x & -x
trailing zeros: __builtin_ctzll(x)
// subsets of mask: for (int s = mask; s > 0; s = (s - 1) & mask)   (add s = 0
separately)
// OR / AND of a growing subarray changes at most ~60 times -> keep the list of
distinct values ending at i

// max over i, j of (a[i] & b[j]) in O((n + m) * B): greedy from the high bit, keep
only candidates having all chosen bits
int maxAndPair(vi a, vi b, int B = 30)
{
    int ans = 0;
    repb(bit, B - 1, 0)
    {
        int want = ans | (1LL << bit);
        vi na, nb;
```

```
        for (int x : a) if ((x & want) == want) na.pb(x);
        for (int x : b) if ((x & want) == want) nb.pb(x);
        if (!na.empty() && !nb.empty()) ans = want, a = na, b = nb;
    }
    return ans;
}
// max (a[i] & a[j]) with i != j inside ONE array: same idea, need at least 2
candidates
int maxAndPairOne(vi a, int B = 30)
{
    int ans = 0;
    repb(bit, B - 1, 0)
    {
        int want = ans | (1LL << bit);
        vi na;
        for (int x : a) if ((x & want) == want) na.pb(x);
        if (na.size() >= 2) ans = want, a = na;
    }
    return ans;
}
// max xor pair: insert into BinTrie (binary_trie.cpp), answer max over x of
t.maxXor(x) before inserting x
// sum over all pairs i < j of (a[i] ^ a[j]): per bit, ones * zeros * 2^bit
int sumPairXor(const vi &a, int B = 31)
{
    int n = a.size(), s = 0;
    rep(bit, 0, B) { int c = 0; for (int x : a) c += x >> bit & 1; s += c * (n - c) *
(1LL << bit); }
    return s;
}
// SOS dp (sum over subsets / supersets): sosSubsets, sosSupersets in misc/common.cpp
```

FILE: templates/misc/common.cpp

```
// small everyday helpers

// coordinate compression: vals sorted unique, idx = lower_bound position
vi compress(vi &a) // replaces a[i] by its rank (0-based), returns the sorted unique
values
{
    vi v = a;
    sort(all(v)), v.erase(unique(all(v)), v.end());
    for (int &x : a) x = lower_bound(all(v), x) - v.begin();
    return v;
}

// longest strictly increasing subsequence in O(n log n). for non-decreasing use
upper_bound
int lis(const vi &a)
{
    vi d;
    for (int x : a)
    {
        auto it = lower_bound(all(d), x);
        if (it == d.end()) d.pb(x); else *it = x;
    }
    return d.size();
}

// binary search on the answer: smallest x in [lo, hi] with ok(x) true (ok is monotone
```

```
false...true), hi + 1 if none
template <class F> int firstTrue(int lo, int hi, F ok)
{
    hi++;
    while (lo < hi)
    {
        int mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}

// ternary search for the max of a unimodal function on integers [lo, hi]
template <class F> int ternaryMax(int lo, int hi, F f)
{
    while (hi - lo > 2)
    {
        int m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1; else hi = m2;
    }
    int best = lo;
    rep(x, lo, hi + 1) if (f(x) > f(best)) best = x;
    return best;
}
// on doubles: repeat 100 times with m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3

// bit tricks
// for (int s = mask; s; s = (s - 1) & mask)  -> every non-empty submask of mask (3^n
total over all masks)
// __builtin_popcountll(x), __builtin_ctzll(x) (lowest set bit index), 63 -
__builtin_clzll(x) (highest)
// sum over subsets (SOS dp): f[mask] = sum of a[sub] over all sub of mask, O(n 2^n)
void sosSubsets(vi &f, int n)
{
    rep(i, 0, n) rep(mask, 0, 1LL << n) if (mask >> i & 1) f[mask] += f[mask ^ (1LL <<
i)];
}
void sosSupersets(vi &f, int n) // f[mask] = sum over supersets
{
    rep(i, 0, n) rep(mask, 0, 1LL << n) if (!(mask >> i & 1)) f[mask] += f[mask | (1LL
<< i)];
}

// prefix sums 2d: P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] - P[i][j]
// rectangle (r1, c1)..(r2, c2) = P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]

// __int128 input / output
__int128 read128()
{
    string s;
    cin >> s;
    __int128 x = 0;
    bool neg = s[0] == '-';
    for (char c : s) if (isdigit(c)) x = x * 10 + (c - '0');
    return neg ? -x : x;
}
string str128(__int128 x)
{
    if (x == 0) return "0";
    bool neg = x < 0;
```

```
    string s;
    while (x) s += char('0' + (neg ? -(x % 10) : x % 10)), x /= 10;
    if (neg) s += '-';
    reverse(all(s));
    return s;
}
```

FILE: templates/misc/fast_io.cpp

```
// very fast input for 1e6+ numbers or huge strings (faster than cin with sync off).
// use readInt() everywhere instead of cin >> for numbers. do not mix with cin on the
same input
static char ibuf[1 << 25];
size_t ipos = 0, ilen = 0;
inline int gc() { if (ipos == ilen) { ilen = fread(ibuf, 1, sizeof(ibuf), stdin); ipos
= 0; if (!ilen) return -1; } return ibuf[ipos++]; }
inline int readInt()
{
    int c = gc();
    while (c != '-' && (c < '0' || c > '9')) c = gc();
    bool neg = false;
    if (c == '-') neg = true, c = gc();
    int x = 0;
    while (c >= '0' && c <= '9') x = x * 10 + (c - '0'), c = gc();
    return neg ? -x : x;
}
inline string readToken() { string s; int c = gc(); while (c == ' ' || c == '\n' || c
== '\r') c = gc(); while (c != -1 && c != ' ' && c != '\n' && c != '\r') s += char(c),
c = gc(); return s; }
// output: build one string / use '\n' not endl, and printf / cout with sync off is
fine
```

FILE: templates/misc/digit_dp.cpp

```
// digit dp skeleton: count x in [0, N] with some digit property (example: digit sum ==
S)
// count(R) - count(L - 1) for a range. memo over (pos, state) only when not tight
string digs;
int S;
int memo[20][200];
bool seen[20][200];
int go(int pos, int sum, bool tight)
{
    if (sum > S) return 0;
    if (pos == (int)digs.size()) return sum == S;
    if (!tight && seen[pos][sum]) return memo[pos][sum];
    int lim = tight ? digs[pos] - '0' : 9, res = 0;
    rep(d, 0, lim + 1) res += go(pos + 1, sum + d, tight && d == lim);
    if (!tight) seen[pos][sum] = 1, memo[pos][sum] = res;
    return res;
}
int countUpTo(int N) // numbers in [0, N] with digit sum S
{
    if (N < 0) return 0;
    digs = to_string(N);
    memset(seen, 0, sizeof seen);
    return go(0, 0, true);
}
// add a "started" flag to the state when leading zeros matter
```

FILE: templates/misc/grundy_brute.cpp

```
// game theory toolkit
// sprague-grundy: g(state) = mex of g(next states). position is LOST (for the player
to move) iff g = 0
// several independent games: xor the grundy values. nim: xor of pile sizes
// unknown game with huge N: brute force g(n) for small n, PRINT the table, then find
the pattern
// (period, parity, powers of two, n mod k ...). This is exactly how to attack "N <=
1e16" games.
int mexOf(vi v) { sort(all(v)); int m = 0; for (int x : v) { if (x == m) m++; else if
(x > m) break; } return m; }
void grundyTable(int upTo)
{
    vi g(upTo + 1, 0);
    rep(n, 0, upTo + 1)
    {
        vi nxt;
        // ---- write the moves of the game here. example: take 1, 2 or 3 stones ----
        rep(k, 1, 4) if (n - k >= 0) nxt.pb(g[n - k]);
        g[n] = mexOf(nxt);
        cout << n << ": " << g[n] << (g[n] ? "  win" : "  LOSE") << nl;
    }
}
// splitting games (a move cuts a segment into two parts): g(n) = mex over moves of
g(left) ^ g(right)
// misere nim (last mover LOSES): if all piles are 1, first wins iff number of piles is
even; else same as normal nim
// win/lose without grundy (single game, small state): win[s] = exists move to a losing
state
```

FILE: templates/misc/stress.sh

```
#!/bin/bash
# stress test: compares sol.cpp with brute.cpp on random inputs from gen.cpp
# gen.cpp reads a seed from argv[1] and prints one test. usage: bash stress.sh 1000
g++ -std=c++14 -O2 -o sol sol.cpp || exit
g++ -std=c++14 -O2 -o brute brute.cpp || exit
g++ -std=c++14 -O2 -o gen gen.cpp || exit
for ((i = 1; i <= ${1:-1000}; i++)); do
    ./gen $i > in.txt
    ./sol < in.txt > out1.txt
    ./brute < in.txt > out2.txt
    if ! cmp -s out1.txt out2.txt; then
        echo "mismatch on test $i"; cat in.txt
        echo "sol:"; cat out1.txt
        echo "brute:"; cat out2.txt
        exit
    fi
done
echo "all $1 tests passed"
```

FILE: templates/misc/gen_example.cpp

Standalone file (has its own includes / main): copy the needed part.

```
// random test generator for stress.sh (seed from argv[1])
#include <bits/stdc++.h>
using namespace std;
int main(int argc, char **argv)
```

```
{
    mt19937 rng(atoi(argv[1]));
    auto rnd = [&](int l, int r) { return (int)(rng() % (r - l + 1)) + l; };
    int n = rnd(1, 8);
    cout << n << "\n";
    for (int i = 0; i < n; i++) cout << rnd(1, 10) << " \n"[i == n - 1];
}
```

17.6 Dynamic programming

KEYWORDS: dynamic programming template code

FILE: templates/dp/dp_patterns.cpp

```
// common dp patterns (LIS is in misc/common.cpp: lis(a))

// 0/1 knapsack, max value with weight <= W, O(nW). loop w DOWN so each item is used
once (UP = unlimited items)
int knapsack(const vi &wt, const vi &val, int W)
{
    vi dp(W + 1, 0);
    rep(i, 0, (int)wt.size()) repb(w, W, wt[i]) dp[w] = max(dp[w], dp[w - wt[i]] +
val[i]);
    return dp[W];
}
// subset sum reachability with bitset, O(n S / 64). which sums of a subset are
possible
const int MAXS = 100001;
bitset<MAXS> subsetSums(const vi &a) { bitset<MAXS> b; b[0] = 1; for (int x : a) b |= b
<< x; return b; }
// count ways to reach each sum (mod MOD), each item once
vi countSubsetSums(const vi &a, int S)
{
    vi dp(S + 1, 0);
    dp[0] = 1;
    for (int x : a) repb(s, S, x) dp[s] = (dp[s] + dp[s - x]) % MOD;
    return dp;
}
// bitmask dp, travelling salesman: dp[mask][v] = min cost to visit mask, ending at v,
start at 0. O(2^n n^2), n <= 20
int tsp(const vvi &c) // c[u][v] cost, returns min cost of a cycle through all vertices
from 0
{
    int n = c.size();
    vvi dp(1LL << n, vi(n, LLONG_MAX / 4));
    dp[1][0] = 0;
    rep(mask, 1, 1LL << n) rep(v, 0, n) if ((mask >> v & 1) && dp[mask][v] < LLONG_MAX
/ 4)
        rep(u, 0, n) if (!(mask >> u & 1)) dp[mask | (1LL << u)][u] = min(dp[mask |
(1LL << u)][u], dp[mask][v] + c[v][u]);
    int best = LLONG_MAX;
    rep(v, 0, n) best = min(best, dp[(1LL << n) - 1][v] + c[v][0]);
    return best;
}
// interval dp: dp[l][r] from smaller intervals. loop by length. example: merge stones,
cost = sum of the merged part
int mergeCost(const vi &a)
{
    int n = a.size();
    vi pre(n + 1, 0);
```

```
    rep(i, 0, n) pre[i + 1] = pre[i] + a[i];
    vvi dp(n, vi(n, 0));
    rep(len, 2, n + 1) rep(l, 0, n - len + 1)
    {
        int r = l + len - 1;
        dp[l][r] = LLONG_MAX;
        rep(k, l, r) dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + pre[r + 1] -
pre[l]);
    }
    return dp[0][n - 1];
}
// longest common subsequence O(nm)
int lcs(const string &a, const string &b)
{
    int n = a.size(), m = b.size();
    vvi dp(n + 1, vi(m + 1, 0));
    rep(i, 1, n + 1) rep(j, 1, m + 1) dp[i][j] = a[i - 1] == b[j - 1] ? dp[i - 1][j -
1] + 1 : max(dp[i - 1][j], dp[i][j - 1]);
    return dp[n][m];
}
// max subarray sum (kadane), non-empty
int kadane(const vi &a) { int best = LLONG_MIN, cur = 0; for (int x : a) cur = max(x,
cur + x), best = max(best, cur); return best; }
// dp over a path split into k blocks with cost(l, r): dp[j][i] = min over p of dp[j-1]
[p] + cost(p, i-1). O(k n^2)
// if cost is "sum |x - median|" of a sorted block: median m = (l + r) / 2, use prefix
sums of x
```

17.7 Data structures

KEYWORDS: data structures template code

FILE: templates/data_structures/dsu.cpp

```
// dsu with union by size + path compression. DSU d(n); d.find(x); d.unite(a, b);
d.size(x); d.comps
struct DSU
{
    vi p, sz;
    int comps;
    DSU(int n) : p(n), sz(n, 1), comps(n) { iota(all(p), 0); }
    int find(int x)
    {
        while (x != p[x]) x = p[x] = p[p[x]];
        return x;
    }
    bool unite(int a, int b) // false if already together
    {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a, sz[a] += sz[b], comps--;
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return sz[find(x)]; }
};
```

FILE: templates/data_structures/dsu_parity.cpp

```
// dsu with parity: unite(a, b, d) forces side(a) xor side(b) = d, returns false on
contradiction
// ParityDSU d(n); d.find(v) = {root, side of v relative to root}. iterative, union by
size
struct ParityDSU
{
    vi p, sz, rel; // rel = side of v relative to p[v] (0 same, 1 different)
    ParityDSU(int n) : p(n), sz(n, 1), rel(n, 0) { iota(all(p), 0); }
    pi find(int v)
    {
        int r = v, x = 0;
        while (p[r] != r) x ^= rel[r], r = p[r];
        int cur = x; // compress: every node on the path points to r
        while (p[v] != v)
        {
            int nxt = p[v], c = rel[v];
            p[v] = r, rel[v] = cur;
            cur ^= c, v = nxt;
        }
        return {r, x};
    }
    bool unite(int a, int b, int d)
    {
        pi A = find(a), B = find(b);
        if (A.first == B.first) return (A.second ^ B.second) == d;
        if (sz[A.first] < sz[B.first]) swap(A, B);
        p[B.first] = A.first, rel[B.first] = A.second ^ B.second ^ d, sz[A.first] +=
sz[B.first];
        return true;
    }
};
```

FILE: templates/data_structures/dsu_rollback.cpp

```
// dsu with rollback (no path compression): unite, then undo the last union with
rollback()
// use: offline dynamic connectivity (segment tree over time), divide and conquer
struct RollbackDSU
{
    vi p, sz;
    vector<pi> hist; // {attached root, old size of the other root}, {-1,-1} for a no-
op
    int comps;
    RollbackDSU(int n) : p(n), sz(n, 1), comps(n) { iota(all(p), 0); }
    int find(int x) { while (x != p[x]) x = p[x]; return x; }
    bool unite(int a, int b)
    {
        a = find(a), b = find(b);
        if (a == b) { hist.pb({-1, -1}); return false; }
        if (sz[a] < sz[b]) swap(a, b);
        hist.pb({b, sz[a]});
        p[b] = a, sz[a] += sz[b], comps--;
        return true;
    }
    void rollback()
    {
        pi h = hist.back();
```

```
        hist.pop_back();
        if (h.first == -1) return;
        int b = h.first, a = p[b];
        sz[a] = h.second, p[b] = b, comps++;
    }
};
```

FILE: templates/data_structures/dsu_offline_edge_deletion.cpp

Needs: data_structures/dsu.cpp above it

```
// offline edge deletions: answer "number of components after each deletion" by adding
edges back in reverse
// input: n m q, m edges (1-indexed), q edge indices to delete (1-indexed). uses DSU
from dsu.cpp
vi offlineDeletions(int n, const vpi &edges, const vi &del) // del = 0-based edge ids
in deletion order
{
    int m = edges.size(), q = del.size();
    vector<char> removed(m, 0);
    for (int id : del) removed[id] = 1;
    DSU d(n + 1);
    int comps = n;
    rep(i, 0, m) if (!removed[i] && d.unite(edges[i].first, edges[i].second)) comps--;
    vi ans(q);
    repb(i, q - 1, 0)
    {
        ans[i] = comps; // components after the i-th deletion
        if (d.unite(edges[del[i]].first, edges[del[i]].second)) comps--;
    }
    return ans;
}
```

FILE: templates/data_structures/segment_tree/point_update.cpp

```
// iterative segment tree: point set, range query (any associative op). SegTree st(n);
st.set(i, v); st.query(l, r) inclusive
struct SegTree
{
    int n;
    vi t;
    const int ID = 0;                                   // identity: 0 for sum, LLONG_MIN
for max, LLONG_MAX for min
    static int op(int a, int b) { return a + b; }     // change to max / min / __gcd
    SegTree(int n) : n(n), t(2 * n, 0) {}
    SegTree(const vi &a) : n(a.size()), t(2 * a.size())
    {
        rep(i, 0, n) t[n + i] = a[i];
        repb(i, n - 1, 1) t[i] = op(t[2 * i], t[2 * i + 1]);
    }
    void set(int i, int v)
    {
        for (t[i += n] = v; i > 1; i >>= 1) t[i >> 1] = op(t[i & ~1LL], t[i | 1]);
    }
    int query(int l, int r) // result of op over [l, r]; op must be commutative here
    {
        int res = ID;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) res = op(res, t[l++]);
```

```
            if (r & 1) res = op(res, t[--r]);
        }
        return res;
    }
};
```

FILE: templates/data_structures/segment_tree/range_update_lazy_sum.cpp

```
// lazy segment tree: range add, range sum. LazySum st(n) (or LazySum st(a)); st.add(l,
r, v); st.sum(l, r), 0-based inclusive
struct LazySum
{
    int n;
    vi t, lz;
    LazySum(int n) : n(n), t(4 * n, 0), lz(4 * n, 0) {}
    LazySum(const vi &a) : LazySum(a.size()) { build(1, 0, n - 1, a); }
    void build(int x, int l, int r, const vi &a)
    {
        if (l == r) { t[x] = a[l]; return; }
        int m = (l + r) / 2;
        build(2 * x, l, m, a), build(2 * x + 1, m + 1, r, a);
        t[x] = t[2 * x] + t[2 * x + 1];
    }
    void apply(int x, int l, int r, int v) { t[x] += v * (r - l + 1), lz[x] += v; }
    void push(int x, int l, int r)
    {
        if (!lz[x]) return;
        int m = (l + r) / 2;
        apply(2 * x, l, m, lz[x]), apply(2 * x + 1, m + 1, r, lz[x]);
        lz[x] = 0;
    }
    void add(int ql, int qr, int v, int x = 1, int l = 0, int r = -1)
    {
        if (r == -1) r = n - 1;
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { apply(x, l, r, v); return; }
        push(x, l, r);
        int m = (l + r) / 2;
        add(ql, qr, v, 2 * x, l, m), add(ql, qr, v, 2 * x + 1, m + 1, r);
        t[x] = t[2 * x] + t[2 * x + 1];
    }
    int sum(int ql, int qr, int x = 1, int l = 0, int r = -1)
    {
        if (r == -1) r = n - 1;
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return t[x];
        push(x, l, r);
        int m = (l + r) / 2;
        return sum(ql, qr, 2 * x, l, m) + sum(ql, qr, 2 * x + 1, m + 1, r);
    }
};
```

FILE: templates/data_structures/segment_tree/range_add_max.cpp

```
// segment tree: range add, global max is seg[0]
// sz = size, seg.assign(4*sz, 0), lz.assign(4*sz, 0), add(0, 0, sz-1, l, r, v)
int sz;
vi seg, lz;
void add(int idx, int l, int r, int ql, int qr, int v)
```

```
{
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr)
    {
        seg[idx] += v, lz[idx] += v;
        return;
    }
    int mid = (l + r) / 2;
    add(2 * idx + 1, l, mid, ql, qr, v);
    add(2 * idx + 2, mid + 1, r, ql, qr, v);
    seg[idx] = max(seg[2 * idx + 1], seg[2 * idx + 2]) + lz[idx];
}
```

FILE: templates/data_structures/segment_tree/custom_node.cpp

```
// segment tree with a CUSTOM node and merge (point update, range query). the merge
does not need to be
// commutative. change Node, ID(), leaf(), merge(). example below: bracket matching (CF
380C style):
// longest correct bracket subsequence in s[l..r] = 2 * (matched pairs)
struct Node { int match, open, close; }; // matched pairs, unmatched '(' , unmatched
')'
Node ID() { return {0, 0, 0}; }
Node leaf(char c) { return c == '(' ? Node{0, 1, 0} : Node{0, 0, 1}; }
Node merge(const Node &a, const Node &b)
{
    int t = min(a.open, b.close); // left's open brackets close with right's close
brackets
    return {a.match + b.match + t, a.open + b.open - t, a.close + b.close - t};
}
struct SegCustom
{
    int n;
    vector<Node> t;
    SegCustom(const string &s) : n(s.size()), t(4 * s.size()) { build(1, 0, n - 1, s);
}
    void build(int x, int l, int r, const string &s)
    {
        if (l == r) { t[x] = leaf(s[l]); return; }
        int m = (l + r) / 2;
        build(2 * x, l, m, s), build(2 * x + 1, m + 1, r, s);
        t[x] = merge(t[2 * x], t[2 * x + 1]);
    }
    void update(int i, char c, int x = 1, int l = 0, int r = -1)
    {
        if (r < 0) r = n - 1;
        if (l == r) { t[x] = leaf(c); return; }
        int m = (l + r) / 2;
        if (i <= m) update(i, c, 2 * x, l, m); else update(i, c, 2 * x + 1, m + 1, r);
        t[x] = merge(t[2 * x], t[2 * x + 1]);
    }
    Node query(int ql, int qr, int x = 1, int l = 0, int r = -1) // left-to-right order
is kept
    {
        if (r < 0) r = n - 1;
        if (qr < l || r < ql) return ID();
        if (ql <= l && r <= qr) return t[x];
        int m = (l + r) / 2;
        return merge(query(ql, qr, 2 * x, l, m), query(ql, qr, 2 * x + 1, m + 1, r));
    }
```

```
};
// other useful nodes: max subarray sum {sum, best prefix, best suffix, best}; count +
min; hash of a range
```

FILE: templates/data_structures/segment_tree/lazy_segment_tree_generic.cpp

Standalone file (has its own includes / main): copy the needed part.

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define nl '\n'
#define rep(i,a,b) for(int i=a;i<b;i++)

//================ Segment Tree ================
// change ONLY: node, identity(), makeLeaf(), merge(), applyNode()
const int maxN = 200005;
int a[maxN];

struct node{
    int sum, mx;
};
node seg[4*maxN];
int lazy[4*maxN];//pending update for children (seg[idx] itself is already updated)

//doesn't affect merge -> returned when completely outside
node identity(){
    return {0, LLONG_MIN};
}

//leaf node from a single value
node makeLeaf(int val){
    return {val, val};
}

//combine left and right child
node merge(node L, node R){
    node res;
    res.sum = L.sum+R.sum;
    res.mx = max(L.mx, R.mx);
    return res;
}

//apply "add val to every element of [l,r]" on this node
void applyNode(int idx, int l, int r, int val){
    seg[idx].sum += (r-l+1)*val;
    seg[idx].mx += val;
    lazy[idx] += val;//children still need it
}

//push pending laziness to children
void push(int idx, int l, int r){
    if(lazy[idx]!=0 && l!=r){
        int mid = l+(r-l)/2;
        applyNode(2*idx+1, l, mid, lazy[idx]);
        applyNode(2*idx+2, mid+1, r, lazy[idx]);
    }
    lazy[idx]=0;
}
```

```
void build(int idx, int l, int r){
    lazy[idx]=0;
    if(l==r){
        seg[idx]=makeLeaf(a[l]);
        return;
    }
    int mid = l+(r-l)/2;
    build(2*idx+1, l, mid);//left child
    build(2*idx+2, mid+1, r);//right child
    seg[idx]=merge(seg[2*idx+1], seg[2*idx+2]);
}

//set a[pos] = val
void pointUpdate(int idx, int l, int r, int pos, int val){
    if(l==r){
        seg[idx]=makeLeaf(val);
        return;
    }
    push(idx, l, r);
    int mid = l+(r-l)/2;
    if(pos<=mid) pointUpdate(2*idx+1, l, mid, pos, val);
    else pointUpdate(2*idx+2, mid+1, r, pos, val);
    seg[idx]=merge(seg[2*idx+1], seg[2*idx+2]);
}

//add val to [ql,qr]
void rangeUpdate(int idx, int l, int r, int ql, int qr, int val){
    if(l>qr || r<ql) return;//completely outside
    if(l>=ql && r<=qr){
        applyNode(idx, l, r, val);//completely inside
        return;
    }
    push(idx, l, r);//overlapping
    int mid = l+(r-l)/2;
    rangeUpdate(2*idx+1, l, mid, ql, qr, val);
    rangeUpdate(2*idx+2, mid+1, r, ql, qr, val);
    seg[idx]=merge(seg[2*idx+1], seg[2*idx+2]);
}

node query(int idx, int l, int r, int ql, int qr){
    if(l>qr || r<ql) return identity();//completely outside
    if(l>=ql && r<=qr) return seg[idx];//completely inside
    push(idx, l, r);
    int mid = l+(r-l)/2;
    return merge(query(2*idx+1, l, mid, ql, qr),
                 query(2*idx+2, mid+1, r, ql, qr));
}

//================ Code starts here ================
void solve()
{
  int n;
  cin >>n;
  rep(i,0,n){
    cin>>a[i];
  }
  build(0,0,n-1);
  int q;
  cin >>q;
  while(q--){
```

```
    int type;
    cin >>type;
    if(type==1){//add val to [l,r]
        int l,r,val;
        cin >>l>>r>>val;
        l--,r--;//0 based
        rangeUpdate(0,0,n-1,l,r,val);
    }
    else if(type==2){//set a[pos]=val
        int pos,val;
        cin >>pos>>val;
        pos--;
        pointUpdate(0,0,n-1,pos,val);
    }
    else{//sum and max of [l,r]
        int l,r;
        cin >>l>>r;
        l--,r--;
        node ans = query(0,0,n-1,l,r);
        cout<<ans.sum<<" "<<ans.mx<<nl;
    }
  }
}

signed main()
{
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  solve();
  return 0;
}
```

FILE: templates/data_structures/segment_tree/segment_tree_max.cpp

```
//segment tree implementation
const int maxN = 100005;
int a[maxN], seg[4*maxN];
void build(int idx, int l, int r){
    if(l==r){
        seg[idx]=a[l];
        return;
    }
    int mid = l+(r-l)/2;
    build(2*idx+1, l,mid);//left child
    build(2*idx+2, mid+1,r);//right child
    seg[idx]=max(seg[2*idx+1],seg[2*idx+2]);
    return;
}
int query(int idx, int l, int r, int ql, int qr){
    if(l>=ql && r<=qr){
        return seg[idx];//completely inside
    }
    if(l>qr || r<ql){
        return LLONG_MIN;//completely outside
    }
    int mid = l+(r-l)/2;
    int left = query(2*idx+1, l,mid, ql, qr);
    int right = query(2*idx+2, mid+1,r, ql, qr);
    return max(left,right);
}
```

```
//================ Code starts here ================
void solve()
{
  int n;
  cin >>n;
  rep(i,0,n){
    cin>>a[i];
  }
  build(0,0,n-1);//build(index, l_range, r_range)
  int q;
  cin >>q;
  while(q--){
    int l,r;
    cin >>l>>r;
    l--,r--;
    cout<<query(0,0,n-1,l,r)<<nl;//search in whole space
  }
}
```

FILE: templates/data_structures/fenwick.cpp

```
// fenwick tree (BIT), 0-based outside. BIT b(n); b.add(i, v); b.sum(r) = a[0..r];
b.sum(l, r)
// b.lowerBound(k): smallest i with a[0..i] >= k (all values non-negative), n if none
struct BIT
{
    int n;
    vi t;
    BIT(int n) : n(n), t(n + 1, 0) {}
    void add(int i, int v) { for (i++; i <= n; i += i & -i) t[i] += v; }
    int sum(int r) { int s = 0; for (r++; r > 0; r -= r & -r) s += t[r]; return s; }
    int sum(int l, int r) { return l > r ? 0 : sum(r) - sum(l - 1); }
    int lowerBound(int k)
    {
        int pos = 0, LOG = 1;
        while ((LOG << 1) <= n) LOG <<= 1;
        for (int step = LOG; step; step >>= 1)
            if (pos + step <= n && t[pos + step] < k) pos += step, k -= t[pos];
        return pos; // 0-based index
    }
};
// range add + range sum with two BITs. RangeBIT b(n); b.add(l, r, v); b.sum(l, r)
struct RangeBIT
{
    BIT b1, b2;
    RangeBIT(int n) : b1(n + 1), b2(n + 1) {}
    void add(int l, int r, int v) { b1.add(l, v), b1.add(r + 1, -v), b2.add(l, v * l),
b2.add(r + 1, -v * (r + 1)); }
    int pref(int r) { return b1.sum(r) * (r + 1) - b2.sum(r); }
    int sum(int l, int r) { return pref(r) - (l ? pref(l - 1) : 0); }
};
// 2d fenwick, point add, rectangle sum (0-based, inclusive)
struct BIT2D
{
    int n, m;
    vvi t;
    BIT2D(int n, int m) : n(n), m(m), t(n + 1, vi(m + 1, 0)) {}
    void add(int x, int y, int v) { for (int i = x + 1; i <= n; i += i & -i) for (int j
= y + 1; j <= m; j += j & -j) t[i][j] += v; }
    int sum(int x, int y) { int s = 0; for (int i = x + 1; i > 0; i -= i & -i) for (int
```

```
j = y + 1; j > 0; j -= j & -j) s += t[i][j]; return s; }
    int sum(int x1, int y1, int x2, int y2) { return sum(x2, y2) - sum(x1 - 1, y2) -
sum(x2, y1 - 1) + sum(x1 - 1, y1 - 1); }
};
// inversion count: compress values, then for each element count bigger ones already
seen
```

FILE: templates/data_structures/sparse_table.cpp

```
// sparse table, O(1) idempotent range query (min / max / gcd). SparseTable st(a);
st.query(l, r) inclusive
struct SparseTable
{
    vvi t;
    static int op(int a, int b) { return min(a, b); } // change to max / __gcd
    SparseTable(const vi &a)
    {
        int n = a.size(), k = 64 - __builtin_clzll(max(n, 1LL));
        t.assign(k, vi(n));
        t[0] = a;
        rep(j, 1, k) rep(i, 0, n - (1LL << j) + 1) t[j][i] = op(t[j - 1][i], t[j - 1][i
+ (1LL << (j - 1))]);
    }
    int query(int l, int r)
    {
        int j = 63 - __builtin_clzll(r - l + 1);
        return op(t[j][l], t[j][r - (1LL << j) + 1]);
    }
};
```

FILE: templates/data_structures/monotonic_stack.cpp

```
// previous / next smaller element with a stack, O(n). -1 / n when none
// prevLess[i] = last j < i with a[j] < a[i];  nextLessEq[i] = first j > i with a[j] <=
a[i]
// (strict on one side, non-strict on the other, so equal values are counted exactly
once)
void prevNext(const vi &a, vi &pl, vi &nl_)
{
    int n = a.size();
    pl.assign(n, -1), nl_.assign(n, n);
    vi st;
    rep(i, 0, n)
    {
        while (!st.empty() && a[st.back()] >= a[i]) nl_[st.back()] = i, st.pop_back();
        pl[i] = st.empty() ? -1 : st.back();
        st.pb(i);
    }
}
// sum over all subarrays of min(subarray): a[i] is the min of (i - pl[i]) * (nl[i] -
i) subarrays
int sumSubarrayMin(const vi &a)
{
    vi pl, nx;
    prevNext(a, pl, nx);
    int s = 0;
    rep(i, 0, (int)a.size()) s += a[i] * (i - pl[i]) * (nx[i] - i); // use % MOD if it
can overflow
    return s;
```

```
}
// sum of max: negate the array, take sumSubarrayMin, negate back
int sumSubarrayMax(vi a) { for (int &x : a) x = -x; return -sumSubarrayMin(a); }
// largest rectangle in a histogram: for each bar, width = nextLess - prevLess - 1 (use
strict on both sides
// or the version above: it still gives the right maximum)
int largestRectangle(const vi &h)
{
    vi pl, nx;
    prevNext(h, pl, nx);
    int best = 0;
    rep(i, 0, (int)h.size()) best = max(best, h[i] * (nx[i] - pl[i] - 1));
    return best;
}
// sliding window minimum (window k): deque of indices with increasing values
vi slidingMin(const vi &a, int k)
{
    deque<int> dq;
    vi res;
    rep(i, 0, (int)a.size())
    {
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.pb(i);
        if (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) res.pb(a[dq.front()]);
    }
    return res;
}
```

FILE: templates/data_structures/interval_assign.cpp

```
// interval assignment ("chtholly tree" / ODT): the array is kept as runs [l, r] of
equal value in a map
// assign(l, r, v) merges [l, r] into one run. with random queries / mostly assigns it
stays small.
// guaranteed fast if every operation that walks runs is an assign on the same range
(runs only drop)
// ODT t(n, init); t.assign(l, r, v); t.sum(l, r); for any other query walk
split(l)..split(r+1)
struct ODT
{
    map<int, int> mp; // run start -> value, runs cover [0, n)
    int n;
    ODT(int n, int init = 0) : n(n) { mp[0] = init, mp[n] = 0; } // sentinel at n
    map<int, int>::iterator split(int pos) // make pos the start of a run, return it
    {
        auto it = mp.lower_bound(pos);
        if (it != mp.end() && it->first == pos) return it;
        --it;
        return mp.insert({pos, it->second}).first;
    }
    void assign(int l, int r, int v) // 0-based inclusive
    {
        auto itr = split(r + 1), itl = split(l);
        mp.erase(itl, itr);
        mp[l] = v;
    }
    int sum(int l, int r)
    {
        auto itr = split(r + 1), itl = split(l);
```

```
        int s = 0;
        for (auto it = itl; it != itr; ++it) s += it->second * (next(it)->first - it-
>first);
        return s;
    }
    int runs(int l, int r) { auto itr = split(r + 1), itl = split(l); return
distance(itl, itr); } // runs inside
};
```

FILE: templates/data_structures/ordered_set.cpp

Standalone file (has its own includes / main): copy the needed part.

Needs: the pb_ds #include lines ABOVE "#define int ll" (uncomment them in the base)

```
// policy based ordered set. the pb_ds #includes MUST stay above "#define int ll"
(uncomment them in the base template),
// otherwise the library breaks. paste only the two lines below.
// os.order_of_key(x) = number of elements < x, *os.find_by_order(k) = k-th smallest
(0-based)
template <class T> using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
template <class T> using ordered_multiset = tree<pair<T, int>, null_type, less<pair<T,
int>>, rb_tree_tag, tree_order_statistics_node_update>; // {value, unique id}
```

FILE: templates/data_structures/binary_trie.cpp

```
// binary trie on numbers < 2^B (BinTrie t(40) for values up to 1e12): insert / erase
(multiset) / max xor with x / min xor with x / kth smallest / countLess / countXorLess
/ values() sorted
struct BinTrie
{
    int B;
    vector<array<int, 2>> nx;
    vi cnt;
    BinTrie(int bits = 30) : B(bits) { nx.pb({0, 0}), cnt.pb(0); }
    void insert(int x, int d = 1) // d = -1 erases one copy
    {
        int v = 0;
        cnt[v] += d;
        repb(b, B - 1, 0)
        {
            int c = x >> b & 1;
            if (!nx[v][c]) { nx[v][c] = nx.size(); nx.pb({0, 0}), cnt.pb(0); }
            v = nx[v][c];
            cnt[v] += d;
        }
    }
    int maxXor(int x) // trie must be non-empty
    {
        int v = 0, res = 0;
        repb(b, B - 1, 0)
        {
            int c = x >> b & 1;
            if (nx[v][c ^ 1] && cnt[nx[v][c ^ 1]] > 0) v = nx[v][c ^ 1], res |= 1LL <<
b;
            else v = nx[v][c];
        }
        return res;
    }
    int minXor(int x)
```

```
    {
        int v = 0, res = 0;
        repb(b, B - 1, 0)
        {
            int c = x >> b & 1;
            if (nx[v][c] && cnt[nx[v][c]] > 0) v = nx[v][c];
            else v = nx[v][c ^ 1], res |= 1LL << b;
        }
        return res;
    }
    // ---------- walks down the trie ----------
    int kth(int k) // k-th smallest stored value (1-indexed, copies count), k <= cnt[0]
    {
        int v = 0, res = 0;
        repb(b, B - 1, 0)
        {
            int l = nx[v][0] ? cnt[nx[v][0]] : 0;
            if (k <= l) v = nx[v][0];
            else k -= l, v = nx[v][1], res |= 1LL << b;
        }
        return res;
    }
    int countLess(int x) // stored values < x
    {
        if (x >> B) return cnt[0];
        int v = 0, res = 0;
        repb(b, B - 1, 0)
        {
            int c = x >> b & 1;
            if (c && nx[v][0]) res += cnt[nx[v][0]]; // whole 0-subtree is smaller
            v = nx[v][c];
            if (!v) break;
        }
        return res;
    }
    int countXorLess(int x, int k) // stored y with (x ^ y) < k, x < 2^B
    {
        if (k >> B) return cnt[0];
        int v = 0, res = 0;
        repb(b, B - 1, 0)
        {
            int c = x >> b & 1, kb = k >> b & 1;
            if (kb && nx[v][c]) res += cnt[nx[v][c]]; // xor bit 0 here while k has 1:
all smaller
            v = nx[v][c ^ kb];
            if (!v) break;
        }
        return res;
    }
    void walk(int v, int b, int val, vi &out) // all stored values in sorted order
(recursive, depth B)
    {
        if (b < 0) { rep(i, 0, cnt[v]) out.pb(val); return; }
        rep(c, 0, 2) if (nx[v][c] && cnt[nx[v][c]] > 0) walk(nx[v][c], b - 1, val | (c
<< b), out);
    }
    vi values() { vi out; if (cnt[0] > 0) walk(0, B - 1, 0, out); return out; }
};
```

FILE: templates/data_structures/trie.cpp

```
// trie on lowercase strings, arrays instead of pointers (fast, no leaks). node 0 =
root (empty string).
// Trie t; t.insert(s); t.erase(s) (multiset); t.count(s); t.prefixCount(p); t.find(p)
= node or -1
// pass[v] = words in v's subtree, ends[v] = words ending exactly at v, dep/par/ch
rebuild the string of v
// traversals: words() = all words sorted (dfs), kth(k) = k-th smallest word, levels()
= nodes by depth (bfs),
// euler() = tin/tout so v's subtree is the range [tin[v], tout[v]] (subtree queries
with a segment tree)
struct Trie
{
    static const int A = 26;
    vector<array<int, A>> nx;
    vi pass, ends, dep, par, ch;
    Trie() { newNode(-1, -1); }
    int newNode(int p, int c)
    {
        nx.pb({}), nx.back().fill(-1);
        pass.pb(0), ends.pb(0), dep.pb(p < 0 ? 0 : dep[p] + 1), par.pb(p), ch.pb(c);
        return nx.size() - 1;
    }
    int size() { return nx.size(); }
    void insert(const string &s, int d = 1) // d = -1 removes one copy (word must
exist)
    {
        int v = 0;
        pass[v] += d;
        for (char c : s)
        {
            int x = c - 'a';
            if (nx[v][x] < 0) { int u = newNode(v, x); nx[v][x] = u; }
            v = nx[v][x], pass[v] += d;
        }
        ends[v] += d;
    }
    void erase(const string &s) { if (count(s)) insert(s, -1); }
    int find(const string &p) // node spelling p, or -1
    {
        int v = 0;
        for (char c : p)
        {
            v = nx[v][c - 'a'];
            if (v < 0 || pass[v] == 0) return -1;
        }
        return v;
    }
    int count(const string &s) { int v = find(s); return v < 0 ? 0 : ends[v]; }      //
copies of s
    int prefixCount(const string &p) { int v = find(p); return v < 0 ? 0 : pass[v]; }
// words starting with p
    int lcpWith(const string &s) // longest prefix of s that is a prefix of some word
    {
        int v = 0, d = 0;
        for (char c : s)
        {
            int u = nx[v][c - 'a'];
            if (u < 0 || pass[u] == 0) break;
```

```
            v = u, d++;
        }
        return d;
    }
    string str(int v) // the string spelled by node v
    {
        string s;
        for (; v > 0; v = par[v]) s += char('a' + ch[v]);
        reverse(all(s));
        return s;
    }
    // ---------- traversals ----------
    vector<string> words(int from = 0) // every word in from's subtree, sorted, with
repeats (iterative dfs)
    {
        vector<string> res;
        string cur = str(from);
        vector<pi> st = {{from, 0}}; // (node, next child to try)
        rep(i, 0, ends[from]) res.pb(cur);
        while (!st.empty())
        {
            int v = st.back().first, &c = st.back().second;
            while (c < A && (nx[v][c] < 0 || pass[nx[v][c]] == 0)) c++;
            if (c == A) { st.pop_back(); if (!cur.empty() && !st.empty())
cur.pop_back(); continue; }
            int u = nx[v][c++];
            cur += char('a' + ch[u]);
            rep(i, 0, ends[u]) res.pb(cur);
            st.pb({u, 0});
        }
        return res;
    }
    string kth(int k) // k-th smallest word (1-indexed, repeats count), k <= pass[0]
    {
        int v = 0;
        string s;
        while (true)
        {
            if (k <= ends[v]) return s;
            k -= ends[v];
            rep(c, 0, A)
            {
                int u = nx[v][c];
                if (u < 0) continue;
                if (k <= pass[u]) { v = u, s += char('a' + c); break; }
                k -= pass[u];
            }
        }
    }
    vi levels() // nodes in bfs order: depth by depth, each depth in lexicographic
order
    {
        vi q = {0};
        rep(i, 0, (int)q.size()) rep(c, 0, A) if (nx[q[i]][c] >= 0) q.pb(nx[q[i]][c]);
        return q;
    }
    vi tin, tout;
    void euler() // preorder positions: subtree of v = [tin[v], tout[v]]. bottom-up dp:
loop levels() backwards
    {
```

```
        int n = size(), t = 0;
        tin.assign(n, 0), tout.assign(n, 0);
        vector<pi> st = {{0, 0}};
        tin[0] = t++;
        while (!st.empty())
        {
            int v = st.back().first, &c = st.back().second;
            while (c < A && nx[v][c] < 0) c++;
            if (c == A) { tout[v] = t - 1, st.pop_back(); continue; }
            int u = nx[v][c++];
            tin[u] = t++, st.pb({u, 0});
        }
    }
};
// examples:
//   for (int v : t.levels()) {...}                       top-down dp (parent before
child)
//   vi o = t.levels(); reverse(all(o)); for (int v : o)  bottom-up dp (children before
parent)
//   t.words(t.find(p))                                   all words with prefix p,
sorted
```

FILE: templates/data_structures/persistent_segment_tree.cpp

```
// persistent segment tree over values [0, V): version k = first k elements inserted
// kth smallest in a[l..r]: build roots by inserting compressed values, then
kth(root[r+1], root[l], k)
struct PST
{
    vi L, R, cnt;
    int V;
    PST(int V, int reserveNodes = 0) : V(V) { L.reserve(reserveNodes),
R.reserve(reserveNodes), cnt.reserve(reserveNodes); L.pb(0), R.pb(0), cnt.pb(0); } //
node 0 = empty
    int insert(int prev, int pos, int l = 0, int r = -1)
    {
        if (r == -1) r = V - 1;
        int v = L.size();
        L.pb(L[prev]), R.pb(R[prev]), cnt.pb(cnt[prev] + 1);
        if (l == r) return v;
        int m = (l + r) / 2;
        if (pos <= m) { int c = insert(L[prev], pos, l, m); L[v] = c; }
        else { int c = insert(R[prev], pos, m + 1, r); R[v] = c; }
        return v;
    }
    int kth(int hiRoot, int loRoot, int k, int l = 0, int r = -1) // k is 1-based,
returns compressed value
    {
        if (r == -1) r = V - 1;
        while (l < r)
        {
            int m = (l + r) / 2, left = cnt[L[hiRoot]] - cnt[L[loRoot]];
            if (k <= left) hiRoot = L[hiRoot], loRoot = L[loRoot], r = m;
            else k -= left, hiRoot = R[hiRoot], loRoot = R[loRoot], l = m + 1;
        }
        return l;
    }
};
```

FILE: templates/data_structures/mo.cpp

```
// mo's algorithm: offline range queries in O((n + q) sqrt n) add/remove operations
// fill qs with {l, r, idx} (0-based, inclusive), write add(i) / remove(i) / answer(),
call mo(qs, q)
struct Query { int l, r, idx; };
void add(int i);
void removeIdx(int i);
int answer();
vi mo(vector<Query> qs, int q)
{
    int B = 512; // block size ~ n / sqrt(q); 512 is fine for n, q ~ 2e5
    sort(all(qs), [&](const Query &a, const Query &b)
    {
        if (a.l / B != b.l / B) return a.l < b.l;
        return (a.l / B & 1) ? a.r > b.r : a.r < b.r;
    });
    vi res(q);
    int cl = 0, cr = -1;
    for (auto &x : qs)
    {
        while (cr < x.r) add(++cr);
        while (cl > x.l) add(--cl);
        while (cr > x.r) removeIdx(cr--);
        while (cl < x.l) removeIdx(cl++);
        res[x.idx] = answer();
    }
    return res;
}
```

FILE: templates/data_structures/li_chao.cpp

```
// li chao tree: add lines y = kx + b, query the minimum at integer x in [lo, hi].
O(log) each
// for maximum: insert (-k, -b) and negate the answer
struct LiChao
{
    struct Line { int k, b; int eval(int x) const { return k * x + b; } };
    int lo, hi;
    vector<Line> t;
    vi L, R;
    LiChao(int lo, int hi) : lo(lo), hi(hi) { newNode(); }
    int newNode() { t.pb({0, LLONG_MAX}), L.pb(-1), R.pb(-1); return t.size() - 1; }
    void add(Line ln, int v = 0, int l = LLONG_MIN, int r = LLONG_MIN)
    {
        if (l == LLONG_MIN) l = lo, r = hi;
        while (true)
        {
            int m = l + (r - l) / 2;
            bool lef = ln.eval(l) < t[v].eval(l), mid = ln.eval(m) < t[v].eval(m);
            if (t[v].b == LLONG_MAX) { t[v] = ln; return; }
            if (mid) swap(t[v], ln);
            if (l == r) return;
            if (lef != mid)
            {
                if (L[v] == -1) { int u = newNode(); L[v] = u; }
                v = L[v], r = m;
            }
            else
```

```
            {
                if (R[v] == -1) { int u = newNode(); R[v] = u; }
                v = R[v], l = m + 1;
            }
        }
    }
    int query(int x)
    {
        int v = 0, l = lo, r = hi, res = LLONG_MAX;
        while (v != -1)
        {
            if (t[v].b != LLONG_MAX) res = min(res, t[v].eval(x));
            int m = l + (r - l) / 2;
            if (x <= m) v = L[v], r = m; else v = R[v], l = m + 1;
        }
        return res;
    }
};
```

FILE: templates/data_structures/treap.cpp

```
// implicit treap (array with split / merge): insert, erase, reverse, sum on ranges in
O(log n)
// root = tmerge(root, tnew(x)) to append. reverse [l, r]: split into a | b | c, flip
b, merge back
mt19937 trng(chrono::steady_clock::now().time_since_epoch().count());
struct TNode { int val, sum, pri, sz; bool rev; int l, r; };
vector<TNode> tp = {{0, 0, 0, 0, false, 0, 0}}; // node 0 = null
int tnew(int v) { tp.pb({v, v, (int)trng(), 1, false, 0, 0}); return tp.size() - 1; }
int tsz(int t) { return tp[t].sz; }
void tpull(int t) { if (t) tp[t].sz = 1 + tsz(tp[t].l) + tsz(tp[t].r), tp[t].sum =
tp[t].val + tp[tp[t].l].sum + tp[tp[t].r].sum; }
void pushT(int t) { if (t && tp[t].rev) { swap(tp[t].l, tp[t].r); if (tp[t].l)
tp[tp[t].l].rev ^= 1; if (tp[t].r) tp[tp[t].r].rev ^= 1; tp[t].rev = 0; } }
void tsplit(int t, int k, int &a, int &b) // first k elements go to a
{
    if (!t) { a = b = 0; return; }
    pushT(t);
    if (tsz(tp[t].l) < k) { tsplit(tp[t].r, k - tsz(tp[t].l) - 1, tp[t].r, b); a = t; }
    else { tsplit(tp[t].l, k, a, tp[t].l); b = t; }
    tpull(t);
}
int tmerge(int a, int b)
{
    if (!a || !b) return a ^ b;
    if (tp[a].pri > tp[b].pri) { pushT(a); tp[a].r = tmerge(tp[a].r, b); tpull(a);
return a; }
    pushT(b); tp[b].l = tmerge(a, tp[b].l); tpull(b); return b;
}
void reverseRange(int &root, int l, int r) // 0-based inclusive
{
    int a, b, c;
    tsplit(root, l, a, b), tsplit(b, r - l + 1, b, c);
    tp[b].rev ^= 1;
    root = tmerge(tmerge(a, b), c);
}
int rangeSum(int &root, int l, int r)
{
    int a, b, c;
    tsplit(root, l, a, b), tsplit(b, r - l + 1, b, c);
```

```
    int s = tp[b].sum;
    root = tmerge(tmerge(a, b), c);
    return s;
}
void toArray(int t, vi &out) { if (!t) return; pushT(t); toArray(tp[t].l, out);
out.pb(tp[t].val); toArray(tp[t].r, out); }
```

FILE: templates/data_structures/sqrt_decomposition/range_min_query.cpp

```
// sqrt decomposition: point update, range min. SqrtMin s(a); s.update(i, x);
s.query(l, r) inclusive
struct SqrtMin
{
    int n, B;
    vi a, blk;
    SqrtMin(const vi &v) : n(v.size()), a(v)
    {
        B = max(1LL, (int)sqrt(n));
        blk.assign(n / B + 1, LLONG_MAX);
        rep(i, 0, n) blk[i / B] = min(blk[i / B], a[i]);
    }
    void update(int i, int x)
    {
        a[i] = x;
        int b = i / B;
        blk[b] = LLONG_MAX;
        rep(j, b * B, min(n, (b + 1) * B)) blk[b] = min(blk[b], a[j]);
    }
    int query(int l, int r)
    {
        int res = LLONG_MAX;
        while (l <= r && l % B) res = min(res, a[l++]);
        while (l + B - 1 <= r) res = min(res, blk[l / B]), l += B;
        while (l <= r) res = min(res, a[l++]);
        return res;
    }
};
```

FILE: templates/data_structures/sqrt_decomposition/sqrt_decomposition_lazy_min.cpp

```
// sqrt decomposition: range "a[i] = min(a[i], x)" and range sum. ChminSum s(a);
s.update(l, r, x); s.query(l, r)
// each block keeps its sorted values + prefix sums, and a lazy cap. O(sqrt(n) log n)
per operation
struct ChminSum
{
    int n, B;
    vi a, cap;
    vvi srt, pre;
    ChminSum(const vi &v) : n(v.size()), a(v)
    {
        B = max(1LL, (int)sqrt(n));
        int k = (n + B - 1) / B;
        cap.assign(k, LLONG_MAX), srt.assign(k, vi()), pre.assign(k, vi());
        rep(b, 0, k) rebuild(b);
    }
    void push(int b) // apply the lazy cap to the real values
    {
```

```
        if (cap[b] == LLONG_MAX) return;
        rep(i, b * B, min(n, (b + 1) * B)) a[i] = min(a[i], cap[b]);
        cap[b] = LLONG_MAX;
    }
    void rebuild(int b)
    {
        srt[b].assign(a.begin() + b * B, a.begin() + min(n, (b + 1) * B));
        sort(all(srt[b]));
        pre[b].assign(srt[b].size() + 1, 0);
        rep(i, 0, (int)srt[b].size()) pre[b][i + 1] = pre[b][i] + srt[b][i];
    }
    int blockSum(int b) // sum of min(v, cap) over the block
    {
        int c = cap[b], k = lower_bound(all(srt[b]), c) - srt[b].begin();
        return pre[b][k] + (c == LLONG_MAX ? 0 : c * ((int)srt[b].size() - k));
    }
    void update(int l, int r, int x)
    {
        int bl = l / B, br = r / B;
        if (bl == br)
        {
            push(bl);
            rep(i, l, r + 1) a[i] = min(a[i], x);
            rebuild(bl);
            return;
        }
        push(bl); rep(i, l, (bl + 1) * B) a[i] = min(a[i], x); rebuild(bl);
        rep(b, bl + 1, br) cap[b] = min(cap[b], x);
        push(br); rep(i, br * B, r + 1) a[i] = min(a[i], x); rebuild(br);
    }
    int query(int l, int r)
    {
        int bl = l / B, br = r / B, s = 0;
        if (bl == br) { rep(i, l, r + 1) s += min(a[i], cap[bl]); return s; }
        rep(i, l, (bl + 1) * B) s += min(a[i], cap[bl]);
        rep(b, bl + 1, br) s += blockSum(b);
        rep(i, br * B, r + 1) s += min(a[i], cap[br]);
        return s;
    }
};
```

FILE: templates/data_structures/xor_mst.cpp

```
// xor minimum spanning tree: sort indices by w into ord, edges.clear(), mst(0, n - 1,
29)
// edges gets the n - 1 tree edges (pairs of indices)
int w[200005];
vi ord;
int tr[200005 * 31][2], tid[200005 * 31], tc;
vpi edges;
void ins(int i)
{
    int v = 0;
    repb(b, 29, 0)
    {
        int c = w[i] >> b & 1;
        if (!tr[v][c]) tr[v][c] = ++tc, tr[tc][0] = tr[tc][1] = 0;
        v = tr[v][c];
    }
    tid[v] = i;
```

```
}
pi best(int i) // {min xor with w[i] in the trie, index}
{
    int v = 0, res = 0;
    repb(b, 29, 0)
    {
        int c = w[i] >> b & 1;
        if (tr[v][c]) v = tr[v][c];
        else v = tr[v][c ^ 1], res |= 1 << b;
    }
    return {res, tid[v]};
}
// xor minimum spanning tree of ord[l..r] (sorted by value), all equal above bit b
void mst(int l, int r, int b)
{
    if (l >= r || b < 0)
    {
        rep(i, l, r) edges.pb({ord[i], ord[i + 1]}); // equal values, any chain works
        return;
    }
    int m = l;
    while (m <= r && !(w[ord[m]] >> b & 1)) m++;
    mst(l, m - 1, b - 1), mst(m, r, b - 1);
    if (m <= l || m > r) return;
    tc = 0, tr[0][0] = tr[0][1] = 0;
    rep(i, l, m) ins(ord[i]);
    pi bst = {LLONG_MAX, 0};
    int who = -1;
    rep(i, m, r + 1)
    {
        pi got = best(ord[i]);
        if (got.first < bst.first) bst = got, who = ord[i];
    }
    edges.pb({who, bst.second});
}
```

17.8 Graphs

KEYWORDS: graphs template code

FILE: templates/graph/bfs_variants.cpp

```
// 0-1 bfs: shortest paths when every weight is 0 or 1, O(n + m). g[u] = {v, w}, w in
{0, 1}
vi bfs01(const vector<vpi> &g, int s)
{
    vi d(g.size(), LLONG_MAX);
    deque<int> dq;
    d[s] = 0, dq.pb(s);
    while (!dq.empty())
    {
        int u = dq.front();
        dq.pop_front();
        for (auto &e : g[u]) { int v = e.first, w = e.second; if (d[u] + w < d[v])
        {
            d[v] = d[u] + w;
            if (w == 0) dq.push_front(v); else dq.pb(v);
        } }
    }
    return d;
```

```
}
// multi-source bfs on a grid: push every source with distance 0 first. '#' = wall
vvi gridBfs(const vector<string> &g, const vpi &src)
{
    int R = g.size(), C = g[0].size();
    vvi d(R, vi(C, -1));
    queue<pi> q;
    for (auto &p : src) d[p.first][p.second] = 0, q.push(p);
    int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
    while (!q.empty())
    {
        int r = q.front().first, c = q.front().second;
        q.pop();
        rep(k, 0, 4)
        {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nc < 0 || nr >= R || nc >= C || g[nr][nc] == '#' || d[nr][nc]
!= -1) continue;
            d[nr][nc] = d[r][c] + 1, q.push({nr, nc});
        }
    }
    return d;
}
// knight moves: dr = {1,2,2,1,-1,-2,-2,-1}, dc = {2,1,-1,-2,-2,-1,1,2}
```

FILE: templates/graph/dijkstra.cpp

```
// dijkstra, O((n + m) log n). g[u] = {v, w}, w >= 0. returns dist (LLONG_MAX =
unreachable), par for the path
vi dijkstra(const vector<vpi> &g, int s, vi *par = nullptr)
{
    int n = g.size();
    vi d(n, LLONG_MAX);
    if (par) par->assign(n, -1);
    priority_queue<pi, vpi, greater<pi>> pq;
    d[s] = 0, pq.push({0, s});
    while (!pq.empty())
    {
        int dv = pq.top().first, v = pq.top().second;
        pq.pop();
        if (dv != d[v]) continue; // outdated entry
        for (auto &e : g[v])
            if (dv + e.second < d[e.first])
            {
                d[e.first] = dv + e.second, pq.push({d[e.first], e.first});
                if (par) (*par)[e.first] = v;
            }
    }
    return d;
}
// 0-1 bfs: weights only 0 or 1, O(n + m)
vi bfs01(const vector<vpi> &g, int s)
{
    vi d(g.size(), LLONG_MAX);
    deque<int> dq;
    d[s] = 0, dq.pb(s);
    while (!dq.empty())
    {
        int v = dq.front(); dq.pop_front();
        for (auto &e : g[v])
```

```
            if (d[v] + e.second < d[e.first])
            {
                d[e.first] = d[v] + e.second;
                if (e.second == 0) dq.push_front(e.first); else dq.pb(e.first);
            }
    }
    return d;
}
```

FILE: templates/graph/bellman_ford.cpp

```
// bellman ford, O(nm). edges {u, v, w}. returns dist from s (LLONG_MAX = unreachable);
// negInf[v] = 1 if v can be made arbitrarily small (reachable from a negative cycle
reachable from s)
vi bellmanFord(int n, const vector<array<int, 3>> &e, int s, vi &negInf)
{
    vi d(n, LLONG_MAX);
    negInf.assign(n, 0);
    d[s] = 0;
    rep(it, 0, n - 1) for (auto &x : e)
        if (d[x[0]] != LLONG_MAX && d[x[0]] + x[2] < d[x[1]]) d[x[1]] = d[x[0]] + x[2];
    rep(it, 0, n) for (auto &x : e) // spread "minus infinity"
        if (d[x[0]] != LLONG_MAX && (negInf[x[0]] || d[x[0]] + x[2] < d[x[1]]))
negInf[x[1]] = 1;
    return d;
}
// any negative cycle in the whole graph (as a vertex list), empty if none
vi findNegCycle(int n, const vector<array<int, 3>> &e)
{
    vi d(n, 0), p(n, -1);
    int x = -1;
    rep(it, 0, n)
    {
        x = -1;
        for (auto &y : e) if (d[y[0]] + y[2] < d[y[1]]) d[y[1]] = d[y[0]] + y[2],
p[y[1]] = y[0], x = y[1];
    }
    if (x == -1) return {};
    rep(i, 0, n) x = p[x];
    vi cyc;
    for (int v = x;; v = p[v]) { cyc.pb(v); if (v == x && cyc.size() > 1) break; }
    reverse(all(cyc));
    return cyc; // first == last
}
```

FILE: templates/graph/floyd_warshall_directed.cpp

```
void solve_directed_apsp() {
    int n, m, q;
    cin >> n >> m >> q;

    const long long INF = (long long)4e18;
    vector<vector<long long>> dist(n+1, vector<long long>(n+1, INF));
    for(int i=1;i<=n;i++) dist[i][i]=0;

    for(int i=0;i<m;i++){
        int u,v; long long w;
        cin >> u >> v >> w;
        dist[u][v] = min(dist[u][v], w); // no dist[v][u]
```

```
    }

    for(int k=1;k<=n;k++)
        for(int i=1;i<=n;i++){
            if(dist[i][k]>=INF/2) continue;
            for(int j=1;j<=n;j++){
                if(dist[k][j]>=INF/2) continue;
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }

    while(q--){
        int u,v; cin >> u >> v;
        cout << (dist[u][v]>=INF/2? -1 : dist[u][v]) << "\n";
    }
}
```

FILE: templates/graph/kruskal.cpp

Needs: data_structures/dsu.cpp above it

```
// kruskal minimum spanning tree. edges {w, u, v}. returns {total weight, chosen
edges}; needs DSU (dsu.cpp)
// total < 0 never happens; if result.second.size() < n - 1 the graph is disconnected
pair<int, vector<array<int, 3>>> kruskal(int n, vector<array<int, 3>> e)
{
    sort(all(e));
    DSU d(n);
    int total = 0;
    vector<array<int, 3>> used;
    for (auto &x : e) if (d.unite(x[1], x[2])) total += x[0], used.pb(x);
    return {total, used};
}
```

FILE: templates/graph/topo_sort.cpp

```
// kahn topological sort. adj[u] = nodes v with edge u -> v, nodes 1..n
// returns topo order; size < n means the graph has a cycle
// lexicographically smallest order: use priority_queue<int, vi, greater<int>> instead
of queue
vi topoSort(int n, vvi &adj)
{
    vi indeg(n + 1, 0), order;
    rep(u, 1, n + 1) for (int v : adj[u]) indeg[v]++;
    queue<int> q;
    rep(u, 1, n + 1) if (indeg[u] == 0) q.push(u);
    while (!q.empty())
    {
        int u = q.front(); q.pop();
        order.pb(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order;
}
```

FILE: templates/graph/scc_tarjan.cpp

```
// tarjan scc (iterative). scc sc(n); sc.add_edge(u, v); sc.build();
struct scc
{
```

```
    int n, timer, nc;
    vvi g, comps;
    vi comp, num, low, stk;
    vector<bool> onstk;

    scc(int n) : n(n), timer(0), nc(0), g(n), comp(n, -1), num(n, 0), low(n, 0),
onstk(n, false) {}

    void add_edge(int u, int v) { g[u].pb(v); }

    void run(int root)
    {
        vpi call;
        num[root] = low[root] = ++timer;
        stk.pb(root);
        onstk[root] = true;
        call.pb({root, 0});
        while (!call.empty())
        {
            int v = call.back().first;
            int i = call.back().second;
            if (i < (int)g[v].size())
            {
                call.back().second = i + 1;
                int to = g[v][i];
                if (!num[to])
                {
                    num[to] = low[to] = ++timer;
                    stk.pb(to);
                    onstk[to] = true;
                    call.pb({to, 0});
                }
                else if (onstk[to]) low[v] = min(low[v], num[to]);
            }
            else
            {
                if (low[v] == num[v])
                {
                    vi cur;
                    while (true)
                    {
                        int u = stk.back();
                        stk.pop_back();
                        onstk[u] = false;
                        comp[u] = nc;
                        cur.pb(u);
                        if (u == v) break;
                    }
                    comps.pb(cur);
                    nc++;
                }
                call.pop_back();
                if (!call.empty()) low[call.back().first] = min(low[call.back().first],
low[v]);
            }
        }
    }

    void build() { rep(i, 0, n) if (!num[i]) run(i); }
};
```

```
// scc sc(n + 1);
// sc.add_edge(a, b);
// sc.build();
// sc.comp[v] = component id of v
// sc.comps   = list of components, sinks first
// sc.g       = the graph (reuse it, don't build a second one)
```

FILE: templates/graph/two_sat.cpp

```
// 2-sat. TwoSat ts(n) for variables 0..n-1. ts.either(a, va, b, vb) adds "(x_a == va)
or (x_b == vb)"
// ts.solve() -> true if satisfiable, then ts.val[i] holds an assignment. iterative
tarjan
struct TwoSat
{
    int n;
    vvi g;
    vi val;
    TwoSat(int n) : n(n), g(2 * n) {}
    int id(int x, bool v) { return 2 * x + (v ? 0 : 1); }
    void either(int a, bool va, int b, bool vb)
    {
        int A = id(a, va), B = id(b, vb);
        g[A ^ 1].pb(B), g[B ^ 1].pb(A);
    }
    void setTrue(int a, bool va) { either(a, va, a, va); }
    void implies(int a, bool va, int b, bool vb) { either(a, !va, b, vb); }
    bool solve()
    {
        int N = 2 * n, timer = 0, nc = 0;
        vi num(N, 0), low(N, 0), comp(N, -1), stk, it(N, 0);
        vector<char> on(N, 0);
        rep(s, 0, N)
        {
            if (num[s]) continue;
            vi call = {s};
            num[s] = low[s] = ++timer, stk.pb(s), on[s] = 1;
            while (!call.empty())
            {
                int v = call.back();
                if (it[v] < (int)g[v].size())
                {
                    int to = g[v][it[v]++];
                    if (!num[to]) num[to] = low[to] = ++timer, stk.pb(to), on[to] = 1,
call.pb(to);
                    else if (on[to]) low[v] = min(low[v], num[to]);
                }
                else
                {
                    if (low[v] == num[v])
                    {
                        while (true)
                        {
                            int u = stk.back(); stk.pop_back();
                            on[u] = 0, comp[u] = nc;
                            if (u == v) break;
                        }
                        nc++;
                    }
```

```
                    call.pop_back();
                    if (!call.empty()) low[call.back()] = min(low[call.back()],
low[v]);
                }
            }
        }
        val.assign(n, 0);
        rep(i, 0, n)
        {
            if (comp[2 * i] == comp[2 * i + 1]) return false;
            val[i] = comp[2 * i] < comp[2 * i + 1]; // tarjan numbers sinks first
        }
        return true;
    }
};
```

FILE: templates/graph/bridges.cpp

```
// bridges + articulation points + 2-edge-connected components (handles multi-edges and
self loops)
// LowLink L(n); L.add(u, v) for every undirected edge; L.run();
// L.isBridge[edge id], L.isCut[v], L.comp2e[v] = 2-edge-connected component id (bridge
tree: join comps by bridges)
// iterative dfs, no stack overflow
struct LowLink
{
    int n, m = 0;
    vector<vpi> g; // {to, edge id}
    vi tin, low, isCut, comp2e;
    vector<char> isBridge;
    vpi edges;
    LowLink(int n) : n(n), g(n) {}
    void add(int u, int v) { g[u].pb({v, m}), g[v].pb({u, m}), edges.pb({u, v}), m++; }
    void run()
    {
        tin.assign(n, -1), low.assign(n, 0), isCut.assign(n, 0), isBridge.assign(m, 0);
        int timer = 0;
        vi it(n, 0), pe(n, -1), kids(n, 0);
        rep(s, 0, n)
        {
            if (tin[s] != -1) continue;
            vi st = {s};
            tin[s] = low[s] = timer++;
            while (!st.empty())
            {
                int v = st.back();
                if (it[v] < (int)g[v].size())
                {
                    int to = g[v][it[v]].first, id = g[v][it[v]].second;
                    it[v]++;
                    if (id == pe[v]) continue; // skip only the edge we came by
                    if (tin[to] != -1) low[v] = min(low[v], tin[to]);
                    else
                    {
                        pe[to] = id, kids[v]++;
                        tin[to] = low[to] = timer++;
                        st.pb(to);
                    }
                }
                else
```

```
                {
                    st.pop_back();
                    if (!st.empty())
                    {
                        int p = st.back();
                        low[p] = min(low[p], low[v]);
                        if (low[v] > tin[p]) isBridge[pe[v]] = 1;
                        if (low[v] >= tin[p] && pe[p] != -1) isCut[p] = 1;
                    }
                }
            }
            if (kids[s] > 1) isCut[s] = 1;
        }
        comp2e.assign(n, -1);
        int c = 0;
        rep(s, 0, n)
        {
            if (comp2e[s] != -1) continue;
            vi st = {s};
            comp2e[s] = c;
            while (!st.empty())
            {
                int v = st.back(); st.pop_back();
                for (auto &e : g[v])
                    if (!isBridge[e.second] && comp2e[e.first] == -1) comp2e[e.first] =
c, st.pb(e.first);
            }
            c++;
        }
    }
};
```

FILE: templates/graph/articulation_points.cpp

```
// articulation points: use LowLink in bridges.cpp (L.isCut[v]). it handles multi-edges
and is iterative.
```

FILE: templates/graph/bridges_tarjan_subtree_size.cpp

```
// simple graphs only (skips every edge to the parent). for multi-edges use LowLink in
bridges.cpp
// bridge -> any edge on who's removal the graph becomes disconnected and no of
connected components +=1
//  time[] - > time of first visit of a node
//  low[] -> lowest time of visit of a node that can be reached from the subtree of
that node (including itself)-> (not including parent)
//  case 1: u-v edge, time[u] < low[v] -> v cannot reach any ancestor of u, so u-v is a
bridge
//  case 2: time[u] >= low[v] -> v can reach an ancestor of u, even if that edge is
removed, so u-v is not a bridge
//  this algorithm is called tarjan's algo
// just find the bridges, the subtree sizes,
//  u->v  sub[v]* (n-sub[v]) hobe reduced pairs

void tarjan_dfs(vvi &adj, vi &subtree_size, vi &time, vi &low, int &timer, int u, int
parent, vpi &bridges)
{
    time[u] = low[u] = timer;
    timer++;
```

```
    subtree_size[u] = 1;
    for (auto &child : adj[u])
    {
        if (child == parent)
            continue;
        if (time[child] == -1)
        {
            tarjan_dfs(adj, subtree_size, time, low, timer, child, u, bridges);
            subtree_size[u] += subtree_size[child];
        }
        low[u] = min(low[u], low[child]);
        if (time[u] < low[child])
        {
            bridges.pb({u, child});
        }
    }
}
```

FILE: templates/graph/biconnected_components.cpp

```
// vertex biconnected components (blocks), handles multi-edges. BCC b(n); b.add(u, v);
b.run();
// b.comps[i] = list of edge ids in block i; b.isCut[v]. isolated vertices belong to no
block.
// recursive dfs: fine for n up to ~2e5 on most judges
struct BCC
{
    int n, m = 0, timer = 0;
    vector<vpi> g; // {to, edge id}
    vi tin, low, isCut, st;
    vvi comps;
    vpi edges;
    BCC(int n) : n(n), g(n) {}
    void add(int u, int v) { g[u].pb({v, m}), g[v].pb({u, m}), edges.pb({u, v}), m++; }
    void dfs(int v, int pe)
    {
        tin[v] = low[v] = timer++;
        int kids = 0;
        for (auto &e : g[v])
        {
            int to = e.first, id = e.second;
            if (id == pe) continue;
            if (tin[to] != -1)
            {
                if (tin[to] < tin[v]) st.pb(id); // back edge, push once
                low[v] = min(low[v], tin[to]);
                continue;
            }
            st.pb(id);
            dfs(to, id);
            kids++;
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v])
            {
                if (pe != -1) isCut[v] = 1;
                vi comp;
                while (true)
                {
                    int x = st.back(); st.pop_back();
                    comp.pb(x);
```

```
                    if (x == id) break;
                }
                comps.pb(comp);
            }
        }
        if (pe == -1 && kids > 1) isCut[v] = 1;
    }
    void run()
    {
        tin.assign(n, -1), low.assign(n, 0), isCut.assign(n, 0);
        rep(v, 0, n) if (tin[v] == -1) dfs(v, -1);
    }
};
```

FILE: templates/graph/finding_simple_cycle.cpp

```
// FINDING ONE SIMPLE CYCLE
// findCycle(g, directed) -> vertices of one simple cycle in order, or {-1} if there is
none.
//   g        : adjacency list, 0- or 1-indexed (unused vertices are fine)
//   directed : false -> g holds each undirected edge in both lists (u in g[v] and v in
g[u])
//              true  -> g[u] holds v for every edge u -> v
// Handles self loops ({u}), multiple edges (undirected u-v twice -> {u, v}),
// disconnected graphs. Iterative dfs, so no stack overflow. O(n + m).
//
// The cycle is returned without repeating the first vertex:
// {a, b, c} means a - b - c - a (directed: a -> b -> c -> a).

vi findCycle(const vvi &g, bool directed)
{
    int n = g.size();
    vi state(n, 0), par(n, -1), it(n, 0);
    // undirected: skip the edge back to the parent only ONCE, so a second parallel
edge still counts
    vector<char> skipped(n, 0);
    auto build = [&](int from, int to) // from is a descendant of to (or equal): walk
parents
    {
        vi cyc;
        for (int v = from; v != to; v = par[v]) cyc.pb(v);
        cyc.pb(to);
        reverse(all(cyc)); // to -> ... -> from, then the closing edge from -> to
        return cyc;
    };
    rep(s, 0, n)
    {
        if (state[s]) continue;
        vi st = {s};
        state[s] = 1; // 1 = on the dfs stack, 2 = finished
        while (!st.empty())
        {
            int u = st.back();
            if (it[u] == (int)g[u].size())
            {
                state[u] = 2;
                st.pop_back();
                continue;
            }
            int v = g[u][it[u]++];
```

```
            if (v == u) return {u}; // self loop
            if (!directed && v == par[u] && !skipped[u])
            {
                skipped[u] = 1; // the tree edge itself, used once
                continue;
            }
            if (state[v] == 0)
            {
                state[v] = 1, par[v] = u;
                st.pb(v);
            }
            else if (state[v] == 1) return build(u, v); // back edge to a vertex on the
stack
            // state 2: undirected -> already seen from the other side; directed ->
cross/forward edge
        }
    }
    return {-1};
}

/*
void solve()
{
    int n, m;
    cin >> n >> m;
    vvi g(n + 1);
    rep(i, 0, m)
    {
        int u, v;
        cin >> u >> v;
        g[u].pb(v);
        g[v].pb(u); // remove this line for a directed graph
    }
    vi cyc = findCycle(g, false);
    if (cyc[0] == -1) { cout << "IMPOSSIBLE" << nl; return; }
    cout << cyc.size() + 1 << nl;
    for (int v : cyc) cout << v << " ";
    cout << cyc[0] << nl;
}
*/
```

FILE: templates/graph/euler_path_hierholzer.cpp

```
// euler path / circuit (hierholzer, iterative, O(n + m)), multi-edges and self loops
ok
// eulerPath(n, edges, directed, start): vertex sequence using every edge once, or {}
if impossible
// start = -1 picks a valid start automatically (odd vertex / out-in = 1, else any
vertex with an edge)
// for a circuit check path.front() == path.back()
vi eulerPath(int n, const vpi &edges, bool directed, int start = -1)
{
    int m = edges.size();
    vector<vpi> g(n);
    vi deg(n, 0), in(n, 0);
    rep(i, 0, m)
    {
        int u = edges[i].first, v = edges[i].second;
        g[u].pb({v, i}), deg[u]++, in[v]++;
        if (!directed) g[v].pb({u, i}), deg[v]++;
```

```
    }
    if (start == -1)
    {
        rep(v, 0, n) if (deg[v]) { start = v; break; }
        if (start == -1) return {0};
        rep(v, 0, n)
        {
            if (directed && deg[v] - in[v] == 1) { start = v; break; }
            if (!directed && deg[v] % 2) { start = v; break; }
        }
    }
    // degree conditions
    int bad = 0;
    rep(v, 0, n)
    {
        int d = directed ? deg[v] - in[v] : deg[v] % 2;
        if (directed && d != 0 && !((d == 1 && v == start) || d == -1)) bad++;
        if (directed && (d > 1 || d < -1)) bad++;
        if (!directed && d && v != start) bad++;
    }
    if (directed ? bad > 0 : bad > 1) return {};
    if (!directed && bad == 1 && deg[start] % 2 == 0) return {};
    vector<char> used(m, 0);
    vi it(n, 0), st = {start}, path;
    while (!st.empty())
    {
        int v = st.back();
        while (it[v] < (int)g[v].size() && used[g[v][it[v]].second]) it[v]++;
        if (it[v] == (int)g[v].size()) { path.pb(v); st.pop_back(); continue; }
        auto e = g[v][it[v]++];
        used[e.second] = 1;
        st.pb(e.first);
    }
    reverse(all(path));
    if ((int)path.size() != m + 1) return {}; // disconnected edges
    return path;
}
```

FILE: templates/graph/bipartite_coloring.cpp

```
//GRAPH COLORING
void solve()
{
    int n, m;
    cin >> n >> m;
    vi adj[n + 1];
    rep(i, 1, m + 1)
    {
        int u, v;
        cin >> u >> v;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    vi colors(n + 1, 0);
    function<void(int)> dfs = [&](int node)
    {
        // this node is visited and assigned a color
        for (int child : adj[node])
        {
            if (colors[child] == 0)
```

```
            {
                // unvisited child,assigning a color
                colors[child] = colors[node] == 1 ? 2 : 1;
                // dfs chalabe ke???
                dfs(child);
            }

            // visited and ancestor
            else
            {
                if (colors[child] + colors[node] != 3)
                {
                    cout << "IMPOSSIBLE" << nl;
                    exit(0);
                }
            }
        }
    };
    rep(i, 1, n + 1)
    {
        if (colors[i] == 0)
        {
            // unvisited so is not assigned color
            colors[i] = 1;
            dfs(i);
        }
    }
    rep(i, 1, n + 1)
    {
        cout << colors[i] << " ";
    }
    cout << nl;
}
```

FILE: templates/graph/kuhn_matching.cpp

```
// kuhn bipartite matching. fill adj[left] with right vertices, matching(nleft, nright)
// after it, mt[right] = matched left vertex or -1
vvi adj;
vi mt, used;
int stamp;
bool kuhn(int v)
{
    if (used[v] == stamp) return false;
    used[v] = stamp;
    for (int u : adj[v])
        if (mt[u] < 0 || kuhn(mt[u]))
        {
            mt[u] = v;
            return true;
        }
    return false;
}
// returns size of max matching, mt[right] = left
int matching(int n, int m)
{
    mt.assign(m, -1), used.assign(n, 0);
    int res = 0;
    rep(v, 0, n)
    {
```

```
        stamp++;
        if (kuhn(v)) res++;
    }
    return res;
}
```

FILE: templates/graph/hopcroft_karp.cpp

```
// hopcroft karp bipartite matching, O(E sqrt V). HopcroftKarp hk(nL, nR); hk.add(l,
r); hk.run()
// after run: ml[l] = matched right vertex or -1, mr[r] = matched left vertex or -1
struct HopcroftKarp
{
    int nL, nR;
    vvi g;
    vi ml, mr, dist;
    HopcroftKarp(int nL, int nR) : nL(nL), nR(nR), g(nL), ml(nL, -1), mr(nR, -1) {}
    void add(int l, int r) { g[l].pb(r); }
    bool bfs()
    {
        dist.assign(nL, -1);
        queue<int> q;
        rep(l, 0, nL) if (ml[l] == -1) dist[l] = 0, q.push(l);
        bool found = false;
        while (!q.empty())
        {
            int l = q.front(); q.pop();
            for (int r : g[l])
            {
                int l2 = mr[r];
                if (l2 == -1) found = true;
                else if (dist[l2] == -1) dist[l2] = dist[l] + 1, q.push(l2);
            }
        }
        return found;
    }
    bool dfs(int l)
    {
        for (int r : g[l])
        {
            int l2 = mr[r];
            if (l2 == -1 || (dist[l2] == dist[l] + 1 && dfs(l2))) { ml[l] = r, mr[r] =
l; return true; }
        }
        dist[l] = -1;
        return false;
    }
    int run()
    {
        int res = 0;
        while (bfs()) rep(l, 0, nL) if (ml[l] == -1 && dfs(l)) res++;
        return res;
    }
};
```

FILE: templates/graph/konig_vertex_cover.cpp

```
// minimum vertex cover in a bipartite graph = maximum matching (konig), WITH the
actual cover vertices
```

```
// left 0..nL-1, right 0..nR-1, adj[l] = right neighbours. also gives max independent
set = all - cover
// cover = (left vertices NOT reached) + (right vertices reached) by alternating paths
from free left vertices
struct Konig
{
    int nL, nR;
    vvi adj;
    vi ml, mr; // ml[l] = matched right or -1, mr[r] = matched left or -1
    vector<bool> seen;
    Konig(int nL, int nR) : nL(nL), nR(nR), adj(nL), ml(nL, -1), mr(nR, -1) {}
    void add(int l, int r) { adj[l].pb(r); }
    bool kuhn(int v)
    {
        for (int r : adj[v]) if (!seen[r])
        {
            seen[r] = true;
            if (mr[r] < 0 || kuhn(mr[r])) { ml[v] = r, mr[r] = v; return true; }
        }
        return false;
    }
    int matching()
    {
        int res = 0;
        rep(v, 0, nL) { seen.assign(nR, false); if (kuhn(v)) res++; }
        return res;
    }
    pair<vi, vi> cover() // call after matching(). {left vertices, right vertices} of a
minimum cover
    {
        vector<bool> visL(nL, false), visR(nR, false);
        vi q;
        rep(v, 0, nL) if (ml[v] < 0) visL[v] = true, q.pb(v);
        rep(i, 0, (int)q.size())
        {
            int v = q[i];
            for (int r : adj[v]) if (!visR[r] && ml[v] != r) // go along a non-matching
edge
            {
                visR[r] = true;
                int l = mr[r];
                if (l >= 0 && !visL[l]) visL[l] = true, q.pb(l); // come back along the
matching edge
            }
        }
        vi L, R;
        rep(v, 0, nL) if (!visL[v]) L.pb(v);
        rep(r, 0, nR) if (visR[r]) R.pb(r);
        return {L, R};
    }
};
```

FILE: templates/graph/hungarian.cpp

```
// hungarian algorithm: min cost assignment, a is 1-indexed n x m with n <= m, O(n^2 m)
// for max cost, negate the matrix
int hungarian(vvi &a, int n, int m, vi &ans)
{
    vi u(n + 1), v(m + 1), p(m + 1), way(m + 1);
    rep(i, 1, n + 1)
```

```
    {
        p[0] = i;
        int j0 = 0;
        vi minv(m + 1, LLONG_MAX);
        vector<char> used(m + 1, 0);
        do
        {
            used[j0] = 1;
            int i0 = p[j0], delta = LLONG_MAX, j1 = 0;
            rep(j, 1, m + 1) if (!used[j])
            {
                int cur = a[i0][j] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            rep(j, 0, m + 1)
            {
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            }
            j0 = j1;
        } while (p[j0] != 0);
        do
        {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    ans.assign(n + 1, 0);
    rep(j, 1, m + 1) ans[p[j]] = j;
    return -v[0];
}
```

FILE: templates/graph/dinic.cpp

```
// dinic max flow. dinic d(n); d.add(u, v, cap, reverse_cap); d.flow(s, t)
// min cut = max flow
struct edge
{
    int to, cap;
};
struct dinic
{
    vector<edge> e;
    vvi g;
    vi lvl, it;
    dinic(int n) : g(n), lvl(n), it(n) {}
    void add(int u, int v, int c, int rc = 0)
    {
        g[u].pb(e.size()), e.pb({v, c});
        g[v].pb(e.size()), e.pb({u, rc});
    }
    bool bfs(int s, int t)
    {
        fill(all(lvl), -1);
        lvl[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty())
```

```
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
                if (e[id].cap > 0 && lvl[e[id].to] < 0) lvl[e[id].to] = lvl[u] + 1,
q.push(e[id].to);
        }
        return lvl[t] >= 0;
    }
    int dfs(int u, int t, int f)
    {
        if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++)
        {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && lvl[v] == lvl[u] + 1)
            {
                int d = dfs(v, t, min(f, e[id].cap));
                if (d > 0)
                {
                    e[id].cap -= d, e[id ^ 1].cap += d;
                    return d;
                }
            }
        }
        return 0;
    }
    int flow(int s, int t)
    {
        int res = 0;
        while (bfs(s, t))
        {
            fill(all(it), 0);
            while (int f = dfs(s, t, LLONG_MAX)) res += f;
        }
        return res;
    }
};
```

FILE: templates/graph/min_cost_flow.cpp

```
// min cost max flow (successive shortest paths with spfa; costs may be negative but no
negative cycles)
// MCMF f(n); f.add(u, v, cap, cost); auto r = f.flow(s, t, maxFlow = INF) -> {flow,
cost}
struct MCMF
{
    struct E { int to, cap, cost; };
    int n;
    vector<E> e;
    vvi g;
    MCMF(int n) : n(n), g(n) {}
    void add(int u, int v, int cap, int cost)
    {
        g[u].pb(e.size()), e.pb({v, cap, cost});
        g[v].pb(e.size()), e.pb({u, 0, -cost});
    }
    pi flow(int s, int t, int maxFlow = LLONG_MAX)
    {
        int fl = 0, cost = 0;
```

```
        while (fl < maxFlow)
        {
            vi d(n, LLONG_MAX), pe(n, -1);
            vector<char> inq(n, 0);
            deque<int> q;
            d[s] = 0, q.pb(s), inq[s] = 1;
            while (!q.empty())
            {
                int v = q.front(); q.pop_front();
                inq[v] = 0;
                for (int id : g[v])
                    if (e[id].cap > 0 && d[v] + e[id].cost < d[e[id].to])
                    {
                        d[e[id].to] = d[v] + e[id].cost, pe[e[id].to] = id;
                        if (!inq[e[id].to]) inq[e[id].to] = 1, q.pb(e[id].to);
                    }
            }
            if (d[t] == LLONG_MAX) break;
            int push = maxFlow - fl;
            for (int v = t; v != s; v = e[pe[v] ^ 1].to) push = min(push,
e[pe[v]].cap);
            for (int v = t; v != s; v = e[pe[v] ^ 1].to) e[pe[v]].cap -= push, e[pe[v]
^ 1].cap += push;
            fl += push, cost += push * d[t];
        }
        return {fl, cost};
    }
};
```

FILE: templates/graph/dominator_tree.cpp

```
// dominator tree (lengauer tarjan). dominator d(n); d.add(u, v); vi idom =
d.build(root)
// idom[root] = root, unreachable vertices get -1
struct dominator
{
    int n;
    vvi g, rg, bucket;
    vi arr, par, rev, sdom, dom, dsu, lbl;
    int t = 0;
    dominator(int n) : n(n), g(n), rg(n), bucket(n), arr(n, -1), par(n), rev(n),
sdom(n), dom(n), dsu(n), lbl(n) {}
    void add(int u, int v) { g[u].pb(v); }
    int find(int u, int x = 0)
    {
        if (u == dsu[u]) return x ? -1 : u;
        int v = find(dsu[u], x + 1);
        if (v < 0) return u;
        if (sdom[lbl[dsu[u]]] < sdom[lbl[u]]) lbl[u] = lbl[dsu[u]];
        dsu[u] = v;
        return x ? v : lbl[u];
    }
    void dfs(int root)
    {
        vpi st = {{root, -1}};
        while (!st.empty())
        {
            int u = st.back().first, p = st.back().second;
            st.pop_back();
            if (arr[u] >= 0) continue;
```

```
            arr[u] = t, rev[t] = u, par[t] = (p < 0) ? 0 : arr[p];
            lbl[t] = sdom[t] = dsu[t] = t;
            t++;
            for (int w : g[u])
            {
                if (arr[w] < 0) st.pb({w, u});
            }
        }
        rep(u, 0, n) if (arr[u] >= 0) for (int w : g[u]) rg[arr[w]].pb(arr[u]);
    }
    vi build(int root)
    {
        dfs(root);
        repb(i, t - 1, 0)
        {
            for (int w : rg[i]) sdom[i] = min(sdom[i], sdom[find(w)]);
            if (i) bucket[sdom[i]].pb(i);
            for (int w : bucket[i])
            {
                int v = find(w);
                dom[w] = (sdom[v] == sdom[w]) ? sdom[w] : v;
            }
            if (i > 1) dsu[i] = par[i];
        }
        rep(i, 1, t) if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
        vi idom(n, -1);
        rep(i, 0, t) idom[rev[i]] = rev[dom[i]];
        idom[root] = root;
        return idom;
    }
};
```

17.9 Trees

KEYWORDS: trees template code

FILE: templates/trees/binary_lifting_lca.cpp

```
const int LOG = 20;
vvi up;
vi depth;

void build_lca(const vvi &g, int root)
{
    int n = g.size();
    up.assign(n, vi(LOG, root));
    depth.assign(n, 0);
    vector<char> vis(n, 0);
    vi st = {root};
    vis[root] = 1;
    while (!st.empty())
    {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
        {
            if (vis[v]) continue;
            vis[v] = 1;
            up[v][0] = u; //2^0 th ancestor = parent
            depth[v] = depth[u] + 1;
```

```
            rep(j, 1, LOG) up[v][j] = up[up[v][j - 1]][j - 1];
            st.pb(v);
        }
    }
}

int kth_ancestor(int u, int k)
{
    if (k > depth[u]) return -1;
    rep(j, 0, LOG) if (k >> j & 1) u = up[u][j]; //19 = 16+2+1
    return u;
}

int get_lca(int u, int v)
{
    if (depth[u] < depth[v]) swap(u, v);
    u = kth_ancestor(u, depth[u] - depth[v]);
    if (u == v) return v; //v is the parent of u, so lca(u,v)=v
    for (int j = LOG - 1; j >= 0; j--)
    {
        if (up[u][j] != up[v][j]) //those ancestors are not the same, so below the lca
        {
            u = up[u][j];
            v = up[v][j];
        }
    }
    return up[u][0]; //now they are just below lca
}

int dist(int u, int v)
{
    return depth[u] + depth[v] - 2 * depth[get_lca(u, v)];
}

/*
void solve()
{
    int n;
    cin >> n;
    vvi g(n + 1);
    rep(i, 0, n - 1)
    {
        int u, v;
        cin >> u >> v;
        g[u].pb(v);
        g[v].pb(u);
    }
    build_lca(g, 1);
    int q;
    cin >> q;
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << get_lca(u, v) << nl;
    }
}
*/
```

FILE: templates/trees/rerooting.cpp

```
// rerooting dp: answer for EVERY root in O(n). example: sum of distances from each
vertex to all others
// step 1: dfs from root 0, down[v] = answer inside v's subtree (here: sum of depths
below v), sz[v]
// step 2: top-down, ans[child] from ans[parent]: moving the root from p to c, the
sz[c] vertices of c's
//         subtree get 1 closer and the other n - sz[c] get 1 farther: ans[c] = ans[p]
- sz[c] + (n - sz[c])
// to adapt: change what down[] stores and the formula that moves the root across one
edge
vi rerootSumDist(const vvi &g) // vertices 0..n-1
{
    int n = g.size();
    vi par(n, -1), order, sz(n, 1), down(n, 0), ans(n, 0);
    order.pb(0), par[0] = 0;
    rep(i, 0, n) for (int u : g[order[i]]) if (u != par[order[i]]) par[u] = order[i],
order.pb(u); // bfs order
    par[0] = -1;
    repb(i, n - 1, 1) { int v = order[i], p = par[v]; sz[p] += sz[v], down[p] +=
down[v] + sz[v]; } // bottom-up
    ans[0] = down[0];
    rep(i, 1, n) { int v = order[i]; ans[v] = ans[par[v]] - sz[v] + (n - sz[v]); } //
top-down
    return ans;
}
```

FILE: templates/trees/tree_diameter.cpp

```
// tree diameter by two bfs. returns {length (edges), {end a, end b}}; works for any
node labels 0..n-1 / 1..n
pair<int, pi> diameter(const vvi &g, int start)
{
    auto bfs = [&](int s, vi &d)
    {
        d.assign(g.size(), -1);
        queue<int> q;
        d[s] = 0, q.push(s);
        int far = s;
        while (!q.empty())
        {
            int v = q.front(); q.pop();
            if (d[v] > d[far]) far = v;
            for (int u : g[v]) if (d[u] == -1) d[u] = d[v] + 1, q.push(u);
        }
        return far;
    };
    vi d1, d2;
    int a = bfs(start, d1), b = bfs(a, d2);
    return {d2[b], {a, b}};
}
// for every v: farthest distance = max(dist(a, v), dist(b, v))
```

FILE: templates/trees/sum_of_all_pair_distances.cpp

```
// sum of distances over all unordered pairs of a tree: every edge is used by sz * (n -
sz) paths
// g = adjacency list (any labels 0..n-1 or 1..n with g.size() = n + 1), root = any
```

```
vertex. iterative
int sumAllPairDistances(const vvi &g, int root, int n)
{
    vi par(g.size(), -1), order = {root}, sz(g.size(), 1);
    rep(i, 0, (int)order.size())
    {
        int v = order[i];
        for (int u : g[v]) if (u != par[v]) par[u] = v, order.pb(u);
    }
    int res = 0;
    repb(i, (int)order.size() - 1, 1)
    {
        int v = order[i];
        sz[par[v]] += sz[v];
        res += sz[v] * (n - sz[v]);
    }
    return res;
}
```

FILE: templates/trees/hld.cpp

```
// heavy light decomposition. HLD h(g, root); positions h.pos[v] in [0, n) (subtree of
v = [pos[v], pos[v] + sz[v] - 1])
// h.path(u, v, f): calls f(l, r) for O(log n) position ranges covering the path
(vertex values)
// for edge values store the edge at its lower vertex and call h.path(u, v, f, true)
struct HLD
{
    int n;
    vi par, heavy, head, pos, dep, sz;
    HLD(const vvi &g, int root) : n(g.size()), par(n, -1), heavy(n, -1), head(n),
pos(n), dep(n, 0), sz(n, 1)
    {
        vi order = {root};
        rep(i, 0, (int)order.size())
        {
            int v = order[i];
            for (int u : g[v]) if (u != par[v]) par[u] = v, dep[u] = dep[v] + 1,
order.pb(u);
        }
        repb(i, (int)order.size() - 1, 1)
        {
            int v = order[i], p = par[v];
            sz[p] += sz[v];
            if (heavy[p] == -1 || sz[v] > sz[heavy[p]]) heavy[p] = v;
        }
        int t = 0; // dfs order, heavy child first, iterative
        vi st = {root};
        while (!st.empty())
        {
            int v = st.back(); st.pop_back();
            for (int u = v; u != -1; u = heavy[u])
            {
                head[u] = v, pos[u] = t++;
                for (int w : g[u]) if (w != par[u] && w != heavy[u]) st.pb(w);
            }
        }
    }
    template <class F> void path(int u, int v, F f, bool edges = false)
    {
```

```
        while (head[u] != head[v])
        {
            if (dep[head[u]] < dep[head[v]]) swap(u, v);
            f(pos[head[u]], pos[u]);
            u = par[head[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (pos[u] + edges <= pos[v]) f(pos[u] + edges, pos[v]);
    }
    int lca(int u, int v)
    {
        while (head[u] != head[v])
        {
            if (dep[head[u]] < dep[head[v]]) swap(u, v);
            u = par[head[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }
};
```

FILE: templates/trees/centroid_decomposition.cpp

```
// centroid decomposition. Centroid cd(g); cd.cpar[v] = parent in the centroid tree (-1
for the root)
// cd.anc[v] = list of {centroid ancestor, distance} (including v itself at distance
0), O(n log n) memory
// typical use: nearest marked vertex -> best[c] = min dist; mark v: for (c, d) in
anc[v] best[c] = min(best[c], d)
struct Centroid
{
    int n;
    const vvi &g;
    vi sz, cpar;
    vector<char> removed;
    vector<vpi> anc;
    Centroid(const vvi &g) : n(g.size()), g(g), sz(n), cpar(n, -1), removed(n, 0),
anc(n) { build(0, -1); }
    void calc(int s) // subtree sizes in the current piece, iterative
    {
        vi order = {s}, pr = {-1};
        for (int i = 0; i < (int)order.size(); i++)
        {
            int v = order[i];
            for (int u : g[v]) if (u != pr[i] && !removed[u]) order.pb(u), pr.pb(v);
        }
        repb(i, (int)order.size() - 1, 0)
        {
            int v = order[i];
            sz[v] = 1;
            for (int u : g[v]) if (u != pr[i] && !removed[u]) sz[v] += sz[u];
        }
    }
    int centroid(int s)
    {
        calc(s);
        int total = sz[s], v = s, p = -1;
        while (true)
        {
            int nxt = -1;
            for (int u : g[v]) if (u != p && !removed[u] && sz[u] < sz[v] && sz[u] * 2
```

```
> total) nxt = u;
            if (nxt == -1) return v;
            p = v, v = nxt;
        }
    }
    void build(int s, int p)
    {
        int c = centroid(s);
        cpar[c] = p;
        vi order = {c}, dist = {0}, pr = {-1}; // distances from c inside the piece
        for (int i = 0; i < (int)order.size(); i++)
        {
            int v = order[i];
            anc[v].pb({c, dist[i]});
            for (int u : g[v]) if (u != pr[i] && !removed[u]) order.pb(u),
dist.pb(dist[i] + 1), pr.pb(v);
        }
        removed[c] = 1;
        for (int u : g[c]) if (!removed[u]) build(u, c);
    }
};
```

17.10 Strings

KEYWORDS: strings template code

FILE: templates/strings/hashing/double_hash.cpp

```
// double hashing (mod 1e9+7 and 1e9+9). call preCompute(max length) once, then Hash
h(s)
// h.getHash(l, r) = hash of s[l..r] (0-based, inclusive), {0, 0} for an empty range
// works for any characters; compare hashes of two strings with the same preCompute
constexpr int MOD1 = 1000000007, MOD2 = 1000000009, p1 = 133, p2 = 137;
vi pow1, pow2;
void preCompute(int n) // do not forget to call it !!
{
    pow1.resize(n + 1), pow2.resize(n + 1);
    pow1[0] = pow2[0] = 1;
    rep(i, 1, n + 1)
    {
        pow1[i] = pow1[i - 1] * p1 % MOD1;
        pow2[i] = pow2[i - 1] * p2 % MOD2;
    }
}
struct Hash
{
    vi hash1, hash2;
    Hash() {}
    Hash(const string &s)
    {
        int n = s.size();
        hash1.assign(n + 1, 0), hash2.assign(n + 1, 0);
        rep(i, 0, n)
        {
            int c = (unsigned char)s[i];
            hash1[i + 1] = (hash1[i] * p1 + c) % MOD1;
            hash2[i + 1] = (hash2[i] * p2 + c) % MOD2;
        }
    }
    pi getHash(int l, int r)
```

```
    {
        if (l > r) return {0, 0};
        int len = r - l + 1;
        int h1 = (hash1[r + 1] - hash1[l] * pow1[len] % MOD1 + MOD1) % MOD1;
        int h2 = (hash2[r + 1] - hash2[l] * pow2[len] % MOD2 + MOD2) % MOD2;
        return {h1, h2};
    }
};
```

FILE: templates/strings/hashing/palindrome_hash.cpp

Needs: strings/hashing/double_hash.cpp above it

```
// forward + backward hash (needs double_hash.cpp above it, and preCompute(n) called
first)
// PalHash ph(s); ph.fwd(l, r) = hash of s[l..r], ph.bwd(l, r) = hash of s[l..r] read
backwards
// ph.isPal(l, r): is s[l..r] a palindrome, O(1). 0-based, inclusive
struct PalHash
{
    int n;
    Hash f, g; // g is built on the reversed string
    PalHash(const string &s) : n(s.size()), f(s), g(string(s.rbegin(), s.rend())) {}
    pi fwd(int l, int r) { return f.getHash(l, r); }
    pi bwd(int l, int r) { return g.getHash(n - 1 - r, n - 1 - l); }
    bool isPal(int l, int r) { return l >= r || fwd(l, r) == bwd(l, r); }
};
```

FILE: templates/strings/hashing/lcp_double_hash.cpp

Needs: strings/hashing/double_hash.cpp above it

```
// lcp with hashing (needs double_hash.cpp above it, and preCompute(max length) called
first)
// works on two different strings or on one string: lcp(ha, i, hb, j) = longest common
prefix of a[i..] and b[j..]
// same string: lcp(h, i, h, j). optional lim caps the answer (e.g. compare only up to
some length)
// a[i..] < b[j..] in dictionary order: k = lcp(...); then compare a[i + k] and b[j +
k] (if one ran out, it is smaller)
int lcp(Hash &ha, int i, Hash &hb, int j, int lim = LLONG_MAX)
{
    int na = (int)ha.hash1.size() - 1, nb = (int)hb.hash1.size() - 1;
    int lo = 0, hi = min({na - i, nb - j, lim});
    while (lo < hi)
    {
        int mid = (lo + hi + 1) / 2;
        if (ha.getHash(i, i + mid - 1) == hb.getHash(j, j + mid - 1)) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
```

FILE: templates/strings/hashing/dynamic_hash.cpp

```
// hashing with point updates: change a character, compare / test palindrome on any
range in O(log n)
// DynHash h(s); h.set(i, c); h.get(l, r) = hash of s[l..r]
// segment tree: each node keeps the hash of its segment, merge = left * B^(length of
right) + right
struct DynHash
```

```
{
    static const int M1 = 1000000007, M2 = 1000000009, B = 131;
    struct node { int h1, h2, len; };
    int n;
    vi pw1, pw2;
    vector<node> seg;
    node identity() { return {0, 0, 0}; }
    node makeLeaf(int c) { return {c % M1, c % M2, 1}; }
    node merge(node L, node R)
    {
        return {(L.h1 * pw1[R.len] + R.h1) % M1, (L.h2 * pw2[R.len] + R.h2) % M2, L.len
+ R.len};
    }
    void build(int idx, int l, int r, const string &s)
    {
        if (l == r) { seg[idx] = makeLeaf((unsigned char)s[l]); return; }
        int mid = l + (r - l) / 2;
        build(2 * idx + 1, l, mid, s);
        build(2 * idx + 2, mid + 1, r, s);
        seg[idx] = merge(seg[2 * idx + 1], seg[2 * idx + 2]);
    }
    void pointUpdate(int idx, int l, int r, int pos, int c)
    {
        if (l == r) { seg[idx] = makeLeaf(c); return; }
        int mid = l + (r - l) / 2;
        if (pos <= mid) pointUpdate(2 * idx + 1, l, mid, pos, c);
        else pointUpdate(2 * idx + 2, mid + 1, r, pos, c);
        seg[idx] = merge(seg[2 * idx + 1], seg[2 * idx + 2]);
    }
    node query(int idx, int l, int r, int ql, int qr)
    {
        if (l > qr || r < ql) return identity();
        if (ql <= l && r <= qr) return seg[idx];
        int mid = l + (r - l) / 2;
        return merge(query(2 * idx + 1, l, mid, ql, qr), query(2 * idx + 2, mid + 1, r,
ql, qr));
    }
    DynHash(const string &s) : n(s.size()), pw1(n + 1, 1), pw2(n + 1, 1), seg(4 *
max(1LL, (int)s.size()))
    {
        rep(i, 1, n + 1) pw1[i] = pw1[i - 1] * B % M1, pw2[i] = pw2[i - 1] * B % M2;
        if (n) build(0, 0, n - 1, s);
    }
    void set(int i, int c) { pointUpdate(0, 0, n - 1, i, c); } // s[i] = c
    pi get(int l, int r) // hash of s[l..r]
    {
        if (l > r) return {0, 0};
        node x = query(0, 0, n - 1, l, r);
        return {x.h1, x.h2};
    }
};
// palindrome check on s[l..r] with updates: keep DynHash f(s) and g(reversed s);
// s[l..r] is a palindrome  <=>  f.get(l, r) == g.get(n - 1 - r, n - 1 - l)
// the same with updates: DynPal p(s); p.set(i, c); p.isPal(l, r); p.bwd(l, r) = hash
of s[l..r] read backwards
struct DynPal
{
    int n;
    DynHash f, g;
    DynPal(const string &s) : n(s.size()), f(s), g(string(s.rbegin(), s.rend())) {}
```

```
    void set(int i, int c) { f.set(i, c), g.set(n - 1 - i, c); }
    pi fwd(int l, int r) { return f.get(l, r); }
    pi bwd(int l, int r) { return g.get(n - 1 - r, n - 1 - l); }
    bool isPal(int l, int r) { return l >= r || fwd(l, r) == bwd(l, r); }
};
```

FILE: templates/strings/hashing/xor_hash.cpp

```
// hashing sets and multisets. give every value a random 64-bit number val(x)
// xor of val over a range: every value appears an even number of times <=> 0 (with
high probability)
// sum of val over a range: equal multisets <=> equal sums. sets {1..k}: compare with
prefix sums of val(1..k)
mt19937_64 hrng(chrono::steady_clock::now().time_since_epoch().count());
map<int, unsigned long long> hval;
unsigned long long val(int x)
{
    auto it = hval.find(x);
    if (it != hval.end()) return it->second;
    return hval[x] = hrng();
}
```

FILE: templates/strings/hashing/hash_2d.cpp

```
// 2D hash: hash rows with base b1, then the column of row hashes with base b2 (one
modulus 2^61-1)
// Hash2D h(grid); h.get(r1, c1, r2, c2) = hash of the sub-rectangle (0-based,
inclusive)
// equal rectangles of the same size give equal hashes; compare only same-size
rectangles
struct Hash2D
{
    typedef unsigned long long ull;
    static const ull M = (1ULL << 61) - 1;
    static ull mul(ull a, ull b) { __int128 r = (__int128)a * b; ull x = (ull)(r & M) +
(ull)(r >> 61); return x >= M ? x - M : x; }
    ull b1 = 131, b2 = 1000003;
    int n, m;
    vector<vector<ull>> h;
    vector<ull> p1, p2;
    Hash2D(const vector<string> &g)
    {
        n = g.size(), m = g[0].size();
        p1.assign(m + 1, 1), p2.assign(n + 1, 1);
        rep(i, 1, m + 1) p1[i] = mul(p1[i - 1], b1);
        rep(i, 1, n + 1) p2[i] = mul(p2[i - 1], b2);
        h.assign(n + 1, vector<ull>(m + 1, 0));
        rep(i, 0, n) rep(j, 0, m)
        {
            ull v = mul(h[i + 1][j], b1) + (unsigned char)g[i][j];
            h[i + 1][j + 1] = v % M;
        }
        rep(i, 0, n) rep(j, 0, m + 1) h[i + 1][j] = (h[i + 1][j] + mul(h[i][j], b2)) %
M;
    }
    ull sub(ull a, ull b) { return a >= b ? a - b : a + M - b; }
    ull get(int r1, int c1, int r2, int c2)
    {
        r2++, c2++;
```

```
        int w = c2 - c1, ht = r2 - r1;
        ull A = sub(h[r2][c2], mul(h[r2][c1], p1[w]));
        ull B = sub(h[r1][c2], mul(h[r1][c1], p1[w]));
        return sub(A, mul(B, p2[ht]));
    }
};
```

FILE: templates/strings/hashing/substring_hash.cpp

```
// single-mod substring hash, O(n) build, O(1) query. for safety prefer double_hash.cpp
// SingleHash h(s); h.get(l, r) = hash of s[l..r] (0-based, inclusive)
struct SingleHash
{
    static const int M = 1000000007, B = 131;
    vi h, pw;
    SingleHash(const string &s)
    {
        int n = s.size();
        h.assign(n + 1, 0), pw.assign(n + 1, 1);
        rep(i, 0, n)
        {
            h[i + 1] = (h[i] * B + (unsigned char)s[i]) % M;
            pw[i + 1] = pw[i] * B % M;
        }
    }
    int get(int l, int r) { return (h[r + 1] - h[l] * pw[r - l + 1] % M + M) % M; }
};
```

FILE: templates/strings/hashing/polynomial_rolling_hash.cpp

```
// hash of a whole string (mod MOD). for substrings use double_hash.cpp
int getHash(const string &key, int p = 131)
{
    int value = 0;
    for (char c : key) value = (value * p + (unsigned char)c) % MOD;
    return value;
}
```

FILE: templates/strings/hashing/palindrome_tracker.cpp

```
struct palindrome_tracker
{
    const int p1 = 31, m1 = 1e9 + 7, inv1 = 129032259;
    const int p2 = 37, m2 = 1e9 + 9, inv2 = 297297300;
    int f1 = 0, r1 = 0, pw1 = 1;
    int f2 = 0, r2 = 0, pw2 = 1;
    string s;

    void push_back(char ch)
    {
        int c = ch;
        f1 = (f1 * p1 + c) % m1;
        f2 = (f2 * p2 + c) % m2;
        r1 = (r1 + c * pw1) % m1;
        r2 = (r2 + c * pw2) % m2;
        pw1 = pw1 * p1 % m1;
        pw2 = pw2 * p2 % m2;
        s.pb(ch);
    }
```

```
    void pop_back()
    {
        int c = s.back();
        s.pop_back();
        pw1 = pw1 * inv1 % m1;
        pw2 = pw2 * inv2 % m2;
        r1 = ((r1 - c * pw1) % m1 + m1) % m1;
        r2 = ((r2 - c * pw2) % m2 + m2) % m2;
        f1 = (f1 - c + m1) % m1 * inv1 % m1;
        f2 = (f2 - c + m2) % m2 * inv2 % m2;
    }

    bool is_palindrome() const { return f1 == r1 && f2 == r2; }
    bool empty() const { return s.empty(); }
    int size() const { return s.size(); }
};
```

FILE: templates/strings/kmp.cpp

```
// prefix function (kmp). pi[i] = longest proper prefix of s[0..i] that is also a
suffix
// find(text, pat) = start positions of every occurrence. period of s = n - pi[n-1] (if
it divides n)
vi prefixFunction(const string &s)
{
    int n = s.size();
    vi p(n, 0);
    rep(i, 1, n)
    {
        int j = p[i - 1];
        while (j > 0 && s[i] != s[j]) j = p[j - 1];
        if (s[i] == s[j]) j++;
        p[i] = j;
    }
    return p;
}
vi kmpFind(const string &text, const string &pat)
{
    vi res;
    if (pat.empty()) return res;
    vi p = prefixFunction(pat + '\x01' + text); // separator must not appear in the
strings
    int m = pat.size();
    rep(i, m + 1, (int)p.size()) if (p[i] == m) res.pb(i - 2 * m);
    return res;
}
// kmp automaton: aut[state][c] for c in 0..25, state = matched length (0..m)
vvi kmpAutomaton(const string &s)
{
    int m = s.size();
    vi p = prefixFunction(s);
    vvi aut(m + 1, vi(26, 0));
    rep(i, 0, m + 1) rep(c, 0, 26)
    {
        if (i < m && s[i] - 'a' == c) aut[i][c] = i + 1;
        else aut[i][c] = i ? aut[p[i - 1]][c] : 0;
    }
```

```
    return aut;
}
```

FILE: templates/strings/z_function.cpp

```
// z function: z[i] = longest common prefix of s and s[i..]; z[0] = n
// occurrences of pat in text: z of pat + '#' + text, positions with z == |pat|
vi zFunction(const string &s)
{
    int n = s.size();
    vi z(n, 0);
    if (n) z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++)
    {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
```

FILE: templates/strings/manacher.cpp

```
// manacher: d1[i] = number of odd palindromes centered at i (radius incl. center),
// d2[i] = number of even palindromes centered between i-1 and i. O(n)
// longest palindrome: max over 2*d1[i]-1 and 2*d2[i]
pair<vi, vi> manacher(const string &s)
{
    int n = s.size();
    vi d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return {d1, d2};
}
// is s[l..r] a palindrome in O(1) after manacher
bool isPal(const vi &d1, const vi &d2, int l, int r)
{
    int len = r - l + 1, mid = (l + r + 1) / 2;
    return len % 2 ? d1[(l + r) / 2] * 2 - 1 >= len : d2[mid] * 2 >= len;
}
```

FILE: templates/strings/palindromic_tree.cpp

```
// palindromic tree (eertree). Eertree t(n); t.add(c) for each char of s in order
('a'..'z')
// t.add returns the node of the longest palindromic suffix. node 0: len -1 root, node
1: len 0 root
```

```
// number of distinct palindromes = t.len.size() - 2. cnt[v] (after t.count()) =
occurrences of node v's palindrome
struct Eertree
{
    vector<array<int32_t, 26>> nx; // int32 keeps memory low
    vi len, link, cnt;
    string s;
    int last = 1;
    Eertree(int n = 0)
    {
        nx.reserve(n + 2);
        nx.pb({}), nx.pb({});
        nx[0].fill(0), nx[1].fill(0);
        len = {-1, 0}, link = {0, 0}, cnt = {0, 0};
    }
    int getLink(int v, int i)
    {
        while (i - 1 - len[v] < 0 || s[i - 1 - len[v]] != s[i]) v = link[v];
        return v;
    }
    int add(char ch)
    {
        s.pb(ch);
        int i = s.size() - 1, c = ch - 'a';
        int v = getLink(last, i);
        if (!nx[v][c])
        {
            int u = nx.size();
            nx.pb({}), nx.back().fill(0);
            len.pb(len[v] + 2), cnt.pb(0);
            link.pb(len.back() == 1 ? 1 : nx[getLink(link[v], i)][c]);
            nx[v][c] = u;
        }
        last = nx[v][c];
        cnt[last]++;
        return last;
    }
    void count() { repb(v, (int)len.size() - 1, 2) cnt[link[v]] += cnt[v]; }
};
```

FILE: templates/strings/aho_corasick.cpp

```
// aho corasick over 'a'..'z'. AhoCorasick ac; id = ac.add(word); ac.build();
// walk: v = ac.go[v][c]. ac.out[v] = number of words ending at v or at any suffix link
of v (after build)
// to list matches, follow ac.dict[v] (next node on the suffix-link chain that ends a
word)
// ac.bfs = nodes in bfs order (increasing depth): to push values up the suffix links,
loop it backwards
struct AhoCorasick
{
    vector<array<int32_t, 26>> go; // int32 keeps memory low
    vi link, out, dict, endHere, bfs;
    AhoCorasick() { node(); }
    int node()
    {
        go.pb({}), go.back().fill(-1);
        link.pb(0), out.pb(0), dict.pb(-1), endHere.pb(0);
        return go.size() - 1;
    }
```

```
    int add(const string &w)
    {
        int v = 0;
        for (char ch : w)
        {
            int c = ch - 'a';
            if (go[v][c] == -1) { int u = node(); go[v][c] = u; }
            v = go[v][c];
        }
        endHere[v]++, out[v]++;
        return v;
    }
    void build() // bfs, turns go[][] into a full automaton
    {
        queue<int> q;
        bfs = {0};
        rep(c, 0, 26)
        {
            if (go[0][c] == -1) go[0][c] = 0;
            else link[go[0][c]] = 0, q.push(go[0][c]);
        }
        while (!q.empty())
        {
            int v = q.front(); q.pop();
            bfs.pb(v);
            out[v] += out[link[v]];
            dict[v] = endHere[link[v]] ? link[v] : dict[link[v]];
            rep(c, 0, 26)
            {
                int u = go[v][c];
                if (u == -1) go[v][c] = go[link[v]][c];
                else link[u] = go[link[v]][c], q.push(u);
            }
        }
    }
};
```

FILE: templates/strings/suffix_array.cpp

```
// suffix array O(n log n) + lcp (kasai). sa = sorted suffix starts, rk = rank of each
suffix,
// lcp[i] = lcp(sa[i], sa[i+1]) for i < n-1. lcp of any two suffixes: range min on lcp
(sparse table)
// distinct substrings = n(n+1)/2 - sum(lcp)
struct SuffixArray
{
    vi sa, rk, lcp;
    SuffixArray(const string &str)
    {
        string s = str + '\x01'; // sentinel smaller than every character
        int n = s.size(), classes = 256;
        sa.assign(n, 0), rk.assign(n, 0);
        vi cnt(max(classes, n), 0), tmp(n);
        rep(i, 0, n) cnt[(unsigned char)s[i]]++;
        rep(i, 1, classes) cnt[i] += cnt[i - 1];
        repb(i, n - 1, 0) sa[--cnt[(unsigned char)s[i]]] = i;
        rk[sa[0]] = 0, classes = 1;
        rep(i, 1, n) rk[sa[i]] = (s[sa[i]] == s[sa[i - 1]]) ? classes - 1 : classes++;
        for (int h = 1; h < n; h <<= 1)
        {
```

```
            rep(i, 0, n) tmp[i] = (sa[i] - h + n) % n;
            fill(cnt.begin(), cnt.begin() + classes, 0);
            rep(i, 0, n) cnt[rk[tmp[i]]]++;
            rep(i, 1, classes) cnt[i] += cnt[i - 1];
            repb(i, n - 1, 0) sa[--cnt[rk[tmp[i]]]] = tmp[i];
            vi nr(n);
            nr[sa[0]] = 0, classes = 1;
            rep(i, 1, n)
            {
                pi cur = {rk[sa[i]], rk[(sa[i] + h) % n]}, prv = {rk[sa[i - 1]],
rk[(sa[i - 1] + h) % n]};
                nr[sa[i]] = (cur == prv) ? classes - 1 : classes++;
            }
            rk = nr;
            if (classes == n) break;
        }
        sa.erase(sa.begin()); // drop the sentinel
        n--;
        rk.assign(n, 0);
        rep(i, 0, n) rk[sa[i]] = i;
        lcp.assign(max(0LL, n - 1), 0);
        for (int i = 0, k = 0; i < n; i++)
        {
            if (rk[i] == n - 1) { k = 0; continue; }
            int j = sa[rk[i] + 1];
            while (i + k < n && j + k < n && str[i + k] == str[j + k]) k++;
            lcp[rk[i]] = k;
            if (k) k--;
        }
    }
};
```

FILE: templates/strings/suffix_automaton.cpp

```
// suffix automaton of one string. SAM sa(s.size()); for (char c : s) sa.extend(c -
'a');
// every substring = one path from state 0; distinct substrings = sum over v>0 of
len[v] - len[link[v]]
// occurrences: call sa.countOcc() -> cnt[v] = how many times the substrings of state v
occur
struct SAM
{
    vector<array<int32_t, 26>> nx; // int32 keeps memory low
    vi link, len, cnt;
    int last = 0;
    SAM(int n = 0)
    {
        nx.reserve(2 * n + 2), link.reserve(2 * n + 2), len.reserve(2 * n + 2),
cnt.reserve(2 * n + 2);
        newState(0, -1);
    }
    int newState(int l, int lk)
    {
        nx.pb({}), nx.back().fill(-1);
        len.pb(l), link.pb(lk), cnt.pb(0);
        return nx.size() - 1;
    }
    void extend(int c)
    {
        int cur = newState(len[last] + 1, 0), p = last;
```

```
        cnt[cur] = 1;
        while (p != -1 && nx[p][c] == -1) nx[p][c] = cur, p = link[p];
        if (p != -1)
        {
            int q = nx[p][c];
            if (len[p] + 1 == len[q]) link[cur] = q;
            else
            {
                int cl = newState(len[p] + 1, link[q]);
                nx[cl] = nx[q];
                while (p != -1 && nx[p][c] == q) nx[p][c] = cl, p = link[p];
                link[q] = link[cur] = cl;
            }
        }
        last = cur;
    }
    void countOcc() // push counts up the suffix links, longest states first
    {
        int n = len.size();
        vi order(n);
        iota(all(order), 0);
        sort(all(order), [&](int a, int b) { return len[a] > len[b]; });
        for (int v : order) if (link[v] >= 0) cnt[link[v]] += cnt[v];
    }
    int distinctSubstrings()
    {
        int res = 0;
        rep(v, 1, (int)len.size()) res += len[v] - len[link[v]];
        return res;
    }
};
```

FILE: templates/strings/generalized_sam.cpp

```
// generalized suffix automaton: for every string set s.last = 0, then s.ins(c) for
each char (c in 0..25)
// ins returns the state of the current prefix; len / link / nx as usual
struct sam
{
    vector<array<int32_t, 26>> nx;
    vector<int32_t> link, len;
    int last = 0;
    sam(int cap)
    {
        nx.reserve(cap), link.reserve(cap), len.reserve(cap);
        add(0, -1);
    }
    int add(int l, int lk)
    {
        array<int32_t, 26> e;
        e.fill(-1);
        nx.pb(e), len.pb(l), link.pb(lk);
        return len.size() - 1;
    }
    int ins(int c) // returns the state of the new prefix
    {
        int p = last;
        if (nx[p][c] >= 0) // the prefix already exists (generalized case)
        {
            int q = nx[p][c];
```

```
            if (len[p] + 1 == len[q]) return last = q;
            int cl = add(len[p] + 1, link[q]);
            nx[cl] = nx[q];
            while (p >= 0 && nx[p][c] == q) nx[p][c] = cl, p = link[p];
            link[q] = cl;
            return last = cl;
        }
        int cur = add(len[last] + 1, 0);
        while (p >= 0 && nx[p][c] < 0) nx[p][c] = cur, p = link[p];
        if (p >= 0)
        {
            int q = nx[p][c];
            if (len[p] + 1 == len[q]) link[cur] = q;
            else
            {
                int cl = add(len[p] + 1, link[q]);
                nx[cl] = nx[q];
                while (p >= 0 && nx[p][c] == q) nx[p][c] = cl, p = link[p];
                link[q] = cl, link[cur] = cl;
            }
        }
        return last = cur;
    }
};
```

FILE: templates/strings/subsequence_automaton.cpp

```
// nxt[i][c] = smallest j >= i with s[j] == c, or n. jump through it to test "is t a
subsequence of s[i..]"
// in O(|t|). lowercase letters; change 26 / 'a' for other alphabets
struct SubseqAutomaton
{
    int n;
    vector<array<int32_t, 26>> nxt;
    SubseqAutomaton(const string &s)
    {
        n = s.size();
        nxt.resize(n + 1);
        nxt[n].fill(n);
        repb(i, n - 1, 0)
        {
            nxt[i] = nxt[i + 1];
            nxt[i][s[i] - 'a'] = i;
        }
    }
    int matchEnd(const string &t, int from = 0) // index right after the greedy match
of t, or -1
    {
        int i = from;
        for (char c : t)
        {
            if (i >= n || nxt[i][c - 'a'] == n) return -1;
            i = nxt[i][c - 'a'] + 1;
        }
        return i;
    }
    bool isSubseq(const string &t) { return matchEnd(t) != -1; }
};
```

FILE: templates/strings/string_dp.cpp

```
// classic string dps
// edit distance (insert / delete / replace, cost 1 each), O(nm)
int editDistance(const string &a, const string &b)
{
    int n = a.size(), m = b.size();
    vvi d(n + 1, vi(m + 1));
    rep(i, 0, n + 1) d[i][0] = i;
    rep(j, 0, m + 1) d[0][j] = j;
    rep(i, 1, n + 1) rep(j, 1, m + 1)
        d[i][j] = min({d[i - 1][j] + 1, d[i][j - 1] + 1, d[i - 1][j - 1] + (a[i - 1] !=
b[j - 1])});
    return d[n][m];
}
// longest common subsequence, returns one LCS string
string lcsString(const string &a, const string &b)
{
    int n = a.size(), m = b.size();
    vvi d(n + 1, vi(m + 1, 0));
    repb(i, n - 1, 0) repb(j, m - 1, 0) d[i][j] = a[i] == b[j] ? d[i + 1][j + 1] + 1 :
max(d[i + 1][j], d[i][j + 1]);
    string res;
    int i = 0, j = 0;
    while (i < n && j < m)
    {
        if (a[i] == b[j]) res += a[i], i++, j++;
        else if (d[i + 1][j] >= d[i][j + 1]) i++;
        else j++;
    }
    return res;
}
// number of distinct subsequences (including the empty one) mod MOD, O(n)
int distinctSubsequences(const string &s)
{
    vi last(256, -1);
    int n = s.size();
    vi dp(n + 1);
    dp[0] = 1;
    rep(i, 1, n + 1)
    {
        unsigned char c = s[i - 1];
        dp[i] = dp[i - 1] * 2 % MOD;
        if (last[c] != -1) dp[i] = (dp[i] - dp[last[c]] + MOD) % MOD;
        last[c] = i - 1;
    }
    return dp[n];
}
```

FILE: templates/strings/brackets.cpp

```
// bracket sequences. balanced(s): every prefix has #'(' >= #')' and the totals are
equal
// matchPairs(s): mt[i] = index of the bracket matched with i, or -1. works for ()[]{}
mixed
bool balanced(const string &s)
{
    int bal = 0;
    for (char c : s)
```

```
    {
        bal += c == '(' ? 1 : -1;
        if (bal < 0) return false;
    }
    return bal == 0;
}
vi matchPairs(const string &s)
{
    int n = s.size();
    vi mt(n, -1), st;
    string open = "([{", close = ")]}";
    rep(i, 0, n)
    {
        if (open.find(s[i]) != string::npos) st.pb(i);
        else if (close.find(s[i]) != string::npos)
        {
            if (!st.empty() && open.find(s[st.back()]) == close.find(s[i])) mt[i] =
st.back(), mt[st.back()] = i, st.pop_back();
            else st.clear(); // a wrong closer breaks everything before it
        }
    }
    return mt;
}
// number of balanced sequences of length 2n = catalan(n) = C(2n, n) / (n + 1)
// sequences of length n starting with a fixed prefix (balance b, open so far o):
reflection,
// C(rem, k) - C(rem, k + b + 1) with rem = n - |prefix|, k = n/2 - o (closers to place
= rem - k)
```

FILE: templates/strings/expression_parser.cpp

```
// recursive descent: + - * / and parentheses, integers, left-to-right, usual priority
// pos must be 0 before evaluate(s). handles unary minus: "-(2+3)*4"
int pos;
int parseExpr(const string &s);
int parseAtom(const string &s)
{
    if (s[pos] == '-') { pos++; return -parseAtom(s); }
    if (s[pos] == '(')
    {
        pos++;
        int v = parseExpr(s);
        pos++; // skip ')'
        return v;
    }
    int v = 0;
    while (pos < (int)s.size() && isdigit(s[pos])) v = v * 10 + (s[pos++] - '0');
    return v;
}
int parseTerm(const string &s)
{
    int v = parseAtom(s);
    while (pos < (int)s.size() && (s[pos] == '*' || s[pos] == '/'))
    {
        char op = s[pos++];
        int r = parseAtom(s);
        v = op == '*' ? v * r : v / r;
    }
    return v;
}
```

```
int parseExpr(const string &s)
{
    int v = parseTerm(s);
    while (pos < (int)s.size() && (s[pos] == '+' || s[pos] == '-'))
    {
        char op = s[pos++];
        int r = parseTerm(s);
        v = op == '+' ? v + r : v - r;
    }
    return v;
}
int evaluate(string s)
{
    s.erase(remove(all(s), ' '), s.end());
    pos = 0;
    return parseExpr(s);
}
```

FILE: templates/strings/bigint.cpp

```
// non-negative big integers as decimal strings (most significant digit first)
// bigAdd, bigSub (a >= b), bigMulSmall (by an int up to ~1e17), bigMul (O(n*m)),
bigCmp, bigDivSmall
int bigCmp(const string &a, const string &b)
{
    if (a.size() != b.size()) return a.size() < b.size() ? -1 : 1;
    return a < b ? -1 : (a > b ? 1 : 0);
}
string strip(string s)
{
    int i = 0;
    while (i + 1 < (int)s.size() && s[i] == '0') i++;
    return s.substr(i);
}
string bigAdd(const string &a, const string &b)
{
    string r;
    int i = a.size() - 1, j = b.size() - 1, c = 0;
    while (i >= 0 || j >= 0 || c)
    {
        int d = c + (i >= 0 ? a[i--] - '0' : 0) + (j >= 0 ? b[j--] - '0' : 0);
        r += char('0' + d % 10), c = d / 10;
    }
    reverse(all(r));
    return r;
}
string bigSub(const string &a, const string &b)
{
    string r;
    int i = a.size() - 1, j = b.size() - 1, c = 0;
    while (i >= 0)
    {
        int d = a[i--] - '0' - c - (j >= 0 ? b[j--] - '0' : 0);
        c = d < 0, r += char('0' + (d + 10) % 10);
    }
    reverse(all(r));
    return strip(r);
}
string bigMulSmall(const string &a, int k)
{
```

```
    if (k == 0) return "0";
    string r;
    __int128 c = 0;
    repb(i, (int)a.size() - 1, 0)
    {
        c += (__int128)(a[i] - '0') * k;
        r += char('0' + (int)(c % 10)), c /= 10;
    }
    while (c) r += char('0' + (int)(c % 10)), c /= 10;
    reverse(all(r));
    return strip(r);
}
string bigMul(const string &a, const string &b)
{
    vi d(a.size() + b.size(), 0);
    repb(i, (int)a.size() - 1, 0) repb(j, (int)b.size() - 1, 0) d[i + j + 1] += (a[i] -
'0') * (b[j] - '0');
    repb(i, (int)d.size() - 1, 1) d[i - 1] += d[i] / 10, d[i] %= 10;
    string r;
    for (int x : d) r += char('0' + x);
    return strip(r);
}
string bigDivSmall(const string &a, int k, int &rem) // quotient, remainder in rem
{
    string q;
    rem = 0;
    for (char c : a)
    {
        rem = rem * 10 + (c - '0');
        q += char('0' + rem / k), rem %= k;
    }
    return strip(q);
}
```

FILE: templates/strings/min_rotation.cpp

```
// lexicographically smallest rotation, O(n). returns the start index
int minRotation(const string &s)
{
    int n = s.size(), i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n)
    {
        char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b) { k++; continue; }
        if (a > b) i += k + 1; else j += k + 1;
        if (i == j) j++;
        k = 0;
    }
    return min(i, j);
}
```

FILE: templates/strings/lyndon.cpp

```
// duval: lyndon factorization of s into s = w1 w2 ... wk with w1 >= w2 >= ... (each wi
is a lyndon word). O(n)
// returns the start index of each factor. smallest rotation: run on s + s, take the
factor start < n with the longest reach
vi duval(const string &s)
{
```

```
    int n = s.size(), i = 0;
    vi starts;
    while (i < n)
    {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j])
        {
            if (s[k] < s[j]) k = i; else k++;
            j++;
        }
        while (i <= k) starts.pb(i), i += j - k;
    }
    return starts;
}
```

FILE: templates/strings/bwt.cpp

```
// burrows-wheeler transform. bwt(s): s must end with a unique smallest char (e.g. '#'
or '$')
// last column of the sorted rotations. inverseBwt(t): give back s (with the marker at
the end)
string bwt(const string &s)
{
    int n = s.size();
    vi r(n);
    iota(all(r), 0);
    string d = s + s;
    sort(all(r), [&](int a, int b) { return d.compare(a, n, d, b, n) < 0; }); // O(n^2
log n): use SuffixArray for big n
    string t;
    for (int i : r) t += s[(i + n - 1) % n];
    return t;
}
string inverseBwt(const string &t, char marker = '#')
{
    int n = t.size();
    vi idx(n);
    iota(all(idx), 0);
    stable_sort(all(idx), [&](int a, int b) { return t[a] < t[b]; }); // idx[i] = row
of last column feeding sorted row i
    int row = find(all(t), marker) - t.begin();
    string s;
    rep(k, 0, n) row = idx[row], s += t[row];
    return s;
}
```

FILE: templates/strings/wildcard_match.cpp

Needs: math/fft.cpp above it

```
// pattern matching with wildcards '?' (in text and/or pattern) using fft (needs
math/fft.cpp: convolve)
// position i matches iff sum over j of p_j t_{i+j} (p_j - t_{i+j})^2 = 0, with '?' = 0
and letters = 1..26
// = sum p^3 t - 2 p^2 t^2 + p t^3 -> three convolutions. returns all i where pat
matches text at i
vi wildcardMatch(const string &text, const string &pat)
{
    int n = text.size(), m = pat.size();
    if (m > n) return {};
```

```
    auto v = [](char c) { return c == '?' ? 0LL : (int)(c - 'a' + 1); };
    vi t1(n), t2(n), t3(n), p1(m), p2(m), p3(m);
    rep(i, 0, n) { int x = v(text[i]); t1[i] = x, t2[i] = x * x, t3[i] = x * x * x; }
    rep(j, 0, m) { int x = v(pat[m - 1 - j]); p1[j] = x, p2[j] = x * x, p3[j] = x * x *
x; } // reversed
    vi a = convolve(p3, t1), b = convolve(p2, t2), c = convolve(p1, t3);
    vi res;
    rep(i, 0, n - m + 1) if (a[i + m - 1] - 2 * b[i + m - 1] + c[i + m - 1] == 0)
res.pb(i);
    return res;
}
// letters only up to 26: values reach 26^4 * m, fine for doubles while m <= ~2e5. for
bigger inputs use bitsets:
// bitset<N> ok; ok.set(); for each j: ok &= (pos[pat[j]] >> j)  (pos[c] = bitset of
positions of c in text)
```

17.11 Geometry

KEYWORDS: geometry template code

FILE: templates/geometry/geometry_2d.cpp

```
// 2d geometry kit
// P  : integer point, everything exact (|coord| <= 1e9 is safe with cross/dot in long
long)
// PD : long double point, for intersection points, distances, circles
struct P
{
    int x, y;
    P operator+(const P &o) const { return {x + o.x, y + o.y}; }
    P operator-(const P &o) const { return {x - o.x, y - o.y}; }
    P operator*(int k) const { return {x * k, y * k}; }
    bool operator<(const P &o) const { return x < o.x || (x == o.x && y < o.y); }
    bool operator==(const P &o) const { return x == o.x && y == o.y; }
};
int cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int cross(P a, P b, P c) { return cross(b - a, c - a); } // > 0: c is left of a->b
(ccw)
int dot(P a, P b) { return a.x * b.x + a.y * b.y; }
int dist2(P a, P b) { return dot(a - b, a - b); }
int sgn(int x) { return (x > 0) - (x < 0); }

bool onSeg(P a, P b, P p) // p on segment ab (endpoints included)
{
    return cross(a, b, p) == 0 && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
bool segInter(P a, P b, P c, P d) // segments ab and cd share at least one point
{
    int c1 = sgn(cross(a, b, c)), c2 = sgn(cross(a, b, d));
    int c3 = sgn(cross(c, d, a)), c4 = sgn(cross(c, d, b));
    if (c1 * c2 < 0 && c3 * c4 < 0) return true; // proper crossing
    return onSeg(a, b, c) || onSeg(a, b, d) || onSeg(c, d, a) || onSeg(c, d, b);
}

int area2(const vector<P> &p) // twice the signed area (ccw > 0), shoelace
{
    int s = 0, n = p.size();
    rep(i, 0, n) s += cross(p[i], p[(i + 1) % n]);
    return s;
```

```
}
int inPoly(const vector<P> &p, P q) // any simple polygon: 0 outside, 1 inside, 2 on
boundary
{
    int n = p.size(), w = 0;
    rep(i, 0, n)
    {
        P a = p[i], b = p[(i + 1) % n];
        if (onSeg(a, b, q)) return 2;
        if ((a.y <= q.y) != (b.y <= q.y))
        {
            int c = cross(a, b, q);
            if ((c > 0) == (b.y > a.y)) w ^= 1;
        }
    }
    return w;
}
int boundaryPoints(const vector<P> &p) // lattice points on the boundary
{
    int n = p.size(), b = 0;
    rep(i, 0, n) b += __gcd(llabs(p[i].x - p[(i + 1) % n].x), llabs(p[i].y - p[(i + 1)
% n].y));
    return b;
}
// pick: area = inside + boundary / 2 - 1  ->  inside = (|area2| - boundary + 2) / 2

int closestPair2(vector<P> p) // smallest squared distance, n >= 2, O(n log n)
{
    sort(all(p));
    set<pi> s; // {y, x}
    int best = LLONG_MAX, j = 0;
    rep(i, 0, (int)p.size())
    {
        int d = ceil(sqrt((long double)best));
        while (j < i && p[i].x - p[j].x >= d) s.erase({p[j].y, p[j].x}), j++;
        auto lo = s.lower_bound({p[i].y - d, LLONG_MIN}), hi = s.upper_bound({p[i].y +
d, LLONG_MAX});
        for (auto it = lo; it != hi; it++) best = min(best, dist2(p[i], {it->second,
it->first}));
        s.insert({p[i].y, p[i].x});
    }
    return best;
}
int diameter2(const vector<P> &h) // farthest pair (squared) of a ccw convex hull,
rotating calipers
{
    int n = h.size(), best = 0;
    if (n == 1) return 0;
    if (n == 2) return dist2(h[0], h[1]);
    for (int i = 0, j = 1; i < n; i++)
    {
        while (cross(h[(i + 1) % n] - h[i], h[(j + 1) % n] - h[j]) > 0) j = (j + 1) %
n;
        best = max({best, dist2(h[i], h[j]), dist2(h[(i + 1) % n], h[j])});
    }
    return best;
}
bool half(P a) { return a.y < 0 || (a.y == 0 && a.x < 0); }
bool angleLess(P a, P b) // sort vectors by polar angle in [0, 2pi), exact
{
```

```
    if (half(a) != half(b)) return half(a) < half(b);
    return cross(a, b) > 0;
}

// ---------------- floating point part ----------------
typedef long double ld;
const ld EPS = 1e-9, PI = acosl(-1);
struct PD
{
    ld x, y;
    PD operator+(const PD &o) const { return {x + o.x, y + o.y}; }
    PD operator-(const PD &o) const { return {x - o.x, y - o.y}; }
    PD operator*(ld k) const { return {x * k, y * k}; }
    PD operator/(ld k) const { return {x / k, y / k}; }
};
ld cross(PD a, PD b) { return a.x * b.y - a.y * b.x; }
ld dot(PD a, PD b) { return a.x * b.x + a.y * b.y; }
ld len(PD a) { return sqrtl(dot(a, a)); }
PD rot(PD a, ld t) { return {a.x * cosl(t) - a.y * sinl(t), a.x * sinl(t) + a.y *
cosl(t)}; }
PD toPD(P a) { return {(ld)a.x, (ld)a.y}; }

PD lineInter(PD a, PD b, PD c, PD d) // lines ab and cd, must not be parallel
{
    return a + (b - a) * (cross(c - a, d - c) / cross(b - a, d - c));
}
bool segInterPoint(P A, P B, P C, P D, PD &res) // true + the point if they meet in
exactly one point
{
    if (!segInter(A, B, C, D)) return false;
    if (cross(B - A, D - C) == 0) // parallel: collinear overlap, a single point only
if they just touch
    {
        for (P q : {A, B})
            for (P r : {C, D})
                if (q == r && !(onSeg(A, B, (q == C ? D : C)) && onSeg(C, D, (q == A ?
B : A)))) { res = toPD(q); return true; }
        return false; // overlap of positive length (infinitely many points)
    }
    res = lineInter(toPD(A), toPD(B), toPD(C), toPD(D));
    return true;
}
ld distPointLine(PD p, PD a, PD b) { return fabsl(cross(b - a, p - a)) / len(b - a); }
ld distPointSeg(PD p, PD a, PD b)
{
    if (dot(p - a, b - a) < 0) return len(p - a);
    if (dot(p - b, a - b) < 0) return len(p - b);
    return distPointLine(p, a, b);
}
vector<PD> circleLine(PD c, ld r, PD a, PD b) // intersection points of circle and line
ab
{
    PD d = b - a;
    PD h = a + d * (dot(c - a, d) / dot(d, d)); // foot of perpendicular
    ld s = r * r - dot(c - h, c - h);
    if (s < -EPS) return {};
    if (s < EPS) return {h};
    PD off = d * (sqrtl(s) / len(d));
    return {h - off, h + off};
}
```

```
vector<PD> circleCircle(PD c1, ld r1, PD c2, ld r2) // intersection points (none if
same circle)
{
    PD d = c2 - c1;
    ld D = len(d);
    if (D < EPS || D > r1 + r2 + EPS || D < fabsl(r1 - r2) - EPS) return {};
    ld a = (r1 * r1 - r2 * r2 + D * D) / (2 * D), h2 = r1 * r1 - a * a;
    PD m = c1 + d * (a / D);
    if (h2 < EPS) return {m};
    PD off = PD{-d.y, d.x} * (sqrtl(h2) / D);
    return {m - off, m + off};
}
```

FILE: templates/geometry/convex_hull.cpp

```
// integer convex hull (monotone chain, ccw, no collinear points)
// inside(h, q): point in convex polygon in O(log n), boundary counts as inside
struct pt
{
    int x, y;
    pt operator-(const pt &o) const { return {x - o.x, y - o.y}; }
    bool operator<(const pt &o) const { return x < o.x || (x == o.x && y < o.y); }
};
__int128 cross(pt a, pt b) { return (__int128)a.x * b.y - (__int128)a.y * b.x; }
__int128 dot(pt a, pt b) { return (__int128)a.x * b.x + (__int128)a.y * b.y; }
vector<pt> hull(vector<pt> p) // ccw, no collinear points, starts at lowest x
{
    sort(all(p));
    p.erase(unique(all(p), [](pt a, pt b) { return a.x == b.x && a.y == b.y; }),
p.end());
    if (p.size() < 3) return p;
    vector<pt> h;
    rep(it, 0, 2)
    {
        int st = h.size();
        for (auto &q : p)
        {
            while ((int)h.size() >= st + 2 && cross(h.back() - h[h.size() - 2], q -
h[h.size() - 2]) <= 0) h.pop_back();
            h.pb(q);
        }
        h.pop_back();
        reverse(all(p));
    }
    return h;
}
bool inside(vector<pt> &h, pt q) // ccw convex polygon, boundary counts as inside
{
    int n = h.size();
    pt o = h[0];
    if (cross(h[1] - o, q - o) < 0 || cross(h[n - 1] - o, q - o) > 0) return false;
    int l = 1, r = n - 1;
    while (r - l > 1)
    {
        int mid = (l + r) / 2;
        if (cross(h[mid] - o, q - o) >= 0) l = mid;
        else r = mid;
    }
    if (cross(h[n - 1] - o, q - o) == 0) return dot(q - o, q - o) <= dot(h[n - 1] - o,
h[n - 1] - o);
```

```
    return cross(h[l + 1] - h[l], q - h[l]) >= 0;
}
```

FILE: templates/geometry/minkowski_sum.cpp

```
// minkowski sum of two ccw convex polygons in O(n + m) + point in convex polygon
// {a + b} for a in p, b in q is again convex; use it for midpoint / translation
questions
struct pt
{
    int x, y;
    pt operator+(const pt &o) const { return {x + o.x, y + o.y}; }
    pt operator-(const pt &o) const { return {x - o.x, y - o.y}; }
    pt operator*(int k) const { return {x * k, y * k}; }
};
int cross(pt a, pt b) { return a.x * b.y - a.y * b.x; }
int dot(pt a, pt b) { return a.x * b.x + a.y * b.y; }
// rotate so the lowest (then leftmost) point comes first
void reorder(vector<pt> &p)
{
    int k = 0;
    rep(i, 1, (int)p.size()) if (p[i].y < p[k].y || (p[i].y == p[k].y && p[i].x <
p[k].x)) k = i;
    rotate(p.begin(), p.begin() + k, p.end());
}
vector<pt> minkowski(vector<pt> p, vector<pt> q) // both ccw convex
{
    reorder(p), reorder(q);
    int n = p.size(), m = q.size();
    p.pb(p[0]), p.pb(p[1]), q.pb(q[0]), q.pb(q[1]);
    vector<pt> res;
    int i = 0, j = 0;
    while (i < n || j < m)
    {
        res.pb(p[i] + q[j]);
        int c = cross(p[i + 1] - p[i], q[j + 1] - q[j]);
        if (c >= 0 && i < n) i++;
        if (c <= 0 && j < m) j++;
    }
    vector<pt> h; // drop collinear points
    int k = res.size();
    rep(t, 0, k)
    {
        pt a = res[(t + k - 1) % k], b = res[t], c = res[(t + 1) % k];
        if (cross(b - a, c - b) != 0) h.pb(b);
    }
    reorder(h);
    return h;
}
bool inside(vector<pt> &h, pt q) // ccw convex, boundary counts
{
    int n = h.size();
    pt o = h[0];
    if (cross(h[1] - o, q - o) < 0 || cross(h[n - 1] - o, q - o) > 0) return false;
    int l = 1, r = n - 1;
    while (r - l > 1)
    {
        int mid = (l + r) / 2;
        if (cross(h[mid] - o, q - o) >= 0) l = mid;
        else r = mid;
```

```
    }
    if (cross(h[n - 1] - o, q - o) == 0) return dot(q - o, q - o) <= dot(h[n - 1] - o,
h[n - 1] - o);
    return cross(h[l + 1] - h[l], q - h[l]) >= 0;
}
```

FILE: templates/geometry/3d_distances.cpp

```
// 3d geometry in long double: point-segment, segment-segment, point-triangle, segment-
triangle distances
// segTri(s, e, a, b, c) = 0 when the segment touches or crosses triangle abc
typedef long double ld;
struct p3
{
    ld x, y, z;
    p3 operator+(const p3 &o) const { return {x + o.x, y + o.y, z + o.z}; }
    p3 operator-(const p3 &o) const { return {x - o.x, y - o.y, z - o.z}; }
    p3 operator*(ld k) const { return {x * k, y * k, z * k}; }
};
ld dot(p3 a, p3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
p3 cross(p3 a, p3 b) { return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y
- a.y * b.x}; }
ld len(p3 a) { return sqrtl(dot(a, a)); }
ld pointSeg(p3 p, p3 a, p3 b)
{
    p3 d = b - a;
    ld t = dot(p - a, d) / dot(d, d);
    t = max((ld)0, min((ld)1, t));
    return len(p - (a + d * t));
}
ld segSeg(p3 a, p3 b, p3 c, p3 d) // closest points of two segments, clamped
{
    p3 u = b - a, v = d - c, w = a - c;
    ld aa = dot(u, u), bb = dot(u, v), cc = dot(v, v), dd = dot(u, w), ee = dot(v, w);
    ld den = aa * cc - bb * bb;
    ld best = min(min(pointSeg(a, c, d), pointSeg(b, c, d)), min(pointSeg(c, a, b),
pointSeg(d, a, b)));
    if (den > 1e-12)
    {
        ld s = (bb * ee - cc * dd) / den, t = (aa * ee - bb * dd) / den;
        if (s >= 0 && s <= 1 && t >= 0 && t <= 1) best = min(best, len((a + u * s) - (c
+ v * t)));
    }
    return best;
}
bool inTri(p3 p, p3 a, p3 b, p3 c, p3 n) // p is on the plane of abc
{
    ld s1 = dot(cross(b - a, p - a), n), s2 = dot(cross(c - b, p - b), n), s3 =
dot(cross(a - c, p - c), n);
    return (s1 >= -1e-9 && s2 >= -1e-9 && s3 >= -1e-9) || (s1 <= 1e-9 && s2 <= 1e-9 &&
s3 <= 1e-9);
}
ld pointTri(p3 p, p3 a, p3 b, p3 c)
{
    p3 n = cross(b - a, c - a);
    ld dn = dot(p - a, n) / len(n);
    p3 q = p - n * (dot(p - a, n) / dot(n, n)); // projection onto the plane
    if (inTri(q, a, b, c, n)) return fabsl(dn);
    return min(pointSeg(p, a, b), min(pointSeg(p, b, c), pointSeg(p, c, a)));
}
```

```
ld segTri(p3 s, p3 e, p3 a, p3 b, p3 c)
{
    p3 n = cross(b - a, c - a);
    ld d0 = dot(s - a, n), d1 = dot(e - a, n);
    if ((d0 < 0 && d1 > 0) || (d0 > 0 && d1 < 0))
    {
        p3 q = s + (e - s) * (d0 / (d0 - d1));
        if (inTri(q, a, b, c, n)) return 0;
    }
    ld res = min(pointTri(s, a, b, c), pointTri(e, a, b, c));
    res = min(res, min(segSeg(s, e, a, b), min(segSeg(s, e, b, c), segSeg(s, e, c,
a))));
    return res;
}
```

18. Ladder problems: statement → solution idea

KEYWORDS: ladder problems solutions ideas index search statement

Every problem from the strings, graphs, number theory and counting ladders. Search a word from your statement; the arrow line is the idea that solves it.

18.1 Number theory: Modular arithmetic & digit tricks

KEYWORDS: modular arithmetic & digit tricks

Divisibility Problem (Codeforces 1328A) — Given a, b (≤ 10^9). One move adds 1 to a. Fewest moves so that b | a?

→ So you need b − a % b more steps… unless a % b is already 0, then you need 0. Answer = (b − a % b) % b. The outer % b handles the 'already divisible' case.

Large Division (LightOJ 1214) — a has up to 200 digits (maybe negative), b is a 32-bit int. Is a divisible by b?

→ Signs don't change divisibility: use |b| and skip a leading '−' in a. r = (r·10 + d) % |b| over all digits; divisible iff r = 0.

Repeated Digit Sum (Dhaka Preli 2017 I) **[Dhaka]** — f(n) = repeated digit sum until one digit. Given a, b < 10^50000 (strings), print f(a^b).

→

Delete, Deduct, and Destroy (Dhaka Preli 2025 A (Gym 106315 A)) **[Dhaka]** — f(x, k) = x − (x with its k-th digit from the right deleted). g(z) = number of pairs (x, k) with f = z. Number z with n ≤ 10^6 digits, q ≤ 10^6 digit updates: print g(z) mod 998244353 each time.

→

Integer Divisibility (LightOJ 1078) — Given n (not divisible by 2 or 5) and a digit d, find the length of the smallest number made only of digit d (d, dd, ddd, …) that is divisible by n.

→ There are only n possible remainders, so within n steps you either hit 0 or loop. Because n shares no factor with 10, you will hit 0. Start with r = d mod n, length 1. While r ≠ 0: r = (r·10 + d) % n, length++. Print the length. O(n) per test.

Zero Remainder Array (Codeforces 1374D) — Array a, number k, and x = 0. A move is either x += 1, or (once per index) a[i] += x and then x += 1. Fewest moves to make every a[i] divisible by k.

→ If c elements need remainder r, the last of them uses x = r + (c − 1)·k. After using it, one more move happens. Answer = max over r of (cnt[r] − 1)·k + r + 1, or 0 if nothing needs fixing. Count with a map.

18.2 Number theory: gcd, lcm, extended Euclid, CRT

KEYWORDS: gcd, lcm, extended euclid, crt

Common Divisors (CSES 1081) — n ≤ $2 \cdot 10^5$ numbers ≤ 10^6. Max gcd over all pairs.
→ Try g from big to small and count how many numbers are multiples of g (walk g, 2g, 3g, …). The first g with count ≥ 2 is the answer. Harmonic loop, O(M log M).

Solutions to an Equation (LightOJ 1306) — Count integer (x, y) with ax + by + c = 0, $x_1 \le x \le x_2$, $y_1 \le y \le y_2$ (all $|\cdot| \le 10^8$).
→ Each of the four box bounds becomes a range of k. Intersect the ranges (be careful with floor/ceil of negatives!). Handle a = 0 or b = 0 separately. Answer = size of the k-range.

Monkey Tradition (LightOJ 1319) — n ≤ 12 congruences $x \equiv r_i \pmod{p_i}$, p_i distinct primes ≤ 40. Smallest positive x.
→ $x = \Sigma\, r_i \cdot (M/p_i) \cdot \text{inv}(M/p_i \bmod p_i)$ — each term is r_i mod p_i and 0 mod the others. The product is < 40^{12}, multiply with __int128. Answer x mod M (positive).

Throne (AtCoder ABC186 E) — N seats in a circle, you are S seats right of the throne; each move goes K right. Min moves to reach the throne, or −1.
→ Divide everything by g; now K/g has an inverse mod N/g (extended Euclid, N isn't prime). x = (−S/g)·inv(K/g) mod (N/g), smallest non-negative; −1 if g ∤ S.

Row GCD (Codeforces 1458A) — Arrays a (n ≤ $2 \cdot 10^5$) and b. For each b_j print $\gcd(a_1+b_j, \ldots, a_n+b_j)$.
→ So the answer is $\gcd(a_1 + b, G)$ where G = gcd of all $|a_i - a_1|$. Compute G once, then each query is one gcd. O(n + m log).

Modified GCD (Codeforces 75C) — a, b ≤ 10^9 and queries [lo, hi]: the largest common divisor of a and b in [lo, hi], or −1.
→ For a query [lo, hi], find the largest divisor ≤ hi with upper_bound. If it's ≥ lo print it, else −1.

Orac and LCM (Codeforces 1349A) — n ≤ 10^5, $a_i \le 2 \cdot 10^5$. gcd of $\text{lcm}(a_i, a_j)$ over all pairs i < j.
→ An easier way to compute it: answer = gcd over i of $\text{lcm}(a_i, \gcd(a_{i+1}, \ldots, a_n))$. Use suffix gcds. O(n log).

Fadi and LCM (Codeforces 1285C) — X ≤ 10^{12}. Find a, b with lcm(a, b) = X minimizing max(a, b).
→ So try every divisor a ≤ √X, set b = X / a, and keep the pair with gcd 1 and the largest a. √X ≤ 10^6 steps.

Strange Function (Codeforces 1542C) — f(i) = smallest positive integer not dividing i. Σ f(i) for i ≤ n (n ≤ 10^{16}) mod 10^9+7.
→ Swap the sums: Σ f(i) = Σ over x of (how many i ≤ n are multiples of lcm(1..x)) = Σ ⌊n / lcm(1..x)⌋. lcm(1..x) passes 10^{16} around x ≈ 41, so it's a tiny loop.

Integers Have Friends (Codeforces 1548B) — Distinct $a_i \le 10^{18}$, n ≤ $2 \cdot 10^5$. Longest subarray where all elements are equal mod some m ≥ 2.
→ Two pointers on d with a sparse table for range gcd: move l right while gcd(l..r) = 1. Answer = longest d-window + 1 (n = 1 → 1). O(n log).

The Football Season (Codeforces 1244C) — n games, p points, win w, draw d (d < w). Find wins x, draws y, losses z, or −1. n ≤ 10^{12}, p ≤ 10^{17}.
→ w ≤ 10^5, so try every y from 0 to w − 1 and check whether (p − y·d) is divisible by w. Then x = (p − y·d)/w and z = n − x − y; print if all are non-negative, else −1.

Two Arithmetic Progressions (Codeforces 710D) — Count x in [L, R] with $x = a_1k + b_1 = a_2l + b_2$, k, l ≥ 0 (all $|\cdot| \le 2 \cdot 10^9$).
→ k, l ≥ 0 means $x \ge \max(b_1, b_2)$. Raise L to that. Count numbers ≡ r mod lcm in [L, R] with floor division (careful with negatives). __int128 inside.

Subset with GCD K (Dhaka Preli 2018 G) **[Dhaka]** — N ≤ 10^5 distinct values ≤ 10^9, Q ≤ 1000 queries K: is there a subset with gcd exactly K?
→

Co Primes (Dhaka Preli 2020 F) **[Dhaka]** — T ≤ 1000: a, b, m ≤ 10^{12}. Count i in [0, m] with gcd(a+i, b+i) = 1.
→

Nezzar and Board (Codeforces 1478D) — Distinct numbers $x_1..x_n$ on a board; you may write $2x - y$ for any written x, y. Can k appear?
→ And all of those really are reachable. YES iff $(k - x_1) \% g == 0$.

Not Adding (Codeforces 1627D) — n distinct numbers ≤ 10^6. Repeatedly add $\gcd(a_i, a_j)$ if it's not present. How many numbers can be added in total?
→ For every d ≤ 10^6 not already present, compute the gcd of its present multiples. Count the d's where it equals d. Harmonic loop, O(M log M).

GCD on Blackboard (AtCoder ABC125 C) — N numbers. You may replace one of them with any integer. Maximise the gcd of all N numbers.
→ Computing each from scratch is $O(N^2)$. Use prefix gcds and suffix gcds. gcd(except i) = gcd(pre[i − 1], suf[i + 1]). Take the maximum. O(N log).

18.3 Number theory: Primes, sieves, divisors, factorization

KEYWORDS: primes, sieves, divisors, factorization

Odd Divisor (Codeforces 1475A) — n ≤ 10^{14}. Does n have an odd divisor greater than 1?
→ So the only bad numbers are powers of two. YES iff n is not a power of two: (n & (n − 1)) != 0.

T-primes (Codeforces 230B) — n ≤ 10^5 numbers, each ≤ 10^{12}. For each, does it have exactly 3 divisors?
→ So x must be a perfect square whose square root is prime. $\sqrt{(10^{12})} = 10^6$, so sieve primes up to 10^6. For each x: r = exact integer sqrt; YES iff r·r = x and r is prime.

Almost Prime (Codeforces 26A) — Count numbers in 1..n (n ≤ 3000) that have exactly two distinct prime divisors.
→ A sieve does it: for each prime p, add 1 to all multiples of p. Count the numbers with exactly 2.

k-th divisor (Codeforces 762A) — n ≤ 10^{15}, k ≤ 10^9. Print the k-th smallest divisor of n, or −1.
→ Careful when n is a perfect square: √n appears once. If k is larger than the total number of divisors, print −1.

Sherlock and his girlfriend (Codeforces 776B) — Items priced 2..n+1. If price p is a prime divisor of price q, they need different colours. Use fewest colours.
→ So colour all primes 1 and all composites 2. If n ≤ 2 there are no composites, so 1 colour is enough. Sieve up to n + 1, print the count of colours and the colouring.

Counting Divisors (CSES 1713) — n ≤ 10^5 queries x ≤ 10^6. Print d(x).
→ Total work = $10^6 \cdot (1 + 1/2 + 1/3 + \ldots) \approx 1.4 \cdot 10^7$ — the 'harmonic loop'. Precompute d[x] for all x, answer queries in O(1).

Div Game (AtCoder ABC169 D) — N ≤ 10^{12}. Repeatedly pick $z = p^e$ (prime power) dividing N, never the same z twice, and divide. Max moves?
→ So the number of moves for that prime is the largest k with 1 + 2 + … + k ≤ e. Factorise N (√N ≤ 10^6) and add up k over its primes.

Help Hanzo (LightOJ 1197) — Count primes in [a, b], 1 ≤ a ≤ b < 2^{31}, b − a ≤ 10^5.
→ For each small prime p, cross out its multiples inside the window [a, b] (start at $\max(p^2$, the first multiple ≥ a)). That's the segmented sieve. Count unmarked numbers ≥ 2. O((b − a) log log b).

Divisor Analysis (CSES 2182) — n given as primes p_i with exponents k_i. Print number, sum and product of its divisors mod 10^9+7.
→ Build it prime by prime: when adding p^k, every old divisor appears k+1 times (prod^(k+1)), and the new factor p appears 0+1+…+k = k(k+1)/2 times per old divisor. prod = prod^(k+1) · p^(k(k+1)/2 · oldCount). Keep oldCount mod (MOD − 1) because it's an exponent (Fermat).

Prime Multiples (CSES 2185) — n ≤ 10^{18}, k ≤ 20 distinct primes. How many of 1..n are divisible by at least one?
→ A subset with product q contributes ±(n / q): + for odd size, − for even size. 2^k subsets. Products can overflow: stop as soon as the product exceeds n (that term is 0). Check with prod > n / p before multiplying.

Happy New Year! (AtCoder ABC284 D) — N = p^2q ≤ $9\cdot10^{18}$ (p, q distinct primes). Find p and q.
→ If i^2 also divides N, then i = p and q = N/i^2. Otherwise i = q and p = √(N/i). Loop ≤ ~$3\cdot10^6$ steps per test, exact integer sqrt.

Almost All Divisors (Codeforces 1165D) — Given all divisors of x except 1 and x. Find smallest such x, or −1.
→ But the list might be a lie. Verify: compute all divisors of x except 1 and x and compare with the input. Enumerate to √x (≤ 10^6), sort, compare. Mismatch → −1. Same idea as Dhaka Preli 2019 J.

Another Problem About Dividing Numbers (Codeforces 1538D) — a, b ≤ 10^9. In one move divide a or b by some c > 1 (must divide). Can they be equal after exactly k moves?
→ Anything between min and max is reachable — except k = 1 needs exactly 'a ≠ b and one divides the other'. Ω by trial division with primes ≤ 31623. YES iff k ≤ Ω(a)+Ω(b) and (k ≥ 2, or k = 1 with that special condition).

Two Divisors (Codeforces 1366D) — n ≤ $5\cdot10^5$ values ≤ 10^7. For each a find divisors d_1, d_2 > 1 with gcd(d_1+d_2, a) = 1, or −1 −1.
→ Easiest split: d_1 = full power of the smallest prime, d_2 = everything else. SPF sieve to 10^7. If d_2 = 1 (a is a prime power) there's no answer → −1 −1.

Prime or Not (SPOJ PON) — Up to 500 numbers below 2^{63}. Print YES if prime.
→ With the 12 prime bases up to 37 it's exact for all 64-bit numbers. Multiply with __int128 to avoid overflow. isp(n) from Lesson 'Big primes'. O(12·log n) per number.

Factorize (Library Checker) — Q ≤ 100 numbers a ≤ 10^{18}. Print all prime factors.
→ Use Miller–Rabin to detect primes so you know when to stop. Recursive: if prime, output; else split with rho and recurse on both parts. Sort at the end.

The Number of Pairs (Codeforces 1499D) — c, d, x ≤ 10^7. Count pairs (a, b) with c·lcm(a,b) − d·gcd(a,b) = x.
→ How many coprime ordered (u, v) with u·v = k? Each distinct prime of k goes fully to u or to v: 2^(number of distinct primes of k). Sieve 'number of distinct primes' up to $2\cdot10^7$, loop divisors g of x. O(√x) per query.

Prime Friendly Numbers (Dhaka Preli 2018 D) **[Dhaka]** — T ≤ 1000, N ≤ 10^{18}. Largest X ≤ N that is prime and uses only digits 2, 3, 5, 7.
→

Good Things Come to Those Who Wait (Dhaka Preli 2019 J) **[Dhaka]** — Given all proper divisors (all except 1 and N) of a composite N ≤ 10^{12}. Find N.
→

Transform the Array (Dhaka Preli 2022 B) **[Dhaka]** — Arrays A, B (Σn ≤ $5\cdot10^5$, values ≤ 10^7). Alice: multiply/divide one element by a divisor of any element. Bob: multiply/divide all by a divisor of all. Can each reach B?
→

Almost Perfect Prime Factor! (Dhaka Preli 2023 A) **[Dhaka]** — $n \le 10^5$, $a_i \le 10^6$, K. Longest subarray whose union of primes has exactly K primes, none of which divides every element.
→

Optimal Balancing Strategy (Dhaka Regional 2025 H (Gym 106270 H)) **[Dhaka]** — Array A ($n \le 2 \cdot 10^5$, $A_i \le 10^6$). Move p | A_i from A_i to A_j at cost p. Need ≥ a elements with no odd prime, ≥ b with no factor 2, and every element one of the two. Min cost.
→ Each element picks a role; the moved primes always have a valid home in the other group. Need x evenly-even elements with $a \le x \le n - b$. Sort by $f_i - s_i$; the first x take f, the rest take s. Prefix sums of f, suffix sums of s, try every x.

Enlarge GCD (Codeforces 1034A) — $n \le 3 \cdot 10^5$ numbers $\le 1.5 \cdot 10^7$. Remove the fewest so that the gcd of the rest becomes larger than the current gcd.
→ Count with a frequency array and a sieve over multiples of each prime (O(M log log M)). Answer n − best, or −1 if all numbers became 1.

GCD of an Array (Codeforces 1493D) — Array $n \le 2 \cdot 10^5$; q queries multiply a_i by x ($\le 2 \cdot 10^5$). After each, print gcd of the whole array mod 10^9+7.
→ On 'multiply a_i by x': for each prime of x, update a_i's exponent; if the minimum went up, multiply the answer by p^(increase). SPF factorisation + multisets. The gcd only grows, so no division needed. O((n+q) $\log^2$).

Make It One (Codeforces 1043F) — $n \le 3 \cdot 10^5$ numbers $\le 3 \cdot 10^5$. Smallest subset with gcd 1, or −1.
→ 'gcd exactly d' = 'all divisible by d' minus 'gcd exactly a bigger multiple of d' → go from big d down to 1. Try k = 1..7; the first k with f[1] > 0 is the answer (count mod a big prime). O(7·M log M).

Make GCD Great Again! (Dhaka Regional 2021 G) **[Dhaka]** — $N \le 10^5$ values $\le 10^6$. You may raise any A_i to $X > A_i$ at cost $X^2 - A_i^2$ (once each). Min total cost to make gcd > 1, and the largest gcd achievable with that cost.
→ Try every g from 2 to max+1: total work Σ max/g = O(M log M) — harmonic again! Keep the minimum cost; among equal costs the largest g. 64-bit is enough.

Bears (Dhaka Regional 2023 C) **[Dhaka]** — Values $A_i \le N$ ($\Sigma N \le 6 \cdot 10^5$). Make at most one rectangle of area A_i for each i; find the longest chain with strictly increasing heights and widths.
→ Longest chain in 2D: sort by h; for equal h, sort w descending so two same-height rectangles never chain. LIS on w with a Fenwick tree for prefix max. O(N $\log^2$ N).

Multiply by 2, divide by 6 (Codeforces 1374B) — From n, a move multiplies by 2 or divides by 6 (if divisible). Fewest moves to reach 1, or −1.
→ Those b divisions remove b twos, so you need a ≤ b, and you add b − a twos first. If rest ≠ 1 or a > b: −1. Otherwise (b − a) + b moves.

18.4 Number theory: Euler totient φ

KEYWORDS: euler totient φ

Euler Totient Function (SPOJ ETF) — Up to 20000 tests. For each $n \le 10^6$, print φ(n).
→ Start ph[i] = i. For each prime p, for every multiple j: ph[j] −= ph[j]/p (multiplying by (1 − 1/p)). Sieve once, answer each query in O(1).

Exponentiation II (CSES 1712) — Up to 10^5 tests: print a^(b^c) mod 10^9+7, with a, b, c ≤ 10^9.
→ Careful when a is a multiple of p: the answer is 0, unless the exponent is truly 0 (b = 0, c > 0 → 1). Two binExp calls: e = b^c mod (p−1), answer = a^e mod p.

Same GCDs (Codeforces 1295D) — Up to 50 tests with $1 \le a < m \le 10^{10}$. Count x in [0, m) with gcd(a, m) = gcd(a + x, m).
→ Write y = g·t: need gcd(t, m/g) = 1 with t in [0, m/g) → exactly φ(m/g) of them. Answer φ(m/g). Compute φ by trial division ($m \le 10^{10}$).

Mathematically Hard (LightOJ 1007) — Up to 10^5 queries $2 \le a \le b \le 5 \cdot 10^6$. Print $\Sigma\, \varphi(i)^2$ for $a \le i \le b$, as an unsigned 64-bit number.
→ Prefix sums of $\varphi(i)^2$ in unsigned long long — the natural overflow is exactly 'mod 2^{64}'. Answer pre[b] − pre[a − 1].

GCD Extreme (I) and (II) (UVa 11424 / UVa 11426 / SPOJ GCDEX) — Many N (up to $2 \cdot 10^5$ for 11424, $4 \cdot 10^6$ for 11426). Print Σ gcd(i, j) over $1 \le i < j \le N$.
→ So $\Sigma_{i<N}$ gcd(i, N) = $\Sigma_{g \mid N, g < N}\ g \cdot \varphi(N/g)$. Harmonic loop over g and multiples, then prefix sums. O(N log N).

LCM Sum (SPOJ LCMSUM) — Up to $3 \cdot 10^5$ tests with $n \le 10^6$. Print $\Sigma_{i=1..n}$ lcm(i, n).
→ Precompute $g[n] = \Sigma_{d \mid n}\ d \cdot \varphi(d)$ for all n with a harmonic loop. Answer $(g[n] + 1) \cdot n/2$. O(1) per query after O(N log N) precompute.

Huge Mods (UVa 10692) — m ≤ 10000 and up to 10 numbers $a_i \le 1000$. Print $a_1\text{^}(a_2\text{^}(a_3\text{^}\ldots))$ mod m.
→ So recurse down the chain m → φ(m) → φ(φ(m)) → …, and keep a capped true value of each sub-tower to know whether e ≥ φ(m). go(i, m): if the capped tower ≥ φ(m) use go(i+1, φ(m)) + φ(m), else the true value; then pow(a_i, that, m).

REQ (Codeforces 594D) — n, q ≤ $2 \cdot 10^5$, values ≤ 10^6. Each query (l, r): print φ(a_l·a_l+1·…·a_r) mod 10^9+7.
→ Sweep r: multiply a_r at r; for each prime p of a_r, undo (p−1)/p at its old last position and put it at r. Fenwick tree of products; query = prefix(r) · inverse(prefix(l−1)). O((n+q) log n · primes).

Pairs Forming GCD (Dhaka Preli 2019 G) **[Dhaka]** — T ≤ 10^5 cases, N ≤ 10^7, P ≤ 10^{15}. Largest G such that at least P pairs $1 \le X \le Y \le N$ have gcd(X, Y) = G, or −1.
→

18.5 Number theory: Möbius & gcd-counting

KEYWORDS: möbius & gcd-counting

Counting Coprime Pairs (CSES 2417) — n ≤ 10^5 numbers, each ≤ 10^6. Count pairs i < j with gcd(a_i, a_j) = 1.
→ cnt[d] = how many numbers are multiples of d (harmonic loop over a frequency array). Answer $\Sigma\, \mu(d) \cdot \text{cnt}[d] \cdot (\text{cnt}[d] - 1)/2$. Sieve μ up to 10^6.

Coprime Subsequences (Codeforces 803F) — n ≤ 10^5, values ≤ 10^5. Count non-empty subsequences (any choice of positions) whose gcd is 1, mod 10^9+7.
→ Answer $\Sigma\, \mu(d) \cdot (2\text{^}\text{cnt}[d] - 1)$. Precompute powers of 2, sieve μ, harmonic loop for cnt. Mod 10^9+7.

Sum of gcd of Tuples (Hard) (AtCoder ABC162 E) — Consider all K^N sequences of length N with values 1..K (N, K ≤ 10^5). Add up the gcd of every sequence, mod 10^9+7.
→ 'Exactly g' = 'all multiples of g' minus 'exactly 2g, 3g, …' → compute from big g down to 1. f[g] = (K/g)^N − Σ f[multiples of g]; answer $\Sigma\, g \cdot f[g]$.

Visible Lattice Points (SPOJ VLATTICE) — Points (x, y, z) with $0 \le x, y, z \le N$, not the origin (N ≤ 10^6). Count the points visible from the origin.
→ Möbius turns 'all multiples of d' into 'gcd exactly 1': $\Sigma\, \mu(d) \cdot ((\lfloor N/d \rfloor + 1)^3 - 1)$. $\lfloor N/d \rfloor$ has few values → floor blocks with prefix sums of μ.

Mike and Foam (Codeforces 547C) — n beers with values ≤ $5 \cdot 10^5$, q ≤ $2 \cdot 10^5$ queries. Each query puts beer x on the shelf or takes it off. After each, print the number of pairs on the shelf with gcd 1.
→ Removing is the same in reverse. Then update cnt[d] for all square-free divisors of x. A number ≤ $5 \cdot 10^5$ has at most 64 square-free divisors → fast.

Winter is here (Codeforces 839D) — n ≤ $2 \cdot 10^5$, values ≤ 10^6. For every subset with gcd > 1, add (subset size) × gcd. Mod 10^9+7.
→ 'gcd exactly d' = that minus 'exactly 2d, 3d, …' → from big d down. Answer $\Sigma_{d \geq 2}$ d·exact[d], mod 10^9+7.

Divide Both (AtCoder ABC206 E) — L ≤ R ≤ 10^6. Count ordered (x, y) in [L, R] where g = gcd(x, y) ≠ 1, x ≠ g and y ≠ g.
→ Then remove pairs where x = g or y = g (only if g ≥ L): there are 2c − 1 of them. Σ over g ≥ 2 of (exact[g] − (g ≥ L ? 2c − 1 : 0)).

Unusual Sequences (Codeforces 900D) — x, y ≤ 10^9. Count sequences of positive integers with gcd = x and sum = y, mod 10^9+7.
→ With all elements multiples of d: 2^(n/d − 1). Möbius: $\Sigma_{d \mid n}$ μ(d)·2^(n/d − 1). Factor n, loop square-free divisors (subsets of its primes).

Steps to One (Codeforces 1139D) — Append random numbers from 1..m (m ≤ 10^5) until the gcd of everything is 1. Expected length, mod 10^9+7.
→ Sum the geometric series over k: $\Sigma_{k \geq 1}$ (c/m)^k = c/(m − c). Answer = 1 − $\Sigma_{d=2..m}$ μ(d)·c/(m − c), with modular inverses.

LCMs (AtCoder AGC038 C) — N ≤ $2 \cdot 10^5$, values ≤ 10^6. $\Sigma_{i<j}$ lcm(A_i, A_j) mod 998244353.
→ Then $\Sigma_{i,j} A_i A_j/\gcd = \Sigma_d w(d) \cdot (S_d)^2$, where S_d = sum of A divisible by d. That counts ordered pairs including i = j: subtract Σ A_i and halve. Mod 998244353.

Three Chairs (Codeforces 1780F) — n ≤ $3 \cdot 10^5$ distinct values ≤ $3 \cdot 10^5$. Count triples whose smallest and largest values are coprime.
→ For each d, walk the sorted positions of its multiples $p_1 < p_2 < \ldots$; Σ (p_b − p_a − 1) with a running prefix sum. O(M log M).

Mocha and Stars (Codeforces 1559E) — n ≤ 50, m ≤ 10^5, ranges [l_i, r_i]. Count sequences with a_i in its range, Σ a_i ≤ m and gcd 1, mod 998244353.
→ That's a bounded knapsack count: dp over the sum, each item adds a value in a range → prefix sums make each step O(m/d). Total Σ_d n·m/d = O(n·m log m). Mod 998244353.

18.6 Number theory: Fast power, Fermat, inverses, discrete log

KEYWORDS: fast power, fermat, inverses, discrete log

Exponentiation (CSES 1095) — n ≤ $2 \cdot 10^5$ queries: a^b mod 10^9+7 (a, b ≤ 10^9).
→ Walk the bits of b: square a each step, multiply it into the answer when the bit is 1. binExp(a, b), mod 10^9+7 every step. $0^0 = 1$ here.

Leading and Trailing (LightOJ 1282) — First three and last three digits of n^k (n < 2^{31}, k ≤ 10^7, n^k has ≥ 6 digits).
→ 10^f is a number in [1, 10) that starts with the same digits as n^k. First three digits = ⌊10^(f + 2)⌋. Use long double.

Discrete Logarithm (Library Checker) — T ≤ 100 queries: smallest x ≥ 0 with $X^x \equiv Y \pmod M$, M ≤ 10^9 (not necessarily prime).
→ If gcd(X, M) > 1, peel the common factor off first (Y must be divisible by it, or check x small directly). Check x = 0, peel gcds, then BSGS. O(√M) per query.

XOR Subset (Dhaka Preli 2015 D) **[Dhaka]** — S = {1..N}, N < 10^10000 (a string). Count unordered pairs of disjoint subsets (X, Y) (XOR of X ≤ XOR of Y), mod 10^9+7.
→

Modular Exponentiation (Codeforces 913A) — Given n and m (≤ 10^8), print m mod 2^n.
→ For small n, compute 2^n with a shift (1LL << n) and take the remainder. if n ≥ 30 print m, else print m % (1LL << n). The trap is computing 2^n for n = 10^8, which overflows.

Integer Sequence Fair (AtCoder ABC228 E) — N, K, M ≤ 10^{18}. Compute M^(K^N) mod 998244353.
→ So e = K^N mod (p − 1) with fast power, then the answer is M^e mod p. The trap: if M is a multiple of p, the answer is 0 (and Fermat doesn't apply). Reduce M mod p before using it.

18.7 Number theory: Factorials, nCr, Legendre, Lucas

KEYWORDS: factorials, ncr, legendre, lucas

Digits of Factorial (LightOJ 1045) — Up to 50000 queries: number of digits of n! in base b (n ≤ 10^6, 2 ≤ b ≤ 1000).
→ Precompute prefix sums of log(i) once for all n. Answer = ⌊pre[n] / log b + ε⌋ + 1.

Trailing Zeroes (III) (LightOJ 1138) — Q ≤ 10^8. Smallest N with exactly Q trailing zeros in N!, or 'impossible'.
→ Find the smallest N with zeros(N) ≥ Q. If zeros(N) = Q print N, else 'impossible'. Search up to 5Q.

Binomial Coefficients (CSES 1079) — n ≤ 10^5 queries C(a, b) mod 10^9+7, a ≤ 10^6.
→ Then every query is just three multiplications. compute_factorials() + ncr(a, b). O(1) per query.

Factorial and Multiple (AtCoder ABC280 D) — K ≤ 10^{12}. Smallest N ≥ 2 with K | N!.
→ That takes at most e steps per prime. Answer = max over primes (at least 2). A leftover big prime p > 10^6 needs N = p.

Modular Stability (Codeforces 1359E) — Count arrays 1 ≤ a_1 < … < a_k ≤ n (n, k ≤ 5·10^5) that are 'stable' (every order gives the same x mod chain).
→ Choose any k − 1 of them: C(⌊n/d⌋ − 1, k − 1). Sum over d, mod 998244353.

Classical Counting (Kattis classicalcounting (ICPC SG Preli 2018)) — N types, M copies of each (N, M, K ≤ 10^5). Ways to take exactly K objects (order doesn't matter, per the samples), mod 10^6+7.
→ Trap! 10^6+7 = 29 × 34483 isn't prime, so factorial inverses don't exist. Compute everything mod 29 and mod 34483 separately. n reaches 2·10^5 > 34483 → Lucas theorem for each prime, then glue the two answers with CRT.

Divisors of the Divisors of An Integer (Dhaka Regional 2018 C (Gym 102040 C)) **[Dhaka]** — ≤ 1000 queries n ≤ 10^6: sndd(n!) = Σ over divisors d of n! of (number of divisors of d), mod 10^7+7.
→ e for each prime from Legendre: n/p + n/p^2 + …. Multiply (e+1)(e+2)/2 over all primes ≤ n. Sieve once.

d(AF(n)) (Dhaka Regional 2017) **[Dhaka]** — AF(n) = 1!·2!·…·n!, n ≤ 5·10^6, ≤ 100 queries. Number of divisors of AF(n) mod 10^8+7.
→ Group by powers of p: every multiple m of p^k adds (n − m + 1). With T = ⌊n/p^k⌋ that's T(n+1) − p^k·T(T+1)/2. O(π(n)·log) per query after one sieve.

Combinations (LightOJ 1067) — Up to 2000 queries: C(n, k) mod 1000003 with n ≤ 10^6.
→ Precompute fact[] and invFact[] once (one fast power, then walk down). Each query is fact[n]·invFact[k]·invFact[n − k] mod 1000003.

Bouquet (AtCoder ABC156 D) — Choose a non-empty subset of n ≤ 10^9 flowers whose size is neither a nor b (a, b ≤ 2·10^5). Count, mod 10^9+7.
→ n is huge, so no factorial table. But a is small: C(n, a) = n(n−1)…(n−a+1) / a!. That's a multiplications. Answer = 2^n − 1 − C(n, a) − C(n, b), kept non-negative mod p.

Close Tuples (hard) (Codeforces 1462E2) — Array of n numbers. Count m-element index sets whose max − min ≤ k. Mod 10^9+7.
→ Binary search the last such position j. There are j − i candidates, choose m − 1 of them. Answer = Σ C(j − i, m − 1) over i, with precomputed factorials. Fixing the minimum avoids double counting.

18.8 Number theory: Combinatorial counting

KEYWORDS: combinatorial counting

Distributing Apples (CSES 1716) — n children, m apples (≤ 10^6). Number of ways to distribute all apples (children may get 0), mod 10^9+7.

→ Choose the positions of the walls among m + n − 1 slots. C(n + m − 1, m) with precomputed factorials. Stars and bars!

Creating Strings II (CSES 1715) — String of length ≤ 10^6. Number of distinct strings formed by rearranging its letters, mod 10^9+7.

→ So divide by every c! (multiply by inverse factorials). n! / (c_1!·c_2!·…), the multinomial coefficient.

Bracket Sequences I (CSES 2064) — Number of valid bracket sequences of length n ≤ 10^6, mod 10^9+7.

→ Paths that do dip can be reflected after their first dip, which matches them with all paths ending at −2. That's C(2k, k + 1) bad paths. Catalan: C(2k, k) − C(2k, k + 1) = C(2k, k) / (k + 1).

Christmas Party (CSES 1717) — n children each bring a gift; gifts are redistributed so nobody gets their own. Count ways mod 10^9+7.

→ D(n) = (n − 1)(D(n−1) + D(n−2)), D(0) = 1, D(1) = 0. O(n) loop mod 10^9+7.

Gerald and Giant Chess (Codeforces 559C) — h×w board (≤ 10^5), n ≤ 2000 black cells. Paths from top-left to bottom-right moving right/down avoiding black cells, mod 10^9+7.

→ dp[i] = paths(start → i) − Σ over earlier black cells j of dp[j]·paths(j → i). Sort cells so 'earlier' makes sense. Treat the target as the last cell. O(n^2). Dhaka Preli 2018 E was this with 'at most B blocked cells'.

Jury Meeting (Codeforces 1569C) — n members with task counts a_i. In a permutation order they each tell one task per round. Count 'nice' orders (nobody tells two tasks in a row), mod 998244353.

→ Among the max person and the k people with max − 1, each is equally likely to be last: probability 1/(k + 1). Answer n! − n!/(k + 1) (0 if k = 0).

Anton and School - 2 (Codeforces 785D) — Bracket string ≤ $2\cdot10^5$. Count subsequences of the form '((…))' (k '(' then k ')', k ≥ 1), mod 10^9+7.

→ Vandermonde collapses that sum to C(L + R − 1, L). Sum C(L + R − 1, L) over every '('. O(n).

Placing Rooks (Codeforces 1342E) — n×n board, n ≤ $2\cdot10^5$. Place n rooks so every empty cell is attacked and exactly k pairs attack each other. Count, mod 998244353.

→ Count ways to put n rooks (one per row) into exactly c columns, none empty: surjections = Σ (−1)^i C(c, i)(c − i)^n. Answer C(n, c)·surjections, ×2 when k > 0 (row-version and column-version differ); 0 if k ≥ n.

Count the Arrays (Codeforces 1312D) — Count arrays of length n with values 1..m that have exactly one pair of equal elements and are strictly increasing up to some peak and strictly decreasing after it. Mod 998244353.

→ Each remaining n − 3 value goes either left or right of the peak (its position on that side is then forced by sorting): 2^(n−3) ways. Answer = C(m, n − 1) · (n − 2) · 2^(n−3) mod p, and 0 when n = 2.

Two Arrays (Codeforces 1288C) — Count pairs of arrays a, b of length m ≤ 10 with values 1..n ≤ 1000, a non-decreasing, b non-increasing, and a[i] ≤ b[i] for all i. Mod 10^9+7.

→ Non-decreasing sequences of length L with values 1..n are 'stars and bars': C(n + L − 1, L). Answer = C(n + 2m − 1, 2m). Pascal's triangle is enough for these sizes.

Blue and Red Balls (AtCoder ABC132 D) — N balls, K blue and N − K red, in a row. For each i = 1..K, count arrangements where the blue balls form exactly i contiguous groups. Mod 10^9+7.

→ The N − K red balls create N − K + 1 gaps (including both ends). The i groups go into different gaps: C(N − K + 1, i). Answer for i = C(N − K + 1, i) · C(K − 1, i − 1). Pascal's triangle up to 2000.

Advertising Agency (Codeforces 1475E) — n bloggers with follower counts a_i. Choose exactly k bloggers with the maximum possible total. In how many ways? Mod 10^9+7.
→ Let total = how many bloggers have value x, and need = how many of them are among the top k. Any 'need' of the 'total' tied ones work. Answer = C(total, need).

18.9 Number theory: Expectation & probability

KEYWORDS: expectation & probability

Red Apple, Green Apple (Dhaka Preli 2024 D) **[Dhaka]** — N ≤ $3 \cdot 10^5$ boxes, each with red r_i and green g_i. Pick K random boxes, then a random apple from each. Expected total weight for every K, mod 998244353.
→

Dice Probability (CSES 1725) — Throw a die n ≤ 100 times. Probability that the sum is in [a, b], 6 decimals.
→ dp[i][s] = Σ over faces f = 1..6 of dp[i − 1][s − f] / 6. Answer = Σ dp[n][s] for s in [a, b]. 100·600·6 operations.

Candy Lottery (CSES 1727) — n children each get a uniform random number of candies in 1..k. Expected maximum, 6 decimals.
→ Then E = Σ x·P(max = x). O(k) with pow, print 6 decimals.

Ilya and Escalator (Codeforces 518D) — n people in a queue; each second the first one steps on with probability p. Expected number on the escalator after t seconds (n, t ≤ 2000).
→ dp[s+1][j+1] += dp[s][j]·p, dp[s+1][j] += dp[s][j]·(1−p); when j = n, it just stays. Answer Σ j·dp[t][j]. O(n·t).

Beautiful Mirrors (Codeforces 1265E) — n mirrors; mirror i says 'beautiful' with probability $p_i/100$. Each day ask from mirror 1; on 'no' restart from 1 next day, on 'yes' continue to the next mirror (stop after n). Expected days mod 998244353.
→ An attempt succeeds with probability $p_1 \ldots p_n$; attempts repeat independently. Expected total = (days per attempt) / (success probability). E = (1 + p_1 + … + $p_1 \ldots p_{n-1}$) / ($p_1 \ldots p_n$), with p_i = given·inv(100) mod 998244353.

Game on Tree (Codeforces 280C) — Rooted tree n ≤ 10^5. Each step pick a uniformly random remaining node and delete its whole subtree. Expected number of steps.
→ All dep(v) of them are equally likely to be first → probability 1/dep(v). Answer Σ 1/dep(v), root has dep 1.

Tree Array (Codeforces 1540B) — Tree n ≤ 200. Mark a random first node, then repeatedly a random unmarked node adjacent to a marked one. Expected number of inversions in the marking order, mod 10^9+7.
→ P[i][j] = probability that a side needing i steps finishes before a side needing j: P[i][j] = (P[i−1][j] + P[i][j−1]) / 2, P[0][j] = 1, P[i][0] = 0. Sum P[dist(l,b)][dist(l,a)] over roots and pairs, divide by n. $O(n^3)$.

Santa's Bot (Codeforces 1279D) — n kids, kid i wants k_i items. The bot picks a random kid x, a random item y from x's list, and a random kid z. Probability that z wants y, mod 998244353.
→ So the probability is Σ over kids x, Σ over items y of x, of (1/n)(1/k_x)(cnt[y]/n). Compute with modular inverses of n and of each k_x. Total work is the total list length.

Little Pony and Expected Maximum (Codeforces 453A) — A fair m-sided die is tossed n times. Expected maximum value (m, n ≤ 10^5), printed as a real number.
→ Expected value = Σ x · P(max = x) for x = 1..m. Use pow on doubles with x/m < 1, so nothing overflows. O(m).

Journey (AtCoder ABC194 D) — N isolated vertices; you stand on vertex 1. Each step, pick a uniformly random vertex (any of the N), add an edge from your vertex to it and move there. Expected number of steps until the graph is connected.

→ Repeat-until-success takes 1/p tries on average, so that stage takes N/(N − k) jumps. Answer = Σ N/(N − k) for k = 1..N − 1 (it's the coupon collector).

18.10 Number theory: Floor blocks, series, interpolation

KEYWORDS: floor blocks, series, interpolation

Sum of Divisors (CSES 1082) — n ≤ 10^{12}. Σ σ(i) for i = 1..n, mod 10^9+7.

→ ⌊n/d⌋ takes only about 2√n different values, in long runs. Handle a whole run [l, r] at once. Each run adds v·(l + … + r); sum l..r with the formula and inverse of 2, mod 10^9+7.

Fraction Floor Sum (AtCoder ABC230 E) — N ≤ 10^{12}. Σ ⌊N/i⌋ for i = 1..N.

→ From i, the same value v = N/i lasts until j = N/v. Jump block to block. ans += (j − i + 1)·v. O(√N).

Geometric Sum (AtCoder ABC293 E) — Σ A^i for i = 0..X−1 mod M (A, M ≤ 10^9, X ≤ 10^{12}). M is not prime.

→ Or a 2×2 matrix [[A, 1], [0, 1]] raised to the X-th power. O(log X). Watch out for M = 1.

Yet Another Counting Problem (Codeforces 1342C) — a, b ≤ 200, queries [l, r] ≤ 10^{18}: count x with ((x mod a) mod b) ≠ ((x mod b) mod a).

→ Prefix sums over the period: cnt(x) = full periods·(bad per period) + pre[x mod L + 1]. Answer per query = cnt(r) − cnt(l − 1).

The Sum of the k-th Powers (Codeforces 622F) — Σ i^k for i = 1..n mod 10^9+7, n ≤ 10^9, k ≤ 10^6.

→ Lagrange interpolation at x = n with points 0..k+1: prefix and suffix products of (n − j) for numerators, factorials for denominators, sign (−1)^(k+1−i). O(k log). If n ≤ k+1, just print S(n).

Bob and Numbers (Dhaka Preli 2023 B) **[Dhaka]** — T ≤ 10^5: A ≤ 10^5, N ≤ 10^{12}. Σ_{i=1..N} |i mod A − i mod (A−1)|.

→

Floor and Mod (Codeforces 1485C) — Count pairs 1 ≤ a ≤ x, 1 ≤ b ≤ y (≤ 10^9) with ⌊a/b⌋ = a mod b.

→ k(k + 1) < a ≤ x, so k ≤ √x — only √x values of k! Σ max(0, min(y, x/k − 1) − k).

Sum of Divisors (AtCoder ABC172 D) — N ≤ 10^7. Compute Σ k·f(k) for k = 1..N, where f(k) is the number of divisors of k.

→ So Σ k·f(k) = Σ over d of d · m(m + 1)/2 with m = N/d. One O(N) loop with 64-bit integers.

Everyone is a Winner! (Codeforces 1263C) — Given n ≤ 10^9, list all distinct values of ⌊n/k⌋ over k ≥ 1 (including 0 for k > n).

→ Collect one q per block, plus 0. Sort and print. O(√n) per test.

18.11 Number theory: Matrix exponentiation

KEYWORDS: matrix exponentiation

Fibonacci Numbers (CSES 1722) — F_n mod 10^9+7 for n ≤ 10^{18}.

→ Raise it to the n-th power with fast exponentiation (square-and-multiply on matrices). M^n = [[F(n+1), F(n)], [F(n), F(n−1)]]. O(8 log n).

Throwing Dice (CSES 1096) — Ways to get sum n (n ≤ 10^{18}) by throwing a die (order matters), mod 10^9+7.

→ First row all 1s (the sum), below it a shifted identity (moving the window). Start from dp[0] = 1, raise the matrix to the n-th power. O(216 log n).

Magic Gems (Codeforces 1117D) — A magic gem takes 1 unit or splits into M normal gems (M units). Ways to fill exactly N units ($N \le 10^{18}$, $M \le 100$), mod 10^9+7.
→ $f(n) = 1$ for $n < M$ (only unsplit gems fit). Matrix power: $100^3 \cdot 60 \approx 6 \cdot 10^7$ operations.

Product Oriented Recurrence (Codeforces 1182E) — $f_x = c^{(2x-6)} \cdot f_{x-1} \cdot f_{x-2} \cdot f_{x-3}$. Given $n \le 10^{18}$ and f_1, f_2, f_3, c, find f_n mod 10^9+7.
→ Exponents only matter mod $p - 1$ (Fermat), so raise the 3×3 matrix mod $p - 1$. $f_n = g_n \cdot c^{(-n)}$. Matrix power + Fermat on exponents.

Jzzhu and Sequences (Codeforces 450B) — $f_1 = x$, $f_2 = y$, $f_i = f_{i-1} - f_{i-2}$. Print f_n mod 10^9+7 ($n \le 2 \cdot 10^9$, x, y can be negative).
→ (When a recurrence repeats, you don't need matrix power at all. If it didn't, a 2×2 matrix would work.) Pick the value from the cycle and print ((v % MOD) + MOD) % MOD for negatives.

18.12 Number theory: Squares, exponent parity, XOR basis

KEYWORDS: squares, exponent parity, xor basis

Sigma Function (LightOJ 1336) — $n \le 10^{12}$. How many i in 1..n have an even σ(i) (sum of divisors)?
→ So σ(i) is odd ⇔ every odd prime has an even exponent ⇔ i is a square or twice a square. Answer = $n - \lfloor\sqrt{n}\rfloor - \lfloor\sqrt{n/2}\rfloor$, with exact integer square roots.

Power Products (Codeforces 1225D) — $n \le 10^5$, $a_i \le 10^5$, $k \le 100$. Count pairs $i < j$ with $a_i \cdot a_j = x^k$ for some integer x.
→ a_j pairs with a_i iff a_j's signature is the 'complement' of a_i's ($k - e$ for every prime). Count signatures in a map while scanning; add count[complement] for each new number.

Square Subsets (Codeforces 895C) — $n \le 10^5$ numbers ≤ 70. Count non-empty subsets whose product is a perfect square, mod 10^9+7.
→ Put all masks into an XOR basis (Gaussian elimination over bits). If the basis has r vectors, exactly $2^{(n-r)}$ subsets XOR to 0. Answer $2^{(n-r)} - 1$ (remove the empty subset).

Odd is real (Dhaka Preli 2018 C) **[Dhaka]** — $T \le 10^5$ ranges [L, R] ≤ 10^{18}. Count numbers with an odd number of odd divisors.
→

Together Square (AtCoder ABC254 D) — Given $N \le 2 \cdot 10^5$, count ordered pairs (i, j), $1 \le i, j \le N$, such that $i \cdot j$ is a perfect square.
→ So j must look like $s \cdot u^2$. How many such j are ≤ N? Exactly $\lfloor\sqrt{N/s}\rfloor$ choices of u. Compute the square-free part of every number with a sieve (divide out d^2 for every d), then sum $\lfloor\sqrt{N/s(i)}\rfloor$ over i. Use an integer square root.

Square-free division (easy) (Codeforces 1497E1) — Array of n numbers ≤ 10^7 (k = 0). Split it into the fewest contiguous segments so that inside each segment no two numbers multiply to a perfect square.
→ Greedy: extend the current segment as long as possible; start a new one only when the next value is already in it. Starting later can never help. Sieve smallest prime factors up to 10^7, compute square-free parts, and sweep with a set. Clear the set when you start a new segment.

18.13 Number theory: Invariants

KEYWORDS: invariants

Omicron Juice Again (Dhaka Preli 2021 B) **[Dhaka]** — Glasses A, B, C; a move transfers exactly K from one glass (with ≥ K) to another. Can all be equal? Values ≤ 20, $T \le 2 \cdot 10^5$.
→

Omicron Juice 2023 (Dhaka Preli 2022 D) **[Dhaka]** — Like 2021 B, but each move pours K from two different glasses into two cups, then back into two different glasses. Values ≤ $2 \cdot 10^6$.
→

Flip the Bits (Codeforces 1504B) — Binary strings a and b of length n. A move picks a prefix of a with equally many 0s and 1s and inverts it. Can a become b?
→ Keep a 'flipped' flag for everything to the left. If a[i] (after the flag) differs from b[i], you must flip prefix i, which is allowed only if it's balanced in the original a. Precompute balance of each prefix of a. Scan i = n..1; on a mismatch require balance[i] = 0 and toggle the flag. O(n).

Magic Stones (Codeforces 1110E) — Arrays c and t of length n. A move picks $1 < i < n$ and sets $c[i] = c[i-1] + c[i+1] - c[i]$. Can c become t?
→ Swapping neighbours lets you reach any order of the differences. So the multiset of differences is the only invariant besides c[1]. Answer Yes iff c[1] = t[1] and the sorted difference arrays are equal. (Then c[n] = t[n] follows.)

Element Extermination (Codeforces 1375C) — A permutation a. A move picks i with a[i] < a[i+1] and deletes one of the two. Can you reduce the array to one element?
→ If a[1] < a[n], you can always make progress: there's always an increasing neighbouring pair, and you can delete in a way that keeps a[1] and a[n]. Answer YES iff a[1] < a[n]. Check the claim with a brute force on all permutations of size 5.

Mere Array (Codeforces 1401C) — Array a. You may swap a[i] and a[j] if gcd(a[i], a[j]) equals the minimum of the whole array. Can you sort it?
→ Multiples of m can be rearranged freely: swap each one with m itself (gcd(m, x) = m), like using m as a 'buffer'. Sort a copy. Answer YES iff every position where a[i] ≠ sorted[i] holds a multiple of m.

18.14 Counting: easy

KEYWORDS: counting easy

Bit Strings (CSES 1617) — Count binary strings of length n, modulo 10^9+7.
→ Product rule: $2 \cdot 2 \cdot \ldots \cdot 2$. Answer 2^n mod 10^9+7 with binExp (or a loop). O(log n). Gotcha: MOD is 10^9+7, not 998244353.

Selection of Personnel (Codeforces 630F) — Count groups of 5, 6 or 7 people chosen from n ≤ 777 candidates (exact answer, no modulus).
→ Answer C(n,5)+C(n,6)+C(n,7). Values fit in 64 bits but the naive n!/... does not. Compute C(n,k) incrementally: c = 1; for i in 0..k−1: c = c·(n−i)/(i+1). Each intermediate value is itself a binomial times a small factor, so it stays below ~2.4·10^17. O(1).

Binomial Coefficients (CSES 1079) — Answer up to 10^5 queries C(a,b) mod 10^9+7 with a ≤ 10^6.
→ fact[] and invFact[] up to 10^6; build invFact backwards from one exponentiation. C(a,b) = fact[a]·invFact[b]·invFact[a−b]. O(N + q). Gotcha: switch MOD to 10^9+7 and use fast I/O.

Creating Strings II (CSES 1715) — Count distinct strings formed by rearranging all characters of a given string (length ≤ 10^6), mod 10^9+7.
→ Multinomial: n! divided by (count of each letter)!. Answer n!·Π invFact[cnt_c] over the 26 letters. O(n).

Distributing Apples (CSES 1716) — Distribute m identical apples among n children (some may get none); count ways mod 10^9+7.
→ Stars and bars with m stars and n−1 bars. Answer C(n+m−1, n−1). Gotcha: factorials must go up to n+m−1 ≈ $2 \cdot 10^6$.

Christmas Party (CSES 1717) — n children each give a gift to someone else (a permutation with no fixed points); count mod 10^9+7.
→ Consider where child 1's gift goes (child k), then whether k gives back to 1. D(n) = (n−1)(D(n−1)+D(n−2)), D(1)=0, D(2)=1. O(n).

Bracket Sequences I (CSES 2064) — Count valid bracket sequences of length n mod 10^9+7.
→ For n = 2k the answer is the k-th Catalan number. C(2k,k) − C(2k,k+1) with factorials up to n. O(n).

Grid Paths (CSES 1638) — Count right/down paths in an n×n grid (n ≤ 1000) avoiding trap cells, mod 10^9+7.
→ dp[i][j] = dp[i−1][j] + dp[i][j−1], and 0 on a trap. O(n^2). Gotcha: if the start cell is a trap the answer is 0.

Chess Queen (UVa 11538) — On an n×m board count ordered placements of a black and a white queen that attack each other.
→ Rows: n·m(m−1). Columns: m·n(n−1). For diagonals assume n ≤ m and count, per diagonal of length L, L(L−1) ordered pairs. Diagonals of lengths 1..n−1 appear twice at each end, length n appears m−n+1 times, and there are 2 directions: 2·(2·Σ_L=1..n−1L(L−1) + (m−n+1)·n(n−1)), with Σ L(L−1) = (n−2)(n−1)n/3. Check 2×2: 4+4+4 = 12. Gotcha: n, m up to 10^6, use 64-bit; swap so n ≤ m.

Modulo Sum (Codeforces 577B) — Given n ≤ 10^6 numbers and m ≤ 1000, decide whether some non-empty subsequence has sum divisible by m.
→ n ≥ m: m+1 prefix sums (with $P_0 = 0$) in m residues → two equal → YES by pigeonhole. Otherwise n < m ≤ 1000: reachable-residue DP in O(m^2). Gotcha: subsequence must be non-empty, so start the DP from each element rather than from residue 0 alone.

Combinations (LightOJ 1067) — Up to 2000 queries of C(n,k) mod 1000003 with n ≤ 10^6.
→ So factorials mod 1000003 never become 0. Precompute fact and invFact mod 1000003 up to 10^6, answer in O(1). If n could exceed the modulus you would need Lucas.

18.15 Counting: standard

KEYWORDS: counting standard

Bouquet (AtCoder ABC156 D) — Count non-empty subsets of n ≤ 10^9 flowers whose size is neither a nor b (a, b ≤ $2\cdot10^5$), mod 10^9+7.
→ 2^n − 1 − C(n,a) − C(n,b). n is too large for factorials. C(n,a) by the falling product n(n−1)…(n−a+1)·(a!)^−1 in O(a). Gotcha: take (x % MOD + MOD) % MOD after subtracting.

Two Arrays (Codeforces 1288C) — Count pairs of length-m arrays a (non-decreasing) and b (non-increasing) with values in [1,n] and a_i ≤ b_i.
→ a_1 ≤ … ≤ a_m ≤ b_m ≤ … ≤ b_1: one non-decreasing array of length 2m. The condition a_m ≤ b_m is enough. Non-decreasing arrays of length 2m over n values: C(n+2m−1, 2m). O(n+m). (An O(n^2m) DP also passes.)

Count the Arrays (Codeforces 1312D) — Count arrays of length n with values in [1,m], exactly one pair of equal elements, strictly increasing up to a peak then strictly decreasing.
→ Choose the values C(m, n−1); choose which non-max value repeats (n−2 ways); it sits once on each side. Each of the other n−3 values goes left or right: 2^n−3. Answer C(m,n−1)·(n−2)·2^n−3; n = 2 gives 0. O(m).

Factorization (AtCoder ABC110 D) — Count sequences of N positive integers whose product is M (N ≤ 10^5, M ≤ 10^9), mod 10^9+7.
→ For p^e, distribute e identical exponents into N distinct positions. Answer Π C(e+N−1, N−1) over primes. Factor by trial division to √M. Factorials up to N+30.

Beautiful Numbers (Codeforces 300C) — With two digits a, b, count length-n numbers using only a and b whose digit sum also uses only a and b.
→ Loop i = number of a's (0..n); sum = i·a + (n−i)·b. If the sum's digits are all a or b, add C(n,i). O(n log n).

Almost Identity Permutations (Codeforces 888D) — Count permutations of n ≤ 1000 with at least n−k fixed points, k ≤ 4.
→ Those i positions must form a derangement among themselves. Σ_i=0..k C(n,i)·D(i), D = 1,0,1,2,9. Exact 64-bit (no modulus). O(k).

Max-Min Sums (AtCoder ABC151 E) — Sum of (max − min) over all K-element subsets of an array, mod 10^9+7.
→ Sort. a_i (0-indexed) is the max of C(i, K−1) subsets and the min of C(n−1−i, K−1) subsets. Σ a_i·(C(i,K−1) − C(n−1−i,K−1)). Equal values are fine because indices break ties. Gotcha: negative a_i: reduce into [0,MOD).

Close Tuples (hard) (Codeforces 1462E2) — Count index m-tuples whose values have max − min ≤ k, mod 10^9+7.
→ For index i, let j be the last index with a_j ≤ a_i+k (two pointers or upper_bound). Add C(j−i, m−1) for each i. O(n log n). Fixing the leftmost sorted index avoids double counting with equal values.

Number of Permutations (Codeforces 1207D) — Count orderings of n pairs where neither the first components nor the second components end up sorted, mod 998244353.
→ |A| = Π (count of each first value)!, |B| similarly. |A∩B|: sort pairs; if seconds are non-decreasing then Π (count of each identical pair)!, else 0. Answer n! − |A| − |B| + |A∩B|. O(n log n).

Ilya and Escalator (Codeforces 518D) — Each second the front person of a queue of n boards with probability p; find the expected number on the escalator after t seconds.
→ dp[s][j] = P(j people boarded after s seconds). If j < n: board with p, stay with 1−p; if j = n stay. Answer Σ j·dp[t][j]. O(nt) with doubles.

Counting Necklaces (CSES 2209) — Count necklaces of n pearls with m colours, rotations considered equal, mod 10^9+7.
→ Rotation by i splits positions into gcd(i,n) cycles. (1/n)·Σ m^gcd(i,n), divide with the inverse of n. O(n log n).

Candy Lottery (CSES 1727) — Each of n children gets a uniform random number of candies in [1,k]; print the expected maximum to 6 decimals.
→ P(max = x) = (x/k)^n − ((x−1)/k)^n. E = Σ x·P(max = x) with pow on doubles. Gotcha: k^n overflows, so work with ratios; print with fixed 6 decimals.

Two Sets II (CSES 1093) — Count ways to split {1..n} into two sets of equal sum, mod 10^9+7.
→ Count subsets with sum S/2: 0/1 knapsack, iterate sums downwards. Every split is counted twice (each side). Fix n on one side: run the knapsack on 1..n−1 only. O(n·S).

18.16 Counting: medium

KEYWORDS: counting medium

The Intriguing Obsession (Codeforces 869C) — Three clusters of islands; bridges between same-colour islands must leave them at distance ≥ 3. Count bridge sets mod 998244353.

→ Rule: one island may have at most one bridge to the red group, at most one to the blue group, and so on. So between red and blue, every island has at most one bridge. This is a matching (a set of pairs where no island is used twice). Example: a=1, b=2: the empty set, or the red island joined to blue 1, or to blue 2. That is 3 ways. The three pairs (red–blue, blue–purple, red–purple) do not affect each other, so multiply. For one pair with sizes a and b: choose k islands on each side and pair them up. Count f(a,b) = Σ_k=0..min(a,b) C(a,k)·C(b,k)·k!. Precompute factorials and inverse factorials up to 5000, mod 998244353. You need them for C(n,k).Why matchings: if island X has two bridges to red islands R1 and R2, then R1–X–R2 has length 2, which is forbidden. Bridges inside one colour are also forbidden (distance 1). So each pair of colours gives a matching.Why independent: the rule only looks at bridges from one island to one colour group. A red–blue bridge never changes what is allowed between blue and purple.Write f(a,b): loop k from 0 to min(a,b). Pick k red islands C(a,k), pick k blue islands C(b,k), pair them in k! ways. Add up.Answer = f(a,b)·f(b,c)·f(a,c) mod p.Check: a=b=c=1. f(1,1)=1+1=2, answer 2·2·2=8. This matches the sample (each of the 3 bridges is on or off).Complexity O(n) per pair after O(n) precomputation. Gotcha: take mod after every multiplication; use 64-bit integers.

Strivore (AtCoder ABC171 F) — Count strings obtainable by inserting K lowercase letters into S, mod 10^9+7.

→ To count each string once, use the greedy (earliest) embedding: match S[1] at the first place it appears, then S[2] at the first place after that, and so on. Example: S="ab", T="aab": greedy uses positions 1 and 3. Every string has exactly one greedy embedding. Say the last letter of S is matched at position i. Before position i, the free (unmatched) letters must not equal the next letter of S we are waiting for (else greedy would take it), so each has 25 choices. After position i, any letter is fine: 26 choices. The N−1 other matched positions can be any of the first i−1 places: C(i−1, N−1). Read K and S. Let N = |S|. Note: the answer depends only on N, not on the letters of S.Precompute factorials up to N+K and powers of 25 and 26, mod 10^9+7.For each i from N to N+K: term = C(i−1, N−1) · 25^i−N · 26^N+K−i.Why 25: at a free position before i, we are waiting for some letter of S. If the free letter were that letter, greedy would match it there. So it must be one of the other 25.Why 26: after the last match, nothing is waiting, so any letter is fine.Sum all terms mod p. Check: K=1, S="a" (N=1). i=1: 1·1·26=26; i=2: 1·25·1=25. Total 51 = 26·2−1 strings of length 2 containing 'a'. Correct.Complexity O(N+K). Gotcha: N+K can be 2·10^6; size arrays well and mod every product.

Tree and Constraints (AtCoder ABC152 F) — Colour each edge of a tree (N ≤ 50) black or white so that each of M ≤ 20 given paths contains a black edge.

→ Use inclusion–exclusion (IE): answer = Σ over subsets S of constraints of (−1)^|S| · (ways where all paths in S are all white). With M ≤ 20 there are only 2^20 ≈ 10^6 subsets. Example: one path of 2 edges in a tree with 3 edges: 2^3 − 2^1 = 6. Store each path as a bitmask: a 64-bit number where bit e is 1 if edge e is on the path (N−1 ≤ 49 edges fit). For a subset S, the forced-white edges are the OR of the masks. The other edges are free: 2^(N−1) − popcount ways (popcount = number of 1-bits). Read the tree. Number edges 0..N−2.For each constraint (u,v), find the path: run DFS from u, keep the parent edge of every vertex, then walk back from v to u. Set those edge bits in mask[i].For every subset S from 0 to 2^M−1: OR together mask[i] for i in S. Let c = popcount of the OR.Add (−1)^|S| · 2^N−1−c to the answer. Why: IE removes colourings where some path is all white, correcting double counts with alternating signs.You can build the OR fast: or[S] = or[S without lowest bit] | mask[lowest bit]. This gives O(2^M) total.Check: N=3 (path 1–2–3), M=1, path 1→3. 2^2 − 2^0 = 3. Correct: only all-white fails.Complexity O(2^M·M) or O(2^M). Gotcha: use 64-bit (unsigned long long, 1ULL<<k). No mod is needed; the answer fits in 64-bit since it is at most 2^49.

Sushi (AtCoder DP J) — N plates with 1–3 sushi; each step a random plate is picked and one piece eaten if present. Expected steps to finish.

→ Use state (c1, c2, c3) = numbers of plates with 1, 2, 3 pieces. Empty plates c0 = N − c1 − c2 − c3. Each step: with chance c1/N we eat from a 1-plate and go to (c1−1, c2, c3); with chance c2/N to (c1+1, c2−1, c3); with chance c3/N to (c1, c2+1, c3−1); with chance c0/N nothing changes (a self-loop). Write E = 1 + (c0/N)·E + (c1/N)·E(c1−1,..) + … . Move the c0 part to the left and solve: E(c1,c2,c3) = (N + c1·E(c1−1,c2,c3) + c2·E(c1+1,c2−1,c3) + c3·E(c1,c2+1,c3−1)) / (c1+c2+c3). E(0,0,0)=0. Count how many plates start with 1, 2, 3 pieces.Make a double array E[c1][c2][c3], size (N+1)^3 (N ≤ 300 so about 27 million doubles; fine, or limit by totals). E[0][0][0]=0.Why the formula: E = 1 + Σ p·E(next) + (c0/N)·E. Then E·(1 − c0/N) = 1 + … . Multiply by N: E·(c1+c2+c3) = N + c1·E(..) + c2·E(..) + c3·E(..).Order: loop c3 from 0 up, then c2 from 0 up, then c1 from 0 up. Why: each transition lowers c3, or keeps c3 and lowers c2, or keeps both and lowers c1. So the needed values are already computed.Skip a term when its count is 0 (avoid index −1). Only compute states with c1+c2+c3 ≤ N.Check: N=1, one plate with 1 piece: E = (1+1·0)/1 = 1. Correct.Print E[start] with about 10 decimals. Complexity O(N^3).

Tricolor Pyramid (AtCoder ARC117 C) — A row of N coloured blocks (B/W/R) builds a pyramid by a fixed rule; find the top colour.

→ Check c = −(a+b) mod 3. Same colours: a=b=1 gives −2 ≡ 1. Good. Different: a=0, b=1 gives −1 ≡ 2, the third colour. Good. Now the pyramid works like Pascal's triangle with an extra minus sign at every level. After N−1 levels, the top = (−1)^N−1 · Σ_i C(N−1, i)·a_i mod 3. We need C(N−1, i) mod 3 for N up to 4·10^5. Use Lucas' theorem: write n and k in base 3; C(n,k) mod 3 = product of C(n_d, k_d) over the digits d (and 0 if some k_d > n_d). Map B→0, W→1, R→2 for each block a_0..a_N−1.Why the sum: each level is c = −(a+b). This is linear, so like Pascal's triangle the top is the sum of a_i times the number of paths C(N−1,i), times (−1) per level.Write a function small C(n,k) for n,k in {0,1,2}: C(0,0)=C(1,0)=C(1,1)=C(2,0)=C(2,2)=1, C(2,1)=2, others 0.Lucas: c=1; while n>0 or k>0: c = c·C(n%3, k%3) % 3; n/=3; k/=3.Sum C(N−1,i)·a_i mod 3. If N−1 is odd, take the negative: s = (3 − s) % 3.Map back 0→B, 1→W, 2→R. Check: N=2, "BW": s = 0+1 = 1, N−1=1 odd → 2 = R. Correct (B and W give R).Complexity O(N log N). Gotcha: you cannot use factorial inverses mod 3, because 3 divides factorials. That is why we need Lucas.

Classy Numbers (Codeforces 1036C) — Count numbers in [L,R] (up to 10^{18}) with at most 3 non-zero digits, for many queries.

→ Let f(X) = how many numbers in [1, X] have at most 3 non-zero digits. Answer = f(R) − f(L−1). To compute f(X), read X's digits from left to right. Example X = 250: numbers with first digit 0 or 1 have a free tail of 2 digits; first digit 2 keeps us "tight" (still equal to X so far). When we put a smaller digit than X's digit at some position, the rest (length len) is free. If we may still use c more non-zero digits, there are Σ_j=0..c C(len, j)·9^j ways: choose j positions for non-zero digits, each has 9 values. This is the same as a digit DP with state (position, non-zero count, tight). Precompute C(n,k) for n ≤ 19 (Pascal) and powers of 9. Use 64-bit.Define ways(len, c) = Σ_j=0..min(c,len) C(len,j)·9^j.f(X): let the digits be d_1..d_m, used = 0. For each position p with digit d > 0: placing 0 here gives ways(m−p, 3−used). Placing 1..d−1 (that is d−1 choices) gives (d−1)·ways(m−p, 3−used−1) if used < 3. Then place d itself: used++. If used > 3, stop.Why: every number below X first differs from X at one position, where it has a smaller digit. After that the tail is free.At the end, if used ≤ 3, add 1 for X itself. This counts 0 as well, which is fine because f(R) − f(L−1) cancels it.Check: f(10) = 0..10 = 11 numbers (all have ≤ 1 non-zero digit). Good.Complexity O(19·4) per query, T ≤ 10^4. Gotcha: R up to 10^18; use long long and handle L−1 = 0.

Chip Move (Codeforces 1716D) — Moves have lengths divisible by k, k+1, k+2, ...; count ways to reach each x ≤ n (n ≤ 2·10⁵), mod 998244353.

→ The sum grows like t^2/2, so t ≤ about √(2n) ≈ 630. So we can do one DP layer per move. Let old[x] = ways to be at x after t moves. Move t+1 has step s = k+t, and its length is j·s for some j ≥ 1. Example k=1: after move 1 (s=1) old[x]=1 for all x ≥ 1. new[x] = old[x−s] + old[x−2s] + … . This is slow, but notice new[x] = new[x−s] + old[x−s]. It is like a prefix sum with jumps of s. Add new[x] to ans[x] for every layer, then set old = new. Start: old[0] = 1, others 0. ans[x] = 0. Let s = k, minimum position lo = 0.Each round: lo += s. If lo > n, stop. Why: no way can reach a position ≤ n with more moves.For x from 0 to n: new[x] = (x ≥ s) ? new[x−s] + old[x−s] : 0. Why: a move of length j·s from x−j·s; either j=1 (old[x−s]) or j ≥ 2, which is the same as new[x−s].Add new[x] to ans[x] for all x, mod 998244353. Then old = new, s++.Check: n=3, k=2. Round 1 (s=2): new[2]=1. Round 2 (s=3): lo=5 > 3, stop. So ans[1]=0, ans[2]=1, ans[3]=0. Correct (only 2 is a multiple of 2 ≤ 3).Print ans[1..n].Complexity O(n√n) time, O(n) memory using two arrays. Gotcha: an n×√n table is too much memory; keep only two layers. Take mod at every addition.

Reordering (AtCoder ABC234 F) — Count distinct non-empty strings formed by choosing some characters of S and permuting them, mod 998244353.

→ Handle letters one by one: first 'a', then 'b', and so on. Let dp[j] = number of different strings of length j using only the letters processed so far. Example: S = "aab". After 'a': dp[0]=1, dp[1]=1 ("a"), dp[2]=1 ("aa"). Add c copies of a new letter to a string of length j. The new string has length j+c, and we choose which c positions are the new letter: C(j+c, c) ways. So dp'[j+c] += dp[j]·C(j+c, c) for 0 ≤ c ≤ cnt (cnt = how many times the letter is in S). Count each letter in S. Precompute factorials and inverses up to n = |S|, mod 998244353.dp[0] = 1.For each letter with count cnt > 0: make dp' all zero. For each j with dp[j] ≠ 0 and each c from 0 to cnt: dp'[j+c] += dp[j]·C(j+c, c). Then dp = dp'.Why distinct: the old letters keep their order, and the positions of the new letter are a set. Different (old string, positions) give different strings.Answer = dp[1] + … + dp[n] (not dp[0], the empty string).Check: S="aab". After 'b': dp'[1]=1+1=2 ("a","b"), dp'[2]=1+C(2,1)=3 ("aa","ab","ba"), dp'[3]=C(3,1)=3. Sum 8. Correct.Complexity: total O(n^2) because Σ cnt = n; fine for n ≤ 5000. Gotcha: mod after each multiply; j only goes up to the letters used so far.

Permutation (AtCoder DP T) — Count permutations of 1..N matching a given string of '<' and '>' between neighbours, mod 10⁹+7.

→ Use only relative ranks. dp[i][j] = number of ways for the first i elements, where the i-th element is the j-th smallest among those i (1 ≤ j ≤ i). Example i=2: "<" gives the order (1,2), so dp[2][2]=1. Add element i+1 with rank j among i+1. For '<' we need the previous rank t < j: dp[i+1][j] = Σ_t<j dp[i][t]. For '>' we need t ≥ j: dp[i+1][j] = Σ_t=j..i dp[i][t]. Use prefix sums so each row is O(N). dp[1][1] = 1.For each i from 1 to N−1, build prefix sums P[t] = dp[i][1] + … + dp[i][t].If s[i] is '<': dp[i+1][j] = P[j−1] for j = 1..i+1. If '>': dp[i+1][j] = P[i] − P[j−1].Why t ≥ j for '>': the new element takes rank j, so old elements with rank ≥ j shift up by one. The previous element was larger exactly when its old rank t ≥ j.Answer = dp[N][1] + … + dp[N][N] mod 10^9+7.Check: N=3, "<>": dp[2] = [0,1]. Row 3 with '>': j=1: 1, j=2: 1, j=3: 0. Sum 2 (132, 231). Correct.Complexity O(N^2). Gotcha: P[i] − P[j−1] can be negative after mod; add the mod back. Use only two rows to save memory.

Bracket Sequences II (CSES 2187) — Count valid bracket sequences of length n that start with a given prefix, mod 10^9+7.

→ After a good prefix we have balance b and k = n − (prefix length) characters left. We must end at balance 0 and never go below 0. Think of it as a path: '(' goes up, ')' goes down, from height b to height 0. Example: b=1, k=1: only ")" works, 1 way. Count all paths from b to 0 in k steps: they have d = (k+b)/2 down steps, so C(k, d). Remove bad paths that touch −1. By the reflection trick (flip the path after the first touch of −1), bad paths match paths ending at −2, which have d+1 down steps: C(k, d+1). Answer = C(k,d) − C(k,d+1). Scan the prefix. If the balance goes below 0, print 0.Let b = final balance, k = n − length. If k+b is odd or b > k, print 0. Why: you need exactly b more ')' than '(', so the parity must match and there must be room.Precompute factorials and inverse factorials up to n, mod 10^9+7.d = (k+b)/2. Answer = C(k,d) − C(k,d+1), with C = 0 when d+1 > k.Why reflection: take a bad path, find its first time at −1, flip every step after it. It now ends at −2. This is a one-to-one match, so bad = C(k, d+1).Check: n=4, prefix "(": b=1, k=3, d=2. C(3,2) − C(3,3) = 3 − 1 = 2 ("(())", "()()"). Correct.Complexity O(n). Gotcha: the subtraction mod p may be negative; add p. Odd n gives 0.

Counting Grids (CSES 2210) — Count n×n black/white grids up to rotation, mod 10^9+7.

→ Use Burnside's lemma: number of classes = average, over the 4 rotations (0°, 90°, 180°, 270°), of the number of grids that do not change under that rotation. Example n=1: every rotation fixes both grids, so (2+2+2+2)/4 = 2. A grid is fixed by a rotation when all cells in one orbit (cells that move into each other) have the same colour. So fixed grids = 2^orbits. 0°: n^2 orbits. 90° and 270°: cells form groups of 4 (plus the centre if n is odd): $\lceil n^2/4 \rceil$ orbits. 180°: pairs (plus centre): $\lceil n^2/2 \rceil$ orbits. Read n. Compute n^2 in 64-bit to be safe (it is the exponent, not reduced mod p).Compute a = 2^n^2, b = 2^$\lceil n^2/4 \rceil$, c = 2^$\lceil n^2/2 \rceil$ with fast power mod 10^9+7. $\lceil x/4 \rceil$ = (x+3)/4 in integer division.Why $\lceil n^2/4 \rceil$: for even n all cells split into groups of 4. For odd n the centre stays alone, giving $(n^2-1)/4 + 1$.Sum = a + 2·b + c (90° and 270° give the same count).Divide by 4: multiply by the modular inverse of 4, which is 4^p−2 mod p (Fermat), here 250000002.Check: n=2. a=16, b=2^1=2, c=2^2=4. (16+4+4)/4 = 6. Correct.Complexity O(log n). Gotcha: do not divide by 4 with normal division after taking mod; use the inverse.

18.17 Counting: hard

KEYWORDS: counting hard

Gerald and the Giant Chess (Codeforces 559C) — Count right/down paths on an h×w board (≤ 10^5) avoiding up to 2000 black cells, mod 10^9+7.

→ Count the bad paths by their first black cell. Every bad path touches some black cell first, so each bad path is counted exactly once. Example: on a 3×4 board with black cells (2,2) and (2,3), there are 10 paths in total; we will remove the ones that touch a black cell first at (2,2) or first at (2,3). Sort the black cells by row, then by column, and put the target (h,w) at the end. Let f[i] = number of paths from (1,1) to cell i that touch no black cell before i. Then f[i] = C(r_i+c_i−2, r_i−1) − Σ f[j]·C(Δr+Δc, Δr) over earlier cells j with r_j ≤ r_i and c_j ≤ c_i, where Δr = r_i−r_j, Δc = c_i−c_j. Precompute factorials and inverse factorials up to h+w, mod 10^9+7, so C(a,b) is O(1). (Inverse = power p−2, by Fermat's little theorem, because the mod is prime.)Read the k black cells, sort by (row, column), and append (h,w) as the last cell. Sorting makes sure any cell that can come before i on a path appears earlier in the list.For each cell i in order: start with f[i] = C(r+c−2, r−1), all paths to i.For each earlier j with r_j ≤ r_i and c_j ≤ c_i: subtract f[j]·C(Δr+Δc, Δr). Why: these are paths whose first black cell is j (f[j] ways), then any path from j to i.The answer is f[last] (the target). It is exactly the paths that touch no black cell.Check on 3×4, black (2,2),(2,3): f=2 for (2,2); f = 3 − 2·1 = 1 for (2,3); target: 10 − 2·3 − 1·2 = 2. Correct.Complexity O(k^2 + h + w). Gotchas: if c_j > c_i, skip (or make C return 0 for negative arguments); keep values in [0, mod) after subtraction by adding mod.

Anton and School - 2 (Codeforces 785D) — Count distinct-position subsequences of a bracket string of the form k '(' followed by k ')', k ≥ 1.

→ Fix the last chosen '(' at position i. Let L = number of '(' at positions ≤ i (including i), and R = number of ')' after i. If the subsequence has t pairs, you must take i, choose t−1 more '(' from the other L−1, and choose t ')' from the R. So the count for i is Σ_t≥1 C(L−1, t−1)·C(R, t). Use Vandermonde's identity: Σ_t C(A, x−t)·C(B, t) = C(A+B, x). Write C(L−1, t−1) = C(L−1, L−t). Then the sum is Σ C(L−1, L−t)·C(R, t) = C(L−1+R, L). So each '(' adds just one binomial. Precompute factorials and inverse factorials up to n, mod 10^9+7.Precompute suffix counts: closeAfter[i] = number of ')' at positions > i.Scan left to right and keep L = number of '(' seen so far (including the current one).At every '(' at position i: R = closeAfter[i]; add C(L−1+R, L) to the answer. Why: by Vandermonde it equals the number of good subsequences whose last '(' is i. Different i give different subsequences, so no double counting.If R = 0, then C(L−1, L) = 0 automatically (you need at least one ')').Check ")(()()": '(' at 2: L=1,R=2 → C(2,1)=2. At 3: L=2,R=2 → C(3,2)=3. At 5: L=3,R=1 → C(3,3)=1. Total 6. Correct.Complexity O(n). Gotcha: n is up to 2·10^5, so factorials up to n are enough; make C(a,b) return 0 when b > a.

Placing Rooks (Codeforces 1342E) — Place n rooks on n×n so every empty cell is attacked and exactly k pairs attack each other; count mod 998244353.

→ Every row has a rook, or every column has a rook. Take the case "every row has one": there are n rooks and n rows, so each row has exactly one rook. Two rooks attack each other only in the same column. If a column has x rooks, it gives x−1 pairs. So with c used columns, pairs = n − c. Example: n=3, k=2 means c=1: all 3 rooks in one column. So c = n − k. Count: choose the c columns, C(n,c), then give each row a column so every chosen column is used. That is a surjection (an onto map) from n rows to c columns. By inclusion–exclusion, surj(n,c) = Σ_j=0..c (−1)^j·C(c,j)·(c−j)^n. Read n and k. Read k as 64-bit: k can be up to n(n−1)/2, which is too big for 32-bit int.If k ≥ n, print 0: c = n − k would be ≤ 0, impossible.Set c = n − k. Precompute factorials and inverse factorials mod 998244353.Compute S = Σ_j=0..c (−1)^j C(c,j)(c−j)^n with fast power. Why: start with all c^n maps, then remove maps missing some column, add back those missing two, and so on.ans = C(n,c)·S.If k > 0, multiply by 2: the "every column has one" case is symmetric and does not overlap (both at once means c = n, so k = 0). For k = 0 the two cases are the same n! placements, so do not double.Check n=3,k=2: c=1, C(3,1)·1 = 3, times 2 = 6. Correct.Complexity O(n log n). Keep negative terms in range by adding the mod.

Tickets (Codeforces 26D) — n buyers pay with 10s, m with 20s, in random order; the seller starts with k tens. Probability the seller can always give change.

→ Draw the order as a path: a 20 is a step up, a 10 is a step right (a lattice path with n right steps and m up steps). Failure means that at some moment (#20s) > (#10s) + k, i.e. the path touches the line y = x + k + 1. All C(n+m, m) orders are equally likely. Reflection trick: take a bad path and mirror (flip) its part before the first touch over that line. This gives a one-to-one match between bad paths and all paths ending at a mirrored end point, so bad orders = C(n+m, m−k−1). So P = 1 − C(n+m, m−k−1)/C(n+m, m). The ratio equals Π_i=0..k (m−i)/(n+1+i). Read n, m, k.If m > n + k, print 0: even at the end there are not enough 10s for all 20s.If m ≤ k, print 1: the starting 10s already cover every 20-buyer.Otherwise compute r = 1.0 and for i = 0..k do r *= (m − i) / (n + 1 + i). Why: C(n+m, m−k−1)/C(n+m, m) = m(m−1)…(m−k) / ((n+1)(n+2)…(n+k+1)), so the big factorials cancel.Print 1 − r with enough digits (for example 6+ decimals).Check n=5, m=3, k=1: r = (3/6)·(2/7) = 1/7, answer 1 − 1/7 ≈ 0.857143. Correct.Complexity O(k), k ≤ 10. Gotcha: never compute the binomials directly (n+m up to 2·10^5 overflows even doubles); use doubles, not integers, in the product.

Team Work (Codeforces 932E) — Compute Σ_x=1..n C(n,x)·x^k mod 10⁹+7 for n ≤ 10⁹, k ≤ 5000.
→ The falling power x^(j) = x(x−1)…(x−j+1) counts ordered choices of j different people from x. The Stirling number of the second kind S(k,j) = ways to split k items into j non-empty groups. Fact: x^k = Σ_j S(k,j)·x^(j). Example: x^2 = x(x−1) + x, since S(2,2)=S(2,1)=1. Now Σ_x C(n,x)·x^(j) = n^(j)·2^n−j: pick a team and j ordered special members = first pick the j ordered members from n, then any subset of the other n−j. So answer = Σ_j=0..min(k,n) S(k,j)·n(n−1)…(n−j+1)·2^n−j. Build S with S(i,j) = j·S(i−1,j) + S(i−1,j−1). Build Stirling numbers up to row k mod 10^9+7 with S(0,0)=1 and S(i,j) = j·S(i−1,j) + S(i−1,j−1). Why: item i either joins one of the j groups, or starts a new group. Use two 1D rows to save memory.For j = 0..min(k,n): keep fall = n(n−1)…(n−j+1) (update fall *= (n−j+1) each step).Add S(k,j)·fall·2^n−j to the answer; get 2^n−j with fast power (or compute 2^n−min once and multiply by 2 going down).The x = 0 term does not matter: 0^k = 0 for k ≥ 1, and S(k,0) = 0.Check n=3, k=2: j=1: 1·3·4 = 12; j=2: 1·6·2 = 12; total 24. Correct.Complexity O(k^2 + k log n). Gotchas: use 64-bit for products before taking mod; stop at j ≤ n (fall becomes 0 anyway).

Beautiful Mirrors (Codeforces 1265E) — Ask mirrors 1..n in order each day; mirror i says yes with probability p_i/100, a no restarts from mirror 1. Expected days until mirror n says yes.
→ E_i = 1 + p_i·E_i+1 + (1−p_i)·E_1, with E_n+1 = 0. Every E is of the form a + b·E_1. Expand from E_1 down the chain to get E_1 = a + b·E_1, then E_1 = a/(1−b). Example: n=1, p=1/2: E = 1 + E/2, so E = 2. Doing the algebra gives a closed form: E = (1 + p_1 + p_1p_2 + … + p_1…p_n−1) / (p_1p_2…p_n). Intuition: one "attempt" lasts 1 + p_1 + p_1p_2 + … days on average, and an attempt succeeds with probability p_1…p_n. Work mod 998244353, with p_i = P_i·inv(100). Let M = 998244353. Compute inv100 = 100^M−2 mod M (Fermat; M is prime).Set num = 0, prod = 1.For i = 1..n: num += prod (this adds p_1…p_i−1, the chance you reach mirror i in an attempt, i.e. one more day); then prod *= P_i·inv100.After the loop, prod = p_1…p_n = chance an attempt succeeds. Why the answer is num/prod: the attempts are independent, so the number of attempts is geometric with mean 1/prod, and each attempt lasts num days on average (a cleaner proof is the a + b·E_1 algebra from hint 2).Answer = num · prod^M−2 mod M.Check 10 20 50: num = 1 + 0.1 + 0.02 = 1.12, prod = 0.01, E = 112. Correct.Complexity O(n + log M). Gotcha: use 64-bit for every multiplication before mod; do not use doubles.

Sasha and Interesting Fact from Graph Theory (Codeforces 1109D) — Count edge-weighted (weights 1..m) labelled trees on n vertices where the distance between a and b is exactly m.
→ For a fixed e (1 ≤ e ≤ min(n−1, m)): the path has e−1 inner vertices, chosen and ordered from the other n−2 vertices: (n−2)!/(n−1−e)! ways. The path weights are positive and sum to m: a composition count, C(m−1, e−1) (stars and bars). The other n−1−e edges can have any weight: m^n−1−e. Now attach the remaining n−e−1 vertices to the path. This is a rooted forest on all n vertices where the e+1 path vertices are the roots. Known formula: forests on n labelled vertices with r given roots = r·n^n−r−1. With r = e+1 this is (e+1)·n^n−e−2, and it equals 1 when e = n−1 (nothing to attach). Precompute factorials and inverse factorials up to max(n, m), mod 10^9+7.For e = 1 .. min(n−1, m) compute:A = (n−2)! / (n−1−e)! — ordered inner vertices of the path.B = C(m−1, e−1) — ways to give the e path edges positive weights summing to m.F = (e+1)·n^n−e−2 if e < n−1, else F = 1 — ways to hang the other vertices (Cayley-type forest formula). W = m^n−1−e — weights of the non-path edges.Add A·B·F·W to the answer. Different e give different trees, so just sum.Check n=3, m=2: e=1: 1·1·2·2 = 4; e=2: 1·1·1·1 = 1; total 5. Correct.Complexity O(n log n) with fast power (or precompute powers for O(n)). Gotcha: avoid the negative exponent n−e−2 = −1 when e = n−1.

Yaroslav and Two Strings (Codeforces 296B) — Two digit templates with '?' of length n; count fillings where the strings are incomparable (some position bigger, some position smaller).
→ Not incomparable means: s_i ≤ w_i at every position, or s_i ≥ w_i at every position. These two events overlap when s = w. So by inclusion–exclusion: answer = total − (all ≤) − (all ≥) + (all equal). Example "90" vs "09": total 1, all ≤ is 0, all ≥ is 0, equal 0, so answer 1. Each of the four counts is a product over positions, because positions are independent. At position i, count digit pairs (a,b) allowed by the templates ('?' = any of 10 digits): tot_i, le_i (a ≤ b), ge_i (a ≥ b), eq_i (a = b). Multiply each one over all i, mod 10^9+7. Set T = LE = GE = EQ = 1.For each position i, loop a over allowed digits of s_i and b over allowed digits of w_i (at most 100 pairs). Count tot, le, ge, eq.Multiply: T *= tot, LE *= le, GE *= ge, EQ *= eq (all mod 10^9+7).Why products: a filling is "all ≤" exactly when every position independently satisfies ≤.Answer = (T − LE − GE + EQ) mod p. Add 2p before taking mod to stay non-negative.Check "11" vs "55": T=1, LE=1, GE=0, EQ=0, answer 0. Correct.Complexity O(100·n), fine for n ≤ 10^5. Gotcha: EQ is added back because strings equal everywhere were removed twice.

Square Subsets (Codeforces 895C) — Count non-empty subsets of an array (values ≤ 70) whose product is a perfect square, mod 10^9+7.
→ For each value, write a 19-bit mask: bit p is 1 if prime p appears an odd number of times. Example: 12 = 2^2·3, mask = {3}; 3 has mask {3}. Multiplying numbers XORs their masks. So 12·3 = 36 is a square because {3} XOR {3} = 0. We need subsets whose masks XOR to 0. Group equal values: value v appears cnt_v times. Taking an even number of copies gives 2^cnt−1 ways and leaves the mask unchanged; taking an odd number also gives 2^cnt−1 ways and XORs mask(v). So do a DP over the 70 values with state = current XOR (2^19 states). List the 19 primes ≤ 70 and compute mask(v) for v = 1..70.Count cnt_v. Precompute powers of 2 up to n.dp[0] = 1, other dp = 0. dp[x] = number of ways to pick from processed values so the XOR is x.For each v with cnt_v > 0: w = 2^cnt−1; new[x] = dp[x]·w + dp[x XOR mask(v)]·w. Why: half of the 2^cnt subsets of the copies have even size, half odd.Answer = dp[0] − 1 (remove the empty subset).Check [1,1,1,1]: mask 0, w = 8, dp[0] = 8+8 = 16, answer 15. Correct.Complexity O(70·2^19) ≈ 3.7·10^7. Gotchas: use two arrays (or careful in-place pairs); mod 10^9+7; 64-bit for products. Alternative: linear basis (Gaussian elimination over XOR), answer 2^n−rank − 1.

Coprime Subsequences (Codeforces 803F) — Count non-empty subsequences whose gcd is 1 (values ≤ 10^5), mod 10^9+7.
→ Let cnt(d) = how many elements are divisible by d. Any non-empty choice of those elements has gcd divisible by d, so there are 2^cnt(d) − 1 such subsequences. Example [1,2,3]: d=1 gives 7, d=2 gives 1, d=3 gives 1. gcd = 1 means no prime divides all chosen elements. Inclusion–exclusion over sets of primes gives signs: this is the Möbius function μ(d): μ(1)=1, μ(d) = (−1)^r if d is a product of r different primes, and μ(d)=0 if some prime squared divides d. Answer = Σ_d μ(d)·(2^cnt(d) − 1). Let V = 10^5. Count freq[x] for each value.cnt(d) = Σ freq[d], freq[2d], freq[3d], … This harmonic loop costs V/1 + V/2 + … = O(V log V).Compute μ with a sieve (linear sieve, or for each prime p flip the sign of multiples of p and zero the multiples of p^2).Precompute pow2[i] up to n mod 10^9+7.ans = Σ_d=1..V μ(d)·(pow2[cnt(d)] − 1). Why: a subsequence with gcd g is counted Σ_d | g μ(d) times, which is 1 if g = 1 and 0 otherwise.Check [1,2,3]: 7 − 1 − 1 + μ(6)·0 = 5. Correct.Complexity O(V log V). Gotchas: negative terms — add the mod; skip d with μ(d) = 0.

Winter is here (Codeforces 839D) — Sum over all subsets with gcd > 1 of (size × gcd), mod 10^9+7.

→ Let c = cnt(d) = number of elements divisible by d. Total size of all subsets of c items = Σ k·C(c,k) = c·2^c−1 (each item is in exactly half of the 2^c subsets). Example [3,3,1], d=3: c=2, sizes 1+1+2 = 4 = 2·2^1. But this counts subsets whose gcd is d, 2d, 3d, … (any multiple of d). Let g(d) = total size of subsets with gcd exactly d. Then g(d) = c·2^c−1 − g(2d) − g(3d) − …. Go from d = V down to 2, so the g of multiples is ready. Answer = Σ_d≥2 d·g(d). Let V = max value (10^6). Count freq[x].For each d, cnt(d) = Σ freq over multiples of d (harmonic loop, O(V log V)).Precompute pow2 up to n mod 10^9+7.For d = V down to 2: g(d) = cnt(d)·pow2[cnt(d)−1] (0 if cnt = 0), then subtract g(2d), g(3d), … ≤ V. Why: every subset of multiples of d has gcd equal to some multiple of d; remove the ones with larger gcd.ans = Σ d·g(d) mod p. d = 1 is skipped because we need gcd > 1.Check [3,3,1]: g(3) = 4, nothing else, ans = 3·4 = 12. Correct.Complexity O(V log V). Gotchas: cnt = 0 means pow2[−1] — handle it; keep g non-negative mod p; use 64-bit.

Steps to One (Codeforces 1139D) — Append uniform random numbers from 1..m until the gcd of the array is 1; find the expected length mod 10^9+7.

→ So E[L] = 1 + Σ_t≥1 P(gcd of the first t numbers > 1). The gcd is > 1 when some d ≥ 2 divides all t numbers. One number is divisible by d with probability c/m, where c = ⌊m/d⌋. Example: m = 6, d = 2: c = 3, so the chance is 3/6. Different d overlap (all numbers divisible by 6 are also divisible by 2 and 3). Fix this with inclusion–exclusion using the Möbius function μ(d): μ(d) = 0 if d has a squared prime factor, otherwise (−1)^number of prime factors. Then P(gcd > 1) = Σ_d≥2 −μ(d)·(c/m)^t. Summing the geometric series over t ≥ 1 gives Σ_t≥1(c/m)^t = c/(m−c). Compute μ(d) for d = 1..m with a sieve (linear sieve or Eratosthenes). Why: we need it for inclusion–exclusion.Start with ans = 1. Why: P(L > 0) = 1, the first number is always added.For each d from 2 to m with μ(d) ≠ 0: let c = ⌊m/d⌋ and subtract μ(d)·c·inverse(m−c) from ans. Why: c/(m−c) is the sum over all t ≥ 1 of (c/m)^t, and the sign −μ(d) comes from inclusion–exclusion.Note m−c ≥ 1 because d ≥ 2, so the inverse always exists.Compute inverse(x) as x^p−2 mod p with fast power, p = 10^9+7.Print ans mod p, keeping it in [0, p).Check m = 2: only d = 2, μ = −1, c = 1, term 1/(2−1) = 1, so ans = 1 + 1 = 2. Correct: each step gives 1 with chance 1/2, so the expected length is 2. Complexity: O(m log p). Gotchas: after subtracting, add p to avoid negative values; use 64-bit for products before taking mod.

Game on Tree (Codeforces 280C) — Repeatedly pick a random remaining vertex and delete its whole subtree; find the expected number of picks.

→ By linearity of expectation, E[picks] = Σ_v P(v is picked). Imagine all n vertices in a random order; you always pick the first vertex in the order that is still alive. Vertex v is picked exactly when it comes before all its ancestors in this order. Example: path 1–2–3 rooted at 1: vertex 3 is picked iff it comes first among {1, 2, 3}, chance 1/3. Let depth(v) = number of vertices on the path from the root to v, with depth(root) = 1. Each of these depth(v) vertices is equally likely to be first, so P(v is picked) = 1/depth(v). The answer is Σ_v 1/depth(v). Read the tree as an adjacency list. Root it at vertex 1.Run BFS (or iterative DFS) from the root to get depth(v), with depth(1) = 1. Why: only the ancestors of v and v itself can remove v.Why P = 1/depth(v): the random process gives the same result as taking a random order of all vertices and picking the first alive one. Only the order among v and its ancestors decides whether v is picked, and each of them is first with the same chance.Add 1.0/depth(v) for all v into a double.Print with enough digits (for example 10 after the point).Check: n = 2 with edge 1–2: 1/1 + 1/2 = 1.5, which matches the sample. Complexity: O(n). Gotchas: use BFS or an explicit stack, because a recursive DFS can overflow the stack on a long path (n up to 10^5); remember to divide as doubles, not integers.

Inverse Coloring (Codeforces 1027E) — Count n×n black/white boards where adjacent rows are equal or opposite, adjacent columns too, and no single-colour rectangle has ≥ k cells.
→ Write the first row as bits r_1…r_n and the first column as bits c_1…c_n. Then the cell (i, j) has colour r_j XOR c_i (XOR: 0 if equal, 1 if different). A one-colour rectangle needs a block of equal r's and a block of equal c's, so the biggest one has size (longest run in r) × (longest run in c). Example: r = 0011, c = 010: biggest rectangle is 2 × 1. Let cnt[l] = number of binary strings of length n whose longest run is exactly l. Let g(l) = number with all runs ≤ l: dp[0] = 1, dp[i] = dp[i−1] + … + dp[i−l] (the last run has length 1..l), and g(l) = 2·dp[n] (2 choices for the first colour). Then cnt[l] = g(l) − g(l−1). Every pair (r, c) and its fully flipped pair give the same board, so the answer is Σ_a·b<k cnt[a]·cnt[b], divided by 2. For each l from 1 to n, compute dp with a sliding window sum (add dp[i−1], remove dp[i−l−1]) in O(n). Set g(l) = 2·dp[n] mod p. Why: a string is a sequence of runs, each of length 1..l, and the colours alternate after the first one.Set g(0) = 0 and cnt[l] = g(l) − g(l−1) (mod p, add p if negative).For every a, b in 1..n with a·b < k, add cnt[a]·cnt[b]. Why: the largest one-colour rectangle is (longest run in the row) × (longest run in the column).Multiply by the inverse of 2, which is (p+1)/2. Why: flipping all r and all c gives the same board, so each board was counted exactly twice.Print the result mod 998244353.Check n = 2, k = 3: cnt[1] = 2 (01, 10), cnt[2] = 2 (00, 11). Pairs with a·b < 3: (1,1), (1,2), (2,1) give 4+4+4 = 12, and 12/2 = 6, which matches the sample. Complexity: O(n²). Gotchas: k can be bigger than n², use 64-bit for products; take differences mod p carefully.

Black and White Rooks (AtCoder ABC242 F) — Place B black and W white rooks on an N×M board (N, M ≤ 50) so no black rook shares a row or column with a white rook.
→ The rows used by black and by white are disjoint, and the same for columns. So choose: r_1 rows and c_1 columns that black uses exactly, and r_2 rows and c_2 columns (from the remaining ones) that white uses exactly. Example: N = M = 2, B = W = 1: black uses 1 row and 1 column (4 ways), white uses the other row and column (1 way), total 4. Let f(r, c, k) = ways to put k rooks in an r×c rectangle so that every row and every column has a rook. Use inclusion–exclusion: choose i rows and j columns that are forced empty, then f(r, c, k) = Σ_i,j (−1)^i+j·C(r, i)·C(c, j)·C((r−i)(c−j), k). Precompute binomials C(a, b) for a up to 2500 (Pascal's triangle 2501×2501, or factorials and inverse factorials) mod 998244353. Why: C((r−i)(c−j), k) can have top up to 50·50.For every r ≤ N, c ≤ M compute fB[r][c] = f(r, c, B) and fW[r][c] = f(r, c, W) by the inclusion–exclusion formula. Why: subtracting placements that leave some row or column empty leaves exactly the ones that use all.Loop over r_1, c_1, r_2, c_2 with r_1+r_2 ≤ N and c_1+c_2 ≤ M.Add C(N, r_1)·C(M, c_1)·C(N−r_1, r_2)·C(M−c_1, c_2)·fB[r_1][c_1]·fW[r_2][c_2]. Why: choose which rows/columns each colour uses, then place rooks inside.Print the sum mod 998244353.Check: N = M = 2, B = W = 1 gives 4·1·1·1 = 4, matching the sample. Complexity: O(N²M²) ≈ 6·10^6. Gotchas: C(x, k) = 0 when k > x; handle the minus sign by adding the modulus; take mod after every multiplication.

Binomial Coefficient is Fun (AtCoder ARC110 D) — Sum Π C(B_i, A_i) over all non-negative sequences B with ΣB_i ≤ M (M up to 10^9), mod 10^9+7.

→ Put M items in a row and cut them into N+1 piles: piles 1..N have sizes B_i, pile N+1 (the "slack") takes the rest. From pile i choose A_i items. Now think of the N cuts as extra marked positions: in a row of M+N positions, mark the N cuts and the chosen items. Example: N = 1, A = (1), M = 1: B can be 0 or 1, the sum is C(0,1) + C(1,1) = 1. Let S = ΣA_i. Any choice of S + N marked positions among M + N gives exactly one valid configuration: going left to right, the first A_1 marks are chosen items, the next mark is cut 1, the next A_2 marks are chosen items, and so on. So the answer is C(M+N, S+N). M is large, so compute it as (M+N)(M+N−1)…(M−S+1) / (S+N)!. Read N, M and A. Compute S = ΣA_i (up to about 4·10^6).If S > M, print 0. Why: you cannot choose more items than exist.Let k = S + N. Compute num = product of (M+N−i) for i = 0..k−1, mod p = 10^9+7. Why: C(n, k) = n(n−1)…(n−k+1)/k!, and n = M+N is too big for a factorial table.Compute den = k! mod p with a simple loop, then its inverse den^p−2 by fast power.Print num·inverse(den) mod p.Why the bijection is right: the order "A_1 chosen, cut, A_2 chosen, cut, …, A_N chosen, cut" uses exactly S + N marks, and the unmarked positions between cuts are the unchosen items of each pile. Check: N = 1, A = (1), M = 1: C(2, 2) = 1, correct. Complexity: O(S + N + log p). Gotchas: reduce M+N−i mod p before multiplying; use 64-bit products; do not build a factorial table of size M.

Placing Jinas (Codeforces 1696E) — A doll at (x, y) splits into (x+1, y) and (x, y+1); clear all white cells of a staircase (row i has a_i white cells). Minimum operations mod 10^9+7.

→ A doll at (x, y) can only come from (x−1, y) or (x, y−1). So the number of dolls reaching (x, y) equals the number of monotone lattice paths from (0, 0), which is C(x+y, x). Example: (1, 1) is reached by 2 dolls (via (0,1) and via (1,0)), and C(2, 1) = 2. Row i has white cells y = 0..a_i−1, so it needs Σ_y<ai C(i+y, i) operations. The hockey stick identity says C(i, i) + C(i+1, i) + … + C(i+t−1, i) = C(i+t, i+1). So row i costs C(i+a_i, i+1), and the answer is Σ_i=0..n C(i+a_i, i+1). Precompute factorials and inverse factorials mod 10^9+7 up to about n + max a + 1 (around 4·10^5).Why each white cell (x, y) needs exactly C(x+y, x) operations: every doll arriving there must be removed, and each removal sends dolls further, so the counts follow Pascal's rule.Why black cells are fine: the staircase is non-increasing, so dolls only leave the white area and never need to be removed again.For each row i from 0 to n, add C(i + a_i, i + 1). Why: hockey stick sums the whole row in one binomial.If a_i = 0, the term C(i, i+1) is 0; your C function must return 0 when the bottom is bigger than the top.Print the sum mod 10^9+7.Check: n = 2, a = (2, 2, 0): C(2,1) + C(3,2) + C(2,3) = 2 + 3 + 0 = 5, matching the sample. Complexity: O(n + V) where V = max a. Gotchas: the array has n+1 values (rows 0..n); the factorial table must cover i + a_i.

Another Filling the Grid (Codeforces 1228E) — Fill an n×n grid with numbers 1..k (k up to 10^9) so every row and every column has minimum 1. Count mod 10^9+7.

→ Choose i rows that are forced to contain no 1 (they may have other values). Now look at each column alone: its cells in those i rows have k−1 choices, other cells have k choices, and the column must still contain a 1. Example: n = 2, k = 2, i = 0: one column has 2·2 = 4 fillings, 3 of them contain a 1. For one column: all fillings are (k−1)^i·k^n−i, and the ones with no 1 are (k−1)^n. So a good column has (k−1)^ik^n−i − (k−1)^n ways, and the n columns are independent. Answer = Σ_i=0..n (−1)^i·C(n, i)·[(k−1)^ik^n−i − (k−1)^n]^n. Precompute C(n, i) for i = 0..n (Pascal's triangle is fine for n ≤ 250).Precompute powers of k and of k−1 mod 10^9+7 up to n. Why: k is up to 10^9, so always use mod arithmetic.For each i, compute col = (k−1)^i·k^n−i − (k−1)^n. Why: total column fillings minus fillings with no 1.Compute term = C(n, i)·col^n with fast power. Why: the columns do not affect each other once the bad rows are fixed.Add the term if i is even and subtract it if i is odd. Why: inclusion–exclusion over the set of rows with no 1.Check n = 2, k = 2: i=0: (4−1)^2 = 9; i=1: −2·(2−1)^2 = −2; i=2: (1−1)^2 = 0. Total 7, matching the sample. Complexity: O(n log n). Gotchas: k = 1 gives k−1 = 0, and 0^0 must be 1 (answer 1); keep col non-negative by adding the modulus.

Sonya and Informatics (Codeforces 1151F) — A 0/1 array of length n ≤ 100 gets k ≤ 10^9 random swaps of a pair (i < j); find the probability it ends sorted.

→ Use one number x = how many zeros are in the first z positions. The array is sorted exactly when x = z. There are at most 101 values of x, which is very few. Example: a = 0 1 0, z = 2, the first two cells are 0 1, so x = 1. In state x there are z−x ones inside the first z cells and z−x zeros outside. Swapping such a 1 with such a 0 makes x+1: $(z-x)^2$ pairs. Swapping a zero inside with a one outside makes x−1: x·(n−2z+x) pairs. All other pairs keep x. Divide each count by C(n, 2) to get probabilities. Since k is up to 10^9, raise this transition matrix to the k-th power with fast exponentiation. Count z, and the starting state x_0 = zeros among the first z cells.Let inv = inverse of C(n, 2) mod 10^9+7. Build a (z+1)×(z+1) matrix T where T[x][x+1] = $(z-x)^2$·inv, T[x][x−1] = x·(n−2z+x)·inv, and T[x][x] = 1 − (the other two). Why: one random swap changes x by at most 1.Compute T^k by repeated squaring. Why: applying T once is one swap; k swaps is T^k.The answer is T^k[x_0][z]: the probability to be sorted at the end.States with negative counts (when n−2z+x < 0) are impossible; they get 0 probability automatically if you only start from a valid x_0.Check: n = 3, k = 2, a = 0 1 0: the sample answer is 1/3, printed as 333333336. Complexity: $O(z^3 \log k)$ ≤ about 10^6·30. Gotchas: use 64-bit when multiplying matrix entries; keep T[x][x] non-negative mod p; if the target state is unreachable the answer is 0.

The Number of Pairs (Codeforces 1499D) — Count pairs (a, b) of positive integers with c·lcm(a,b) − d·gcd(a,b) = x, for up to 10^4 queries with values ≤ 10^7.

→ You get g·(c·L − d) = x. So g must be a divisor of x, and for each divisor g you need c·L = x/g + d, so L = (x/g + d)/c must be a whole number. Example: c = 1, d = 1, x = 3: g = 1 gives L = 4, g = 3 gives L = 2. Now count pairs with gcd g and lcm g·L. Write a = g·p, b = g·q with gcd(p, q) = 1 and p·q = L. Every prime of L must go fully into p or fully into q, so there are 2^ω(L) pairs, where ω(L) = number of distinct primes of L. L is at most (x + d)/c ≤ 2·10^7, so precompute ω with a sieve. Sieve up to 2·10^7: compute the smallest prime factor, then ω[v] = ω[v / spf] + (1 if spf of v/spf differs from spf of v, else 0). Or just add 1 to ω[m] for every multiple m of each prime. Why: needed for 2^ω(L).For each query, list the divisors g of x by trying i from 1 to √x (both i and x/i).For each g: let t = x/g + d. If t mod c ≠ 0, skip. Else L = t/c. Why: from g·(cL − d) = x.Add 2^ω(L) to the answer. Why: distribute each prime power of L to a or to b.Print the answer as a 64-bit number (no modulus).Check: c = 1, d = 1, x = 3: L = 4 gives 2^1 = 2, L = 2 gives 2. Total 4, matching the sample. Complexity: sieve O(V log log V), then O(√x) per query. Gotchas: the sieve array of 2·10^7 must be small (use char or int8 for ω); the answer can exceed 32 bits.

Jzzhu and Numbers (Codeforces 449D) — Count non-empty subsets of up to 10^6 numbers (< 2^{20}) whose bitwise AND is 0, mod 10^9+7.

→ Let f(mask) = number of elements a_i with (a_i AND mask) = mask (the element is a superset of mask). Then the number of non-empty subsets whose AND contains mask is 2^f(mask) − 1. Example: a = (1, 3), mask = 1: both contain bit 0, f = 2, so 3 subsets. By inclusion–exclusion, answer = Σ_mask (−1)^popcount(mask)·(2^f(mask) − 1), where popcount = number of 1 bits. To get all f quickly use SOS DP (sum over subsets/supersets): start with f[v] = count of v, then for each bit b and each mask without bit b, do f[mask] += f[mask | (1<<b)]. Read the numbers and set f[v] += 1 for each value v (array of size 2^20).For b = 0..19, for every mask: if bit b is not in mask, f[mask] += f[mask | (1<<b)]. Why: after processing all bits, f[mask] counts all values that are supersets of mask.Precompute pow2[i] = 2^i mod 10^9+7 for i = 0..n.For each mask, t = pow2[f[mask]] − 1. Add t if popcount(mask) is even, subtract if odd. Why: inclusion–exclusion over the set of bits that stay 1 in the AND; only subsets whose AND is 0 are counted once in total.Print the result mod p, adding p to fix negative values.Check a = (0): f[0] = 1, all other f = 0, answer 2^1 − 1 = 1 (the subset {0}). Correct. Complexity: O(20·2^20 + n). Gotchas: f values are at most 10^6, so int is fine but use the pow2 table, not floats; reading 10^6 numbers needs fast input.

Crossword Expert (Codeforces 1194F) — Crossword i takes t_i or t_i+1 seconds with probability 1/2 each; with T seconds, find the expected number solved mod 10^9+7.
→ Let S_i = t_1 + … + t_i. The real time is S_i + j, where j = number of slow crosswords (each one adds 1 second). j has C(i, j) ways out of 2^i. So P = Σ_j=0..m C(i, j) / 2^i with m = min(i, T − S_i). Example: i = 2, m = 1: (1 + 2)/4 = 3/4. Computing each sum from zero is too slow. Keep sum = Σ_j≤m C(i, j) and update it. Going from i to i+1 with the same m: by Pascal's rule, sum' = 2·sum − C(i, m). Then m only goes down (S grows), so remove the top terms C(i+1, m) one by one. Total moves of m are O(n), so it is amortised O(n). Precompute factorials, inverse factorials and inv2^i up to n, mod 10^9+7.Keep i (crosswords used), m (current upper limit of j) and sum = Σ_j=0..m C(i, j). Start with i = 0, m = 0, sum = 1.For each next crossword: update sum = 2·sum − C(i, m) and i = i+1. Why: Σ_j≤m C(i+1, j) = Σ_j≤m(C(i, j) + C(i, j−1)).Let m' = min(i, T − S_i). If m' < 0, stop: no more crosswords can finish. While m < m': m = m+1 and add C(i, m). While m > m': subtract C(i, m) and m = m−1. Why: m grows by at most 1 per step and otherwise only goes down, so the total work is O(n).Add sum·inv2^i to the answer. Why: this is P(first i finish).Check: n = 3, T = 5, t = (2, 2, 2): i=1: m = 1, P = 1; i=2: m = 1, P = 3/4; i=3: S = 6 > 5, stop. E = 7/4, printed as 750000007, matching the sample. Complexity: O(n) after O(n) precomputation. Gotchas: T up to 2·10^14, use 64-bit for S; keep sum non-negative mod p.

18.18 Strings: String basics: counting, two pointers, parsing

KEYWORDS: string basics: counting, two pointers, parsing

Palindrome Reorder (CSES 1755) — Rearrange the letters of a string (A–Z, length ≤ 10^6) into a palindrome, or print NO SOLUTION.
→ So if two or more letters have an odd count, it's impossible. Otherwise build the left half by writing each letter count/2 times, put the odd letter (if any) in the middle, and mirror the left half on the right. Count with an array of 26. If more than one count is odd, print NO SOLUTION. Else half = each letter repeated cnt/2 times, answer = half + middle + reverse(half). O(n).

Vasya and String (Codeforces 676C) — A string of 'a'/'b' (n ≤ 10^5). You may change at most k letters. Maximise the length of a block of equal letters.
→ Both ends only move forward, so the whole scan is O(n). Do it once targeting 'a' and once targeting 'b'. best(keep) = longest window with at most k letters ≠ keep (two pointers). Answer = max(best('a'), best('b')).

They Are Everywhere (Codeforces 701C) — n flats in a row, each with a Pokémon type (a letter). Find the shortest contiguous segment that contains every type that appears in the row.
→ Every time the window contains all types, record its length, then try to shrink it from the left. need = number of distinct letters in the whole string. Two pointers with cnt[] and have; while have == need update the answer and move l. O(n).

a-Good String (Codeforces 1385D) — A string of length n = 2^k is 'c-good' if it's just c (n = 1), or one half is all c and the other half is (c+1)-good. Minimum changes to make it 'a'-good.
→ Each level of recursion looks at every character once, and there are log n levels, so the total is O(n log n). go(l, r, c): if l == r return s[l] != c. Else m = mid, cost of making each half all-c, return min(leftBad + go(right, c+1), rightBad + go(left, c+1)).

Go Latin (2018 ICPC Asia Seoul D) **[Dhaka]** — Change each word's ending according to a table (for example -a → -as, -ne → -anes), otherwise append -us.
→ For each rule, remove the old ending and add the new one. Check 'ne' first, then the last letter against the table, otherwise add 'us'.

Repetitions (CSES 1069) — A DNA string (n ≤ 10^6). Print the length of the longest block of one repeated letter.
→ The answer is the biggest block length you ever saw. One pass with run and best. O(n).

Creating Strings (CSES 1622) — Print every different string you can make by rearranging the letters of s (|s| ≤ 8), first the count, then in sorted order.
→ Start from the sorted string (the smallest arrangement) and call it until it returns false. sort(s); do res.pb(s); while (next_permutation(all(s))). O(n! · n).

String Reorder (CSES 1743) — Rearrange s (A–Z, n ≤ 10^6) into the lexicographically smallest string with no two equal adjacent letters, or print −1.
→ Careful: the next position also can't repeat the letter you just placed. So after placing c, c itself may have at most ⌊r/2⌋ copies left (it can't take the first slot), and every other letter at most ⌈r/2⌉. For each position, try letters from A to Z (not the previous one), check the two conditions with the current maximum count. O(26 · n).

Chasing Subs (NWERC 2014 (practice session)) **[Dhaka]** — An encrypted text and a decrypted fragment (≤ $2.5 \cdot 10^5$ each). In how many positions can the fragment be the text under some one-to-one letter substitution? If exactly one, print that piece of text.
→ Inside a window, a distance that points before the window's start should count as 0. So run KMP on these numbers, comparing a text value v at matched length j as (v > j ? 0 : v). Encode both strings, KMP with that clipped comparison, count matches and remember the last one. O(n + m).

Update Query Problem (Dhaka Preli 2021 H) **[Dhaka]** — Strings P and Q with range assignments. Query: count the distinct strings X + Y, where X is a prefix of P[L1..R1] and Y a suffix of Q[L2..R2] (both may be empty).
→ Those pairs number Σ over letters c of cntP(c)·cntQ(c), counted over the two pieces. So answer = (a+1)(b+1) − Σ cntP(c)·cntQ(c). Two lazy segment trees (range assign a letter) keeping 26 letter counts per node. O(26 log n) per operation.

18.19 Strings: Hashing: polynomial, double, with updates

KEYWORDS: hashing: polynomial, double, with updates

Good Substrings (Codeforces 271D) — A string (n ≤ 1500), a list of which letters are bad, and k. Count distinct substrings that contain at most k bad letters.
→ For each start i, extend j to the right while the bad count stays ≤ k, and record the hash of s[i..j] together with its length. Then sort the list and count unique values. Double hash (your Hash struct), O(n^2) hashes, sort + unique. O(n^2 log n). Single 32-bit hashing would risk collisions on 10^6 strings.

Watto and Mechanism (Codeforces 514C) — n stored strings over {a, b, c}. For each query string, is there a stored string of the same length that differs in exactly one position? Total length ≤ $6 \cdot 10^5$.
→ So new hash = old hash + (new char − old char) · B^(L−1−i), in O(1). Use a double hash so a collision can't produce a false YES. Store {hash pair, length} in a set. For each query position and each of the two other letters, adjust the hash in O(1) and look it up. O(total length · log).

Palindrome Queries (CSES 2420) — A string with two kinds of queries: change one character; is s[a..b] a palindrome? n, m ≤ $2 \cdot 10^5$.
→ The reversed string gets its own segment tree, and every update is applied to both (position k in s is position n−1−k in the reverse). Keep DynHash f(s) and g(reverse(s)). Update: f.set(k, x), g.set(n−1−k, x). Query: compare f.get(a, b) with g.get(n−1−b, n−1−a). O(log n) each.

Petr# (Codeforces 113B) — Count distinct substrings of t (|t| ≤ 2000) that start with s_begin and end with s_end (the two may overlap).
→ There are at most $4 \cdot 10^6$ such pairs. Hash each t[i..j], collect, sort, count unique. Double hash every valid (i, j), store {hash, length}, sort + unique. O(n^2 log n).

Permutation (Codeforces 452F) — A permutation of 1..n ($n \le 3 \cdot 10^5$). Is there a pair a, b such that (a+b)/2 is an integer that appears between a and b in the permutation?
→ The seen-array changes one bit at a time, so use hashing with updates: keep hashes of seen[] and of its reverse in segment trees (DynHash). For each x (before marking it), take len = min(x, n−1−x) and test whether seen[x−len..x+len] is a palindrome with DynHash; if not, print YES. Then set seen[x] = 1. O(n log n).

Lazy Squirrel (Dhaka Preli 2021 D) **[Dhaka]** — A rooted tree ($N \le 10^6$) with a letter on each node. For how many nodes x is the string from the root to x a palindrome? Print as a reduced fraction of N.
→ Use two moduli so collisions can't fool you. (This is the kind of problem where a weak hash costs the contest.) BFS order (no recursion for 10^6 nodes), count good nodes, print good/N reduced by gcd.

Palindromic Partitions (CEOI 2017) — Cut a string ($\le 10^6$) into as many pieces as possible so that the sequence of pieces reads the same forwards and backwards (piece i equals piece k+1−i).
→ Comparing the two chunks each time must be O(1): use substring hashes. Two pointers + getHash comparisons; add 1 at the end if a middle piece remains. O(n).

Kefa and Watch (Codeforces 580E) — A digit string ($n \le 10^5$) with two operations: set every digit in [l, r] to c, and ask whether [l, r] has period d.
→ A range filled with digit c has hash $(c+1)(1 + B + \ldots + B^{(len-1)})$. Precompute those sums so the lazy assignment is O(1) per node. Lazy segment tree of (double-mod) hashes; query = compare two range hashes. O((n + q) log n).

Almost A Palindrome? (Dhaka Preli 2020 C) **[Dhaka]** — Point updates on S. Query [L, R]: 0 if it's a palindrome, else the smallest index whose removal makes it a palindrome, else −1.
→ The deleted letter must be one of the two mismatching letters i, j. Try deleting i (check s[i+1..j]), then j. Deleting any letter of a run of equal letters gives the same string, so move to the start of the run (not before L) for the smallest index. DynPal + binary search: $O(\log^2 n)$ per query.

Almost Pattern Matching II (Dhaka Preli 2019 A) **[Dhaka]** — Count substrings of S ($\le 1.2 \cdot 10^6$) that differ from P in at most K ≤ 4 positions.
→ After a jump you're at a mismatch: count it, step over it, jump again. After K+1 mismatches stop. lcp(hs, i + j, hp, j) from your template, at most K+1 jumps per window. O(n·K·log n).

Anti Hash (Dhaka Preli 2017 E) **[Dhaka]** — Given S ($N \ge 10^5$), base B and prime modulus $M < 2^{31}$, output a different string T of the same length with the same polynomial hash.
→ Use the last ~40 positions. Split them in two halves: store the effect of $2 \cdot 10^5$ random changes of the first half in a hash map, then try random changes of the second half until one needs an effect −e that is stored. Meet in the middle on random changes; output S with both halves replaced (skip the rare case T = S). Expected $\sim 10^4$ tries.

18.20 Strings: Hashing sets, multisets and grids

KEYWORDS: hashing sets, multisets and grids

Special Substrings (CSES 2186) — Count substrings in which every letter that appears in the whole string appears the same number of times.
→ When a letter c (not the first) arrives, the key grows by val(c). When the first letter arrives, every difference drops by 1, so subtract Σ val(other letters). map from key to count, add the count before inserting. O(n log n).

The Number of Subpermutations (Codeforces 1175F) — Count subarrays that are a permutation of 1..length (n ≤ $3 \cdot 10^5$).
→ That covers windows whose maximum is right of the 1. Reverse the array and repeat for the left side; windows of length 1 are counted separately. Prefix xor + two sweeps. Each position is visited from at most one 1 per direction: O(n).

Prefix Equality (AtCoder ABC 250 E) — Arrays a and b. Queries (x, y): is the set of the first x elements of a equal to the set of the first y elements of b?
→ Two sets are equal exactly when their hashes are equal (with overwhelming probability, using 64-bit randoms). Prefix set hashes for a and b; each query is one comparison. O((n + q) log n).

Rearrange Query (AtCoder ABC 367 F) — Queries: can a[l..r] be rearranged into b[L..R]? (n, q ≤ $2 \cdot 10^5$)
→ Prefix sums make each range hash O(1). Overflow mod 2^{64} is fine: it's still a valid hash. Prefix sums of val for both arrays; compare differences. O(n + q).

Matrix Matcher (UVa 11019) — Count how many times an X×Y pattern occurs in an N×M grid of letters (N, M ≤ 1000, X, Y ≤ 100).
→ Any sub-rectangle's hash comes from four prefix values (like a 2D prefix sum, but with powers). Hash2D on both, compare every placement in O(1). O(NM).

Animal Classification (ICPC Singapore preliminary 2015) **[Dhaka]** — Two binary trees on leaves 1..N (N ≤ 10^5), given in parenthesis form. Count the groups (leaf sets of subtrees) that appear in both trees.
→ Collect all hashes of the first tree, sort them, and binary-search every group of the second tree. Parse with a stack (no recursion, depth can be 10^5), sort, count matches. O(N log N).

Three Occurrences (Codeforces 1418G) — Count subarrays in which every value that appears appears exactly three times.
→ (b) For each right end, the valid left ends form a suffix: move a left pointer whenever some value appears 4 times. Count equal keys only among prefixes inside the window. Prefix keys + two pointers + map of keys in the window. O(n log n).

18.21 Strings: Prefix function & KMP

KEYWORDS: prefix function & kmp

String Matching (CSES 1753) — Count the occurrences of a pattern in a text (both ≤ 10^6).
→ π equals |pattern| exactly where a full occurrence ends. The separator stops a match from crossing into the pattern. kmpFind(text, pattern).size(). O(n + m).

Finding Borders (CSES 1732) — Print all border lengths of a string (a border is a proper prefix that is also a suffix).
→ Keep jumping k → π[k−1] until 0; that lists every border from longest to shortest. Compute π, follow the chain, reverse, print. O(n).

Password (Codeforces 126B) — Find the longest substring that is a prefix, a suffix, and also appears somewhere strictly inside the string.
→ Let inner = max π[i] over i < n−1. The answer is the longest border with length ≤ inner. Walk the border chain from the longest until length ≤ inner; print it or 'Just a legend'. O(n).

Prefixes and Suffixes (Codeforces 432D) — For every prefix that is also a suffix, print its length and how many times it occurs in the string.
→ So count cnt[π[i]]++ for all i, then push counts down: for L from n to 1, cnt[π[L−1]] += cnt[L]. Finally add 1 for the prefix itself. O(n) after the prefix function; print the border chain from short to long with cnt.

Compress Words (Codeforces 1200E) — Merge words left to right: when adding a word, delete its longest prefix that equals a suffix of the current result. Total length ≤ 10^6.
→ Taking only that tail keeps the total work linear: each word costs O(|w|). For each word: k = π of (w + '#' + last min(|w|, |res|) chars), append w.substr(k). O(total length).

Required Substring (CSES 1112) — Count strings of length n (n ≤ 1000) over A–Z that contain a given pattern (length ≤ 100), mod 10^9+7.
→ dp[state] = number of prefixes ending in that state. Never allow the state to reach m (that would contain the pattern). kmpAutomaton(pattern); n rounds of dp over m states × 26 letters. Answer = 26^n − Σ dp. O(n·m·26).

Camp Schedule (Codeforces 1137B) — Rearrange binary string s to contain as many occurrences of t as possible.
→ So greedily write t, then t[k..] again and again while you still have the needed 0s and 1s; dump the remaining digits at the end. Write characters following t, jumping back to position k after each complete copy; if a needed digit runs out, the rest is forced. O(|s|).

Power Strings (Waterloo local contest · Kattis powerstrings / UVa 10298) **[Dhaka]** — For each string s, find the largest n such that s = a^n for some string a. Input ends with a line '.'.
→ But a period only splits s evenly if it divides n. Otherwise s isn't a power at all, and the answer is 1. p = n − π[n−1]; print n / p if n % p == 0 else 1. O(n).

MUH and Cube Walls (Codeforces 471D) — Count positions where the wall b (heights) matches a part of wall a, if b may be shifted up or down as a whole.
→ So it's plain pattern counting on integer arrays: prefix function of diff(b) + separator + diff(a). Careful when w = 1: every position matches. KMP on vectors of differences. O(n + w).

War of the Corporations (Codeforces 625B) — Replace as few characters of s as possible with '#' so that t no longer occurs in s.
→ Any occurrence starting before that '#' is already broken. kmpFind, then count occurrences that start at or after the last placed '#'. O(n + m).

Obsessive String (Codeforces 494B) — Count ways to choose k ≥ 1 non-overlapping substrings of s, each containing t as a substring (mod 10^9+7).
→ Before a piece starting at a, you can have nothing (1 way) or any choice ending before a: dp[i] = Σ_{a=1..q} (1 + S[a−1]), with S = prefix sums of dp. Keep prefix sums of S too. dp[i] = q + SS[q−1]; answer = S[n]. O(n + m).

Lucky Common Subsequence (Codeforces 346B) — Find the longest common subsequence of a and b that does not contain virus as a substring (all ≤ 100).
→ dp[i][j][state] = best length from a[i..], b[j..]. Rebuild the string by following the choices. kmpAutomaton(virus) + 3D DP. O(n · m · |virus|).

String Compression (Codeforces 825F) — Write s (≤ 8000) as pieces 'count + string' (e.g. 3ab = ababab), minimising the total length.
→ For a fixed start j, one prefix function of s[j..] gives the period of every s[j..j+len) at once: len − π[len−1]. n prefix functions, $O(n^2)$ total.

Radio Transmission (BOI 2009) **[Dhaka]** — A message is repeated forever; you received a piece S of the stream. Find the shortest possible message length.
→ Shortest period = n − (longest border) = n − π[n−1]. One prefix function. O(n).

Clock Pictures (NCPC 2014) **[Dhaka]** — Two pictures of an n-hand clock, rotated arbitrarily. Can they show the same time?
→ So the pictures match exactly when one gap list is a cyclic shift of the other. Cyclic shift check = search b's gaps in a's gaps written twice. Sort, take gaps (mod 360000), KMP of b in a + a. O(n log n).

Sequence (Dhaka Preli 2023 D) **[Dhaka]** — Count distinct sequences $x_1..x_k$ such that $P[1..x_i]$ occurs in Q ending at strictly increasing positions y_i (mod 998244353).
→ The first part is (path length)·F[y−1]. For the second you need, per node on the path, the last time it was on a path; then all of them become y. 'Sum over a root path, then overwrite it' = heavy-light + a map of equal-value intervals. KMP + border tree + HLD with interval map. Answer = Σ f[y]. O(n log² n).

18.22 Strings: Z-function

KEYWORDS: z-function

Finding Periods (CSES 1733) — Print all periods of a string (p is a period if s[i] = s[i+p] for all valid i).
→ So p is a period iff z[p] = n − p (the whole suffix matches). n itself is always a period. Compute z, print every p with z[p] == n − p, plus n. O(n).

Tavas and Malekas (Codeforces 535D) — Count strings of length n where pattern p appears at every given position $y_1 < y_2 < \ldots$ (it may also appear elsewhere), mod 10^9+7.
→ If all neighbours agree, every forced cell is fixed, and every other cell is free (26 choices). Check consecutive positions with the Z-function; count covered cells; answer 26^(free) or 0. O(n + |p|).

Om Nom and Necklace (Codeforces 526D) — For each prefix of s, decide whether it has the form ABAB…ABA with exactly k copies of AB (A, B may be empty).
→ So for each L that passes, all prefix lengths from kL to kL + min(L, z[kL]) are good: a range. Mark ranges with a difference array. For L = 1.. while kL ≤ n: check z[L], then mark [kL, kL + min(L, z[kL])]. O(n).

Test (Codeforces 25E) — Find the shortest string containing three given strings (each ≤ 10^5) as substrings.
→ Both checks are prefix-function (or Z) computations on b + '#' + a. min over 6 orders of |merge(merge(x, y), z)|. O(total length).

Lazy Printing (2022 ICPC Latin America L) **[Dhaka]** — A machine prints by repeating a chosen string of length at most D as long as it matches the target. Minimise the number of instructions.
→ lcp of two suffixes comes from hashing + binary search (or Z / suffix array). From i, try every k ≤ D, take the farthest reach k + lcp(i, i+k), jump there, count one instruction.

Z Algorithm (Library Checker zalgorithm) — Print the Z-array of a string (length ≤ $5\cdot10^5$), with z[0] = n.
→ Then extend character by character and move the box when you pass r. zFunction(s). If you get TLE, you're probably restarting from 0 inside the box.

String Functions (CSES 2107) — Print the Z-function and the prefix function of a string.
→ Both functions are O(n). zFunction(s), z[0] = 0, prefixFunction(s).

Division + LCP (hard version) (Codeforces 1968G2) — For every k in [l, r]: split s into exactly k pieces to maximise the longest common prefix of all pieces.
→ cnt(L) needs at most n/L jumps. Process L from n down to 1, adding positions with z = L to a set, and use lower_bound for each jump: Σ n/L = n log n jumps. Z-function + set + harmonic sum. O(n log² n).

Martian Strings (Codeforces 149E) — For each of m words (≤ 100 words, each ≤ 1000) decide whether it equals s[a..b] + s[c..d] with b < c (|s| ≤ 10^5).
→ Do the same on reversed strings for the suffixes. Then try all split points. Two Z-functions per word. O(m (|s| + |w|)).

18.23 Strings: Palindromes & Manacher

KEYWORDS: palindromes & manacher

Longest Palindrome (CSES 1111) — Print the longest palindromic substring of a string of length ≤ 10^6.

→ manacher returns d1 (odd lengths) and d2 (even lengths) for every centre in O(n). Take the centre with the largest 2·d1 − 1 or 2·d2 and print that substring.

Palindrome Degree (Codeforces 7D) — Degree of s: 0 if not a palindrome, else 1 + degree of its first half. Sum the degrees of all prefixes (length ≤ $5 \cdot 10^6$).

→ No arrays of hashes are needed: keep three running numbers (forward, backward, power). deg[i] = fw == bw ? deg[i/2] + 1 : 0, sum them. O(n).

Prefix-Suffix Palindrome (Hard) (Codeforces 1326D2) — Find the longest palindrome t = (prefix of s) + (suffix of s) with |t| ≤ |s|.

→ Longest palindromic prefix of m: the prefix function of m + '#' + reverse(m) gives it directly. answer = outer + best(longest pal prefix of mid, longest pal suffix of mid) + outer. O(n).

Queries for Number of Palindromes (Codeforces 245H) — String of length ≤ 5000 and up to 10^6 queries: how many palindromic substrings lie inside s[l..r]?

→ cnt[l][r] = cnt[l+1][r] + cnt[l][r−1] − cnt[l+1][r−1] + pal[l][r] (inclusion–exclusion on rectangles). Both tables in $O(n^2)$; answer each query in O(1). Use compact types (char, int32) to fit memory.

Palisection (Codeforces 17E) — Count pairs of palindromic substrings that overlap (share a position), mod 51123987. n ≤ $2 \cdot 10^6$.

→ Manacher gives every centre's radius; the palindromes of one centre start at a contiguous range of positions and end at a contiguous range. Add those with difference arrays. Total T = Σ st; answer = C(T, 2) − Σ st[i]·prefixEnds(i−1), all mod. O(n).

Extend to Palindrome (UVa 11475 (local contest)) **[Dhaka]** — For each line, append the fewest characters to the end so that the result is a palindrome; print it.

→ Longest palindromic suffix = the prefix function of reverse(s) + '#' + s at the last position. k = π.back(); print s + reverse(s).substr(k). O(n) per line.

Enumerate Palindromes (Library Checker enumerate_palindromes) — For each of the 2n − 1 centres (characters and gaps), print the length of the longest palindrome centred there.

→ Odd index i means the gap between i/2 and i/2 + 1: the length is 2·d2[i/2 + 1]. manacher(s), then print in the order above.

Palindromic Characteristics (Codeforces 835D) — A k-palindrome is a palindrome whose left half is a (k−1)-palindrome. For each k, count substrings that are k-palindromes (n ≤ 5000).

→ A k-palindrome is also a (k−1)-palindrome, so count each degree and take suffix sums. Interval table of small numbers (uint8 to save memory). $O(n^2)$.

Non-Palindromic Substring (Codeforces 1943B) — For each query [l, r], sum all k such that some length-k substring of s[l..r] is not a palindrome.

→ So: all equal → 0. Alternating → all even k below the length. Otherwise → every k from 2 to len−1. Then add len itself if the whole range isn't a palindrome (Manacher). Precompute 'next place where s[i] ≠ s[i+1]' and 'where s[i] ≠ s[i+2]', plus Manacher. O(n + q).

Palindrome Again? Arghh! (Dhaka Preli 2019 E) **[Dhaka]** — A string over the first K ≤ 5 letters. Minimum number of swaps (any two positions) to make it a palindrome, or −1.

→ One swap can fix two edges (a,b),(b,a). A cycle of c edges (a,b),(b,c),…,(·,a) needs c − 1 swaps. So the answer is edges − (number of cycles), with as many cycles as possible. Take 2-cycles first, then 3-cycles, 4, 5 (K ≤ 5, enumerate them). Checked against BFS on small strings.

18.24 Strings: Brackets, expressions & big numbers

KEYWORDS: brackets, expressions & big numbers

Parentheses (2018 ICPC Asia Seoul F) **[Dhaka]** — Decide whether an expression is an error, or whether its parentheses are exactly a full parenthesisation (proper) or not (improper).
→ Proper: the whole thing is X or X op X, and every parenthesis pair contains exactly one X op X. Two recursive-descent parsers on the string without spaces.

Bracket Sequences I (CSES 2064) — Count balanced bracket sequences of length n (mod 10^9+7).
→ Odd n gives 0. ncr(2k, k) · inverse(k + 1). O(n) for factorials.

Bracket Sequences II (CSES 2187) — Count balanced sequences of length n that start with a given prefix.
→ Bad paths touch −1. Reflect the part before the first touch: they become paths from −b−2 to 0, counted by C(rem, up + b + 1). Answer = all − bad. Check the prefix, then the reflection formula. O(n).

Longest Regular Bracket Sequence (Codeforces 5C) — Find the longest balanced substring and how many there are of that length.
→ Start with −1 on the stack as the first wall. len = i − top; keep the best and its count; if none, print '0 1'. O(n).

Serval and Parenthesis Sequence (Codeforces 1153C) — Replace each '?' so the string is balanced, but no proper prefix is balanced.
→ Then check: every proper prefix must have balance > 0 and the whole must end at 0. Fill '?' with '(' until there are n/2 of them, rest ')'; verify. O(n).

Sereja and Brackets (Codeforces 380C) — Queries: length of the longest balanced subsequence of s[l..r].
→ That merge is associative, so it fits in a segment tree. Segment tree of (match, open, close); answer = 2 · match. O((n + q) log n).

Replace To Make Regular Bracket Sequence (Codeforces 612C) — Four kinds of brackets <>{}[]() . An opening bracket can be replaced by any opening one, closing by closing. Minimum replacements to make s regular, or Impossible.
→ Pop on each closing bracket and compare types. Stack, count mismatched pairs. O(n).

Bracket Subsequence (Codeforces 1023C) — From a regular sequence, pick a regular subsequence of length exactly k.
→ Since s itself is regular, you'll always find enough ')'. One pass. O(n).

Bracket Substring (Codeforces 1015F) — Count regular bracket sequences of length 2n ($n \le 100$) that contain s ($|s| \le 200$) as a substring.
→ dp[balance][state], 2n steps, two choices each. kmpAutomaton on s, DP. O(n · n · |s|).

Transform the Expression (SPOJ ONP) — Convert infix expressions (letters, + − * / ^, parentheses) into Reverse Polish Notation.
→ '(' is pushed, ')' pops until the matching '('. One stack, one pass. O(n).

Vanya and Brackets (Codeforces 552E) — An expression of digits with + and * (at most 15 '*'). Add one pair of brackets to maximise its value.
→ Evaluate each candidate with a normal expression parser (respecting * before +). Try all placements with evaluate() from your template. $O(17^2 \cdot n)$.

Pieces of Parentheses (NAIPC 2017) **[Dhaka]** — Choose some of n pieces (≤ 300, each ≤ 300 long) and order them to form the longest balanced string.
→ After sorting, it's a knapsack: dp[balance] = longest length, and a piece can be appended only if balance + its minimum ≥ 0. Exchange-argument sort + knapsack over balance. O(n · total).

18.25 Strings: Subsequences

KEYWORDS: subsequences

Remove the Substring (hard) (Codeforces 1203D2) — t is a subsequence of s. Remove one contiguous substring of s, as long as possible, so that t is still a subsequence.
→ For a split after t[i], the hole goes from L[i] + 1 to R[i+1] − 1. Also try removing a whole prefix (before R[0]) or a whole suffix (after L[m−1]). Answer = max(R[0], n − 1 − L[m−1], max over i of R[i+1] − L[i] − 1). O(n).

Subsequences (hard version) (Codeforces 1183H) — Collect k distinct subsequences of s (n ≤ 100, k ≤ 10^{12}); a subsequence of length L costs n − L. Minimum total cost, or −1.
→ Cap the counts at about 10^{12} so nothing overflows. Then take lengths from n down to 0 greedily. O(n^2).

Future Generation (2018 ICPC Asia Jakarta J) **[Dhaka]** — For each of N names choose a non-empty subsequence so the chosen strings are strictly increasing; maximise the total length.
→ Sort each name's subsequences. The valid previous ones form a prefix of the sorted list, so keep a running maximum with a moving pointer. Sorted lists + two pointers + prefix maximum; answer = best in the last row, or −1.

Distinct Subsequences (CSES 1149) — Count different non-empty subsequences of s (mod 10^9+7).
→ dp[i] = 2·dp[i−1] − dp[last[c]], with dp[0] = 1 (the empty one). distinctSubsequences(s) − 1. O(n).

Shortest Subsequence (CSES 1087) — Find a shortest DNA string that is NOT a subsequence of s.
→ Build it: the last letter of each block (the one that took longest to show up) and finally a letter missing from the unfinished tail. Greedy blocks with a set. O(n).

Obtain The String (Codeforces 1295C) — Build t by concatenating subsequences of s. Minimum number of subsequences, or −1.
→ If a letter of t doesn't appear in s at all, the answer is −1. SubseqAutomaton, jump; when nxt is n, count++ and restart at 0. O(26|s| + |t|).

Newspaper Headline (Codeforces 91A) — Glue copies of s1 and erase letters to get s2. How many copies are needed?
→ Unknown letters → −1. Subsequence automaton. O(26|s1| + |s2|).

Number of Subsequences (Codeforces 1426F) — Every '?' can be a, b or c. Over all 3^k strings, count the total number of 'abc' subsequences.
→ A '?' triples everything (three versions of each string), and in the 'a' version a += cnt, in the 'b' version ab += a, in the 'c' version abc += ab. Four running values. O(n).

Emoticons (Dhaka Preli 2015 E) **[Dhaka]** — A string of '^' and '_' (length ≤ 10^5). Find the maximum number of disjoint subsequences "^_^".
→ Pair the i-th left eye with the i-th right eye, and give each pair the earliest unused '_' after its left eye. It must come before the right eye. Binary search k; check greedily with two pointers. O(n log n).

18.26 Strings: Trie & binary trie (XOR)

KEYWORDS: trie & binary trie (xor)

Word Combinations (CSES 1731) — Count the ways to split a string (length ≤ 5000) into words from a dictionary (total length ≤ 10^6), mod 10^9+7.
→ The walk stops as soon as the trie has no edge, so it's at most the longest word length per i. Build a trie; for i from n−1 down to 0 walk it and sum dp. O(n · longest word).

Vasiliy's Multiset (Codeforces 706D) — A multiset with insert, delete and 'max (x xor y) over y in the set' (0 is always in the set).
→ For deletions keep a counter on every node and only walk into children with count > 0. BinTrie: insert(x), insert(x, −1), maxXor(x). O(30) per query.

Perfect Security (Codeforces 923C) — Given A and a multiset P, pair each A[i] with a different element of P to make the sequence A[i] xor P[...] lexicographically smallest.
→ Then remove that p from the trie so it can't be used again. BinTrie with counts: best = minXor(A[i]); erase best ^ A[i]; print best. O(30n).

Choosing The Commander (Codeforces 817E) — Events: add a warrior with personality p, remove one, or ask how many warriors have (p xor p_j) < l.
→ Where l has a 0, the xor bit must be 0 too, so follow that branch only. Binary trie with subtree counts; O(30) per event.

A Lot of Games (Codeforces 455B) — Players build a word letter by letter; it must always be a prefix of some given string; whoever can't move loses. k games, the loser starts the next one. Who wins the last game?
→ If the first player can't win at all, the second wins every game. If they can choose either outcome, they lose on purpose until the last game. If they can only win, starters alternate, so it depends on k's parity. Two-flag game DP on the trie (leaf: win = false, lose = true). Decide with the three cases above.

Phone List (NCPC 2007 · Kattis phonelist) **[Dhaka]** — For each test, are the phone numbers consistent, meaning no number is a prefix of another?
→ So just compare each number with its neighbour. Sort; for each i check whether a[i] is a prefix of a[i+1]. O(total length · log).

Sausage Maximization (Codeforces 282E) — Split the array into a prefix, a middle and a suffix (any may be empty). Maximise (xor of prefix) ^ (xor of suffix).
→ Go j from 0 to n, insert pre[j] first (prefix may end right before j), then query maxXor(suf[j]). Values up to 10^{12}, so 40 bits. BinTrie t(40). O(40 n).

Beautiful Subarrays (Codeforces 665E) — Count subarrays with xor ≥ k (n ≤ 10^6).
→ At the end, the remaining node is 'equal to k', which also counts. Trie with subtree counts (int32 arrays for memory). O(30 n).

Vitya and Strange Lesson (Codeforces 842D) — Queries: xor every element with x (cumulative), then print the mex.
→ If you reach a missing child, the rest of y is 0. Binary trie of distinct values with subtree counts. O(19 (n + m)).

Hyper Prefix Sets (UVa 11488) — Binary strings. For a set of strings, value = (length of their common prefix) × (size of the set). Maximise.
→ Answer = max over nodes of depth × counter. Insert all strings into a trie, track the max while inserting. O(total length).

XOR Inverse (Codeforces 1416C) — Choose x to minimise the number of inversions of b_i = a_i ^ x (smallest x on ties).
→ Pick x's bit to take the smaller total, then split groups by this bit and go to the next lower bit. This is a trie traversal written with groups. 30 levels of splitting. O(30 n).

Vugol Search (Dhaka Preli 2018 I) **[Dhaka]** — N words with smartness values. For each query word A, print the maximum over words B of LCP(A, B) + (A and B are anagrams ? smartness(B) : 0).
→ Anagrams of A are exactly the words with the same sorted letters. Build one trie per group; each node keeps the largest smartness of words below it. Walking A in its group, at depth d the best score is d + that maximum. Global trie + map from sorted word to a group trie. O(total length · log).

18.27 Strings: Aho–Corasick

KEYWORDS: aho–corasick

Finding Patterns (CSES 2102) — A text and k patterns (total length ≤ $5 \cdot 10^5$). For each pattern: does it appear in the text?

→ Push the marks up the suffix links, processing states from deepest to shallowest (reverse BFS order). Build ac, walk the text marking states, spread marks with ac.bfs reversed, answer hit[node[i]]. O(total length · 26).

Counting Patterns (CSES 2103) — Same input; for each pattern print how many times it occurs in the text.

→ Add counts upward along suffix links in reverse BFS order (children before parents). cnt[state]++ while walking; for v in reverse bfs: cnt[link[v]] += cnt[v]; print cnt[node[i]]. O(total · 26).

Pattern Positions (CSES 2104) — Same input; for each pattern print the first position where it occurs (1-based) or −1.

→ The pattern starts at end − |p| + 1. first[state] = min index; spread minimums up the links in reverse bfs order. O(total · 26).

You Are Given Some Strings... (Codeforces 1202E) — Text t and strings $s_1..s_n$ (total ≤ $2 \cdot 10^5$). Sum over all ordered pairs (i, j) of the number of occurrences of $s_i + s_j$ in t.

→ 'Ending at p' comes from Aho–Corasick on t with out[] counts. 'Starting at p' is the same thing on the reversed text with reversed patterns. Two automata, two arrays e and st; answer = Σ e[p]·st[p+1]. O(total · 26).

e-Government (Codeforces 163E) — k surnames; operations: activate/deactivate a surname, or query a text: sum over active surnames of their occurrence counts.

→ Activating pattern p adds 1 to every state in p's subtree. With an Euler tour that subtree is a range, so a lazy segment tree with range add and point query handles it (your LazySum). Euler tour the link tree; toggle = range add on [tin, tout] of the pattern's node; query = sum of point values over the states visited. O((text + updates) log).

Boring Problem (2020 ICPC Asia Macau B) **[Dhaka]** — Random letters (given probabilities) are appended until the string contains one of n patterns. For each prefix of R as the start, print the expected final length (mod p).

→ Process states by depth and express each state's E as a linear combination of only n unknowns (the root plus extra children). Then n leftover equations remain. Solve the n×n system ($O(n^3)$), then answer each prefix of R by walking the automaton. All arithmetic modulo the prime.

String Multimatching (Kattis) — Several patterns (any printable characters). Print every position of every pattern in the text.

→ For each text position, walk the dictionary links from the current node: every node on that chain is a pattern ending here. Start = end − length + 1. O(text + patterns + output).

Frequency of String (Codeforces 963D) — Queries (k, m), all m different: shortest substring of s containing m at least k times.

→ Aho–Corasick over all patterns and walk the dict links to list every match. Aho + sliding window per pattern. O(n√n).

x-prime Substrings (Codeforces 1400F) — Delete the fewest digits so that no x-prime substring remains (x ≤ 20).

→ dp over the string: dp[state] = most characters kept. Each character is deleted (state stays) or kept (state moves, must not be forbidden). Generate + Aho + DP. O(n · states).

Strange Typewriter (Dhaka Preli 2020 G) **[Dhaka]** — Each key types a whole string. Minimum number of keystrokes to type S (total lengths ≤ 10^6), or impossible.
→ Store each node's string length (walk every key once). Different key lengths are at most $\sqrt{(2 \cdot 10^6)}$, so dict chains are short. AhoCorasick + DP over positions. Print impossible if dp[|S|] is unreachable.

18.28 Strings: Suffix array + LCP

KEYWORDS: suffix array + lcp

Distinct Substrings (CSES 2105) — Count the distinct substrings of a string (length ≤ 10^5).
→ So each suffix contributes n − sa[i] − lcp[i−1] new substrings. n(n+1)/2 − Σ lcp. O(n log n).

Repeating Substring (CSES 2106) — Find the longest substring that occurs at least twice (or −1).
→ So the answer is the maximum lcp value, taken at the suffix where it occurs. max over lcp[i]; print s.substr(sa[i], lcp[i]). O(n log n).

Substring Order I (CSES 2108) — Find the k-th smallest distinct substring.
→ Inside the batch, the answer is the prefix of that suffix with length lcp[i−1] + k. One pass over the suffix array. O(n log n).

Substring Distribution (CSES 2110) — For each length 1..n, count the distinct substrings of that length.
→ Add all ranges with a difference array, then take prefix sums. O(n log n).

Longest Common Substring (SPOJ LCS) — Longest common substring of two strings (each ≤ $2.5 \cdot 10^5$).
→ The '#' (smaller than letters, appears once) stops common prefixes from crossing between the strings. max lcp[i] where sa[i] and sa[i+1] are on different sides. O(n log n). (SAM solves it too: see the next lesson.)

Fake News (hard) (Codeforces 802I) — Sum over all distinct substrings p of $\text{occ}(p)^2$ (T ≤ 10 tests, |s| ≤ 10^5).
→ So you need the sum of the minimum over all subarrays of the lcp array. That's a classic monotonic-stack sum. answer = n(n+1)/2 + 2 · Σ over subarrays of min(lcp). O(n log n).

Life Forms (Waterloo local contest · UVa 11107) **[Dhaka]** — n DNA strings (n ≤ 100, length ≤ 1000). Print, sorted, every longest substring shared by more than half of them (or '?').
→ For the best L, collect the substrings whose count is more than n/2, deduplicate, sort. Hashing + binary search, O(total · log). The suffix array way: glue strings with separators and slide a window over the lcp array.

Suffix Array (Library Checker suffixarray) — Print the suffix array of a string of length ≤ $5 \cdot 10^5$.
→ Stop early when all ranks are different. SuffixArray sa(s); print sa.sa.

Inverse Suffix Array (CSES 3225) — Given a suffix array, build any string (a–z) that has it, or print −1.
→ Use the smallest letter possible: increase only when forced. If you need more than 26 letters, print −1. rank array (with the empty suffix as smallest), one pass. O(n).

Match & Catch (Codeforces 427D) — Shortest string that occurs exactly once in a and exactly once in b, or −1.
→ Shortest such length = max(lcp[i−1], lcp[i+1]) + 1, if it's ≤ lcp[i]. Suffix array + lcp. O(n log n).

Number of Substrings (Library Checker) — Count distinct substrings (n ≤ $5 \cdot 10^5$).
→ So the answer is n(n+1)/2 − Σ lcp. SuffixArray + Kasai. O(n log n).

Find the Substrings (Dhaka Preli 2018 F) **[Dhaka]** — String S ($\leq 10^6$). Queries (N, M): how many lowercase strings with length in [N, M] are NOT substrings of S (mod 10^9+7)?
→ That's a range of lengths, so +1 on it with a difference array. A prefix sum over L gives distinct(L) for all L. SuffixArray + difference array + prefix sums of 26^L − distinct(L). O(n log n + Q).

18.29 Strings: Suffix automaton

KEYWORDS: suffix automaton

Substring Order II (CSES 2109) — Find the k-th smallest substring when all n(n+1)/2 substrings are listed with repeats.
→ Walk from the start: for letters a..z, skip a branch if k > cnt[u] + dp[u], else enter it and subtract cnt[u]. SAM + countOcc + dp + greedy walk. O(n · 26).

String (Codeforces 128B) — Print the k-th substring (with repetitions) in lexicographic order, or 'No such line.'. |s|, k $\leq 10^5$.
→ Otherwise use the SAM path counts and walk greedily letter by letter. Identical walk to Substring Order II.

Longest Common Substring II (SPOJ LCS2) — Longest common substring of up to 10 strings (each $\leq 10^5$).
→ Per state, the common length over all strings is the minimum of these values. Answer = max over states of min over strings. O(Σ length · 26).

Cyclical Quest (Codeforces 235C) — For each query x, count substrings of s that are cyclic rotations of x (counting occurrences).
→ Different rotations can be equal, so count each state only once per query (a visited stamp). Sum cnt[state] over distinct states found. O(|x|) per query (amortised).

Good Substrings (Codeforces 316G3) — Count distinct substrings t of s such that for every rule (p, l, r), the number of occurrences of t in p is between l and r.
→ A state is good if it occurs in s and all rule counts fit; it contributes len[v] − len[link[v]] distinct substrings. Generalized SAM + one count array per string. O(total length · rules).

LCS Spanning Tree (2021 ICPC Asia Macau I) **[Dhaka]** — n strings; the edge (i, j) weighs the longest common substring of s_i and s_j. Find the maximum spanning tree weight.
→ Process states by decreasing len; union one representative of each child subtree and the strings ending there, adding len per successful union. Generalized SAM + DSU, each union adds the current len. O(total length · 26).

Lexicographical Substring Search (SPOJ SUBLEX) — Queries: the k-th smallest distinct substring.
→ For a query, from the root try letters a..z: if k fits inside this edge's paths, go there; otherwise subtract and try the next letter. SAM + path counts. O(26 · answer length) per query.

Forbidden Indices (Codeforces 873F) — Maximise |a| · (number of occurrences of a that end at a non-forbidden index).
→ Push counts up the links; each state offers len[v] · cnt[v]. SAM with modified counts. O(n).

String (Codeforces 123D) — Sum over all distinct substrings x of occ(x)(occ(x)+1)/2.
→ So the answer is Σ (len[v] − len[link[v]]) · cnt(cnt+1)/2. SAM + countOcc. O(n).

Expensive Strings (Codeforces 616F) — Strings t_i with costs c_i. f(s) = |s| · Σ c_i · occ(s in t_i). Maximise over all s.
→ All strings in a state share that weight; the longest one (len[v]) is best. Generalized SAM + weighted counts. O(total length).

Pairdrome (Dhaka Preli 2021 E) **[Dhaka]** — Count the ways to pick a substring of X and a substring of Y whose concatenation is a palindrome (mod 10^9+7).
→ W(e): a substring of X equals the reverse of a substring of Y ⇔ it occurs in reverse(Y). Walk X through the suffix automaton of reverse(Y); the sum of occurrence counts of all matched lengths is cnt·(extra length) + a precomputed sum along suffix links. Case |B| > |A| is the same with X' = reverse(Y), Y' = reverse(X) and a non-empty palindrome. Palindrome starts from Manacher + difference array. O(n).

18.30 Strings: Palindromic tree (eertree)

KEYWORDS: palindromic tree (eertree)

Palindromes and Super Abilities (Timus 1960) — For every prefix of s, print how many distinct palindromes it contains.
→ So the answer for each prefix is just the number of nodes (minus the two roots). Feed letters into the Eertree, print len.size() − 2 after each. O(n).

Palindrome Partition (Codeforces 932G) — Count ways to split s into an even number of parts $p_1..p_k$ with $p_i = p_{k-i+1}$, mod 10^9+7.
→ Palindromic suffixes form O(log n) groups with the same difference (series links). The eertree stores the sum for a whole group, so each step is O(log n). Eertree + series links + dp over even i. O(n log n).

Palindromes (APIO 2014 · oj.uz APIO14_palindrome) **[Dhaka]** — For each palindromic substring, its value is length × number of occurrences. Print the maximum value.
→ So push counts along suffix links from the newest nodes to the oldest. Eertree, count(), answer = max len[v]·cnt[v]. O(n).

All Palindromes (CSES 3138) — For each position, print the length of the longest palindrome ending there.
→ add returns that node; print its len. Eertree. O(n).

Virus Synthesis (CERC 2014 G) **[Dhaka]** — Build a DNA string from empty using: add a letter at either end, or append the reversed copy of the current string. Minimum operations.
→ You need an extra 'half link' per eertree node, found like the suffix link but with a length limit. Eertree + half links + DP. O(n).

18.31 Strings: String DP

KEYWORDS: string dp

Edit Distance (CSES 1639) — Minimum insertions, deletions and replacements to turn string a into b (each ≤ 5000).
→ Base cases: turning a prefix into the empty string costs its length. editDistance(a, b) from your template. O(nm).

Longest Common Subsequence (CSES 3403) — Two arrays (≤ 1000 each). Print the length of their longest common subsequence and one such subsequence.
→ To print the sequence, walk from (0, 0): take matching elements, otherwise move to the neighbour with the bigger d. O(nm) table + O(n + m) reconstruction (lcsString in your template does the same for strings).

Easy Problem (Codeforces 1096D) — Delete characters (each with a cost) so that the string no longer contains 'hard' as a subsequence. Minimise the cost.
→ Never let the state reach 4. dp over 4 states, O(4n).

Anthem of Berland (Codeforces 808G) — Replace every '?' in s with a letter to maximise the number of (possibly overlapping) occurrences of t. $|s|\cdot|t| \le 10^7$.
→ dp[state] = most occurrences so far. Whenever the automaton reaches the full length, add 1 (the automaton then continues from the right border, so overlaps work). kmpAutomaton(t); dp over $|s| \times (|t|+1) \times 26$. $O(26\cdot|s|\cdot|t|)$.

Clear the String (Codeforces 1132F) — One operation deletes a contiguous block of equal letters. Minimum operations to delete the whole string ($n \le 500$).
→ In the second case the cost is dp[l+1][m−1] + dp[m][r] (s[l] joins s[m]'s deletion for free). $O(n^3)$.

Edit Distance (2018 ICPC Asia Jakarta A) **[Dhaka]** — Given a binary string S, output any binary string T of the same length with edit distance from S greater than |S|/2.
→ When the counts are equal, all-ones gives exactly half, which isn't enough. Change the first character to break the tie. Majority 0s: all 1s; majority 1s: all 0s; equal: if S starts with 0 print 1 then zeros, else 0 then ones.

Empty String (CSES 1080) — Repeatedly delete two adjacent equal letters. Count the different ways to delete everything ($n \le 500$).
→ The two sequences of operations can interleave: choose positions of the (k−l+1)/2 left operations among all (r−l+1)/2: a binomial coefficient. Σ over k: dp[l+1][k−1] · dp[k+1][r] · C(total, left). $O(n^3)$.

18.32 Strings: Rotations, FFT / bitset matching, BWT, de Bruijn

KEYWORDS: rotations, fft / bitset matching, bwt, de bruijn

Minimal Rotation (CSES 1110) — Print the lexicographically smallest rotation of a string (length $\le 10^6$).
→ Each step moves i or j forward past the compared part, so the total is O(n). minRotation(s) from your template (Duval / Lyndon factorization gives the same answer).

Fuzzy Search (Codeforces 528D) — DNA strings S and T ($\le 2\cdot10^5$). T occurs at i if every T[j] equals some S character within distance k of i + j. Count the positions.
→ Use bitsets: ok &= (near[T[j]] >> j). That's m operations of n/64 words. About $2\cdot10^5 \cdot 2\cdot10^5 / 64 \approx 6\cdot10^8$ simple word operations, fast with bitset. (FFT works too.)

Yet Another String Matching Problem (Codeforces 954I) — Letters a–f. For every window of S of length |T|, the distance to T = minimum number of 'replace letter x by y everywhere' operations to make them equal.
→ 'Does pair (a, b) occur in window i?' is a convolution of the indicator of a in S with the reversed indicator of b in T: 30 FFTs. 30 convolutions + a tiny DSU per window. O(30 n log n).

Smart Thief (2018 ICPC Asia Jakarta C) **[Dhaka]** — Build the shortest string over M digits that contains at least K distinct substrings of length N.
→ Generate it with the FKM (Lyndon word) recursion and stop early. de Bruijn prefix of length N + K − 1 (wrap around if the cycle is shorter).

String Transform (CSES 1113) — Undo the Burrows–Wheeler transform (the string had '#' appended).
→ Start at the row whose last character is '#' and follow that mapping n times, reading the characters. stable_sort indices + walk. O(n log n).

Bit Substrings (CSES 2115) — For every k, count substrings of a bit string with exactly k ones.
→ k = 0 is special (a prefix with itself): count runs of zeros directly. FFT convolve. O(n log n).

De Bruijn Sequence (CSES 1692) — Shortest bit string that contains every length-n bit string as a substring.
→ Make it non-cyclic by repeating its first n − 1 characters at the end. FKM (Lyndon words) from your template. $O(2^n)$.

Substring Reversals (CSES 2073) — Apply m range reversals to a string, print the final string.
→ The flag swaps children when pushed down. reverseRange from your treap template. O((n + m) log n).

Rusty String (Codeforces 827E) — A string of V, K, ?. Fill the ?'s: which periods d are possible?
→ For each d, check all multiples of d: Σ n/d = n log n checks. FFT + harmonic loop. O(n log n).

Cyclic Palindrome (Dhaka Preli 2022 E) **[Dhaka]** — String of letters (case matters) and '?'. Sum over all cyclic shifts of the number of ways to replace '?' by letters (the samples allow all 52) so the shift is a palindrome.
→ Count conflicting pairs for every σ at once: Σ p·q·(p−q)² with letters 1..52 and '?' = 0 is a cyclic self-convolution (Lesson 15, Step 2). '?'-'?' pairs: convolve the '?' indicator with itself. Three FFTs, fold indices mod n, then loop over shifts. O(n log n).

18.33 Graphs and trees: DSU power-ups

KEYWORDS: dsu power-ups

Path Queries (Codeforces 1213G) — Weighted tree, n, m ≤ $2 \cdot 10^5$. For each query q, count pairs (u, v), u < v, whose path has max edge weight ≤ q.
→ How many connected pairs are there? Every time two islands of sizes a and b glue together, every city of the first can now reach every city of the second: a·b brand-new pairs. Keep a running counter. Sort edges by w, sort queries by q (remember their original index). Pointer j over edges: before answering q, union every edge with w ≤ q, adding s[a]·s[b] to the counter for each successful union. Store counter as that query's answer, then print answers in the original order. O((n+m) log n).

Destroying Array (Codeforces 722C) — Array of n ≤ 10^5 non-negative numbers. Elements are destroyed one by one in a given order. After each destruction print the max sum of a contiguous block of still-alive elements.
→ The best block sum can only go up while we revive (blocks only grow). So keep one variable best = max over all blocks so far. In the backward movie, the answer for 'after the i-th destruction' is exactly best just before we revive element i. Go i = n−1 down to 0: first write res[i] = best, then revive p[i]: sum = a[p[i]], union with ON neighbours (adding sums), best = max(best, sum of its group). Print res[0..n−1]. O(n α(n)).

Connecting To One (Dhaka Preli 2017 D) **[Dhaka]** — Graph N ≤ $4 \cdot 10^4$, E ≤ $1.5 \cdot 10^5$ (weights may be negative). Q ≤ 10^5 queries C: if all edges with weight < C are dropped, how many nodes (other than 1) are reachable from node 1?
→ Before answering C, union every edge whose weight ≥ C (a pointer moving through the sorted edges). The nodes reachable from 1 are exactly node 1's DSU group. Careful with the sample: node 1 itself is not counted. Offline sweep: sort edges desc, queries desc, pointer j. For each query add edges with w ≥ C, answer = size(find(1)) − 1, store by original index, print per case. O((E+Q) log).

The Number of Imposters (Codeforces 1594D) — n players, each is honest crewmate or lying imposter. m statements “i says j is imposter/crewmate”. Max possible number of imposters, or −1 if contradictory.
→ If x and x+n end up in the same group, then 'x is crewmate' and 'x is imposter' must both be true — impossible, print −1. Otherwise every group has a mirror group (swap all the +n's). For each pair you choose which one is true. Let cnt[root] = how many imposter-nodes (index ≥ n) are in that group. For each pair of mirror groups (a, b), add max(cnt[a], cnt[b]) — pick the side with more imposters. Mark both as used so you add each pair once. O((n+m) α).

The Social Network (Dhaka Preli 2019 B) **[Dhaka]** — N ≤ 10^5 users, Q ≤ $3 \cdot 10^5$ online queries: 0 U V make friends (networks merge), 1 U T user posts at time T, 2 U L R count posts in U's whole network with time in [L, R].

→ When two groups merge, pour the smaller bag into the bigger one (small-to-large). Each post moves only when its bag at least doubles, so a post moves at most log N times. Cheap! pbds tree of (time, unique id) per root. Query 0: union with small-to-large merge. Query 1: insert into find(U)'s tree. Query 2: order_of_key({R, +∞}) − order_of_key({L, −∞}). O(Q log² N). (Uncomment the pb_ds lines at the top of your template.)

Number of Connected Components (Dhaka Regional 2016 F) **[Dhaka]** — N ≤ 10^5 nodes with labels ≤ 10^6. Nodes are joined if gcd(labels) > 1. Count connected components.

→ Factorise each label quickly with an SPF (smallest prime factor) sieve. For each prime p, remember the first node that entered room p; every later node with p just unions with that first node. comps = N; every successful union does comps−−. Reset the 'first node of prime p' table using a list of the primes you touched (don't clear 10^6 entries each test). Label 1 has no primes and stays alone. O(N log A).

Bipartite Checking (Codeforces 813F) — n, q ≤ 10^5. Each query toggles an edge (add if absent, remove if present). After each query print whether the graph is bipartite.

→ Undo needs a DSU without path compression (union by size only) plus a stack of changes to roll back. For bipartiteness store each node's colour-parity to its parent: an edge inside the same group with the same parity = odd cycle. DFS over the time-segment-tree with rollback DSU; if adding an edge makes an odd cycle, every moment under this node is NO — mark them and don't go deeper. O(q log² q).

0-1-Tree (Codeforces 1156D) — Tree n ≤ $2 \cdot 10^5$, each edge 0 or 1. Count ordered pairs (x, y), x ≠ y, whose path never goes along a 0-edge right after a 1-edge.

→ Mixed paths (0s then 1s) switch at exactly one vertex v: start anywhere in v's 0-group (not v), end anywhere in v's 1-group (not v). That's (s0(v)−1)·(s1(v)−1) pairs. Answer = Σ over 0-groups s(s−1) + Σ over 1-groups s(s−1) + Σ over vertices (s0(v)−1)(s1(v)−1). O(n α).

Mocha and Diana (Hard) (Codeforces 1559D2) — Two forests on the same n ≤ 10^5 vertices. Add as many edges as possible (each edge added to both) so both stay forests. Print them.

→ After that pass, every vertex is already with 1 in at least one forest. Collect L = vertices NOT with 1 in forest A, R = vertices NOT with 1 in forest B. A vertex from L and one from R can always be joined (both join 1's tree in different forests). Two pointers over L and R: skip vertices that meanwhile joined 1 (check with find), otherwise add (u, v) and union both into 1's trees. Print all edges. O(n α).

Graph and Queries (Codeforces 1416D) — n ≤ $2 \cdot 10^5$ vertices with distinct values, m edges. Queries: (1 v) print the max value in v's component and set it to 0; (2 i) delete edge i.

→ Magic fact: the original vertices under that remembered node are exactly v's component at that query time (in forward time). And leaves under one node form a contiguous range in the DFS order of this tree! DFS the reconstruction forest → each node gets a leaf range. Segment tree (max value + where it is) over that order. Go forward through type-1 queries: range max on the remembered node's range, print, set that leaf to 0. O((m+q) log n).

18.34 Graphs and trees: Shortest paths on state graphs

KEYWORDS: shortest paths on state graphs

Flight Discount (CSES 1195) — Directed weighted graph, n ≤ 10^5. You may use one coupon to halve (rounded down) one flight. Cheapest route 1 → n.

→ Now it's just plain Dijkstra on 2n nodes! dist[v][0] and dist[v][1]. Relax: (u,0)→(v,0) cost w, (u,1)→(v,1) cost w, (u,0)→(v,1) cost w/2. Answer dist[n][1] (using the coupon never hurts). This 'add the missing info to the node' idea solves tons of problems.

Investigation (CSES 1202) — Directed weighted graph. Print: min price 1 → n, number of min-price routes (mod 10^9+7), min and max number of flights among min-price routes.

→ Relaxing u→v: if the new distance is better, v copies u's info (cnt = cnt[u], mn = mn[u]+1, mx = mx[u]+1). If it's equal, v merges: cnt += cnt[u], mn = min, mx = max. Standard Dijkstra with those updates (mod 10^9+7 on cnt). Print d[n], cnt[n], mn[n], mx[n]. Positive weights make this 'shortest path DAG' merging correct.

Labyrinth (Codeforces 1063B) — n×m maze, start cell. At most x moves left and y moves right (up/down unlimited). How many cells are reachable?

→ 0-1 BFS: use a deque. A 0-cost neighbour goes to the front, a 1-cost neighbour goes to the back. Each cell gets its minimum L. After 0-1 BFS, count cells with $L \le x$ and $L + (j - c) \le y$. O(n·m).

Minimum Path (Codeforces 1473E) — Undirected weighted graph. Path weight = sum − max + min of its edges. Min weight from 1 to every vertex.

→ State = (vertex, removed yet?, doubled yet?) → 4 layers. Taking an edge of weight w: normal +w, remove it +0, double it +2w, or both on the same edge +w. Dijkstra on 4n states, answer dist[v][both used] for every $v \ge 2$. Same 'layered map' idea as Flight Discount.

Pac-Takahashi (AtCoder ABC301 E) — Grid ≤ 300×300 with S, G, walls and ≤ 18 candies. Reach G within T moves; maximise distinct candies visited (or −1).

→ Visiting candies in some order with a step budget = mini travelling salesman. dp[mask][i] = fewest steps to collect the candies in mask, standing on candy i. dp over $2^{18}\cdot 18$ states, transitions to an unvisited candy j. A state is valid if dp[mask][i] + dist(i, G) ≤ T → answer max popcount(mask) (or 0 if just S→G fits, −1 if not even that).

Chemical Reaction (Dhaka Preli 2025 H (Gym 106315 H)) **[Dhaka]** — n initial chemicals, m ≤ 1000 rules (x, y → z). Reactants are not used up. How many distinct chemicals exist after 10^{100} seconds?

→ Don't rescan all rules every second. Use a to-do queue: when a chemical appears, look only at the rules that use it. If both reactants exist and the product doesn't, the product appears → push it into the queue. BFS-style worklist over chemicals; each rule is checked a couple of times only. Answer = number of chemicals present at the end. O(m log m) with the map.

A Lonely Queen (Dhaka Preli 2021 F) **[Dhaka]** — n×m board ($\Sigma\, n\cdot m \le 10^5$) with obstacles and one-way tunnels. A queen move of any length in direction i costs C_i (can't jump over obstacles). Min cost S → D.

→ So each cell has 9 states: 8 'sliding' states + 1 'standing' state. Tunnels go from standing to standing with their cost. Obstacles simply block the next step. Dijkstra on 9·n·m states ($\Sigma\, n\cdot m \le 10^5$, fine). Answer dist(D, standing), or −1.

Greg and Graph (Codeforces 295B) — Complete weighted digraph n ≤ 500. Vertices are deleted one by one; before each deletion print the sum of shortest paths between all remaining pairs.

→ After adding k, update every pair d[i][j] = min(d[i][j], d[i][k] + d[k][j]), then sum d[i][j] over the vertices added so far. Process the deletion order backwards, one Floyd step each, record the sum; print in the original order. $O(n^3)$.

Cycle Finding (CSES 1197) — Directed graph with possibly negative weights, n ≤ 2500, m ≤ 5000. Find any negative cycle or print NO.

→ The node relaxed in round n might just hang off the cycle. Walk its parent pointers n times — now you're surely standing on the cycle. From there follow parents until you return, then reverse to print the cycle in edge order. O(n·m).

The Shortest Statement (Codeforces 1051F) — Connected weighted graph, n, m ≤ 10^5, m − n ≤ 20. q ≤ 10^5 shortest-distance queries.

→ There are at most 42 such special endpoints. Run Dijkstra from each one. Answer = min(tree distance via LCA, min over special s of d_s[u] + d_s[v]). O(42·m log n + q·42).

Edge Deletion (Codeforces 1076D) — Weighted graph n, m ≤ $3 \cdot 10^5$. Keep at most k edges so that as many vertices as possible keep their shortest distance from 1. Print kept edges.
→ Dijkstra pops vertices exactly top-down! When a vertex is popped (finalised), its parent edge connects it to an already-good vertex. Record the parent edge of each vertex when it's popped; output the first min(k, n−1) of them.

Bicycles (Codeforces 1915G) — n, m ≤ 1000. Each city sells a bike with slowness s_i ≤ 1000; a road of length w costs w·(slowness of your bike). Cheapest trip 1 → n.
→ At city v you may swap to v's bike for free: new slowness = min(current, s_v). Riding road (v, u, w) costs w × slowness. Dijkstra on (city, slowness); the first time you pop city n, that's the answer.

Shortest Path (Codeforces 59E) — n ≤ 3000, m ≤ $2 \cdot 10^4$, k ≤ 10^5 forbidden triples (a, b, c): you may not visit a, b, c consecutively. Shortest path 1 → n and the path.
→ BFS over states: from (a, b) you may go to (b, c) unless (a, b, c) is forbidden. Store forbidden triples in a set/hash. BFS from (0, 1); the first state with current city n gives the length. Follow saved parents to print the path.

Legacy (Codeforces 786B) — n, q ≤ 10^5 portal plans: v → u, v → every planet in [l, r], or every planet in [l, r] → v, each with a cost. Shortest distances from s.
→ Build an 'up tree': children have 0-cost edges to their parent, so any planet in a range can climb to the node covering it, then leave. Leaves of both trees are the planets (linked both ways with 0 cost). Type 2: v → down-tree nodes of [l, r]. Type 3: up-tree nodes of [l, r] → v. Dijkstra from s over ~8n nodes. Classic 'segment tree as a graph'.

18.35 Graphs and trees: MST tricks

KEYWORDS: mst tricks

Make It Connected (Codeforces 1095F) — n ≤ $2 \cdot 10^5$ vertices with values a_i. Adding edge (x, y) costs a_x + a_y, or w for m special offers. Min cost to connect everything.
→ So the only useful normal edges are the n−1 'star' edges to v_0, plus the m special offers. Kruskal on those n−1+m edges. O((n+m) log).

Minimum spanning tree for each edge (Codeforces 609E) — Connected weighted graph, n, m ≤ $2 \cdot 10^5$. For every edge, the weight of the cheapest spanning tree that contains it.
→ So the answer is W − (max edge on the tree path u–v) + w. If the edge is already in the MST, this gives W. Kruskal + binary lifting that stores 'max edge on the jump' (like LCA). Each edge queried in O(log n).

New Roads Queries (CSES 2101) — n cities, m roads built one per day. q queries (a, b): first day after which a and b are connected, or −1.
→ a and b get connected exactly on the day of the latest road on their forest path. DSU to build the forest, binary lifting with max-edge, −1 if they're in different trees, 0 if a = b.

GCD and MST (Codeforces 1513D) — Array a, n ≤ $2 \cdot 10^5$, number p. Edge (i, j) costs min(a[i..j]) if that equals gcd(a[i..j]); also adjacent i, i+1 cost p. MST weight.
→ Spread left and right from i while values are multiples of x, joining each gap between neighbours. Stop at a gap already joined — it's already in a cheaper group. Each gap gets joined once by the smallest useful value (cost x); the remaining gaps cost p each. O(n log n).

18.36 Graphs and trees: Tree DP and rerooting

KEYWORDS: tree dp and rerooting

Tree Matching (CSES 1130) — Tree with n ≤ 2·10⁵ nodes. Max number of edges with no shared endpoint.
→ So go from the deepest nodes up to the root (reverse BFS order). Whenever a node and its parent are both free, pair them. Greedy from leaves up, O(n). (DP version: dp[v][0/1] = v unmatched/matched — good to write once too.)

Tree Distances I (CSES 1132) — Tree n ≤ 2·10⁵. For every node, the maximum distance to any other node.
→ Then for every v, its farthest node is a or b. Three BFS: from 1, from a, from b. Answer = max(dist(v, a), dist(v, b)). O(n).

Tree Distances II (CSES 1133) — Tree n ≤ 2·10⁵. For every node, the sum of distances to all other nodes.
→ The other n − sz[v] nodes each get 1 step farther. So ans[v] = ans[p] − sz[v] + (n − sz[v]). One pass for subtree sizes (bottom-up), one pass top-down using the formula. This is the rerooting template — learn it by heart!

Tree with Maximum Cost (Codeforces 1092F) — Tree with vertex values a_i. Choose a root v to maximise Σ dist(v, i)·a_i.
→ So ans[v] = ans[p] + S − 2·sub[v]. Compute ans[1] = Σ depth·a, push the formula down the tree, take the max. O(n). Dhaka Preli 2021 G uses the same rerooting idea.

Maximum White Subtree (Codeforces 1324F) — Tree, each vertex white or black. For each v: max (white − black) over connected subgraphs containing v.
→ The best blob for v may also go upward through the parent. The parent's best without v's part is res[p] − max(0, dp[v]). res[v] = dp[v] + max(0, res[p] − max(0, dp[v])). Bottom-up for dp, top-down for res. O(n).

Distance in Tree (Codeforces 161D) — Tree n ≤ 5·10⁴, k ≤ 500. Count pairs of vertices at distance exactly k.
→ Merge children one by one. For a new child ch: pairs += Σ_d c[v][k−1−d]·c[ch][d] (one end already merged, other end in ch). Then add c[ch] into c[v] shifted by 1. O(n·k). Use int32 arrays to save memory (5·10⁴ × 501). Same 'count at the top node' idea as Dhaka Preli 2016 D.

Alice, Bob and Tree (Dhaka Preli 2021 G) **[Dhaka]** — Tree N ≤ 3·10⁴ with weights W. Choose root r maximising Σᵢ (W_r − Wᵢ)·(−1)^dist(r,i) (smallest label on ties).
→ Σ (W_r − W_i)(±1) = W_r·(cnt_same − cnt_other) − (sum_same − sum_other). Count nodes and weight sums per colour once, evaluate every root in O(1), keep the max (smallest label on ties). O(N). Rerooting also works, but colouring is simpler.

Double Trouble (Dhaka Preli 2016 D) **[Dhaka]** — Tree N ≤ 10⁵, each node 0 or 1. Count ordered pairs (home, office) of 0-nodes whose path has exactly two 1-nodes.
→ At v, merge children one by one keeping acc[k] (v itself counts as an endpoint if it's 0). New pairs: acc[k1]·f[c][k2] whenever k1 + k2 + a[v] = 2. Then add f[c] into acc. f[v][k + a[v]] = acc[k] (ignore counts above 2). Ordered pairs → multiply by 2. O(N·9).

Christmas Tree (exactly K children) (Dhaka Regional 2017) **[Dhaka]** — Rooted tree N ≤ 1000, K ≤ 100. Delete whole subtrees so every non-leaf has exactly K children. Max nodes left.
→ If keeping K children, obviously keep the K children with the biggest f. f(v) = 1 + sum of top K f(child) if v has ≥ K children, else 1. Answer f(1). O(N log N).

Road Improvement (Codeforces 543D) — Tree, n ≤ 2·10⁵. For every capital x: number of ways to 'improve' a set of roads so that every path from x has at most one bad road. Mod 10⁹+7.
→ Trap: you can't divide by (dp[c] + 1) because it can be 0 mod p! Use prefix and suffix products over v's children instead. ans[v] = dp[v]·up[v], with up[root] = 1. O(n).

Tree Painting (Codeforces 1187E) — Tree n ≤ 2·10⁵. You paint vertices one by one (each new one adjacent to a painted one); each paint scores the size of the white component containing it. Max total.
→ Moving the root p → v: v's subtree size becomes n, p's becomes n − sz[v], all others stay the same. ans[v] = ans[p] − sz[v] + (n − sz[v]). Take the max over all v. O(n).

Sergey and Subway (Codeforces 1060E) — Tree n ≤ 2·10⁵. Add an edge between every two vertices at distance 2. Sum of all pairwise distances in the new graph.
→ Σ d over all pairs = Σ over edges sz·(n − sz). Odd distance ⇔ the two nodes are in different depth-parity colours. Answer = (S + even·odd) / 2. O(n).

The Fair Nut and the Best Path (Codeforces 1083A) — Tree n ≤ 3·10⁵ with gas w_i at vertices and road lengths. Choose a path; gain Σw − Σroads. Max gain.
→ At each v, candidate answer = w[v] + top1 + top2 of (best[c] − edge), counting only positive ones. One bottom-up pass, O(n). The 'never negative gas' rule doesn't change the maximum.

Parsa's Humongous Tree (Codeforces 1528A) — Tree, vertex v gets a value in [l_v, r_v]. Maximise Σ over edges |a_u − a_v|.
→ dp[v][x] = Σ over children max(dp[c][0] + |val_v − l_c|, dp[c][1] + |val_v − r_c|). Answer max(dp[root][0], dp[root][1]). O(n).

Serval and Rooted Tree (Codeforces 1153D) — Rooted tree; internal nodes are max or min of children; leaves get 1..k (k leaves) in some order. Maximise the root.
→ Max node: just follow the best child → min of dp[child]. Min node: all children must be big → sum of dp[child]. Leaf: 1. Answer = k − dp[root] + 1. O(n).

Diameter Cuts (Codeforces 1499F) — Tree n ≤ 5000, k. Count sets of edges to delete so every resulting tree has diameter ≤ k (mod 998244353).
→ Merging child c: cut edge v–c → multiply by (sum of all dp[c]). Keep it → only if a + b + 1 ≤ k, and the new depth is max(a, b + 1). Bound array sizes by subtree height so the total is O(n²). Answer = Σ dp[root].

18.37 Graphs and trees: Tree greedy and contribution

KEYWORDS: tree greedy and contribution

Linova and Kingdom (Codeforces 1336A) — Tree, capital 1. Choose exactly k cities as industry; each envoy goes to the capital, and its happiness = tourism cities on its path. Maximise total.
→ So each city has a fixed value: value[v] = depth[v] − (sz[v] − 1). Sort values, sum the top k. O(n log n).

Maximum Distributed Tree (Codeforces 1401D) — Tree n ≤ 10⁵. Put positive integers on edges whose product is k (given as primes), with as few 1s as possible. Maximise Σ over all pairs of path sums.
→ Weights must multiply to k. If there are more primes than edges, multiply the extra biggest primes into the top edge. If fewer, pad with 1s. Sort c desc (by real value, not modded!), weights desc, pair them, sum mod 10⁹+7.

Trees Are Beautiful (Dhaka Regional 2019 E) **[Dhaka]** — Weighted tree N ≤ 2·10⁴. Operation: +1 on a negative edge. Min operations so the sum of all-pairs distances is ≥ 0.
→ To close the gap with as few operations as possible, use the most important edges first. Sort negative edges by c descending; take k = min(|w|, ceil(need / c)) from each until need ≤ 0.

Orient the Tree (Dhaka Preli 2018 J) **[Dhaka]** — Tree N ≤ 10⁵. Direct every edge to minimise the length of the longest directed path. Print the edges.
→ Colour the tree with 2 colours. Direct every edge from colour 0 to colour 1: every node is then only a source or only a sink — no middle nodes! BFS colouring, print u→v with col[u] = 0. O(N).

Tree with Small Distances (Codeforces 1029E) — Tree n ≤ 2·10⁵. Add the fewest edges so that every vertex is within distance 2 of vertex 1.
→ So go deepest-first. When a vertex has distance > 2, connect its parent: parent's distance becomes 1, and all its neighbours get ≤ 2. Greedy, O(n). Same flavour as Dhaka Regional 2025 D.

Save the Wonderland (Dhaka Regional 2025 D (Gym 106270 D)) **[Dhaka]** — Tree N ≤ 10⁵, K guards with the same radius R; a guard covers all nodes within distance R. Smallest R so K guards cover the tree.
→ Each node carries two numbers: (farthest uncovered node below me, nearest guard below me). If they 'meet' (sum ≤ R), the uncovered one is already safe. Post-order with those two values, place a guard when uncovered distance = R, one more at the root if something is left. Count ≤ K? O(N log N).

18.38 Graphs and trees: LCA and path tricks

KEYWORDS: lca and path tricks

Company Queries II (CSES 1688) — Company tree n, q ≤ 2·10⁵ (boss of each employee given). Each query: lowest common boss of a and b.
→ First lift the deeper person up to the same depth as the other, using the bits of the depth difference (13 = 8 + 4 + 1 jumps). If they meet, that's the answer. Otherwise jump both up together with the biggest jumps that keep them different; at the end their parent is the LCA. O(log n) per query — practise until you can type it in 3 minutes.

Distance Queries (CSES 1135) — Tree n, q ≤ 2·10⁵. Distance between a and b for each query.
→ Add them: dep[a] + dep[b] − 2·dep[lca]. Binary lifting LCA + the formula. Weighted tree? Use distance-from-root instead of depth.

Tree Queries (Codeforces 1328E) — Rooted tree. Each query gives k vertices: is there a root-to-u path such that each given vertex is on it or adjacent to it?
→ That root path should end at the deepest of them, and every other one must be an ancestor of that deepest vertex. Euler times: x is an ancestor of y ⇔ tin[x] ≤ tin[y] and tout[y] ≤ tout[x]. Check all, O(k) per query.

Counting Paths (CSES 1136) — Tree n, m ≤ 2·10⁵. m paths are given. For each node, how many paths contain it?
→ For path a–b with l = lca: +1 at a, +1 at b, −1 at l, −1 at parent(l). After subtree sums, exactly the nodes on the path got +1. Process all paths, then one bottom-up pass adding each node's value to its parent. O((n+m) log n).

Misha, Grisha and Underground (Codeforces 832D) — Tree n, q ≤ 10⁵. Given three stations a, b, c: choose start s, finish f, other start t (all among them) to maximise the number of stations on both s→f and t→f.
→ Its length in edges is (d(s,f) + d(t,f) − d(s,t)) / 2, and the number of stations is that + 1. Three LCA-distance formulas per query, take the max. This 'shared part' formula is the heart of path-intersection problems (Dhaka 2018, 2024).

Path Intersection (Dhaka Regional 2018 F (Gym 102040 F)) **[Dhaka]** — Tree N ≤ 10⁴, Q queries, each gives K ≤ 50 paths. Print the number of nodes common to all K paths.
→ When is it empty? If the two deepest are the same node and that node is above lca(a,b) or above lca(c,d). Keep a current path (a, b), intersect with each new path; answer dist(a, b) + 1, or 0 once empty.

Passable Paths (hard) (Codeforces 1702G2) — Tree n ≤ 2·10⁵, q queries; each gives a set of vertices (Σ ≤ 2·10⁵). Is there a simple path containing all of them?
→ Now test each x: x lies on path a–b ⇔ dist(a, x) + dist(x, b) = dist(a, b) (no detour). LCA distances for every vertex in the query. O(k log n) per query.

Fools and Roads (Codeforces 191C) — Tree n ≤ 10^5, k paths. For each road (in input order), how many paths use it?
→ Path a–b: +1 at a, +1 at b, −2 at lca (both halves stop at the lca, and the lca's own upward edge is not used). Subtree sums; the answer for an edge is the value at its child endpoint. Print in input order.

A and B and Lecture Rooms (Codeforces 519E) — Tree n ≤ 10^5, queries (x, y): how many vertices are at equal distance from x and y?
→ Same depth: mid is the lca; everything except the two branches toward x and toward y works → n − sz(branch to x) − sz(branch to y). Different depths: answer = sz[mid] − sz[the vertex just below mid toward x]. Binary lifting for 'k-th ancestor'.

18.39 Graphs and trees: Euler tour, HLD, small-to-large

KEYWORDS: euler tour, hld, small-to-large

Subtree Queries (CSES 1137) — Tree with node values, n, q ≤ $2 \cdot 10^5$. Change a value; ask the sum of a subtree.
→ That's exactly what a Fenwick tree (BIT) does. Iterative DFS for tin/tout (recursion $2 \cdot 10^5$ deep may crash), BIT over positions. O(log n) per query.

Path Queries (CSES 1138) — Tree with values. Change a value; ask the sum on the path from the root to s.
→ And 'ask s' = read the single position tin[s]. Range add + point query = Fenwick on a difference array. O(log n).

New Year Tree (Codeforces 620E) — Tree n ≤ $4 \cdot 10^5$, colours ≤ 60. Queries: paint a whole subtree one colour; count distinct colours in a subtree.
→ Segment tree storing OR of masks, with lazy 'assign this colour' on ranges. Range assign + range OR query, answer = popcount. O(log n) per operation.

Lomsat gelral (Codeforces 600E) — Rooted tree n ≤ 10^5, colours. For every vertex, the sum of all colours that appear the most times in its subtree.
→ An element moves only when its table at least doubles, so each moves at most log n times. Keep, with each table, the best count and the sum of colours having that count; update while pouring. O(n $\log^2$ n).

Lazy Squirrel (Dhaka Preli 2021 D) **[Dhaka]** — Rooted tree N ≤ 10^6 with a letter on each node. For how many nodes x is the root→x string a palindrome? Print as a reduced fraction of N.
→ Use two moduli (or $2^{61}-1$) so collisions don't fool you. BFS order (N = 10^6, no recursion), count good nodes, print good/N reduced by gcd.

Dominant Indices (Codeforces 1009F) — Tree n ≤ 10^6. For each vertex, the depth (below it) with the most vertices in its subtree; smallest such depth on ties.
→ Merge the other children element by element. Each element is merged once per 'long chain', so the total is O(n). Store arrays reversed so 'add depth 0' is just push_back. Track the best depth while merging (smallest depth on ties).

Water Tree (Codeforces 343D) — Rooted tree n ≤ $5 \cdot 10^5$. Operations: fill v's subtree; empty v and all its ancestors; ask if v is full.
→ Emptying u also empties all ancestors of u. So v is affected by an emptying of any node in v's subtree! Store empty time at tin[u]; v's last empty = max over [tin[v], tout[v]]. Full ⇔ lastFill > lastEmpty. Two segment trees, O(log n) each.

Vasya and a Tree (Codeforces 1076E) — Tree n, m ≤ $3 \cdot 10^5$. Each query adds x to all vertices in v's subtree at depth ≤ dep(v) + d. Print final values.
→ Fenwick over depth: add x at dep(v), subtract at dep(v)+d+1 when entering v; undo when leaving. Each node reads the prefix sum at its own depth. Iterative DFS, O((n+m) log n).

Tree Requests (Codeforces 570D) — Tree n, m ≤ $5 \cdot 10^5$ with letters. Query (v, h): can the letters at depth h inside v's subtree be rearranged into a palindrome?
→ For each depth keep (tin, prefix XOR of masks). Binary search the run [tin[v], tout[v]] and XOR two prefixes. popcount(mask) ≤ 1 → Yes. O((n+m) log n).

Path Queries II (CSES 2134) — Tree n, q ≤ $2 \cdot 10^5$. Change a node value; ask the max value on the path a–b.
→ Lay chains out contiguously in one array. Query: while a and b are on different chains, take the one whose chain head is deeper, query that chain piece, jump above its head. Segment tree over HLD order. O($\log^2$ n) per query. Your regional template should have HLD.

18.40 Graphs and trees: DAG, SCC, functional graphs

KEYWORDS: dag, scc, functional graphs

Game Routes (CSES 1681) — DAG n ≤ 10^5. Number of routes from 1 to n mod 10^9+7.
→ Kahn's algorithm: repeatedly take a node with in-degree 0. ways[1] = 1; for u in topo order, ways[v] += ways[u] for every edge u→v. O(n+m).

Coin Collector (CSES 1686) — Directed graph with coins in rooms. Each room's coins can be taken once. Max coins on a route.
→ Now it's 'max coins on a path in a DAG' = DP in topological order. Kosaraju gives components already in topological order; best[x] = coins[x] + max best of successors. O(n+m).

Planets Queries II (CSES 1160) — Every planet has one teleporter (functional graph). Queries: minimum teleports from a to b, or −1.
→ If b is on a tree part, b must be on a's way down: d[a] ≥ d[b] and jumping a forward d[a] − d[b] steps lands on b (binary lifting). If b is on a's cycle: d[a] + (pos[b] − pos[entry]) mod len. Otherwise −1.

Reachability Queries (CSES 2143) — Directed graph n ≤ $5 \cdot 10^4$, m, q ≤ 10^5. Can a reach b?
→ For a block of 64 sources, give each one bit, then push masks along DAG edges in topological order (mask[v] |= mask[u]). Answer queries of this block by reading one bit. O((n+m)·n/64). Dhaka Regional 2024 A was built on this kind of reachability.

Checkposts (Codeforces 427C) — Directed graph n ≤ 10^5 with costs. A checkpost at i protects j if i → j → i. Protect all with min cost; also count min-cost ways.
→ The number of ways = product over SCCs of how many junctions tie for that minimum cost. Kosaraju + per-component min and count, ways mod 10^9+7. O(n+m).

Substring (Codeforces 919D) — Directed graph n, m ≤ $3 \cdot 10^5$ with a letter per node. Path value = max frequency of a letter on it. Max value, or −1 if infinite.
→ Use Kahn's algorithm. If it can't process every node, there's a cycle. dp[u][c] = max(dp[u][c], dp[v][c] + [letter of u = c]). O(26(n+m)), int32 to save memory.

18.41 Graphs and trees: Bridges, articulation points, 2-SAT

KEYWORDS: bridges, articulation points, 2-sat

Necessary Roads (CSES 2076) — Connected graph. Print all roads whose removal disconnects it (bridges).
→ Skip the edge you arrived by using its id, not its endpoint, so parallel edges are handled correctly. Iterative DFS (10^5 depth). O(n+m).

Necessary Cities (CSES 2077) — Connected graph. Print all cities whose removal disconnects it (articulation points).
→ The root is special: it's critical ⇔ it has 2 or more DFS children. Iterative DFS, O(n+m).

We Need More Bosses (Codeforces 1000E) — Connected graph. Choose s, t to maximise the number of edges that every s→t path must use.
→ Now: the s–t path with the most edges in that tree = its diameter. Bridges → components → bridge tree → two BFS. O(n+m).

Giant Pizza (CSES 1684) — m toppings, n wishes 'a or b' (each with + or −). Choose toppings satisfying everyone, or IMPOSSIBLE.
→ Otherwise choose x true when its SCC comes later in topological order than not-x. Implication graph on 2m literals, Kosaraju, compare components. O(n+m).

Pursuit For Artifacts (Codeforces 652E) — Connected graph; some edges hold an artifact. Every edge breaks after you cross it. Can you go from a to b and pick an artifact on the way?
→ In a tree there's only one way from a's block to b's block, and you must follow it. YES ⇔ some block or bridge on that tree path has an artifact. Bridges + BFS on the bridge tree.

18.42 Graphs and trees: Matching, König, flow

KEYWORDS: matching, könig, flow

School Dance (CSES 1696) — n boys, m girls (≤ 500), k ≤ 1000 allowed pairs. Max number of dance pairs; print them.
→ Use a visited stamp per attempt so each girl is tried once per boy. O(V·E), fine for 500. Print mt[girl] for matched girls.

SAM I AM (UVa 11419) — R×C grid with N targets. A cannon shot clears a whole row or column. Fewest shots to clear all targets, and which rows/columns.
→ To print the cover: from every unmatched row, walk alternating paths (any edge to a column, then the matching edge back to a row). Cover = rows NOT visited + columns visited. Kuhn + König construction. Exactly what Dhaka Preli 2024 B asked you to print.

Download Speed (CSES 1694) — Directed network n ≤ 500, m ≤ 1000 with capacities. Max flow 1 → n.
→ Dinic does this fast: BFS levels, then DFS only along increasing levels, with a pointer per node. Capacities up to 10^9 → long long. Keep Dinic in your notebook.

Police Chase (CSES 1695) — Undirected graph. Remove the fewest streets so that 1 and n are disconnected; print them.
→ Every original street with one end on each side is in the cut. Dinic + one BFS on the residual graph.

Network Disentanglement (Dhaka Preli 2024 B) **[Dhaka]** — M ≤ 1000 wire segments, each owned by ISP X or Y. Remove the fewest wires so no X-wire touches/crosses a Y-wire; print which.
→ The segment test must include touching endpoints and collinear overlaps — use cross products in __int128 and bounding-box checks. 10^6 segment tests, Kuhn matching, König construction (unvisited left + visited right), print sorted. This was the 2024 gate problem.

Maximize Mex (Codeforces 1139E) — n students (potential $p_i < 5000$, club c_i). Each day one student leaves, then you pick one student per club to maximise mex of picked potentials.
→ Backwards, the answer only grows: keep trying to match the next potential (mex) with Kuhn. Each potential gets matched at most once overall → about O(5000·E).

Petya and Graph (Codeforces 1082G) — n, m ≤ 1000. Vertex weights a, edge weights w. Choose a subgraph (edge needs both ends); maximise Σw − Σa.
→ Cutting source→edge means 'give up this edge'; cutting vertex→sink means 'pay for this vertex'. The min cut is the cheapest combination. Answer = Σw − maxflow. Dinic.

Distinct Routes (CSES 1711) — Directed graph n ≤ 500, m ≤ 1000. Max number of 1 → n routes that share no road; print them.
→ Walk from 1 following roads with flow, deleting each road as you use it, until you reach n. Repeat k times. Dinic + path peeling.

18.43 Graphs and trees: Centroid decomposition and XOR on trees

KEYWORDS: centroid decomposition and xor on trees

Fixed-Length Paths I (CSES 2080) — Tree n ≤ $2 \cdot 10^5$, k. Count paths with exactly k edges.
→ Through c: go branch by branch. cnt[d] = nodes at depth d in earlier branches (c itself at depth 0). For a node at depth d in the new branch, add cnt[k − d]; then add the branch into cnt. Recursion depth is log n → O(n log n).

Xenia and Tree (Codeforces 342E) — Tree n, m ≤ 10^5. Node 1 is red. Queries: paint a node red; distance from a node to the nearest red node.
→ Paint v: for every centroid ancestor c, best[c] = min(best[c], dist(v, c)). Ask v: min over c of best[c] + dist(v, c). Each node has only log n centroid ancestors — precompute (centroid, distance) lists. O(log n) per operation.

XOR Distance Counting (Dhaka Regional 2017) **[Dhaka]** — Tree n ≤ 10^5, 16-bit edge weights, distance = XOR of path weights. For every x in [0, 2^{16}), count pairs at distance x.
→ Now: count pairs of numbers with a given XOR. cnt[a] = how many nodes have x = a. Pairs with XOR t = Σ cnt[a]·cnt[a ⊕ t] — an XOR-convolution. Fast Walsh–Hadamard transform: transform, square, transform back; subtract n at 0 (pairs with themselves) and halve.

Mr. Dumb the Police in charge (Dhaka Preli 2022 H) **[Dhaka]** — Tree N ≤ 10^5, M requests (time, city). The officer always goes to the nearest pending city (ties: earliest request), only choosing again on arrival. Print the visiting order.
→ Each centroid's set holds (distance to the centroid, request time, city). For a query from pos: over pos's centroid ancestors, take each set's smallest and add dist(pos, centroid); keep the best (distance, time). Insert/erase in all O(log) ancestor sets. O((N+M) $\log^2$ N).

Fixed-Length Paths II (CSES 2081) — Tree n ≤ $2 \cdot 10^5$. Count paths whose length is between k1 and k2.
→ Keep earlier depths in a Fenwick tree and ask a range count. O(n $\log^2$ n).

18.44 Graphs and trees: Tree isomorphism and hashing

KEYWORDS: tree isomorphism and hashing

Tree Isomorphism I (CSES 1700) — Two rooted trees (root 1), n ≤ 10^5. Are they the same shape?
→ Compute bottom-up: a leaf's list is empty → always the same id. Compare the two root ids. O(n log n).

Tree Isomorphism II (CSES 1701) — Two unrooted trees. Are they isomorphic?
→ Compare with the rooted method from Tree Isomorphism I. O(n log n). Dhaka Regional 2024 F (flower shapes) uses this kind of shape hashing.